

Transformers IN ACTION

Nicole Koenigstein

MEAP



MANNING





MEAP Edition
Manning Early Access Program
Transformers in Action
Version 7

Copyright 2024 Manning Publications

For more information on this and other Manning titles go to
manning.com

welcome

Thank you for purchasing the MEAP for *Transformers in Action*, your comprehensive guide to understanding and implementing Transformer models across various domains.

Transformers have established themselves as a indispensable tool in the field of machine learning and artificial intelligence as the research and deployment of Large Language Models (LLMs) continues to expand.

This book will take you on a fascinating journey through the applications of Transformers, which have, in recent years, evolved from their initial use in natural language processing (NLP) to a wide array of domains. These include, but is not limited to, computer vision, speech recognition, reinforcement learning, mathematical operations, and the study of biological systems such as protein folding. The most notable innovations have been the emergence of decision Transformers and multimodal models. These groundbreaking models have the potential to reshape our understanding of deep learning and broaden its horizons.

This book is designed for a diverse audience: ML engineers, data scientists, researchers, students, and AI practitioners who are eager to harness the potential of Transformer models in various domains.

Throughout this book, we will use a hands-on approach to help you build a strong knowledge for Transformers and their applications. Each chapter comes with detailed examples, visualizations, and case studies, accompanied by an extensive repository of worked examples to assist you in mastering the concepts. We will guide you through fine-tuning transformers for your own use cases, optimizing their performance, and deploying them efficiently, ensuring you have the practical skills to achieve the best results. To further support your learning, we have prepared a detailed guide on the next page, outlining how to effectively run the accompanying code notebooks throughout this book.

To get the most out of this book, you should have intermediate Python skills, a basic understanding of machine learning, deep learning fundamentals, and essential mathematical concepts related to ML, such as linear algebra, statistics, and calculus. It is also helpful to have a basic understanding of NLP concepts, PyTorch, and Python tools like NumPy, Pandas, and Matplotlib. Don't worry if you need to review these basics, our examples and case studies will give you plenty of chances to learn and practice these skills.

We have designed this book with you, our reader, in mind. Your feedback is invaluable in helping us improve and enhance the content. Please share your thoughts, questions, comments, and suggestions in the [liveBook's Discussion Forum](#) for this book. We hope you enjoy this journey through the exciting world of Transformers and emerge with the knowledge and skills to tackle real-world challenges using these powerful models. Welcome aboard!

brief contents

PART 1: INTRODUCTION TO TRANSFORMERS

- 1 The need for transformers*
- 2 A deeper look into transformers*

PART 2: TRANSFORMERS FOR FUNDAMENTAL NLP TASKS

- 3 Text summarization*
- 4 Machine translation*
- 5 Text classification*

PART 3: ADVANCED MODELS AND METHODS

- 6 Text generation*
- 7 Controlling generated text*
- 8 Multimodal models*
- 9 Optimize and evaluate large language models*
- 10 Ethical and responsible large language models*

APPENDIX

Get the most out of this book -how to run the code

Part 1

Introduction to transformers

Since their introduction at the 31st Conference on Neural Information Processing Systems (NIPS 2017), transformers have become a popular topic of interest. The transformer is a type of machine learning model that, unlike its predecessors, can understand the context of words in a sentence by focusing on different parts of the sentence simultaneously. Exactly this fact makes transformers highly efficient in a variety of natural language processing (NLP) tasks, including text classification, summarization, and generation. They are being extensively used across many applications, underscoring the importance of understanding these models in full depth.

While you may be eager to get started and learn how to build your own ChatGPT model that can answer any questions you might have and even make predictions, it is critical to first grasp the principles of a transformer model. This is because, in order to be successful in this field and to use more advanced models, you must first establish a solid foundation.

Think of learning how to leverage transformers for your own use cases to be similar to growing a garden. You must first prepare the soil and establish a good foundation before you can grow your fancy and exotic plants. The first part of this book serves a similar function, providing you with the essential background knowledge to establish a solid foundation. That said, part 1 gives you a deeper look at the essential concepts and the basic mathematical foundations of the transformer, explaining why they are built the way they are and how they work. And with that, this part prepares you for the more complex and modern models as well as complex approaches in parts 2 and 3 of the book.

The need for transformers



This chapter covers

- How transformers revolutionized NLP
- Attention mechanism - the transformers key architectural component
- How to use transformers
- When and why you want to use transformers

The field of natural language processing (NLP) has undergone a revolutionary change with the invention of a new class of neural networks called transformers. These models, famous for their capacity to understand and generate natural language, are the backbone of widely-used applications such as Google Translate or ChatGPT.

Transformers, along with their derivatives like large language models (LLMs), are distinctive due to their unique architectural approach. Unlike their predecessors, like Long Short Term Memory (LSTMs), transformers incorporate an innovative architectural component called the "attention mechanism." This mechanism empowers the model to concentrate on different segments of the input data to varying extents, thereby enhancing its ability to process and comprehend complex sequential data. This capability is not only relevant to LLMs processing natural language, but also applies to the broader use of transformers in processing audio streams, images, and even video.

Let's start by comparing transformers with their predecessors, the LSTM models, and examine each component of a transformer in more detail.

1.1 The transformers breakthrough

Transformers are a type of deep learning model that use the attention mechanism to increase processing speed and improve performance in tasks involving sequential data. Introduced in the groundbreaking paper "Attention is all you need" (1), transformers marked a significant shift from the previously popular recurrent neural network architectures, such as LSTMs. Before the invention of transformers, LSTMs, a particular type of Recurrent Neural Networks (RNNs), were the predominant choice for processing sequential data, including natural language. RNNs, as the name suggests, handle sequences by iterating through elements and maintaining a form of "memory" about the information processed so far. To illustrate this, suppose we want to translate the sentence "I don't speak French" into "Je ne parle pas français". To translate the sentence, with an RNN, the architectural component of this neural network, the encoder, would process each word in the sentence "I don't speak French" one word at the time, updating its state with each word. Producing then a so-called context vector of the English sentence. This context vector encapsulates the semantics of the input sentence and serves as the bridge to the next component, the decoder, of the RNN. The decoder then uses this context vector to generate the output sequence: "Je ne parle pas français", again one word at a time, using its internal (recurrent) state to remember the previously generated words. Throughout this process, the recurrent nature of the architecture allows each step to build upon the previous ones, thereby crafting a coherent translation. This sequence-based processing enables the RNN to capture temporal dependencies within the sentence but can also lead to challenges in handling long-distance relationships between words. This process is visualized in figure 1.1.

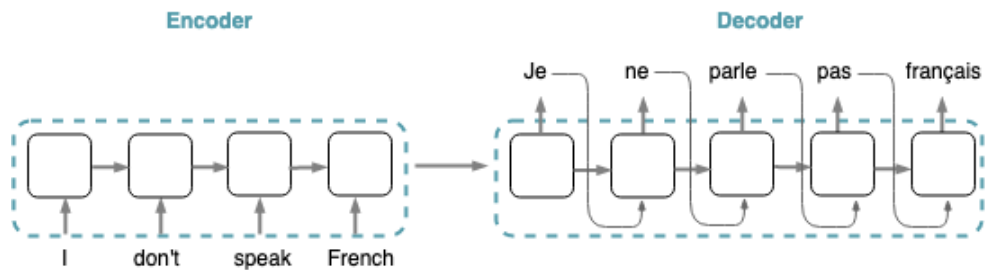


Figure 1.1 A high-level overview of the general flow of sequential data within an RNN. The encoder in an RNN processes the input sentence one word at a time, updating its internal state with each word. This culminates in a final context vector representing the entire sentence. The decoder then uses this context vector to generate the output sentence, again one word at a time, using its internal (recurrent) state to remember the previously generated words.

As we can see in figure 1.1, the "recurrent state" is updated for each item in the output sequence, capturing a contextual information about the parts of the sequence it has processed so far. However, RNNs come with some drawbacks, they struggle with long sequences, which makes them less efficient in capturing long-term dependencies in the data.

HOW ARE TRANSFORMERS DIFFERENT?

Transformers, on the other hand, pioneered a radically different strategy. They are built around the "attention mechanism," a powerful concept that enables them to take into account multiple parts of the input sequence concurrently when processing each individual part. Instead of processing the sequence element by element and carrying a single recurrent state forward, the transformer model calculates an attention score for each element in the context of all other elements in the sequence.

This fundamental shift allows transformers to be much more parallelizable and thus more efficient. It also dramatically enhances the model's capacity to understand intricate patterns and long-term dependencies in the data. Unlike traditional sequence-to-sequence models, transformers leverage this attention mechanism to create a more interconnected understanding of the entire sequence, which enhances the model's ability to make accurate predictions.

Moreover, the transformer's architecture enables the simultaneous processing of all elements in the sequence, significantly reducing computation time. To illustrate this process, again, suppose we want to translate the sentence "I don't speak French" into "Je ne parle pas français." Now, with the transformer model, the encoder extracts features from the input sentence, "I don't speak French." These features are then processed by the transformer, with the help of the attention mechanism, to calculate attention scores for each element. These scores effectively capture the contextual relationship between each word and the rest of the sentence. The decoder, another component of the transformer model, uses these attention scores and the extracted features to generate the translated sentence, "Je ne parle pas français." This high-level functionality of the translation with a transformer model is illustrated in figure 1.2.

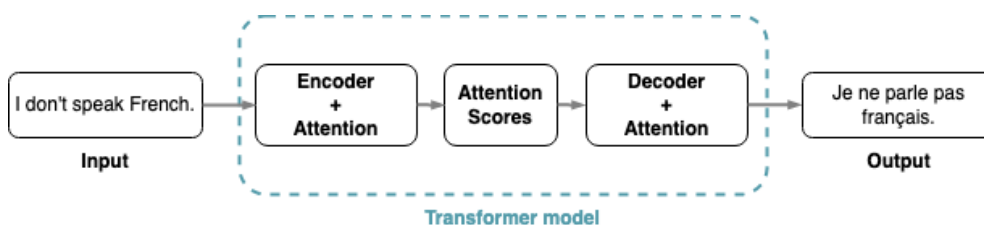


Figure 1.2 A high-level overview of the general flow of sequential data within a transformer architecture. Starting from the left, an input sentence is passed into the transformer, where it is first processed by the encoder along with the attention mechanism. The result, depicted as attention scores (also called contextual representations), are passed to the decoder with its own attention mechanism. The output is then the translated sentence.

As figure 1.2 demonstrates, the attention mechanism within the transformer model allows for the concurrent processing of the entire sequence, rather than updating a "recurrent state" for each item as in RNNs. This design enables the transformer to effectively capture the contextual relationship between all parts of the sequence, overcoming the challenges faced by traditional models like RNNs that can struggle with long sequences. The result is

a more efficient and powerful model, capable of handling intricate patterns and long-term dependencies in the data.

1.1.1 Unveiling the attention mechanism

The attention mechanism in transformers enables the model to weigh different positions of a sequence when computing a representation of that sequence. In essence, within the encoder-decoder architecture, the attention mechanism assesses the relevance of various input vectors and assigns higher weights to the most important ones. This stands in contrast to RNNs such as LSTM models, which process input sequences one item at a time.

To make this more clear, let's revisit our previous translation example where we translated the sentence "I don't speak French" into "Je ne parle pas français". An ideal translation model needs to comprehend each word in the context of the others in the sentence. In this instance, the translation of the word "speak" into "parle" is influenced by its surrounding words "I", "don't", and "French". This is illustrated in figure 1.3 where this connection is represented by the thickness of the edges, which represent the attention scores assigned by the model, demonstrating how the words relate to each other.

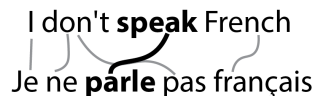


Figure 1.3 A sample translation made by a transformer model with attention. The thickness and color of the edges demonstrate the attention scores assigned by the model.

This attention mechanism is a key part of the transformer model, providing it the ability to process sequences with complex relationships between elements. However, our example sentence, while straightforward, demands the model to consider relationships between various words in the sequence through attention. In addition, as sequences grow more intricate and tasks more nuanced, the importance of advanced attention strategies, such as multi-head attention, becomes evident. Lets take a look at multi-head attention as we consider a more complicated NLP task: sentiment classification.

1.1.2 The power of multi-head attention

The transformer model introduces a powerful extension to the attention mechanism known as "multi-head attention." Multi-head attention enhances the models ability to capture multiple relationships within the sequence. This approach allows the model to focus on different positions of the input simultaneously, capturing various aspects of the information. This means the model can maintain multiple "perspectives" to better understand complex patterns in the data.

To illustrate the benefits of multi-head attention, let us take the sentence, "The movie was not bad". In this case, the transformer's multi-head attention allows the model to parse the interaction of "not" and "bad" simultaneously, thereby understanding that the

overall sentiment is positive. Figure 1.4 visualizes this interaction between the words in the sentence.

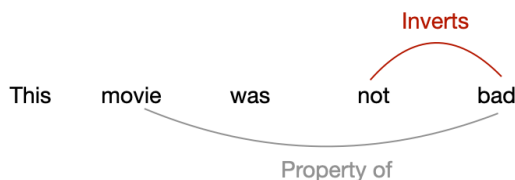


Figure 1.4 Example of different relations in a sentence.

In contrast, recurrent models like LSTMs often struggle with identifying such long-term dependencies within a sentence, leading to potential difficulties in recognizing these subtle nuances. Transformers, with their attention-based architecture, bypass this problem, capturing complex interrelationships within textual data efficiently.

In a nutshell, multi-head attention allows transformers to understand various aspects of the input simultaneously, marking a significant leap over previous models like LSTMs. Using this technique, the transformer was able to reach a score of 41 for a metric called BLEU (BiLingual Evaluation Understudy). This is a very high score and indicates that the generated translation from the transformer model is very similar to the reference text. Even more fascinating is that the first transformer model achieved this result after only 3.5 days of training. This was only a small fraction of the time it took for earlier state-of-the-art networks, like LSTMs, to achieve a similar high score for the same English-to-French translation task.

Moreover, in comparison to other RNN-based models like LSTMs, the success of the transformer and LLMs is more evident in terms of their revolutionary impact on the field of NLP. Although LSTMs were significant advancements for sequence processing, the speed at which LLMs and transformers conquered the field and state-of-the-art models are being continuously developed is unparalleled. As a result of the transformers' fast-paced and groundbreaking success, it was eventually possible to build such advanced neural networks such as ChatGPT just five years after the introduction of the first transformer model.

1.2 *How to use transformers*

Starting your journey with transformers is greatly simplified through the Transformers library of the machine learning platform Hugging Face. This library provides a vast amount of pre-trained models, facilitating tasks that range from language translation to text generation, text classification and sentiment analysis.

The term "pre-trained" refers to models that have been previously trained on extensive datasets, usually containing millions of documents, encompassing a broad spectrum of topics. This exposure allows the models to learn the nuanced patterns of language, including syntax, semantics, and context. By using these pre-trained models, you essentially gain the ability to leverage their learned knowledge, eliminating the need for training a model from scratch. This approach saves considerable computational resources and time, because

you now only have to train the model for your specific task at hand. For instance, using a labeled dataset consisting of pairs of sentences and the labels neutral, negative or positive, to perform a sentiment analysis.

Another factor to consider when using transformers is the processing power required. Transformers, given their complexity, can be computationally intensive. This is where GPUs come into play. GPUs are designed to carry out many operations concurrently. This makes them well-suited for the matrix calculations and parallel computations that are commonplace in training and using a transformer. Even if you don't have a high-powered GPU at your disposal, many cloud services, such as Google Colab or Paperspace, offer access to GPUs, making the use of transformers accessible to a wide audience.

Hence, with pre-trained models available through Hugging Face's Transformers library and the processing power offered by cloud services, you are well-equipped to harness the power of transformers for your own NLP tasks.

1.3 When and why you'd want to use transformers

Transformers have become an integral part of the modern machine learning landscape. They especially offer unparalleled capabilities in natural language processing, but their potential reaches even far beyond this field. As transformers have shown promise in domains like computer vision and audio recognition, hinting at a future where they may become a more general-purpose machine learning architecture.

However, the real charm of transformers lies in their accessibility. The Hugging Face's Transformers library is home to a diverse range of pre-trained models, making it easier for practitioners to get started. Unlike LSTM-based architectures, which often require training from scratch, transformers are via Hugging Face readily available in pre-trained forms, saving considerable time and computational resources.

This advantage is further magnified by the active community surrounding the Hugging Face's Transformers library. Being open-source, it benefits from constant contributions and improvements by machine learning enthusiasts worldwide. Consequently, it's often a matter of a few hours to fine-tune a pre-trained transformer model for a task like sentiment analysis.

Furthermore, the rise of zero-shot and few-shot learning techniques has expanded the applicability of transformers. Zero-shot learning refers to a model's ability to handle tasks it was never specifically trained for. Essentially, it can understand and perform unseen tasks based on its broad training. Few-shot learning, on the other hand, implies that the model can quickly learn to perform new tasks after being trained on a very small amount of data related to that task. This technique takes advantage of the model's pre-existing knowledge from its extensive training. These advanced techniques, combined with the resourcefulness of the community, make transformers an appealing choice for a broad array of tasks in the field of NLP.

Nonetheless, as impressive as these diverse models are, they do come with some limitations. If we consider the so-called billion-parameter models, which refers to the scale and learnable parameters or "weights" of these transformer models. For instance, early

transformer models had 110 million parameters but as advancements in the field continues, this has led to even larger models, possessing 175 billion parameters and more.

These "billion-parameter models," exhibit an impressive ability to generate coherent and contextually relevant responses in a conversation. However, despite their vast scale and impressive performance, these models do have their limitations. Their practical applicability might diminish in certain specialized domains like finance or healthcare, where domain-specific context is crucial. The computational and memory demands of these models can make their deployment challenging, especially in real-time systems that require quick and accurate results.

Nevertheless, the potential of these different variations of transformer models cannot be underestimated. They are pushing the boundaries of what's possible in natural language understanding and generation, text classification, translation, and more. But as with any tool, the key lies in a balanced, problem-specific approach. As we move forward, we will explore this balance in more depth, focusing on how to leverage the strengths of these different types of models while navigating their specific limitations.

1.4 Summary

- A transformer model employs attention and multi-head attention mechanisms. Using these tools, it expertly navigates through various parts of a sentence, shining a spotlight (signifying more attention) on the words that are pivotal in shaping the overall narrative and context comprehension.
- Attention allows the model to focus on key portions of the input and emphasizes the most important information.
- The Transformer model's multi-head attention component helps it to identify numerous links between words in a sequence, leading to its popularity as one of the most extensively used NLP models.
- Transformers have excelled in NLP tasks because of their capacity to handle long-term dependencies in sequential data.
- They revolutionized the field by making it possible to train a transduction model in a matter of days rather than weeks or months, yet outperforming state-of-the-art networks.
- Advanced techniques like zero-shot or few-shot learning allow large models to infer and generalize about new tasks based on their pre-existing training, without needing explicit retraining, enabling a more efficient use of resources and time.
- Despite their impressive capabilities, extremely large language models aren't without limitations. A potential issue is the decreased effectiveness in certain specialized domains like finance or healthcare. Furthermore, their computational and memory demands can make deployment challenging. Thus, selecting the most suitable model requires careful consideration of the specific task at hand, balancing the trade-off between model complexity and practical applicability.

Bibliography

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *CoRR*, vol. abs/1706.03762, 2017. 3

A deeper look into transformers

This chapter covers

- Sequence modeling before transformers
- Core components of a transformer model
- Attention mechanism and its variants
- How transformers can help stabilize gradient propagation

Now that we have a high-level understanding of the transformer model's benefits and that one of the primary goals for developing the transformer architecture was to reduce the sequential computation from previously used networks like LSTMs, we can delve deeper. This chapter aims to demystify the seemingly complex and daunting transformer model by breaking it down into its foundational elements. We'll dissect the model, highlighting simple concepts such as matrix multiplication operations, which are pivotal to the attention mechanism and its variant, the multi-head attention. As we journey through the architecture of the transformer, we'll unravel the importance of its building blocks, including feed-forward networks and positional encoding, and their contribution to its effectiveness in natural language processing tasks. But before we delve deeper into transformers, let's first briefly take a closer look at LSTMs and sequence-to-sequence models to better understand the unique advancements and benefits of transformers in more detail.

RNN-based and sequence-to-sequence (seq-2-seq) models have been widely employed in NLP tasks, such as machine translation. These models have the ability to take in variable length sequences as input and produce variable length sequences as output. To illustrate this, let us use our sentence from chapter 1 : "I do not understand French", which we want to translate into: "Je ne comprends pas le français". We can see that our input sequence is shorter (5 words) than our output sequence (6 words). This examples makes it clear that we need to have a model architecture which can handle variable length input and output sequences. Let us examine in detail how these models are constructed and where the transformer comes in. **Note:** all code examples of this chapter can be found in the book's repository, <https://github.com/Nicolepcx/Transformers-in-Action>, in folder CH02.

2.1 From seq-2-seq models to transformers

As you are aware, from chapter 1, the structure of most competitive neural sequence transduction models is encoder-decoder based. The RNN encoder-decoder architecture is used in machine translation tasks to read the source language sentence and construct a fixed-length representation, z , of it, which is then passed to the decoder, as shown in figure 2.1.

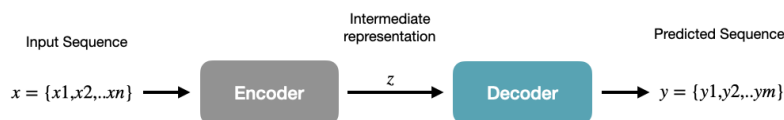


Figure 2.1 Illustration of Sequence to Sequence learning.

The decoder constructs the target language sentence by predicting one word at a time in an autoregressive mode, based on the previous words and the encoder's fixed-length representation. Sequence-to-sequence modeling is a process central to tasks where context capture is vital. The key advantage of this approach is its ability to comprehend the contextual semantics of a sentence, a crucial aspect in translation tasks. For example, the word "date" may refer to both a fruit and a social engagement, and I'm sure you don't want these two to be mixed up in a translation.

2.1.1 The difficulty of training RNNs

In training RNN-based encoder and decoder architectures, we face significant challenges. These issues can make improvements in BLEU scores difficult, when using LSTM-based encoder-decoder models. This holds true, especially when the model must translate between languages with vastly different syntax or vocabulary. To give you some more background on the BLEU score, it is a metric used in machine translation to quantitatively evaluate the quality of translated text by comparing it to a set of reference translations; a higher score indicates a closer match to the reference.

These challenges in training RNNs, particularly deep RNNs, are thoroughly analyzed in the paper "On the difficulty of training Recurrent Neural Networks". As explained by the authors of the paper, difficulties often arise due to the propagation of errors over extended

time sequences. To mitigate these issues, they suggest careful initialization and the use of non-saturating activation functions that avoid getting stuck in specific ranges. This allows for efficient gradient flow during backpropagation and batch normalization, contributing to improved network stability. Within LSTMs, "stability" refers to the model's capability to accurately and reliably learn and represent data patterns and relationships, without becoming overly sensitive to minor changes or noise in the input.

Nevertheless, it is often necessary to employ a deep neural network design, so that the model is capable of modeling the data's complex patterns and relationships and to effectively capture long-term dependencies in sequential data. This is usually accomplished, in the case of LSTMs, by stacking multiple recurrent layers on top of one another, enabling the network to learn increasingly complex representations of the input over time. However, as the number of layers in the network grows, it becomes more difficult to successfully propagate error signals through the network during training, which, in turn, results in the vanishing gradient problem and other stability issues. In the following section, section 2.1.2, we will look more closely at the vanishing gradient problem.

Hence, even though RNNs like LSTMs are designed to capture long-term dependencies in sequential data by employing a memory cell that can selectively remember or forget information from earlier time steps, they can still struggle to learn these dependencies effectively. One solution to this problem is the introduction of attention mechanisms for LSTMs. These mechanisms enable the models to account for long-term dependencies; however, the attention mechanism itself is different and less effective than the one used in transformers. Moreover, in attention-enhanced LSTMs, the input is still fed into the network sequentially, which makes training and inference slow.

In contrast, transformers employ an attention architecture that allows for more efficient processing of long-range dependencies, as shown in figure 2.2. Another advantage of the transformer architecture is the use of multi-head attention, which enables the model to capture different aspects of the input data in parallel, further enhancing its ability to process long-range dependencies effectively.

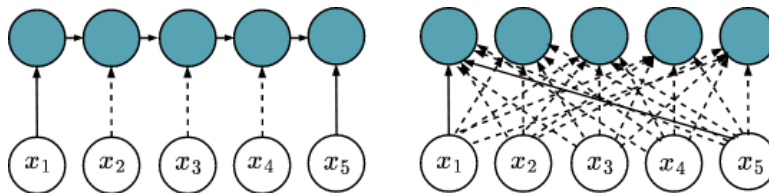


Figure 2.2 Comparing RNN (left), and self-attention (right) architectures. As illustrated, transformers, with their use of self-attention, can directly attend to all positions in the sequence, regardless of their position in time. Dashed lines represent the flow of information or dependencies. The variables x_1 to x_5 represent data input, and the circles above represent the state of the network at each timestep for the RNN. In contrast, for the self-attention mechanism, the circles above denote how each state considers information from all other states, not just the immediate predecessor.

As figure 2.2 demonstrates, transformers provide an alternative method for capturing long-term relationships in sequential data that does not require sequential processing or deep layer stacking.

Even with the rise of transformers, which have shown remarkable effectiveness in many tasks, it's important to acknowledge the ongoing relevance of RNNs and their variants like LSTMs. These models are still useful in a variety of applications where specific sequential data characteristics and temporal dynamics are at play, such as in certain time-series predictions. As we explore the details of transformer models, understanding the strengths and limitations of RNNs illuminates why the newer techniques represent a significant leap in the field of deep learning.

2.1.2 Vanishing gradients: transformer to the rescue

The vanishing gradient problem, in particular, can make it difficult to propagate errors through the network and update the model parameters during backpropagation. This can make learning long-term dependencies and correctly modeling sequential data difficult, which is required for many sequence-to-sequence transduction tasks. And like I just pointed out in the previous section, RNN-based models suffer from the vanishing gradient problem (figure 2.3).

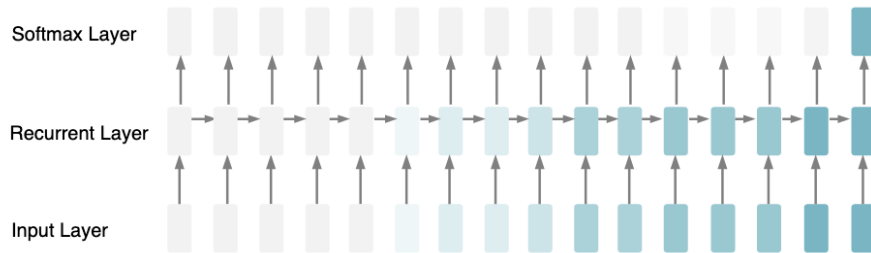


Figure 2.3 Simplified example of the vanishing gradient problem, where the gradient contribution from earlier steps (left) become insignificant.

The vanishing gradient problem is a common problem in neural networks, referring to the problem that the gradients get very small, and, therefore, the weights of the network do not update effectively, leading to slow training and poor performance.

This is where the transformer architecture comes in, offering a solution to this problem by using the attention mechanism to capture dependencies between all positions in an input sequence, as shown in figure 2.2. Figure 2.4 compares the gradients for an LSTM and a transformer architecture based on the sentence: "The quick brown fox jumps over the lazy dog and runs through the fields, while the lazy dog barks loudly and then chases the quick brown fox through the forest", to illustrate how the gradients in an LSTM become very small when the sentence is lengthy. I invite you to run the code provided in the books repository to see how various sentence lengths affect the gradients of the two different architectures.

```

Gradients for RNN model:
tensor([[-1.1910e-08, -5.4378e-09, -1.7932e-08, -1.0601e-07,  4.1367e-08,
         -9.4355e-08,  3.2711e-07,  2.2510e-08, -2.8239e-07, -1.7337e-06,
        -3.7372e-06, -3.9611e-07,  3.1367e-06, -2.0029e-05, -1.4689e-05,
         3.0543e-05,  6.7980e-05,  9.9472e-05,  4.7608e-06, -4.2451e-04,
        -6.8603e-06,  1.0076e-03,  6.5036e-04,  3.5185e-03,  8.8883e-03,
         4.6364e-03,  5.6579e-03,  1.5272e-03, -1.9715e-02, -3.8232e-01]])

Gradients for self-attention model:
tensor([[-0.0010, -0.0011, -0.0007, -0.0013, -0.0019, -0.0011, -0.0010, -0.0014,
        -0.0017, -0.0009, -0.0010, -0.0010, -0.0010, -0.0012, -0.0008, -0.0012,
        -0.0014, -0.0017, -0.0016, -0.0015, -0.0009, -0.0009, -0.0014, -0.0010,
        -0.0011, -0.0007, -0.0013, -0.0010, -0.0010,  0.0032]])

```

Figure 2.4 Comparison of gradients for an LSTM and transformer architecture based on the sentence: "The quick brown fox jumps over the lazy dog and runs through the fields, while the lazy dog barks loudly and then chases the quick brown fox through the forest".

Now that we understand that the attention mechanism in the transformer helps to alleviate the vanishing gradient problem, let us look at its overall architecture in the following section. We'll start by looking at the transformer's two main components, the encoder and the decoder, and how they work together to transform an input sequence into an output sequence. Then we look more closely at the transformer architecture's key innovation, the self-attention mechanism, and finally how positional encoding works.

Before concluding this section, it's important to highlight the other side of the gradient problem, the exploding gradient problem. This is a situation where the gradients during backpropagation become excessively large, causing model weights to update too rapidly and potentially leading to unstable learning and poor performance. This issue often arises when dealing with long sequences or deep architectures, where the compounded effects of the calculations can cause the gradient values to explode. Similar to the vanishing gradients problem, the exploding gradient problem can also interfere with the model's ability to learn long-term dependencies, but for the opposite reason. In section 2.3, after we've learned more about the transformer's architecture, we'll explore this problem in more depth and see how the modular design of transformers mitigates some of the most common problems that arise during training. This will provide us with a better understanding of how the transformer architecture addresses these issues, improving both network stability and performance.

2.2 *Model architecture*

This section distills the essence of the original transformer model as presented in the groundbreaking paper, "Attention Is All You Need". By using simplified terms and concepts, I aim to make this innovative architecture more accessible and easier to grasp. My focus is on the core components that make the transformer what it is - the encoder, the decoder, and the unique attention mechanism that underpins both.

By the end of this section, you should have gained a comprehensive and intuitive understanding of the transformer's architecture. Therefore, this section is not an academic exercise; it is your entry ticket into the fascinating world of transformer models. It equips

you with the knowledge necessary for exploring the many transformer variants discussed in the subsequent chapters.

With this understanding, you can join the ongoing conversation around transformer-based models and their applications in the field of artificial intelligence. The discussion surrounding these models is vibrant, growing, and fundamental to the future of AI.

It's crucial to note that the transformer model, despite being a radical departure from traditional RNN, still adheres to the encoder-decoder framework at its core. This adherence is a testament to the robustness of the encoder-decoder paradigm, which continues to serve as a solid foundation for cutting-edge models.

However, the transformer model achieves its unique capabilities by deploying stacked attention and point-wise, fully connected layers for both the encoder and decoder. These architectural choices, as shown in the left and right portions of figure 2.5, result in a highly flexible and scalable model that excels in a wide array of sequence-to-sequence prediction tasks. This scalability and flexibility underline the true power of the transformer architecture, making it a cornerstone in the rapidly evolving landscape of artificial intelligence.

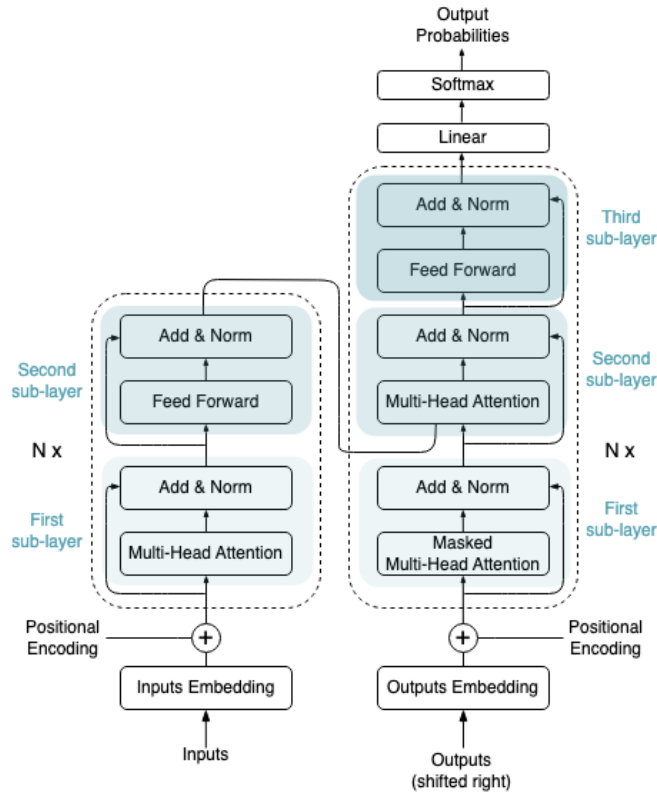


Figure 2.5 The encoder of the transformer architecture is depicted on the left side of the figure, and the decoder is depicted on the right side.

As we have now a visual representation of the transformer architecture, let us look more deeply into each component. We will begin by discussing the encoder and its particular structure and functions before moving on to other important aspects of the transformer model.

2.2.1 Encoder and decoder stacks

The transformers model structure is split into an encoder and a decoder part, as shown in figure 2.5. Let us start with the encoder before moving on to other important parts of the model architecture.

ENCODER PART OF THE TRANSFORMER

The encoder plays a vital role in processing the input sequence in the transformer architecture. It consists of a stack of layers, with each layer having two primary sub-layers: the multi-head self-attention mechanism and the fully connected feed-forward network. The multi-head self-attention mechanism processes the input sequence by determining the significance or "attention" that should be allocated to different parts of the text. Following this, the feed-forward network applies a transformation to the processed sequence from the attention mechanism.

A notable feature in the encoder's design is the use of residual connections. Instead of merely passing the output of one sub-layer to the next, the encoder also merges this output with its original input. This procedure is akin to adding information from the original text sequence to the processed sequence at every step. By doing this, the transformer ensures that the initial context of the input sequence is preserved and integrated throughout the encoding process. This mechanism helps maintain continuity in the processed sequence, ensuring that the model does not lose the inherent meaning and relationships present in the original text.

Let us delve deeper into the encoder's architecture. The encoder is composed of $N = 6$ identical layers. Each of these layers contains the aforementioned sub-layers: the multi-head self-attention mechanism and a position-wise fully connected feed-forward network, which we'll look at in detail in this section.

Building upon the encoder's integration of residual connections, these connections facilitate a direct flow of information from the input through different parts of the network. This flow aids in circumventing some transformations, ensuring that the model retains essential input details. Practically, each sub-layer's output is formulated as $LayerNorm(x + Sublayer(x))$, with $Sublayer(x)$ representing the function executed by the sub-layer. Additionally, to enhance model robustness and curtail overfitting, dropout is applied prior to finalizing these connections.

In a programming sense, this concept is illustrated in listing 2.1, which presents a simplified implementation of the encoder layer.

Listing 2.1 Simplified encoder layer example

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward, dropout=0.1):
```

```

super().__init__()
① multi-head attention
self.self_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
self.feed_forward = nn.Sequential(② feed-forward network
    nn.Linear(d_model, 2 * dim_feedforward),
    GLU(input_size=dim_feedforward, output_size=d_model),
    nn.Dropout(dropout)
)
self.norm1 = LayerNorm(d_model) ③ layer normalization (separate class)
self.norm2 = LayerNorm(d_model)
self.dropout1 = nn.Dropout(dropout) ④ dropout
self.dropout2 = nn.Dropout(dropout)

def forward(self, x, mask=None)
    # self-attention layer ⑤ residual connection
    attn_output, _ = self.self_attn(x, x, x, attn_mask=mask)
    x = x + self.dropout1(attn_output)
    x = self.norm1(x)

    # feed-forward layer ⑥ residual connection
    ff_output = self.feed_forward(x)
    x = x + self.dropout2(ff_output)
    x = self.norm2(x)

    return x

```

All the sub-layers and the embedding layers in the model produce outputs of the same size, $d_{model} = 512$. This uniformity facilitates the seamless use of residual connections, enabling the information from the input to flow directly to the model's output, bypassing individual layers. By adding the original input to the output of a network layer, the resulting output equals the sum of the initial input and the layer's output. This enables the model to learn more effectively and provides a more stable training process, leading to improved performance.

DECODER PART OF THE TRANSFORMER

As with the encoder, the decoder is another crucial component in the transformer architecture. But this part of the architecture is now responsible for generating the output sequence based on the information processed by the encoder. Each layer in the decoder has primary components similar to the encoder: the self-attention mechanism and the feed-forward network. However, the decoder also incorporates a third component: a multi-head attention mechanism that operates over the output of the encoder's last layer. This added component allows the decoder to utilize information from the encoder, enabling it to focus on different parts of the input text while generating the output.

Residual connections, like in the encoder, are present in the decoder. These connections facilitate the merging of each sub-layer's output with its input, ensuring that contextual information is preserved throughout the decoding process. Layer normalization further accompanies these residual connections, enhancing the stability of the model and assisting in training.

A distinguishing feature of the decoder is its masked self-attention mechanism. This masking ensures that while generating an output for a particular position in the sequence, the model is restricted to using only previously known outputs, thereby maintaining the order of sequence generation and ensuring causality in the model's predictions.

The decoder comprises $N = 6$ identical layers. Unlike the encoder, which has two sub-layers, the decoder introduces a third sub-layer to attend over the encoder's output. This unique attention mechanism enriches the decoder's output by providing it with a broader context from the input sequence.

Figures 2.6 and 2.7 provide visual representations of the attention masking and the distinct components of the decoder, respectively.

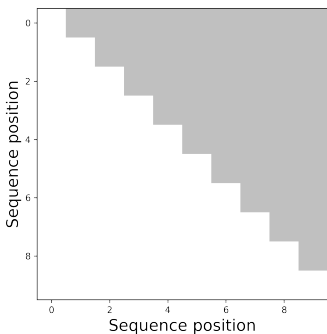


Figure 2.6 Illustration of multi-head attention masking.

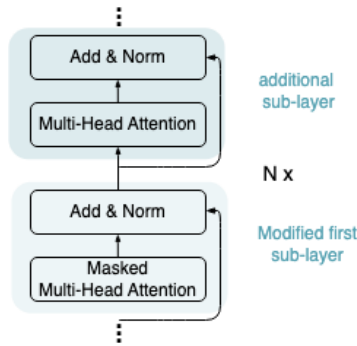


Figure 2.7 Detailed structure of the decoder components.

Masking is fundamental for sequence-to-sequence tasks, where the model generates outputs token-by-token based on preceding tokens in the sequence. Different types of masking, such as padding masking or sequence masking, can be applied depending on the specific application or use case.

2.2.2 Attention

A pivotal element that sets transformer-based models apart is the attention mechanism, particularly its self-attention variant. This component has been instrumental in propelling advancements in natural language processing tasks. In this section, we will delve into the intricacies of self-attention and multi-head attention, aiming to demystify these concepts and demonstrate their role in the transformer architecture's capability.

The term self-attention refers to the fact that the attention weights are computed within a single sequence. When an input sequence is passed through a multi-head self-attention layer, the attention weights are computed between different positions within the same sequence. This mechanism, named "self-attention", allows each element in the input sequence to relate to every other element, including itself, in the sequence. Therefore, the output generated is a weighted representation of the entire sequence.

SCALED DOT-PRODUCT BASICS

If you are now wondering why I mentioned self-attention in the intro of this section self-attention and not scaled dot-product attention, let me first clarify. Self-attention and scaled dot-product attention are two related concepts that are used in the transformer architecture to allow for efficient and effective learning of relationships between elements within a sequence.

To understand both, let us first look at scaled dot-product attention by reducing it to its simplest components. To best understand this concept, we'll start with a graphical illustration. Let's consider a scenario where we have a sequence of five inputs and five outputs, as shown in figure 2.9. This visual representation will allow us to explore the mechanics of scaled dot-product attention and understand how it shapes the interactions within the transformer.



Figure 2.8 Illustration of a sequence of five inputs and five outputs.

To compute y_3 , we use the vector x_3 to determine the associated weights. This is achieved by calculating the dot product of x_3 with each vector in the sequence, starting from x_1 and continuing through to x_5 . As shown in figure 2.9.

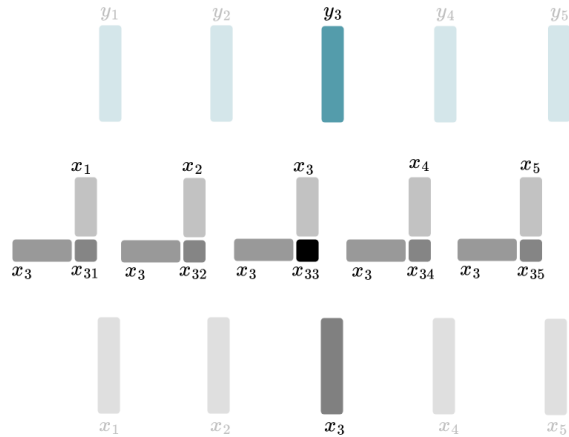


Figure 2.9 Graphical explanation of taking the dot product to compute the weights for y_3 .

After computing these five weights, we take the softmax so that it sums up to one. Then we multiply each input vector by the weights we just computed and sum them all up and this gives us then the vector y_3 . This process is shown in figure 2.10.

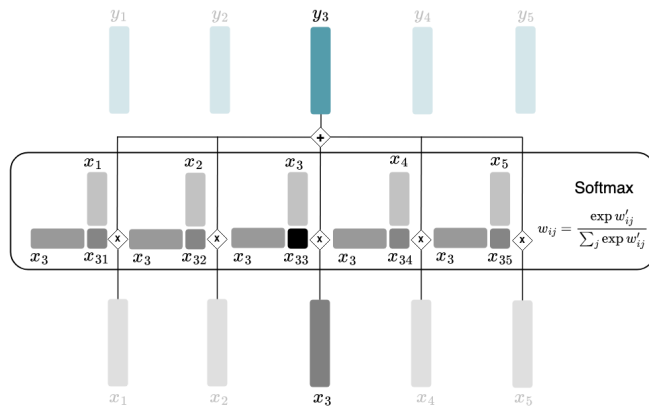


Figure 2.10 To compute y_3 , we take first the softmax function for all weights, so it sums up to 1 and then we multiply each input vector by the weights and sum it up.

Moreover, we process every input vector x_i in three ways to fulfill the three roles with matrix multiplication.

- Compare to every other vector to compute attention weights for its own output y_i , which is the **query**.
- Compare to every other vector to compute attention weight w_{ij} for output y_i , which is the **key**.

- Sum up with the other vectors to form the result of the attention weighted sum, which is the **value**.

To understand this in an intuitive way, using our text example from chapter 1, "the movie was not bad", this just means:

- The **query** represents what we're "asking" about or looking for. It's like a "search term." In our example, if we're interested in understanding the importance relationship between "not," "bad," and "movie," we can imagine them as our queries.
- The **key** represents the "features" or "identifiers" for every word in the sentence. The keys determine how well each word in the sentence responds to the query. If our query is "not," then the keys for every word will determine how related or relevant each word in the sentence is to "not."
- The **value** contains the "content" we want to retrieve or weigh. Once we've determined how relevant each word (via its key) is to the query, the values give us the actual content we'd retrieve or weigh. In many implementations, the initial value representations are just the input embeddings, but as layers of attention stack, they capture more nuanced contextual representations.

So, in our example with "the movie was not bad": If "not" is the query, the attention mechanism might assign high importance (or weight) to "bad" because "not bad" is a common phrase. The values corresponding to both "not" and "bad" would then be summed up with their respective weights to produce the final output for the word "not."

However, in actual models, all words in a sequence simultaneously act as queries, keys, and values. The self-attention mechanism computes a weighted sum of values for each word in the sequence, based on the attention scores between its query representation and all key representations in the sequence. This allows every word to gather information from all other words, based on their relevancy, to produce new contextual embeddings.

Now, mathematically this is represented as follows:

- Query: $\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i$
- Key: $\mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i$
- Value: $\mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$

Therefore, mathematically speaking, we can think of the processes as *simple* matrix multiplication. We then introduce a softmax function to make sure it sums to one, as shown in equation 2.1. So, to summarize all you have to do is to multiply each input vector by these three matrices and then combine the results in the aforementioned three different ways to produce the output y of i .

$$\begin{aligned}
w'_{ij} &= \mathbf{q}_i^\top \mathbf{k}_j \\
w_{ij} &= \text{softmax} \left(w'_{ij} \right)_{\square} \\
\mathbf{y}_i &= \sum_j w_{ij} \mathbf{v}_j
\end{aligned} \tag{2.1}$$

From this explanation, it's clear that the heart of the attention mechanism, or attention function, depends on matrix multiplication. However, it's important to understand that it's the specific combination of these matrix operations - the generation of queries, keys, and values - and their application through the dot product and softmax functions, that enables the model to capture complex relationships within sequences. This gives the transformer model its ability to perform remarkably in NLP tasks.

SCALED DOT-PRODUCT ATTENTION

With a foundational understanding of the basics - query, key, and value matrices - of dot-product attention, we can now examine the more intricate aspect of this attention mechanism, the scaled dot-product attention. Figure 2.11 provides a graphical representation of the overall design of scaled dot-product attention.

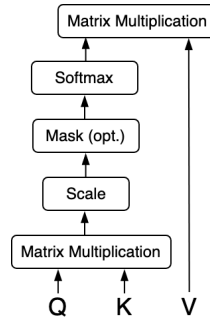


Figure 2.11 Illustration of scaled dot-product attention.

Scaled dot-product attention, a specific type of self-attention mechanism utilized in the transformer architecture, gets its name from the way attention weights are computed. In this mechanism, the dot product of every pair of query and key vectors is calculated and divided by the square root of the dimension of the key vectors, as demonstrated in equation 2.2. This normalization occurs before the application of the softmax function.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \tag{2.2}$$

The importance of scaled dot-product attention lies in its ability to stabilize gradients during the backpropagation phase of training. By limiting the size of the attention scores via scaling the dot product by $\sqrt{d_k}$, softmax saturation can be prevented, and gradient explosion can be avoided. The effect of such scaling is illustrated in Figure 2.12.

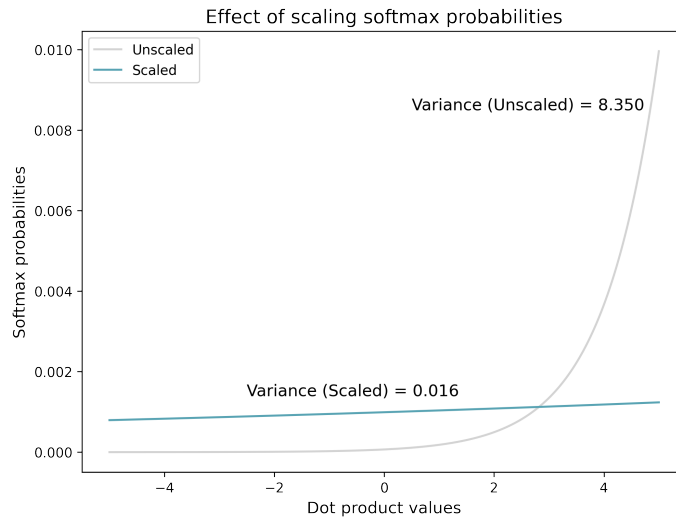


Figure 2.12 This figure shows that the variance of the scaled dot product values is smaller than the variance of the unscaled dot product values, indicating that scaling helps to control the variance of the dot product values. This, in turn, helps prevent the gradients from exploding during backpropagation, improving the stability and convergence of the training process.

Hence, if the dot product values become excessively large, the resulting gradients may also become disproportionately large, resulting in unstable training due to the notorious exploding gradient problem. By controlling the variance of the dot product values through scaling, the magnitude of the gradients during backpropagation is influenced. The scaling factor used in the transformer architecture is the square root of the key dimension (i.e., $\sqrt{512}$ for a key dimension of 512), which has been found to yield good results in practice.

To illustrate the scaled dot-attention concept more clearly, I have prepared a simplified version of it in listing 2.2. For simplicity, we assume we have already learned the weight matrices during training, and we just assign the word embeddings. But in practice, these word embeddings would have been created by the encoder.

Listing 2.2 Simplified scaled dot-attention implementation

1 Defining word embeddings

```
embed_1 = np.array([0, 1, 0])
embed_2 = np.array([1, 0, 1])
embed_3 = np.array([0, 1, 1])
embed_4 = np.array([1, 1, 0])
```

2 stacking the word embeddings into a single array

```
embeddings = np.array([embed_1, embed_2, embed_3, embed_4])
```

3 Initialize weight matrices

```
Wq = rand(3, 3)
Wk = rand(3, 3)
```

```
Wv = rand(3, 3)
```

④ **compute queries, keys and values**

```
Q = embeddings.dot(Wq)
```

```
K = embeddings.dot(Wk)
```

```
V = embeddings.dot(Wv)
```

⑤ **Compute scaled attention scores**

```
attention_scores = softmax(Q.dot(K.T) / sqrt(K.shape[1]), axis=1)
```

⑥ **Compute weighted sum of values**

```
attention_output = attention_scores.dot(V)
```

To recap, this scaling is crucial for model stability and effectiveness, particularly when the key vectors' dimensions are large. In such scenarios, the dot product can grow large in magnitude, leading to very large pre-softmax values. This can lead to two potential problems: saturation of the softmax function, where very large inputs are mapped to the endpoint of the function, resulting in a loss of the original values' information, and large gradients during the backpropagation phase, leading to instability in learning, known as the exploding gradient problem.

By reducing the dot product's magnitude by the square root of the dimensionality, the magnitude of the values entering the softmax function is effectively controlled, mitigating these issues. Therefore, scaled dot-product attention not only aids in stabilizing the gradients during the training phase but also prevents saturation of the softmax function, preserving the original relationships between the inputs.

SELF ATTENTION

Self-attention is a type of attention mechanism that allows a sequence-to-sequence model to focus on different parts of the input sequence when generating an output sequence. Figure 2.13 shows the computed weights of a self-attention matrix using our sentence from chapter 1, "the movie was not bad".



Figure 2.13 Weight distribution of self-attention.

Self-attention operates by allowing a sequence-to-sequence model to weigh different portions of an input sequence differently when producing an output sequence. Instead of applying distinct rules or weights to each individual element in the sequence as in other attention mechanisms, self-attention evaluates the importance of each element based on its relation to every other element in the sequence.

Specifically, in the transformer architecture, each element (or word) in the input sequence can attend to all positions in the sequence, enabling the model to determine which positions are crucial for a given context. This is achieved using the same set of parameters, making the process consistent across different positions. Thus, self-attention provides the model with the capability to focus on segments of the input sequence that are most relevant to the current processing step, ensuring that relevant contextual information is retained and emphasized.

The self-attention mechanism in the transformer architecture uses a singular matrix to calculate the queries, keys, and values. As explained in the original paper "Attention is All You Need" (1), this singular matrix is obtained by concatenating the weight matrices of the linear transformations applied to the queries, keys, and values. The resulting matrix is then divided into multiple heads, with the attention mechanism applied to each. Concatenating the attention results from each head and passing them through a linear layer yields the output.

Furthermore, self-attention is more numerically effective because it eliminates the need to compute a distinct matrix for each point in the input sequence, as in other mechanisms. Second, in addition to its numerical efficiency, self-attention also provides greater freedom to model long-term relationships by allowing each part in the input sequence to respond to any other position. This valuable attention mechanism will be further explored in the next section, where we will discuss its relevance to the network's stability.

MULTI-HEAD ATTENTION

Instead of just using one attention function, the authors of the original transformer paper, "Attention Is All You Need", introduced multi-head attention to project queries, keys, and values h -times with different learned linear projections into:

$$d_k = d_v = d_{\text{model}} / h = 64, \quad (2.3)$$

where $h = 8$, and d_k refers to the dimension of the keys and d_v refers to the dimension of the values and d_{model} to the model's dimension.

These values are then combined and projected again to obtain the final values. This approach is called multi-head attention because it allows the model to look at information from different representations or "views" (referred as "subspaces" in the original transformer paper) of the input simultaneously.

One of the key reasons for adopting multi-head attention is that the different heads can learn to recognize different types of relationships in the data. Each head could potentially focus on a different type of interaction, for example, syntactic versus semantic, or short-term versus long-term dependencies. This diversified perspective enables the model to

capture a richer set of information compared to a single head, which would have a limited, averaged view of the input.

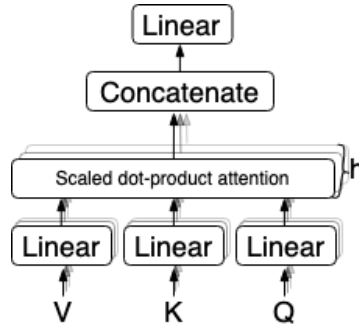


Figure 2.14 Illustration of multi-head attention.

Figure 2.14 and equation 2.4 make it clear that, with multi-head attention, we are splitting the queries, keys, and values into multiple heads and computing their attention separately.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2.4)$$

where $\text{head} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$

Let us transfer this to actual code, to make this concept more explicit. For this, we can use the functional API from PyTorch, as shown in listing 2.3

Listing 2.3 Simplified multi-head attention implementation

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super(MultiHeadAttention, self).__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, query, key, value, mask=None):
        batch_size = query.size(0)

        query = self.W_q(query).view(batch_size, -1,
```

```

self.num_heads, self.d_k).transpose(1, 2)
key = self.W_k(key).view(batch_size, -1,
self.num_heads, self.d_k).transpose(1, 2)
value = self.W_v(value).view(batch_size, -1,
self.num_heads, self.d_v).transpose(1, 2)

scores = torch.matmul(query, key.transpose(-2, -1)) /
        math.sqrt(self.d_k)

if mask is not None:
    scores = scores.masked_fill(mask == 0, -1e9)

attention = F.softmax(scores, dim=-1)
output = torch.matmul(attention, value).transpose(1,
2).contiguous().view(batch_size, -1, self.d_model)
output = self.W_o(output)

return output

```

To explain the code in more detail, first we define a *MultiHeadAttention* class, which is a subclass of `nn.Module`, PyTorch’s base class for all neural network modules. This class is initialized with the dimensions of the model and the number of heads. The assert statement ensures that the model’s dimension is evenly divisible by the number of heads.

In the `__init__` method, we define the linear transformation matrices for the queries, keys, values, and output. Each of these matrices is implemented as a fully connected (*nn.Linear*) layer, which performs a linear transformation of the input data.

The forward method is where the actual computation happens. This method accepts the query, key, and value matrices as input (and optionally a mask), and returns the output of the multi-head attention mechanism.

First, the query, key, and value matrices are independently transformed using the respective weight matrices (*self.W_q*, *self.W_k*, *self.W_v*). They are then reshaped and transposed to have the shape (*batch_size*, *num_heads*, *sequence_length*, *depth*), to accommodate for the multiple heads.

Next, the scaled dot-product attention is computed. The attention scores are calculated by taking the dot product of the query and key matrices, then dividing by the square root of the depth of the key to scale the scores. If a mask is provided, it is applied to the scores. The scores are then passed through a softmax function to obtain the attention weights.

The output is computed by taking the dot product of the attention weights and the value matrix. This output is then reshaped and passed through the output weight matrix (*self.W_o*).

The implementation also includes a usage example, which illustrates how to instantiate the *MultiHeadAttention* class and use it to compute the multi-head attention of some random input data as shown in listing 2.4

Listing 2.4 Simplified multi-head attention implementation

```
❶ Assuming we have some data
batch_size = 32
sequence_length = 100
d_model = 512 ❷ Embedding dimension
num_heads = 8

❸ Instantiate the model
multi_head_attn = MultiHeadAttention(d_model, num_heads)

❹ Create some random data for input
input_data = torch.rand(batch_size, sequence_length, d_model)

❺ We use the same input for query, key and value for self-attention
output = multi_head_attn(input_data, input_data, input_data)
```

If we would print the shape of the output, we would get 32, 100, 512, that is the `batch_size`, `sequence_length` and `d_model`, respectively. This result demonstrates that despite the seemingly complex processing of information during multi-head attention, the output retains the original sequence structure and the model's dimensionality.

To summarize this section, with multi-head attention, the model can attend to different aspects of the input and process them in parallel, thus enabling it to capture more complex relationships in the data. When we compare this with using a single attention head, a single attention head would average the information and would not be able to take advantage of the various elements in the input. This would restrict the model's ability to understand and reflect complicated patterns in the data. Therefore, multi-head attention is an essential component of the transformer architecture, allowing the model to accomplish cutting-edge success in a broad variety of NLP tasks.

2.2.3 Position-wise feed-forward networks

FFNs (position-wise feed-forward networks) are a type of neural network frequently employed in NLP tasks, specifically designed to transform fixed-length vectors. This transformation process is useful because it can convert input data (like words or sentences represented as fixed-length vectors) into more abstract representations. These abstract representations can capture complex patterns in the input data, like the semantics of a sentence or the context of a word. This conversion is achieved through two fully connected layers separated by a nonlinear activation function.

Each element of the vector is individually fed through the network and transformed to a higher dimensional space. This transformation, aimed at increasing capacity, allows for more intricate interactions between the vector's components. The output is then reverted back to its initial dimension using another fully connected layer, ensuring the output maintains a consistent shape with the original input, which is beneficial when stacking multiple layers or components together in a neural network.

In the context of machine learning, a fully connected layer is a type of neural network layer in which each neuron is connected to every neuron in the previous layer, as illus-

trated in figure 2.15. By transforming input vectors in this manner, FFNs are able to extract and leverage higher-level features from the input data, thus improving the model's understanding of the underlying patterns in the data.

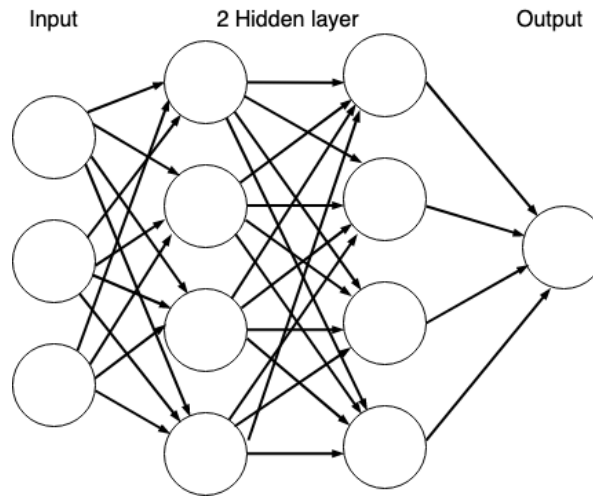


Figure 2.15 Fully connected neural network with two hidden layer.

An implementation of a position-wise FFNs is illustrated in listing 2.5.

Listing 2.5 Simple position-wise feed-forward network

```
class PositionwiseFeedforward(nn.Module):
    def __init__(self, input_dim, hidden_dim, dropout=0.1):
        super(PositionwiseFeedforward, self).__init__()
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.dropout = nn.Dropout(dropout)
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, input_dim)

    def forward(self, x):
        x = self.dropout(torch.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```

NOTE The given code example presents a basic position-wise feedforward network with a single output layer. However, the complete transformer architecture, including multi-head attention and other components, handles sequences of inputs and outputs, not just single ones.

The word "position-wise" relates to the fact that the same transformation is done independently to each element in the vector, regardless of its location within the sequence. This

method is used because it enables the network to understand more complex patterns in the input and makes training on large datasets simpler. Additionally, position-wise FFNs are numerically effective since each member of the array is handled separately in parallel.

2.2.4 Positional encoding

The transformer lacks recurrence, which is a commonly used mechanism to understand the order of tokens in a sequence. Because of this absence, the transformer incorporates positional encoding, a technique for determining the relative or absolute position of tokens within a sequence. Without positional encoding, the transformer wouldn't be able to differentiate the order of tokens, leading to potential misinterpretations in the sequence. Recognizing the position of each token in the sequence allows the model to accurately infer relationships and meaning between tokens. To achieve this, the positional encoding is added to the sequence's input embeddings by encoding each dimension of the position using a sinusoidal function. This encoding enables the model to attend to the relative positions of the words in the input sequence, using a linear function of the position index pos and the dimension index i . The mathematical formula for computing these positional encodings is shown in equation 2.5.

$$\begin{aligned} PE_{(pos, 2i)} &= \sin\left(pos/10000^{2i/d_{\text{model}}}\right) \\ PE_{(pos, 2i+1)} &= \cos\left(pos/10000^{2i/d_{\text{model}}}\right) \end{aligned} \quad (2.5)$$

The model can then use this information to better identify the context and meaning of each token in the sequence by summing the positional encoding with the input embeddings. Listing 2.6 shows an example implementation for positional encoding, using the formula from equation 2.5.

Listing 2.6 Example implementation for positional encoding

```
class PositionalEncoding(nn.Module):
    """Positional encoding class."""
    def __init__(self, num_hiddens, max_len=1000):
        super().__init__()
        self.dropout = nn.Dropout(0.1)
        # Create a positional embedding matrix
        position = torch.arange(max_len, dtype=torch.float32).reshape(-1, 1)
        div_term = torch.exp(
            torch.arange(0, num_hiddens, 2, dtype=torch.float32) *
            -(math.log(10000.0) / num_hiddens))
        pe = torch.zeros((1, max_len, num_hiddens))
        pe[0, :, 0::2] = torch.sin(position * div_term)
        pe[0, :, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe)

    def forward(self, X):
        X = X + self.pe[:, :X.shape[1], :].to(X.device)
        return self.dropout(X)
```

The positional encoding and how it adds a sine wave depending on its position is shown in figure 2.16. Note, for each dimension, the wave's frequency and offset are different.

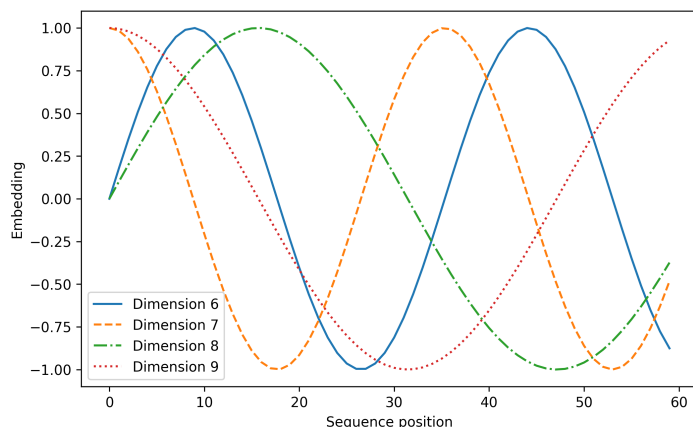


Figure 2.16 Positional encoding example

If you want to get a better grasp of positional encoding, I recommend using the `PositionalEncoding` class, which is also available in the book's repository, <https://github.com/Nicolepcx/Transformers-in-Action>, and changing the model's inputs to see how this changes the plots output.

In this section, we explored the transformer architecture's encoder and decoder components and their collaborative process in transforming input sequences into output sequences. We delved into the key innovation, the self-attention mechanism, and discussed position-wise feed-forward networks (FFNs) used for processing self-attention outputs. Lastly, we examined the role of positional encoding in conveying input sequence order, providing a comprehensive understanding of the transformer architecture and its contributions to cutting-edge performance in natural language processing tasks.

2.3 *Building on the basics: a world of possibilities awaits!*

With a better understanding of the transformer architecture and the role of self-attention, we can now delve deeper into how this type of architecture overcomes some of the issues that arise in RNNs. By comparing the two architectures more closely, we can gain valuable insights into the strengths and weaknesses of each approach.

2.3.1 *Methods to stabilize the training of RNNs*

In this subsection, we'll delve into the challenges of training recurrent neural networks and discuss some widely recognized solutions. A notable reference is "On the difficulty of training Recurrent Neural Networks", which offers a comprehensive view of these challenges and proposes practical solutions. Additionally, other strategies for increasing RNN stability exist, including equal kernel and recurrent kernel initializers, along with orthogonal matrix initializer for recurrence weights. These methods, as described in various studies, are

designed to alleviate the vanishing and exploding gradient problem in RNNs by carefully initializing the weights. The following are key techniques developed to enhance RNN stability.

- Gradient clipping: this method limits the gradient values during backpropagation to prevent them from becoming too large.
- Initialization strategies: careful initialization of the network parameters can also help to prevent the exploding gradient problem.
- Layer normalization: this technique involves normalizing the activations of each layer in the network, which can help improve the stability and convergence of the training process.
- Skip and residual connections: these connections enable the gradient signal to propagate directly through the network without being reduced by activation functions or other transformations. This can prevent gradients from becoming too large or too small, because the network can better propagate the gradient during backpropagation.
- Adaptive learning rate algorithms: optimizers such as Adam, which use adaptive learning rates for each parameter, can help stabilize the learning process and mitigate the vanishing and exploding gradient problem by adjusting the learning rate based on the magnitude of the gradient. Moreover, by using a moving average of past gradient information, Adam reduces oscillations and improves the stability of the learning process.

2.3.2 The transformer architecture: a paradigm shift in neural network stability

In contrast to the RNN based networks, the transformer architecture, with its self-attention mechanism and the way the residual connections and layer normalization are set up, is constructed to naturally alleviate the vanishing and exploding gradient problem. Moreover, because of these design choices and the fact that it uses more efficient feed-forward layers, it is less sensitive to a specific initialization of weights and tuning of hyperparameters than RNNs. In the following are the key aspects on which the transformer builds and how self-attention and the overall architecture design help to stabilize the training.

- Self-attention: the self-attention mechanism allows the model to directly capture dependencies between all positions in an input sequence, regardless of their position in time. This helps the model to better learn long-term dependencies, which can be difficult for traditional RNNs.
- Layer normalization: unlike RNNs that typically use batch normalization, the transformer uses layer normalization, which can help to improve the stability and convergence of the training process and prevent the exploding gradient problem.
- Residual connections: the use of residual connections in the transformer architecture helps to improve the flow of information through the model and prevent the vanishing gradient problem. Note, unlike LSTMs, which add residual connections to the hidden states of each recurrent layer, the transformer adds the input of each sub-layer to

its output using residual connections. This enables the input signal to easily pass through the layer and helps to avoid the vanishing gradient problem, even in deeper architectures. Furthermore, this design supports parallel computation of the sub-layers, which can speed up the training process.

- Positional encoding: the use of positional encoding allows the transformer to take into account the relative positions of elements in an input sequence, which can help to better capture temporal relationships and prevent the vanishing gradient problem.

Overall, the transformer architecture addresses many of the limitations of traditional RNNs, and these improvements help to make the training process more stable and the models more effective in capturing complex dependencies in sequential data.

But it's important to remember that hyperparameters like the learning rate and the number of layers can still affect how well the transformer works, which we'll look at in more detail in chapter 9. However, unlike LSTMs, the transformer does not require a deep network structure to capture long-term dependencies, which further simplifies the gradient computation and results in a simpler computational graph of the transformer than for an LSTM. The simpler computational graph of the transformer can make it easier to optimize hyperparameters and tune the model for better performance. To make this more evident, I provide you some classes and functions in the books repository in folder CH02 that let you plot and compare the computational graphs of both networks, the transformer and the LSTM, respectively.

I hope that I could convince you with this chapter that even though transformers look complex and difficult at first sight, they can be broken down into simple building blocks that use basic mathematical operations. Remember, the attention mechanism, which is the key ingredient of the transformer, is basically a simple matrix multiplication operation. The same concept drives multi-head attention, a variant of the attention mechanism used in transformers. These models are a powerful yet adaptable design for natural language processing and other tasks due to the clever mix of these fundamental matrix multiplication operations, as well as other components such as feed-forward networks and positional encoding. This knowledge is a great foundation for the more advanced variants of the transformer model in this book.

2.4 Summary

- The transformer model is divided into an encoder and decoder. The encoder processes the input sequence into a context or memory, which the decoder then uses to generate the output sequence.
- The pivotal part of the transformer is the attention mechanism. Where the query matrix, in a simplified way, just acts as a way of "retrieving" the key (matrix) with the value (matrix).
- The transformer model focuses on various portions of the input sequence using a self-attention mechanism and a feedforward neural network to transform the attention

outputs. These components are stacked in multiple layers, with residual connections and layer normalization.

- Self-attention is computationally efficient because it allows for parallel processing of the input sequence. Unlike RNNs, which require a distinct set of learnable parameters for each point in the sequence, self-attention is flexible and effective for various NLP tasks.
- Positional encoding is a way of guiding the transformer to differentiate the order of tokens (words) within a sequence.
- While various methods have been developed to improve RNN stability during training, the transformer architecture, with its self-attention mechanism, layer normalization, and residual connections, naturally mitigates both vanishing and exploding gradient problems, making it easier to train and optimize than RNNs.

Bibliography

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL: <http://arxiv.org/abs/1706.03762> 25

Part 2

Transformers for Fundamental NLP Tasks

Transformers have become the driving force behind the most recent breakthroughs in the ever-evolving field of natural language processing. Since these advancements have not only changed the way we engage with technology, such as ChatGPT or its freely accessible versions like Dolly, Falcon and RedPajama, but have also opened up new possibilities for tackling complex language tasks.

In Part 1, we examined the architecture and inner workings of Transformer models, revealing their unique design decisions and mechanisms. With a solid grasp of their basis, we are now well-prepared to investigate the powerful applications enabled by this remarkable architecture. As we venture into this fascinating world, we will uncover the complexities of various NLP tasks and learn how Transformers have revolutionized our approach to solving these challenges.

In this second part of the book, we will look at fundamental NLP applications, revealing the techniques and strategies to unlock the full potential of Transformer models. While also providing you with some fundamental background of the driving forces, i.e. neural machine translations, behind this groundbreaking invention. Moreover, by examining real-world examples, we will gain valuable insights into the practical implementation and fine-tuning of these architectures for diverse use cases. Through case studies and hands-on examples, you will develop a deeper understanding of how to harness the power of Transformer models for your own tasks at hand.

That said, this part of the book will provide you with the tools you need to improve and refine your NLP projects, unlocking new levels of performance and efficiency as well as grounding you in fundamental background knowledge. As a result, you will be well-positioned to advance your work in the rapidly changing landscape of NLP.

Furthermore, by tailoring the power of Transformer models to the particular needs of your own tasks, you will be well equipped to address a wide variety of NLP challenges with the insights and techniques presented in this part of the book. The lessons learned in this

part will be a useful resource for optimizing your own NLP solutions, whether you are working on text summarization, machine translation or classification. Let's embark on this adventure not only to explore the fascinating world of NLP applications but also to learn how to apply these techniques to your own projects.

Text summarization

This chapter covers

- How to choose a baseline and what it entails
- Outline of transformers for summarization
- Overview of evaluation metrics for text summarization
- Evaluation of various models for a summarization task
- Fine-tuning a summarization model

In the previous chapter, we looked at the specific design and building blocks of transformers. This chapter of the book builds on that base and broadens your understanding so that you can create your own solutions for text summarization applications.

Moreover, the text-to-text application provides a distinct framework for further analyzing the transformer's design. Also, understanding how to summarize text can be useful, as there may be situations where you need to summarize a document effectively, whether it is a legal document, a research paper, a news story, or a financial report. In addition, summarizing articles, social media feeds, and other external sources of information can provide businesses with useful insights into market trends, consumer views, and competitor activities. Such information can be used to inform decision-making processes, enhance marketing strategies, and create novel products and services.

To provide you the necessary knowledge on using transformers for text summarization, we build, in this chapter, on the encoder-decoder structure of these seq-2-seq models and explore how you can leverage a pre-trained model and fine-tune it for your own documents.

3.1 *Getting started with text summarization*

Transformers have taken us from the early, simple days of NLP to the big games of advanced language processing technologies. In the past, extractive summarization was the norm. Extractive summarization identifies and extracts the most significant phrases or sentences from the source text, preserving their original form. But now we can easily create abstractive summaries with transformers. This means that we can create a condensed version by rephrasing and synthesizing the content, providing a more concise and coherent summary. It is important to note that for this kind of tasks, RNN-based models were usually applied. But LSTMs have trouble with long sequences, whereas transformer models can handle challenging tasks with ease. This makes it possible for NLP to be used in new and exciting ways and create more comprehensive summaries.

However, before we dive into transformers, let's first examine extractive summarization, as this kind of summarization allows us to establish a solid baseline against which we can later evaluate the performance of our more advanced transformer models.

3.1.1 *Extractive text summarization*

Summarization systems aim to produce concise and fluent summaries that convey key information from texts or documents. Extractive summarization involves selecting important phrases and sentences from the original text and compiling them into a summary. The benefits of this approach are:

- Less prone to misrepresenting information
- Gramatically correct

In spite of its benefits, extractive summarization is generally more suitable for shorter texts like news articles, where important information is often explicitly stated in the text. Simple techniques such as TextRank, a graph-based ranking algorithm, can be effective for these types of summarization tasks.

3.1.2 *Text summarization techniques*

In extractive summarization, several methods such as keyword-based, graph-based, and topic-based are used. These include the ranking of words based on importance, constructing graphs to denote documents, and extracting relevant topics from the text. The paper "Text Summarization Techniques: A Brief Survey" (1) offers a detailed examination of these various techniques. Among them are:

- An early approach prioritizes sentence selection based on the frequency of significant words. (14)
- Another method incorporates the use of cue words and document structure to guide the summarization process. (6)

- The first trainable method used the principles of Naïve Bayes for its operation. (10)
- LexRank represents text as a graph and calculates sentence importance akin to Google’s PageRank algorithm. (7)
- TextRank also uses a graph representation of the text but applies the PageRank algorithm in a slightly different manner, focusing on the number of common words between sentences. (15)

NOTE While both LexRank and TextRank are graph-based techniques for extractive text summarization that take into account semantic connections between words, they vary in some ways. LexRank uses a technique comparable to Google’s PageRank algorithm to calculate sentence importance. It assumes that sentences that are more semantically comparable to other sentences in the text are more significant. TextRank, on the other hand, employs a distinct similarity metric between sentences based on the number of common words between them. It then uses a scoring method similar to PageRank, where nodes indicate sentences and connections are weighted by sentence similarity. As shown in figure 3.1.

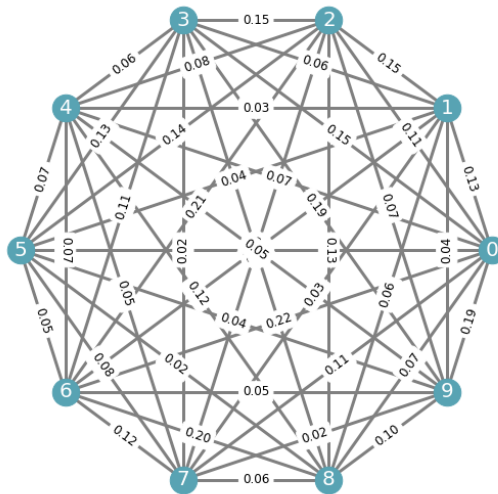


Figure 3.1 This illustration shows a circular graph where each node represents a sentence from the input text, which is the second paragraph of the paper “Attention Is All You Need”. The numbers displayed on the edges are the weights of the edges, representing the similarity scores between the connected sentences. These scores range from 0 to 1, where 0 indicates no similarity and 1 indicates a perfect match (identical sentences). A higher number implies a stronger connection between the two sentences, meaning that they share more words or terms in common. You can explore this further in the accompanying code, located in the “ch03” folder.

In this chapter, we will focus on TextRank as a baseline. Nevertheless, it might be worthwhile for you to investigate LexRank as an alternative, since both methods share a

similar level of computational complexity. If you are familiar with both methods, you can make more informed choices about which is most beneficial to you.

A CLOSER LOOK INTO TEXTRANK

Before we establish our baseline, let us take a closer look at how TextRank works. TextRank is a graph-based ranking algorithm for NLP tasks like summarization and keyword extraction. To give you a high-level imagination, how this works, let us look at a vivid example.

Imagine you're at a party where several groups of people are having separate conversations. You're new to the town, and you want to find out who the most influential people are and with whom you want to "hang out" in the future. One strategy could be to listen in on conversations and keep a score of who is mentioned the most.

The TextRank algorithm works much like this. The party is the text, the groups of people are the sentences (nodes), and the people being mentioned are the words. Each time a word (person) is mentioned in a sentence (conversation), an edge (relationship) is established between that word and the sentence.

Now, consider the influencers at the party. These are likely to be the people most often mentioned, right? This is because the more a person is mentioned, the more important they're likely to be. Likewise, the more frequently a word appears in a text, the more significant it's likely to be, according to TextRank.

However, the story doesn't end here. Not all mentions are equal. Suppose one person (let's call him Bob) is mentioned often, but mostly by less-known individuals at the party. Another person (let's say Alice) is mentioned less frequently, but those who talk about her are themselves very well-known and important and frequently mentioned by others.

In the TextRank world, Alice would have a higher ranking than Bob, because her mentions come from more *important* or *influential* sentences. In this sense, TextRank not only considers the frequency of a word but also the *importance* of the sentences in which the word appears.

By listening to the whole party (analyzing the whole text), you can create a map (a graph) of who knows who and who is talked about the most. This gives you a good idea of who the influencers are. Similarly, TextRank maps the text, identifies the most frequently mentioned and contextually important words, and uses these to rate the significance of the sentences.

So, TextRank is like an eavesdropper at a party, trying to figure out who the most important attendees are, based on who is mentioned the most and who is talked about by the most influential people.

Now, let us attend to the mathematical and technical part of TextRank. Here, text is represented as a graph, with nodes representing textual units (sentences or words) and edges representing the relationships between them. The scores for each node are calculated on the basis of the scores of its neighbors and the weights of the edges that connect them. First, the algorithm applies the PageRank equation as an iteration:

$$p_i^{(k+1)} = \frac{1 - \delta}{N} + \delta \cdot \sum_{\text{edges } ji} \frac{p_j^{(k)}}{c_j} \quad (3.1)$$

where δ is the damping factor (probability of following a link in PageRank), which is usually set to 0.85, N is the total number of nodes (sentences or words in the case of TextRank), p is the rank score, and c is the number of edges for a node. ω_{ji} is the weight of the connection between vertices V_j and V_i

Then, as a second step, the PageRank is replaced by the rank r of the vertices V_i , which can stand for words, phrases or complete sentences.

$$r(V_i^{(k+1)}) = \frac{1 - \delta}{N} + \delta \cdot \sum_{\text{edges}_{ji}} \frac{\omega_{ji} \cdot r(V_j^{(k)})}{\sum_{\text{edges}_{ji}} \omega_{ji}} \quad (3.2)$$

Additionally, to each edge a weight w_{ji} is attached in order to have more flexibility to characterize the *strength* of the connection between two vertices V_j and V_i . Depending on the use case, these edges can represent different types of connection, lexical relations, or contextual relations.

To make the math less abstract, let us consider a simple example. The provided pagerank function from listing 3.1 reflects the second equation, 3.2, where ω_{ji} is the weight of the connection between vertices V_j and V_i (i.e., `adjacency_matrix[j, i]`). The function calculates the rank $r(V_i^{(k+1)})$ for each vertex V_i in the graph, considering the edge weights and the out-degree of each node.

Listing 3.1 Detailed code example for TextRank algorithm

```
def pagerank(self, G, delta=0.85, max_iter=100, tol=1e-6):
    N = len(G)
    ranks = np.ones(N) / N
    adjacency_matrix = nx.to_numpy_array(G)

    ❶ Calculate the out-degree for each node
    out_degrees = np.sum(adjacency_matrix, axis=1)

    for _ in range(max_iter):
        ❷ (1 - delta) divided by N term
        new_ranks = np.ones(N) * (1 - delta) / N
        for i in range(N):
            for j in range(N):
                ❸ check edge between nodes j and i
                if adjacency_matrix[j, i] > 0:
                    ❹ summation part in both equations
                    new_ranks[i] += delta * ranks[j] * adjacency_matrix[j, i] /
out_degrees[j]
                ❺ This line checks for convergence
            if np.linalg.norm(new_ranks - ranks) < tol:
                break
        ranks = new_ranks

    return ranks
```

Now let us apply our function. We will use the following text:

```
TEXT = """Machine learning is a subfield of artificial intelligence.  
Deep learning is a technique used in machine learning.  
Supervised learning is a popular machine learning method.  
Unsupervised learning is another machine learning technique."""
```

Then we instantiate our class and call the functions:

Listing 3.2 Using our custom TextRank class

```
tr = TextRank() ❶ instantiate class  
sentences = tr.split_sentences(text) ❷ split sentences  
G = tr.create_graph(sentences) ❸ create the graph  
tr.print_sentence_weights(G, sentences) ❹ print out weights  
tr.plot_graph(G) ❺ plot constructed graph from step 3
```

The following weights represent the similarity scores between the pairs of sentences. The scores range from 0 to 1, where 0 indicates no similarity and 1 indicates a perfect match (identical sentences). A higher number implies a stronger connection between the two sentences, meaning that they share more words or terms in common.

- Sentence 0: Machine learning is a subfield of artificial intelligence.
Sentence 3: Unsupervised learning is another machine learning technique.
Weight: 0.2469
- Sentence 0: Machine learning is a subfield of artificial intelligence.
Sentence 2: Supervised learning is a popular machine learning technique.
Weight: 0.2469
- Sentence 0: Machine learning is a subfield of artificial intelligence.
Sentence 1: Deep learning is a technique used in machine learning.
Weight: 0.2210
- Sentence 0: Machine learning is a subfield of artificial intelligence.
Sentence 0: Machine learning is a subfield of artificial intelligence.
Weight: 1.0000
- Sentence 1: Deep learning is a technique used in machine learning.
Sentence 3: Unsupervised learning is another machine learning technique.
Weight: 0.4522
- Sentence 1: Deep learning is a technique used in machine learning.
Sentence 2: Supervised learning is a popular machine learning technique.
Weight: 0.4522
- Sentence 1: Deep learning is a technique used in machine learning.
Sentence 1: Deep learning is a technique used in machine learning.
Weight: 1.0000

- Sentence 2: Supervised learning is a popular machine learning technique.
Sentence 3: Unsupervised learning is another machine learning technique.
Weight: 0.5051
- Sentence 2: Supervised learning is a popular machine learning technique.
Sentence 2: Supervised learning is a popular machine learning technique.
Weight: 1.0000
- Sentence 3: Unsupervised learning is another machine learning technique.
Sentence 3: Unsupervised learning is another machine learning technique.
Weight: 1.0000

This results in the graph as shown in figure 3.2

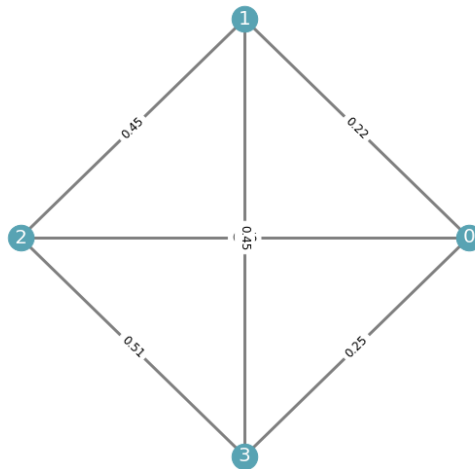


Figure 3.2 These scores range from 0 to 1, where 0 indicates no similarity and 1 indicates a perfect match (identical sentences). A higher number implies a stronger connection between the two sentences, meaning that they share more words or terms in common.

NOTE If you want to experiment with the code, the whole custom TextRank class can be found in the accompanying code, located in the "utils" folder of the repo.

Figure 3.2 visualizes the graph generated by our TextRank algorithm. Each node represents a sentence from the text and the edges between nodes represent the calculated similarity scores. The higher the numbers, the higher the similarity between the sentences. Note how sentences that share terms like "machine learning" and "technique" are more strongly connected.

3.1.3 Establishing a baseline: TextRank

You may be wondering why it is necessary to create a baseline. The reason for this is that it enables you to assess not only the efficacy of advanced models such as transformers, but

also whether a simpler strategy would suffice for a specific task. Creating a firm baseline is an important step in determining the real value that a more complicated model offers to your particular use case. Listing 3.3 shows how to leverage a library called "summa", which is freely available via <https://pypi.org/project/summa/>.

Listing 3.3 Using TextRank from the summa library

```
❶ function to summarize the text
def summarize_text(text, words=80): ❷ set max. words to 80
    summary = summarizer.summarize(text, words=words)
    return summary

❸ Parallelize the TextRank summarization
with Pool() as pool:
    summarized_articles = pool.map(summarize_text, articles_np_1)
```

Since it is unsupervised, simple to apply, and computationally efficient, TextRank is an excellent option as a baseline algorithm for text summarization. It offers a basic yet effective method for extracting important words and ranking sentences based on their significance within the text. Compared to other baseline choices, such as lead-3 (selecting the first three sentences) or simple frequency-based methods, TextRank offers a more sophisticated approach by capturing the relationships between sentences and their relevance to the overall text, providing a better starting point for comparison with advanced methods such as transformers. Transformer-based models are mostly designed for abstractive summarization, though there are exceptions, which we will look at in section 3.3. But first let us understand more intuitively what abstractive summarization is and what it entails. We will later, in section 3.5.1 use the function from listing 3.3 as a baseline and compare the resulting metrics as well as the text summarization results.

3.1.4 Abstractive text summarization

Abstractive summarization is generally considered more difficult and has been the focus of recent research in the field. Abstractive summarization entails creating a new summary from scratch, enabling the model to combine knowledge from various sections of the original text and convey it in its "own words". This method is more adaptable than extractive summarization, which uses only present sentences from the source text to create the summary. Furthermore, an abstractive summary produces more succinct and cohesive summaries than an extractive summarization. In summary, the pros for choosing an abstractive summarization are the following.

- Summary sounds more natural
- Higher compression rates

You might be wondering what the phrase "higher compression rates" means. Let me take a minute to explain what it means.

Abstractive summaries have higher compression rates because they generate summaries by synthesizing new, concise sentences that capture the main ideas of the original text, in-

stead of merely extracting existing sentences. This allows for a more compact representation of the source material while maintaining the essential information.

NOTE LSTMs and other RNN-based models can be used for abstractive summarization (16), but they may struggle with longer documents or more complex relationships between different parts of the text. Transformer-based models have shown better performance in these cases and are often the preferred choice for abstractive summarization tasks. Transformers are designed to handle long sequences of text and can capture complex relationships between them. And with that, they are well-suited for the task of abstractive summarization, leading to more efficient and coherent summaries.

This is crucial when handling longer documents or articles. Although abstractive summarization requires more data and resources to train advanced models, the benefits may justify the investment in specific applications. Let's briefly examine a network combining both extractive and abstractive summarization before exploring how to use transformers for our summarization tasks.

3.1.5 Pointer-generator networks

Now that we understand the concepts of extractive and abstractive summaries, let us briefly discuss pointer-generator networks (21). The network is shown in figure 3.3.

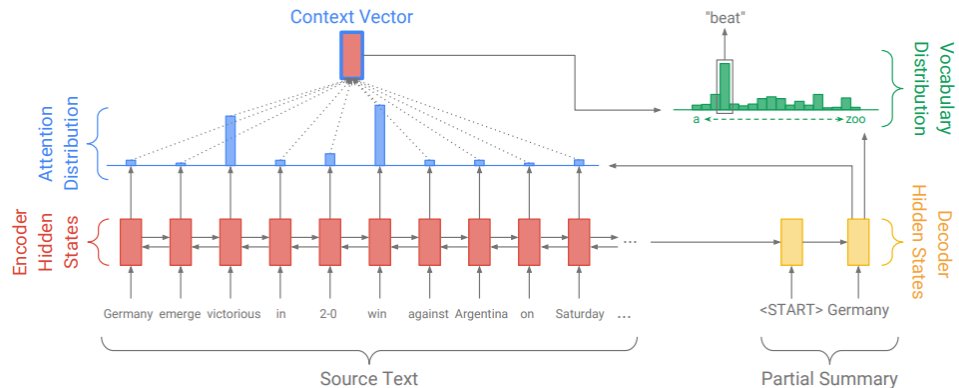


Figure 3.3 Pointer-generator sequence-to-sequence model with attention. The model may attend to relevant words in the source text to generate new words, e.g., to produce the new word *beat* in the abstractive summary *Germany beat Argentina 2-0* the model may attend to the words *victorious* and *win* in the source text. Image was taken from the paper of "Get To The Point: Summarization with Pointer-Generator Networks"

These networks are designed for text generation tasks such as summarization and effectively combine the benefits of both extractive and abstractive approaches. They incorporate attention mechanisms in their architecture to weigh the importance of different parts of the input text, which helps to obtain more meaningful context vectors.

These context vectors are then used to generate output text that is more focused and coherent. Pointer-generator networks employ a pointer mechanism to copy words from the input text while also generating new terms based on the learned vocabulary using a generative mechanism. In a loose sense, the tasks of abstractive text summarization and image generation share some conceptual similarities. Both tasks involve generating new outputs based on previously acquired patterns and relationships in their respective domains (text for abstractive summarization and images for image generation). Although they operate on different principles and cater to different modalities, both share the generative feature of creating new content based on their training data. Even though this form of summarization is impressive, transformer-based models have further advanced the field of text summarization, building upon and surpassing the capabilities of earlier models such as the pointer-generator network. This is partially because pre-trained transformer models can leverage large-scale unsupervised learning, which helps them better understand language and generate more coherent and contextually accurate summaries. Let's explore how transformers advanced the field of text summarization.

3.2 *Text-to-text transformer models*

In this section we will explore various text-to-text transformer models that have revolutionized the landscape of text summarization. Each model has unique characteristics and architecture designs that contribute to its overall performance and versatility. Figure 3.4 shows how the flow of information is in such a text-to-text model.

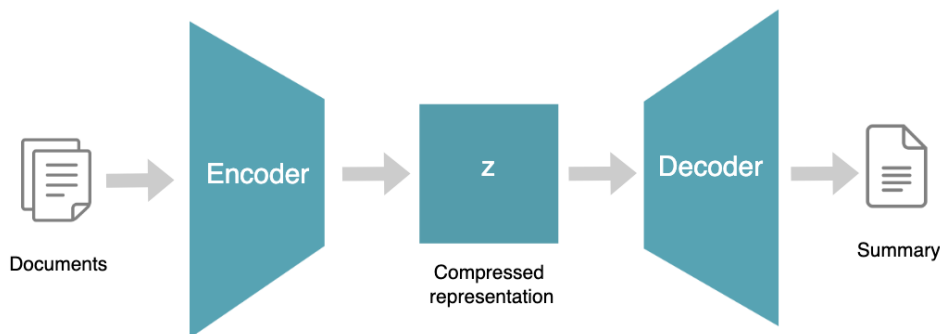


Figure 3.4 The encoder processes the input text to produce a compressed representation "z." This compressed representation captures the important information from the input text. The decoder then utilizes this representation to produce the desired output text, our summary.

You might notice that this architecture resembles an Autoencoder. While both have similar high-level structures and involve encoding input data into a compressed representation, their objectives and applications differ. Autoencoders are used for dimensionality reduction and unsupervised learning, while text-to-text models generate coherent output text based on input text. Despite sharing the principle of preserving essential features, their specific goals and use cases are distinct.

STATE-OF-THE-ART SUMMARIZATION MODELS

BART, T5, ProphetNet, Pegasus, Longformer, and BigBird are some of the most commonly used and cutting-edge text summarization algorithms. They have achieved remarkable results in a variety of summary benchmarks and have been built with particular enhancements or features that make them well-suited for producing summaries. Although there are other methods for summarizing, these are currently some of the most widely used and effective options in the field. Before we go deeper into each of these models and how these architectures differ from each other, here is a brief overview of their main differences, before we discuss these models in more detail in section 3.3.

- The BART (Bidirectional and Auto-Regressive Transformers) (11), model blends the original Transformer architecture's bidirectional encoding with GPT's auto-regressive decoding. This combination enables BART to excel at a wide variety of NLP tasks, such as text summarization and translation. **139 - 406 million parameters**
- T5 (Text-to-Text Transfer Transformer) (20), is another text-to-text model that reformulates all NLP tasks into a unified text-to-text framework. T5 streamlines the fine-tuning process by turning tasks into a text-in, text-out structure, allowing it to perform extremely well on a variety of tasks, including summarization. **60 million - 11 billion parameters**
- ProphetNet, a new transformer model (23), offers future n-gram prediction as a new self-supervised goal. This method enables the model to acquire more global sequence information, resulting in better success on sequence-to-sequence tasks such as summarization and translation. **50 -200 million parameters**
- Pegasus (Extracted Gap-sentence Pre-training for Abstractive Summarization) (26), is a model developed especially for abstractive summarization. Pegasus pre-trains the model by hiding complete phrases in the input text, causing the model to produce more logical and fluent summaries. **568 million - 1.3 billion parameters**
- Longformer (2), and BigBird (25), are transformer models that can handle significantly longer documents than standard models. Longformer makes use of moving windows, whereas BigBird makes use of different attention mechanisms. Both models can handle longer sequences efficiently, making them ideal for summarizing large documents. **Longformer: 148 - 409 million parameters, BigBird: 110 - 337 million parameters.**

GPT-2, GPT-3 AND GPT-4 Despite the fact that GPT-2, (19), and GPT-3, (3), are strong language models, they are not addressed in this chapter because they are less suitable for summarization tasks. In contrast to the models mentioned above, which are specifically intended for summarization, their auto-regressive character can limit their ability to create clear and coherent summaries. While GPT-3 is an advance over GPT-2, it still encounters the same difficulties in producing high-quality abstractive summaries in comparison to models such as BART, T5, ProphetNet, Pegasus,

Longformer, and BigBird. I have chosen to cover these models in a subsequent chapter dedicated to text generation tasks, rather than focusing on them in this chapter specifically about summarization. Moreover, GPT-4 is a multimodal model and it will be covered in chapter 7.

3.3 Model overview

As we explore each of the text-to-text transformer models, you will gain insights into their distinct design choices, strengths, and limitations. This knowledge not only aids in selecting the most appropriate model for your specific summarization needs but also sets the stage for our subsequent "test-spin" with these models in section 3.5. Moreover, by understanding and comparing these models side-by-side, you get a comprehensive grasp of their functionalities, eliminating the need to sift through scattered information across various sections. This direct comparison encourages a deeper understanding, enabling you to critically assess their performance and differentiate between their unique capabilities.

3.3.1 BART

In their paper "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension", the authors propose BART (Bidirectional and Auto-Regressive Transformers). BART demonstrates cutting-edge success on a variety of natural language processing tasks such as abstractive summarization, machine translation, and question answering. The methods used in BERT, (5) and GPT, (18), as highlighted in figures 3.5, 3.6 and 3.7, respectively, inspired BART.

BERT is a masked language model (MLM) that forecasts missing tokens using both left and right context, whereas GPT is an auto-regressive model that only uses the left context. By pre-training a sequence-to-sequence model with a denoising autoencoder, BART combines the strengths of both models.

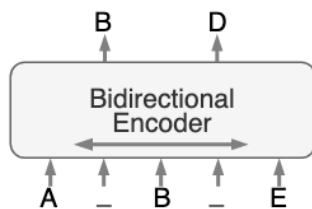


Figure 3.5 BERT: Masks are used to substitute random tokens, and the document is encoded bidirectionally. Because missing tokens are predicted separately, BERT cannot be used for generation.

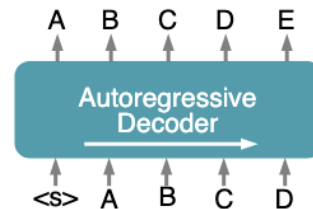


Figure 3.6 GPT: Tokens are auto-regressively predicted, implying that GPT can be used for generation. However, because words can only condition on the left, it cannot acquire bidirectional interactions.

The authors used a collection of noise functions, such as token masking, token deletion, text infilling, and sentence permutation, to produce a corrupted version of the input text for their study. The goal of the pre-training is to reconstruct the original input text from the

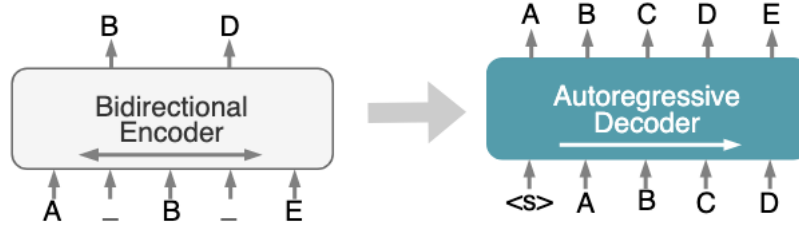


Figure 3.7 BART: Encoder inputs do not have to be aligned with decoder outputs, enabling for arbitrary noise transformations. A document has been corrupted in this case by substituting text spans with mask symbols. A bidirectional model is used to encode the corrupted document (left), and an autoregressive decoder is used to determine the likelihood of the original document (right).

corrupted form. BART learns to model the conditional probability $p(y|x)$ given a corrupted input sequence x , where y is the initial input sequence. The training goal is to minimize the negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{(x,y) \in \mathcal{D}} \log p_{\theta}(y|x) \quad (3.3)$$

Where $\mathcal{D}$ is the set of corrupted-original text pairs, and θ represents the model parameters.

It is worth noting that BART employs the standard transformer design for both its encoder and decoder. The encoder analyzes the corrupted input text and produces context representations, which the decoder uses to reconstruct the original text. The fine-tuned BART demonstrates that it outperforms prior cutting-edge models such as T5, BERT, and RoBERTa (13), on these tasks.

BART is an important contribution to the field of NLP, as this emphasizes the importance of denoising pre-training and bidirectional context modeling for sequence-to-sequence tasks. BART is widely used for different NLP applications and acts as a foundation for subsequent models.

3.3.2 T5

The authors of the paper "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" present T5 (Text-to-Text Transfer Transformer), a powerful pre-training framework for NLP tasks. T5 gets cutting-edge results on a variety of NLP benchmarks, including GLUE, SuperGLUE, SQuAD, and CNN/Daily Mail summarization. We will look more closely at the GLUE benchmark in 5.2. T5 expands on the success of previous models, such as BERT and GPT, by pre-training a large-scale language model in text-to-text format. The authors suggest that all NLP tasks should be viewed as text-to-text problems, with both input and output converted into natural language text. Figure 3.8 illustrates this approach in a simple diagram. With minimal changes in the task-specific architecture, this unified approach allows transfer learning across a wide range of tasks.

T5 is pre-trained by the authors using a denoising autoencoder, which is comparable to the previous mentioned BART model. Moreover, they apply a masking operation to

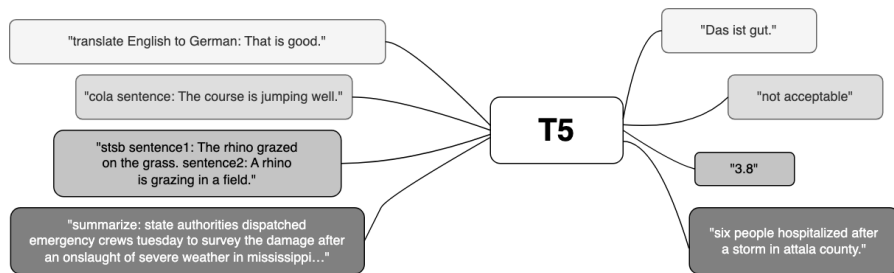


Figure 3.8 The text-to-text structure of T5 is depicted here in diagram form. Each task (translation, question answering, and classification) is viewed as a cast in which the model text is fed as input and trained to produce some target text. This enables to apply the same model, loss function, hyperparameters, and so on to a diverse collection of tasks. “T5” refers to “Text-to-Text Transfer Transformer”

corrupt input sequences and teach the model to reconstruct the original text. Given a corrupted input sequence x , the model learns to model the conditional probability $p(y|x)$, where y is the original input sequence. As we have seen with BART, the training goal is to minimize the identical negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{(x,y) \in \mathcal{D}} \log p_{\theta}(y|x) \quad (3.4)$$

Where $\mathcal{D}$ is the set of corrupted-original text pairs, and θ represents the model parameters.

Like BART, T5 uses the transformer architecture introduced in chapter 2, with minor modifications to enable the text-to-text format. To enhance its performance, the model makes use of relative position embeddings and layer normalization techniques. The authors experimented with a wide range of model sizes, from small (60 million parameters) to extra-large (11 billion parameters). Larger models consistently outperform smaller models on NLP tasks, which suggests that increasing model size can be an effective strategy for improving performance. T5 stresses the importance of a unified text-to-text framework as a key contribution to NLP transfer learning. This method has established a new standard for NLP models, inspiring further study and development. T5’s influential design acted as a foundation for subsequent models and has been extensively adopted across a variety of NLP applications.

3.3.3 ProphetNet

ProphetNet, introduced in the paper "ProphetNet: Predicting Future N-grams for Sequence-to-Sequence Pre-training", uses a new method to enhance the performance of various NLP tasks. It focuses on predicting future n-grams, or sequences of n words, which can be viewed as forecasting what words will come next in a series. Where n-grams refers to a continuous sequences of n items in a given sample of text.

This innovative technique works by modeling the prediction of these n-grams based on what has already been presented in the target sequence and the source sequence. By predicting multiple future tokens, ProphetNet is able to better grasp the context between different words, improving its performance in sequence-to-sequence tasks.

Using a unique self-supervised learning method and a masked n-gram language model, ProphetNet generates text that is diverse, coherent, and has improved contextual understanding.

The model's design revolves around the token span masking technique and relative positional encoding. The n-gram masking approach predicts multiple future tokens, enabling the model to learn more effectively about the context between words. ProphetNet's n-stream attention mechanism, displayed in figure 3.9, is influenced by the n-stream self-attention from XLNet (24).

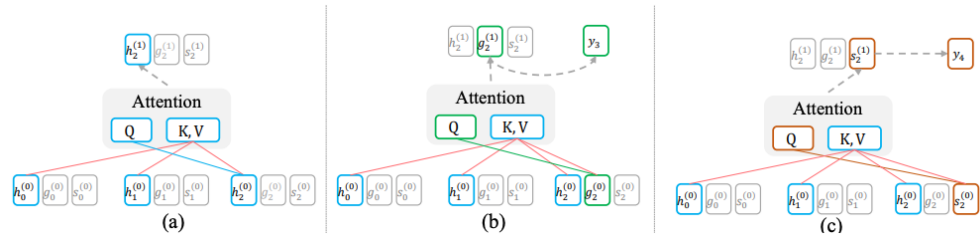


Figure 3.9 Mechanism of self-attention of stream N with a primary stream and prediction of n streams. For simplicity, the scenario employs two streams of self-attention ($n = 2$). Part (a) of the figure depicts the attention process of the primary stream self-attention, while parts (b) and (c) depict the attention processes of the first and second predicting streams, respectively. The image was taken from the ProphetNet paper.

ProphetNet's n-stream attention mechanism extends the multi-head attention mechanism by employing a novel n-stream self-attention approach. Traditional multi-head attention involves a model focusing on different parts of the input when generating each output word. ProphetNet takes this a step further by introducing multiple streams of self-attention, allowing the model to anticipate and predict n-grams or sequences of n words.

In other words, ProphetNet not only pays attention to different parts of the input but also makes predictions about multiple future tokens in the output. This enhanced attention mechanism helps ProphetNet to capture more complex dependencies between tokens and to generate more coherent sequences of text.

Applying these new concepts, ProphetNet outperforms previous state-of-the-art models in several NLP benchmarks, including machine translation, summarization, and question answering. The major contributions of ProphetNet to the field of NLP are the introduction of future n-gram prediction and the development of a unique sequence-to-sequence pre-training approach.

3.3.4 Pegasus

PEGASUS (Pre-training with Extracted Gap-sentences for Abstractive Summarization) represents a specialized pre-training approach specifically designed for abstractive text summarization tasks. The model relies on a unique self-supervised objective, termed "gap sentence generation," to prepare training data. This method prompts the model to predict

masked sentences within a given document, with these masked sentences functioning as a compact summary of the document. The underlying concept promotes the notion that if the model can be taught to predict significant sentences within a document, it can be fine-tuned for more effective summarization tasks. This technique draws its inspiration from masked language modeling approaches, which mask words and contiguous spans to train the model. A visual representation of the architecture is provided in Figure 3.10.

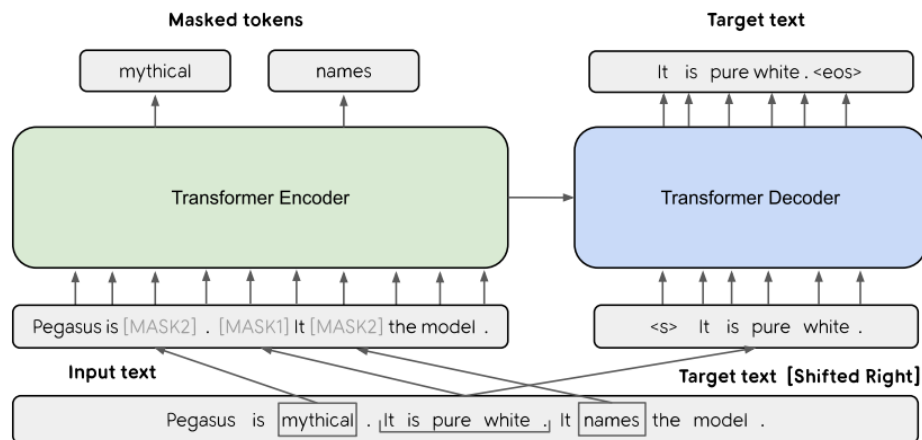


Figure 3.10 PEGASUS's core design is a conventional transformer encoder-decoder. Both GSG and MLM are applied concurrently to this example as pre-training goals. Originally, there are three lines. A phrase is hidden with [MASK1] and used as target generation text (GSG). The other two lines stay in the input, but some tokens are arbitrarily masked by [MASK2] (MLM). The image was taken from the PEGASUS paper.

PEGASUS selects and masks whole sentences from documents, and these gap sentences are concatenated to create a pseudo-summary. This special pre-training method concentrates on extracting the most important information from the input text. The authors of PEGASUS considered three distinct methods to optimize the selection and masking process, with the goal of maximizing the model's ability to learn and produce meaningful abstractive summaries.

- Random: Uniformly select m sentences at random.
- Lead: Select the first m sentences.
- Principal: Select the top- m scored sentences according to importance. As a method to calculate the importance, ROUGE1-F1 is applied between the sentence and the rest of the document, $s_i = \text{rouge}(x_i, D \setminus \{x_i\})$, $\forall i$.

To generate summaries, the authors use standard sequence-to-sequence training techniques to fine-tune PEGASUS on different summarization datasets. The resulting model outperforms earlier models such as BART and T5 on multiple summarization benchmarks, demonstrating cutting-edge performance.

PEGASUS made a significant contribution by introducing the gap sentence generation pre-training objective, which is specifically intended to help in abstractive summarization. PEGASUS learns to recognize and generate meaningful summaries by masking and predicting key sentences in a document during pre-training. This innovative pre-training method has been shown to be effective for abstractive summarization tasks.

3.3.5 Longformer

Longformer is a transformer architecture intended to efficiently handle long input sequences. It extends the original Transformer paper "Attention Is All You Need" initial self-attention mechanism to address the challenges of processing long texts in NLP tasks.

The self-attention mechanism of the Longformer combines local windowed attention with global attention. Longformer captures local context effectively by using a sliding window with or without dilation for local windowed attention. The dilated sliding window can capture long-term dependencies more effectively by covering a bigger context without increasing computational complexity.

Longformer carefully applies global attention to a few tokens in the input sequence for global attention. This enables the model to focus on important tokens that contain more global information. (e.g., special tokens, such as [CLS] in BERT).

Recall that the original transformer model computes attention scores as follows given the linear projections Q , K , and V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.5)$$

where Q , K , and V are the query, key, and value matrices, and d_k is the key vector dimension. Longformer keeps this attention computation but modifies how it is applied to the input sequence by combining local windowed attention and global attention.

Longformer is also related to autoregressive or left-to-right language modeling, which is an essential aspect of its design. This link enables the model to better understand and produce coherent text by accounting for token dependencies in the input sequence.

In summary, Longformer's main contribution lies in its ability to handle long input sequences efficiently by combining local windowed attention, dilated sliding window attention, and global attention, while building upon the original self-attention mechanism and relating to autoregressive language modeling. This advancement enables the model to effectively capture both local and global context while keeping computational efficiency, allowing it to be applied to a broader range of NLP tasks that involve processing long documents.

3.3.6 BigBird

BigBird, as detailed in the paper "Big Bird: Transformers for Longer Sequences," introduces a novel attention mechanism that efficiently manages longer input sequences. This generalized attention mechanism is applied to each layer of the model operating on an input sequence X of length n .

The heart of this mechanism is a directed graph D , with each vertex representing a position in the input sequence. The directed edges of this graph reveal the interrelations considered by the attention mechanism. For each node i in D , its set of out-neighbors, denoted as $N(i)$, reflects the attention span of that node.

The BigBird attention mechanism can be mathematically defined as a function that processes the input sequence $\mathbf{X}$ and modifies each element x_i based on its attention span $N(i)$. It employs different roles for the input sequence elements, represented by the query, key, and value functions. The scoring function σ (such as softmax) is used to determine the weights of these interactions.

The attention mechanism in BigBird combines three types of attention patterns - sliding window attention, global attention, and random attention - forming what's known as sparse attention. This sparse attention pattern allows the model to effectively capture both local and global context in an efficient manner, enabling it to process longer input sequences.

As illustrated in figure 3.11 the sparse attention pattern in Big Bird consists of a combination of sliding window attention, global attention, and random attention.

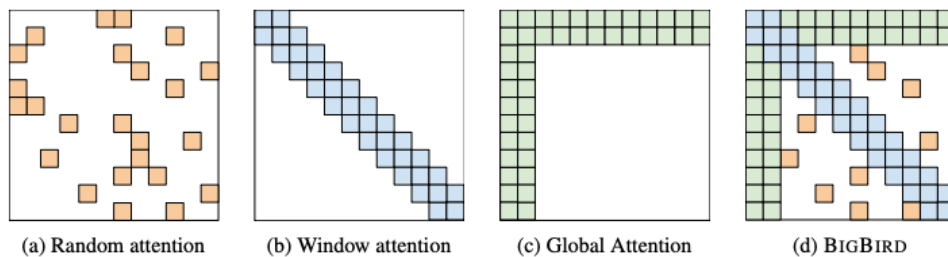


Figure 3.11 Building blocks of the attention mechanism used in BIGBIRD. The white color indicates the absence of attention. (a) random attention with $r = 2$, (b) sliding window attention with $w = 3$ (c) global attention with $g = 2$. (d) the combined BIGBIRD model. The images are taken from the paper: "Big Bird: Transformers for Longer Sequences".

To give you an intuition how each single attention works in BigBird, I listed their main functionalities.

- Random attention: Big Bird includes random attention, where a fixed number of random tokens are attended to by every token in the sequence. This helps capture long-range dependencies that might be missed by the sliding window and global attention mechanisms.
- Window attention: This attention pattern restricts the attention to a fixed-size window around each token, allowing the model to capture local context efficiently.
- Global attention: Big Bird selectively applies global attention to a few tokens in the input sequence, typically critical tokens such as the [CLS] token or tokens related to a specific task. This enables the model to attend to important tokens carrying more global information.

One of Big Bird's main contributions is its ability to process longer input sequences efficiently by employing a generalized attention mechanism that operates on the adjacency matrix A of the graph D . This innovation allows the model to effectively capture both local and global contexts while maintaining computational efficiency, enabling its application to a wider range of tasks and domains.

3.4 Metrics to evaluate generated text

Before we proceed to the next section, where we will train and evaluate various transformer models, let us first explore the tools and metrics available to assess their performance. We will concentrate on the most common ones to evaluate automatic summarization, ROUGE and BLEU, respectively. We will use these metrics to evaluate the performance of our models in section 3.5.

3.4.1 ROUGE

ROUGE, or Recall-Oriented Understudy for Gisting Evaluation, is a set of metrics used to evaluate automatic summarization and machine translation software in natural language processing. The metrics compares an automatically produced summary or translation against a reference or a set of references (human-produced) summary or translation. Note that ROUGE is case insensitive, meaning that upper case letters are treated the same way as lower case letters. ROUGE was introduced in the paper titled "ROUGE: A Package for Automatic Evaluation of Summaries" (12).

ROUGE-N:

This metric measures the n-gram overlap between the generated summary and the reference summary. It is based on the recall of n-grams in the generated summary with respect to the reference summary. An n-gram is a contiguous sequence of n items (such as words or characters) from a given text or speech, used to analyze and model language patterns.

The formula for ROUGE-N is:

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Reference Summaries}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{Reference Summaries}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)} \quad (3.6)$$

where gram_n represents n-grams, $\text{Count}_{\text{match}}(\text{gram}_n)$ is the number of n-grams co-occurring in both the generated and reference summaries, and $\text{Count}(\text{gram}_n)$ is the count of n-grams in the reference summaries.

EXAMPLE

Let us consider ROUGE-1 as a simple example, which focuses on unigrams (single words).

CANDIDATE cat mat

REFERENCE 1 The cat is on the mat.

REFERENCE 2 A cat sits on the mat.

ROUGE-1 = (Sum of matched unigram counts) / (Sum of unigram counts in Reference summaries) Here, the sum of matched unigram counts is 4 (2 matches for each

reference summary), and the sum of unigram counts in both reference summaries is 12 (6 unigrams in each summary). This results in a ROUGE-1 = $\frac{4}{12} = \frac{1}{3}$

NOTE The important words are underlined.

ROUGE-1 AND ROUGE-2

are special cases of ROUGE-N, where $n = 1$ (unigram) and $n = 2$ (bigram), (one- and two-word sequence), respectively.

ROUGE-L: This metric measures the longest common subsequence (LCS) between the generated summary and the reference summary. The formula for ROUGE-L recall is as follows:

$$\text{ROUGE-L} = \frac{\sum_{S \in \text{Reference Summaries}} \text{LCS}(S, \text{Candidate Summary})}{\sum_{S \in \text{Reference Summaries}} \text{len}(S)} \quad (3.7)$$

where $\text{LCS}(S, \text{Candidate Summary})$ is the length of the longest common subsequence between the generated summary (candidate summary) and the reference summary S , and $\text{len}(S)$ is the length of the reference summary.

ROUGE-LSUM

This metric is an extension of ROUGE-L that computes the longest common subsequence for each sentence in the generated summary and the reference summary and then takes the sum of the longest common subsequences. It is designed to handle cases where the summaries have multiple sentences.

NOTE It is good practice to evaluate your model on different metrics and also compare it to state-of-the-art models using the same dataset, if available.

ROUGE scores usually range from 0 to 1, with 1 being a perfect match between the generated summary and the reference summary. In practice, scores closer to 1 are rare, especially for abstractive summarization tasks. For a simple baseline model like TextRank, a "good" ROUGE score can be around 0.3 to 0.4. For more advanced transformer-based models like BART, T5, or PEGASUS, a "good" ROUGE score can range from 0.4 to 0.6, depending on the task and dataset. State-of-the-art models can achieve even higher scores, but it is important to keep in mind that ROUGE scores are just one aspect of evaluating the quality of generated summaries.

3.4.2 BLEU

The BLEU (BiLingual Evaluation Understudy) (17), metric is used to automatically assess machine-translated text. The BLEU score, a value between 0 and 1, signifies the degree of similarity between the machine-translated text and a set of high-quality reference translations. Although originally designed for machine translation, BLEU can also be applied to text summarization, as it effectively measures the alignment between generated summaries and human-written reference summaries, providing insight into the quality and coherence

of the produced summaries. The mathematical formula for BLEU is very similar to the one we have seen for ROUGE.

$$p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{n\text{-gram} \in C} \text{Count}_{\text{clip}}(n\text{-gram})}{\sum_{S \in \text{Candidates}} \sum_{n\text{-gram}' \in C'} \text{Count}(n\text{-gram}')} \quad (3.8)$$

where p_n is the n-gram precision, and C is the candidate machine-generated translation. The term $\text{count}_{\text{clip}}(n\text{-gram})$ represents the clipped count of an n-gram in the machine-generated translation. This clipped count is the minimum of two values: the count of the n-gram in the candidate translation and the maximum count of the n-gram in any of the reference translations. And C' refers to the set of reference translations corresponding to the candidate translation C . While n-gram' is an n-gram in the reference translation C' and $\text{count}(n\text{-gram}')$ is the count of the n-gram' in the reference translation, accordingly. In addition, BLEU has a brevity penalty (BP).

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases} \quad (3.9)$$

where c is the length of the generated translation or summary, r is the effective reference length (usually the length of the reference summary closest to the length of the generated summary).

Then,

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (3.10)$$

BP is the brevity penalty (to penalize shorter generated translations or summaries), N is the maximum order of n-gram (usually set at 4), w_n is the weight for each order of n-gram (usually set at $\frac{1}{N}$), p_n is the precision modified of n-gram for each order of n gram.

EXAMPLE

CANDIDATE the the the the the the the.

REFERENCE 1 The cat is on the mat.

REFERENCE 2 There is a cat on the mat.

This results in a **modified unigram precision** of $\frac{2}{7}$.

NOTE The important words for computing precision are underlined.

For a simple baseline model, a "good" BLEU score could be around 0.1 to 0.2. For transformer-based models, a "good" BLEU score could range from 0.2 to 0.4, again depending on the task and dataset. State-of-the-art models can achieve higher scores, but just like with ROUGE, it's essential to consider other factors when evaluating the quality

of generated summaries. Moreover, keep in mind that these are rough guidelines, and the definition of a "good" score can vary depending on the specific domain, dataset, and evaluation criteria. It is always a good idea to compare your model's scores with scores from existing state-of-the-art models on the same dataset to get a better understanding of its performance.

NOTE It can be beneficial to use both ROUGE and BLEU metrics for evaluation, as they focus on different aspects of the generated summaries. ROUGE is recall-oriented and mainly focuses on the coverage of important content, while BLEU is precision-oriented and evaluates the fluency and correctness of the generated text.

3.5 Applications and worked examples

In this section, we will look at various applications and worked examples of text summarization using transformer models. We will investigate and evaluate models such as BART, T5, ProphetNet, Pegasus, Longformer, and BigBird for summarization tasks ranging from articles and science papers to legal documents.

In addition, we will cover techniques and methods for fine-tuning and applying these models to particular areas, allowing you to produce summaries that are tailored to the specific needs of your applications. By analyzing these applications and worked scenarios I aim to provide you a thorough grasp of how transformer models have revolutionized text summarization, showing their versatility and efficacy in tackling the challenges presented by an ever-increasing volume of textual data. To learn more about the different uses, it is important to first look at the different datasets used in the cases that follow. This examination will give us useful information about how each dataset is different, which is important for figuring out which model to use for the task at hand; and making sure that your chosen transformer model works as expected.

3.5.1 Evaluating different summarization models

Alright, now it's time to get our hands dirty! With our good understanding on how transformer-based summary models work and how we can measure their effectiveness. We're gonna dive in and see how they actually perform on real-world data. By comparing them, we can better validate what each model's strengths and weaknesses are, and with that, we can decide which one is best for our own task.

For this assessment, we use the robust functionality of Hugging Face's pipelines, a high-level, user-friendly API from the Hugging Face platform, that offers seamless access to pre-trained models. This powerful tool allows us to focus our energies on the vital task of evaluating the models' effectiveness and pinpointing the optimal choices for our summarization needs.

To make our evaluation more concrete, we'll introduce the datasets within the context of worked examples, starting with the well-regarded CNN/Daily Mail dataset. By understanding and comparing these models side-by-side on tangible tasks, it promotes a more cohesive and comprehensive comprehension of how they function in real-world scenarios.

Selecting your data

Examining different datasets and understanding their variations enables us to comprehend the importance of choosing the right model for the context and requirements of a particular summarization task. The variety of datasets emphasizes the significance of understanding the underlying data and selecting transformer models that are tailored to the specific requirements of each application.

Keeping this in mind, it is critical to test models on various datasets before implementing them for a particular use case. The reasoning behind this strategy is simple: by testing the models on a variety of datasets, we can guarantee their adaptability and efficacy in dealing with data nuances, eventually leading to better performance and accuracy in the final application. In the following are the key facts why you should test your model with different data.

- Model performance can vary across datasets: a model that works well on one dataset might not work as well on another. Testing on multiple datasets makes sure that the model is strong and can be used with different kinds of data.
- Adaptation to your domain: if the pre-trained model was trained on a dataset that wasn't related to your domain, testing and fine-tuning it on a data set that is related to your use case will help the model understand and adapt to the language and ideas that are specific to your domain.
- Evaluate model limitations: testing the model on different data sets lets you see its strengths and weaknesses, so you can decide if it is right for your use case.

Overall, it is important to test models on different data sets to make sure they are suitable and work well when applied to your specific use case. Additionally, you should consider using publicly available data because it provides you with a benchmark to compare the performance of various models. These openly available datasets are used by many researchers and practitioners to train, validate, and test their models. As a consequence, performance metrics derived from these datasets can serve as a good reference point for comparing models.

With such a benchmark, you can compare the performance of one model compared to others, allowing you to make more informed decisions about which model to use for your specific applications. Furthermore, if you decide to fine-tune an existing model or create your own, having these benchmarks enables you to compare the performance of your model to that of existing state-of-the-art solutions.

Now, let's dive into the world of text summarization with Transformer models on diverse datasets.

COMPARING THE MODEL SUMMARIES ON CNN/DAILY MAIL DATASET

For our first deep-dive, I've selected the CNN/Daily Mail dataset. The goal of the CNN/Daily Mail dataset (22), (8) is to help in the development of models that can summarize long sections of text in just a couple of lines. The dataset contains long news articles with associated human-generated summaries (highlights), making it a great resource for

benchmarking and comparing various summarization models. The articles in the dataset are the CNN stories published between April 2007 and April 2015. And the Daily Mail articles published between June 2010 and April 2015. It should be noted that any summarizations created by models trained on this dataset are not representative of the language used in the articles, but are instead written mechanically. Because version 3.0 is not anonymized, people's identities can be discovered in the information. The dataset does not contain any information about its original author.

DATA SPLITS: train: 287,113 examples; validation: 13,368 examples; test: 11,490 examples

To analyze various models, we define a function `generate_summary()` that takes a `model_name` as input and generates a summary of the input text using our specified model. Listing 3.4 show this function in more detail.

Listing 3.4 Generate summaries with various models on CNN/Daily mail dataset

```
def generate_summary(model_name, sample_text):
    model_dict = { ❶ dict to chose models from
        "BART": "facebook/bart-large-cnn",
        "T5": "sysresearch101/t5-large-finetuned-xsum-cnn",
        "ProphetNet": "microsoft/prophetnet-large-uncased-cnndm"
    }
    if model_name == "Pegasus": ❷ use Pegasus model
        summarization_pipeline = pipeline("summarization",
            model="google/pegasus-cnn_dailymail")
        summarization_output = summarization_pipeline(sample_text)
        summary = summarization_output[0][
            "summary_text"].replace(" .<n>", ". ")
    elif model_name in model_dict.keys(): ❸ chose model from dict
        tokenizer = AutoTokenizer.from_pretrained(
            model_dict[model_name])
        model = AutoModelForSeq2SeqLM.from_pretrained(
            model_dict[model_name])
        ❹ generate summary with specified model
        summarization_pipeline = pipeline("summarization",
            tokenizer=tokenizer, model=model)
        summarization_output = summarization_pipeline(
            sample_text, max_length=100)
        summary = summarization_output[0]["summary_text"]
    else:
        raise ValueError(f"Model {model_name} is not supported.")

    return summary
```

NOTE Some transformer models have a maximum input length constraint. To handle this, we need to break down the input text into smaller chunks before passing

it to the pipeline. You will find a modified version of this function in the notebook *ch03_text_summarization_eval.ipynb* handling exactly this use case.

With that function, we can simply get the summaries as demonstrated in the listing 3.5.

Listing 3.5 Compute the evaluation metrics for each model

```
summaries = {} ① dictionary to store our results
sample_text = sample[0] ② get one sample
summaries["BART"] = generate_summary("BART", sample_text) ③ get summary
```

For all the models we want to evaluate, including our baseline TextRank model, we generate summaries and store them in our *summaries* dictionary. Using our custom *SummarizationMetrics()* class, we can efficiently evaluate the performance of these models on the CNN/Daily Mail dataset. The code for this process is illustrated in listing 3.6 and the class can be found in the book repository. Using this systematic approach, we can effectively compare the various models and gain valuable insights into their performance in the context of our chosen dataset.

Listing 3.6 Compute the evaluation metrics for each model

```
① create instance of the class
evaluator = SummarizationMetrics()
② get the evaluation data frame
metrics_df = evaluator.compute_sum_metric(summaries, reference)
```

In figure 3.12 we have the comparison of the four different models including our baseline algorithm, for one text example from the CNN/Daily Mail dataset.

	rouge1	rouge2	rougeL	rougeLsum	google_bleu
TextRank (Baseline)	0.436975	0.324786	0.386555	0.436975	0.196133
Pegasus	0.800000	0.692308	0.800000	0.800000	0.656627
BART	0.613636	0.372093	0.545455	0.568182	0.315534
T5	0.550725	0.417910	0.376812	0.521739	0.341772
ProphetNet	0.446602	0.257426	0.407767	0.407767	0.105590

Figure 3.12 Comparison of different models incl. our baseline model on the CNN/Daily Mail dataset.

Upon examining the results, we can clearly observe that PEGASUS outperforms the other models. Interestingly, our baseline algorithm manages to surpass ProphetNet. To gain a deeper understanding of the differences between the generated summaries, let's conduct a comparison of these summaries, which will help us better comprehend the variations between them.

GROUND TRUTH Harry Potter star Daniel Radcliffe gets 20M fortune as he turns 18 Monday. Young actor says he has no plans to fritter his cash away. Radcliffe's earnings from first five Potter films have been held in trust fund.

TEXTRANK (BASELINE) [‘LONDON, England (Reuters) – Harry Potter star Daniel Radcliffe gains access to a reported 20 million (\$41.1 million) fortune as he turns 18 on Monday, but he insists the money won't cast a spell on him. Daniel Radcliffe as Harry Potter in "Harry Potter and the Order of the Phoenix" To the disappointment of gossip columnists around the world, the young actor says he has no plans to fritter his cash away on fast cars, drink and celebrity parties.’]

PEGASUS Harry Potter star Daniel Radcliffe gains access to a reported 20 million fortune. Young actor says he has no plans to fritter his cash away. Radcliffe's earnings from the first five Potter films have been held in a trust fund.

BART Harry Potter star Daniel Radcliffe turns 18 on Monday. He gains access to a reported 20 million (\$41.1 million) fortune. Radcliffe's earnings from the first five Potter films have been held in a trust fund. Details of how he'll mark his landmark birthday are under wraps.

T5 Harry Potter star Daniel Radcliffe says he has no plans to fritter away his fortune as he turns 18 . "I don't think I'll be particularly extravagant," he says.

PROPHETNET harry potter star daniel radcliffe gains access to a reported \$ 41 . 1 million fortune. at 18 , radcliffe will be able to gamble in a casino , buy a drink in a pub. details of how he ' ll mark his landmark birthday are under wraps . his earnings from the first five potter films have been held in a trust fund .

Upon examining the summaries generated by the different models, we can observe the following:

GROUND TRUTH provides a concise summary of the main points, mentioning Daniel Radcliffe's fortune, his responsible attitude, and the trust fund holding his earnings from the first five Harry Potter films.

TEXTRANK (BASELINE) produces a longer summary, but with similar information as the ground truth, providing more details and context, such as the fact that he is turning 18.

PEGASUS offers a concise and accurate summary that closely resembles the ground truth. It captures the key points about Radcliffe's fortune, responsible attitude, and the trust fund.

BART presents a summary that covers the main points, including Radcliffe’s fortune, his trust fund, and his upcoming 18th birthday. However, it omits his responsible attitude towards his wealth.

T5 generates a brief summary that focuses on Radcliffe’s responsible attitude but does not mention the size of his fortune or the trust fund.

PROPHETNET creates a summary that includes information about Radcliffe’s fortune, his 18th birthday, and the trust fund. However, it also introduces extraneous details, such as the fact that he will be able to gamble in a casino and buy a drink in a pub, which are not as relevant to the main topic.

In summary, PEGASUS appears to provide the most accurate and concise summary, closely reflecting the ground truth. The other models produce summaries with varying degrees of detail and focus, highlighting the importance of choosing the right model for a given summarization task. However, it is crucial to understand that evaluating a summarization model based on just one text example from a dataset is not enough to determine its overall performance. The reason behind this is that the complexity of the text samples can vary significantly within a dataset. Some text samples might be easier for a model to summarize, while others could be more challenging, containing complex sentence structures, domain-specific jargon, or long and convoluted narratives. That said, to get a more comprehensive sense of a model’s performance, it is essential to evaluate it on a larger number of samples from the testing dataset. By doing so, we can ensure that we are assessing the model’s ability to handle a diverse set of inputs, including those that differ in complexity, topic, and structure. This approach provides a more reliable and generalizable evaluation of the model’s performance, helping us to better understand its strengths and weaknesses. I will showcase such an approach using the arXiv and BillSum datasets and examine how these models perform on these datasets and discuss the results.

COMPARING MULTIPLE SUMMARIES ON THE ARXIV DATASET

In this comparison, we will use the models specifically designed to handle longer sequences, such as Longformer and BigBird, that I introduced in section 3.3. This is because the arXiv dataset consists of considerably longer texts compared to the CNN/Daily Mail dataset, making these models more suitable for this task. That said, the arXiv dataset (4), is a collection of scientific papers from the arXiv.org preprint repository, covering a wide range of disciplines, including physics, mathematics, computer science, and more. This dataset aims to support the development of models that can effectively summarize scientific papers, a challenging task due to the technical and specialized nature of the content. Compared to the CNN/Daily Mail dataset, the arXiv dataset usually includes longer texts. Scientific papers are typically more detailed, with parts such as an introduction, literature analysis, methods, findings, and conclusion. This adds to the intricacy and difficulty of successfully summarizing them.

Moreover, some summarization models, especially those with limited capacity for managing long text sequences, such as BART, T5, and other related models, may struggle with

the length of the text in the arXiv dataset. These models have a certain maximum input token length, which could require truncation or other techniques to successfully manage longer documents. Chunking longer texts like those in the arXiv dataset is one approach, however, the model may not catch crucial information across parts or correctly reflect the document's structure and flow, resulting in less coherent and useful summaries. When using summarization models that cannot handle long text segments, the downside of chunking should be considered.

Another reason the arXiv dataset is useful for measuring and comparing text summarization models is that it evaluates the models' ability to manage more prolonged and more complex texts while still producing clear and helpful summaries. The performance of these models on the arXiv dataset can provide insight into their suitability for different summarization tasks and helps in the selection of the best model for a particular application.

The dataset contains full-text articles along with their abstracts, which serve as human-generated summaries. This makes it an excellent resource for benchmarking and comparing various summarization models in the context of scientific literature. The arXiv dataset presents unique challenges as the language used in scientific papers is often complex, dense, and domain-specific.

DATA SPLITS: **train:** 203,037 examples; **validation:** 6,436 examples; **test:** 6,440 examples

NOTE All used datasets are available via Hugging Face's extensive library of pre-built datasets, making it a convenient choice. This ease of access and implementation allows for fast integration into our projects, streamlining the process of experimenting with various models and datasets, while minimizing the need for manual data wrangling and pre-processing.

Using the unique capabilities of Longformer and BigBird for handling extended texts, we can expect better performance and more coherent summaries. Listing 3.7 illustrates the code to generate summaries out of a range of different text samples from the arXiv dataset.

Listing 3.7 Generating multiply summaries with Longformer

```
def generate_avg_summary(article):
    input_ids = tokenizer(article, padding="max_length",
                           max_length=16384, return_tensors="pt",
                           truncation=True).input_ids.to("cuda")
    global_attention_mask = torch.zeros_like(input_ids)
    global_attention_mask[:, 0] = 1 ❶ global attention on <s> token

    sequences = model.generate(input_ids,
                               global_attention_mask=global_attention_mask,
                               max_length=512, num_beams=4).sequences
    summary = tokenizer.batch_decode(sequences,
                                     skip_special_tokens=True)

    return summary
```

```

dataset_small = test_dataset.select(range(10)) ❷ select range
df = dataset_small.to_pandas() ❸ convert to pandas dataframe
❹ generate summaries for each article
df["Longformer"] = df["article"].apply(generate_avg_summary)

```

To understand why this particular approach is so important, let us compare the different results of the models on producing just one summary out of the dataset, as shown in 3.13 and 3.14, where we decided to only compare our baseline and the Longformer, since BigBird did not perform very well for one sample.

	rouge1	rouge2	rougeL	rougeLsum	google_bleu
TextRank (Baseline)	0.388158	0.132450	0.210526	0.282895	0.128118
bigbird-pegasus	0.338462	0.087629	0.266667	0.323077	0.110609
Longformer	0.583815	0.308140	0.335260	0.491329	0.269841

Figure 3.13 Comparison of Longformer and BigBird incl. our baseline model on one sample from the arXiv dataset.

	rouge1	rouge2	rougeL	rougeLsum	google_bleu
TextRank (Baseline)	0.305858	0.094233	0.170277	0.241837	0.100998
Longformer	0.454593	0.235658	0.317249	0.395121	0.179504

Figure 3.14 Comparison of Longformer and our baseline model on ten samples from the arXiv dataset.

On the basis of the metrics for longer sequences and one sample, we can observe the following results.

- TextRank (Baseline) performs relatively well, especially considering it's a non-neural network approach.
- BigBird-Pegasus, designed to handle longer sequences, shows a lower ROUGE-1 score but has better performance on ROUGE-L and ROUGE-Lsum. Its BLEU score is slightly lower than the baseline.
- The Longformer, another model for longer sequences, outperforms the other models across most metrics. It achieves the highest scores in ROUGE-1, ROUGE-2, ROUGE-Lsum, and BLEU, while having a slightly lower ROUGE-L score.

These results show that Longformer performs best for summarization among the compared models, especially with longer texts. However, it's crucial to consider each task's specific requirements and context when choosing a model. Now, let's evaluate the models on the final dataset, BillSum, comparing results from one and ten samples.

COMPARING MULTIPLE SUMMARIES ON THE BILLSUM DATASET

The BillSum dataset (9), like the CNN / Daily Mail data set, helps in the creation of models that can successfully summarize texts, but now with a legal perspective. The collection contains human-generated descriptions of US Congressional legislation from the 103rd

to the 115th Congress, as supplied by the Congressional Research Service. This dataset is a useful resource for measuring and analyzing different summarization algorithms, particularly in the realm of legislative text.

Due to the complicated and formal language used in legislative documents, the BillSum dataset concentrates on a specialized area and poses distinct challenges. Note that while the summaries produced by models trained on this dataset may not completely convey the subtleties and complexities of the original legislative texts, they can still serve as a decent starting point for comparison.

The data is divided into three sections: US instruction bills, US test bills, and California test bills. The US notes were obtained under a CC0-1.0 license from the Govinfo service offered by the United States Government Publishing Office (GPO). California legislation from the 2015-2016 legislative term is accessible on the legislature's website.

DATA SPLITS: **train:** 18949 examples; **ca_test**(California bills): 1237 examples; **test:** 3269 examples

Due to the specialized character of the BillSum dataset, which consists of legislative bills with complicated law language and structure, text summarization poses distinct challenges. Furthermore, the substance of these laws can be lengthy and complex, making it challenging for models to produce concise and accurate summaries.

Penalizing the model

In some instances, penalizing specific tokens during summarization may be beneficial to prevent the model from producing redundant or irrelevant information. Some tokens, for example, may appear frequently in the source text but add little to the meaning or overall summary. Penalizing these tokens can help to make the generated summary more concise, concentrated, and instructive.

Furthermore, certain tokens may be delicate, offensive, or biased, hence, it's preferable to avoid including them in the summary. By penalizing such tokens, the model can be directed to produce summaries that are more in line with ethical concerns and user standards. Listing 3.8 shows how to penalize the "section" token for the BillSum dataset.

Listing 3.8 Penalizing tokens for summarization

```
❶ tokenize the input text
input_ids = tokenizer.encode(input_text, return_tensors="pt")
tokens_to_avoid = "section" ❷ define tokens to avoid in the summary
❸ encode tokens to their corresponding IDs
tokens_to_avoid_ids = [
    tokenizer.convert_tokens_to_ids(token) for token in tokens_to_avoid]

❹ modify the generate() function
summary_ids = model.generate(input_ids, num_beams=4,
```

```
no_repeat_ngram_size=2, ⑤ set this value based on your preference
bad_words_ids=[tokens_to_avoid_ids], ⑥ Apply penalties
max_length=512, ⑦ set an preferred max_length
min_length=400) ⑧ set an preferred max_length
```

⑧ **decode the generated summary**

```
summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

To do our evaluation of the BillSum dataset, we will use a similar function as we used in listing 3.4 at the beginning of this section. Figures 3.15 and 3.16 show the results from the fine-tuning of our models with one sample and ten samples, respectively.

	rouge1	rouge2	rougeL	rougeLsum	google_bleu
TextRank (Baseline)	0.414815	0.094293	0.177778	0.340741	0.128283
Pegasus	0.493151	0.224771	0.287671	0.438356	0.218349
BART	0.374456	0.197962	0.220610	0.328012	0.143989
T5	0.460829	0.203704	0.230415	0.368664	0.184557
ProphetNet	0.401198	0.156627	0.179641	0.317365	0.139651
bigbird-pegasus	0.118280	0.000000	0.107527	0.118280	0.030905

Figure 3.15 Comparison of various Transformer models incl. our baseline model on the BillSum dataset with one sample tested.

	rouge1	rouge2	rougeL	rougeLsum	google_bleu
TextRank (Baseline)	0.450525	0.200710	0.246021	0.370348	0.184799
BART	0.449934	0.206406	0.232390	0.311584	0.166654
T5	0.475323	0.212825	0.238760	0.321936	0.170961
Pegasus	0.440725	0.192450	0.222895	0.307402	0.154844

Figure 3.16 Comparison of various Transformer models incl. our baseline model on the BillSum dataset with ten samples tested. Note that we only included the ones that performed the best.

Considering the performance metrics for single and ten sample summaries from the BillSum dataset, we see that BART performs well, especially in terms of ROUGE-2 and ROUGE-L scores. Although T5 has slightly higher scores, BART has reasonably balanced performance, making it a promising candidate for further improvement.

Given these findings, it is reasonable that fine-tuning BART on the BillSum dataset could improve its overall performance. The fine-tuning process would allow BART to better adjust to the specific characteristics of the legislative texts, such as the complex legal language and intricate structure, resulting in more accurate and coherent summaries. By optimizing BART for this specialized dataset, we can see if the model's performance can be improved further and provide more useful results for the text summarization task, which we will now do in our last section of this chapter.

3.6 Fine-tuning a summarization model

In this section, we will look at how to fine-tune a summarization model to improve its performance on a specific dataset. As previously discussed, BART achieved competitive results on the BillSum dataset, making it a viable candidate for fine-tuning.

3.6.1 Utilizing the `model.config` function

Examining the model configuration with `model.config` can provide useful insights into the design and hyperparameters of the model we are fine-tuning. The listing 3.9 demonstrates how you can access the model configuration.

Listing 3.9 Print model configuration

```
from transformers import BartConfig, BartForConditionalGeneration

❶ load the configuration
config = BartConfig.from_pretrained("facebook/bart-large-cnn")

❷ load the model
model = BartForConditionalGeneration.from_pretrained("facebook/bart-large-cnn", config=config)

print(model.config) ❸ print the model configuration
)
```

Figure 3.17 shows an excerpt of the resulting model configuration print-out for the BART large model.

```
BartConfig {
  "_name_or_path": "facebook/bart-large-cnn",
  "_num_labels": 3,
  "activation_dropout": 0.0,
  "activation_function": "gelu",
  "add_final_layer_norm": false,
  "architectures": [
    "BartForConditionalGeneration"
  ],
```

Figure 3.17 Small excerpt of the model config print-out for the BART large model.

This data is critical for comprehending the model's capabilities, constraints, and potential areas for improvement. For the time being, we will use this information to acquire a better understanding of the model, and we will revisit this topic in greater detail later in the chapter on hyperparameter tuning. However, by analyzing the configuration, we can make informed decisions when adjusting the model's parameters or modifying its architecture, ultimately leading to more efficient and effective fine-tuning.

Now that we have a better understanding of our model's configuration, let us prepare our data for our model. This will entail pre-processing the data according to the model's requirements, such as tokenization, padding, and generating attention masks, ensuring that the input is correctly formatted and ready for fine-tuning.

3.6.2 Data pre-processing and subset selection

Here we define a preprocessing function and apply it to a subset of the BillSum dataset for both training and testing purposes. The primary objective of this code is to prepare the data for the fine-tuning process of the Facebook/BART-large-cnn model, which we will use for legislative text summarization. The function, as shown in the listing 3.10, takes an example from the dataset, tokenizes the input text and the corresponding summary, and returns the input and target encodings in the required format. The input text is truncated or padded to a maximum length of 1024 tokens, while the summary is limited to 128 tokens. We use the `select(range())` method to select a custom quantity of training and testing data. We can manage the resource needs for training and evaluating the model by using various quantities of data. Larger files will require more GPU RAM and may force the use of smaller batch sizes to work within the available resources. To guarantee effective model training and testing, it is critical to find a balance between the size of the dataset and the available processing resources.

Listing 3.10 Pre-processing data

```
def preprocess_function(example):  
    ❶ tokenize input sequence  
    input_encoding = tokenizer(example["text"], max_length=1024,  
                               truncation=True, padding="max_length", return_tensors="pt")  
  
    ❷ tokenize output sequence  
    target_encoding = tokenizer(example["summary"], max_length=128,  
                                truncation=True, padding="max_length", return_tensors="pt")  
  
    ❸ include input ids, attention mask, labels  
  
    return {"input_ids": input_encoding["input_ids"][0],  
            "attention_mask": input_encoding["attention_mask"][0],  
            "labels": target_encoding["input_ids"][0]}  
  
    ❹ select train and test samples  
    subset_train = dataset["train"].select(range(800)).map(preprocess_function)  
    subset_test = dataset["ca_test"].select(range(400)).map(preprocess_function)  
  
    ❺ set format to PyTorch tensors  
    subset_train.set_format(type="torch", columns=["input_ids",  
                                                    "attention_mask", "labels"])  
    subset_test.set_format(type="torch", columns=["input_ids",  
                                                  "attention_mask", "labels"])
```

To fully understand, what the function from listing 3.10 does, let us look at an illustrative example. For that let us consider the following text and summary:

- Text: "Transformers are a type of neural network architecture that have been proven to be highly effective in handling a variety of NLP tasks. The transformer model relies on self-attention mechanisms and positional encodings to process sequences of data."

- Summary: "Transformers excel in NLP tasks thanks to self-attention and positional encodings."

Now, let's consider these through the steps of our *preprocess_function*:

- **Step 1:** Tokenizes the input text, which might look like this: ["transformers", "are", "a", "type", "of", ..., "sequences", "of", "data"]. If this tokenized text exceeds the maximum length of 1024 tokens, it truncates it. If the tokenized text is shorter, it pads it until it reaches 1024 tokens. The function then returns a tensor of these tokenized inputs.
- **Step 2:** Tokenizes the corresponding summary: ["transformers", "excel", "in", "NLP", "tasks", "thanks", "to", "self-attention", "and", "positional", "encodings"]. Again, this is truncated or padded to a maximum length of 128 tokens.
- **Step 3:** Returns a dictionary containing "input_ids", "attention_mask", and "labels". "input_ids" are the indices of the tokens in the vocabulary, for the tokenized input text. "attention_mask" is a binary tensor indicating the positions of the input tokens that the model should pay attention to (i.e., the tokens that are not padding). "labels" is a tensor containing the indices of the tokens in the summary.

Finally, the function applies these preprocessing steps to the selected train and test samples (step 4) and converts the dataset format to PyTorch tensors (step 5).

This simple yet vital preprocessing function ensures that our data is appropriately formatted and dimensioned, which is crucial for the smooth training and operation of our Transformer model.

Having prepared our data in this manner, we are now ready to use the Hugging Face Trainer class for model fine-tuning.

3.6.3 Using the Hugging-Face Trainer class

The Hugging Face Trainer class offers an intuitive and powerful interface to fine-tune and train our models, streamline, and accelerate the process. One of the primary advantages of using the Trainer class is its simple API, which streamlines model training by abstracting away much of the underlying intricacy.

We can concentrate on our particular task and data by utilizing the Trainer class, while the class handles the different training and evaluation loops. This contains optimizers, learning rate schedulers, and other training-related components, which saves us from having to do it manually.

Another significant benefit of the Trainer class is its seamless interaction with the Hugging Face ecosystem, which includes the extensive collection of pre-trained models and tokenizers accessible in the Transformers library. With this, we can quickly import our preferred model and tokenizer, preprocess the data, and begin training with only a few lines of code. Listing 3.11 shows how you first set up the training arguments for the class and listing 3.12 demonstrates how you instantiate the TrainerClass with the defined training arguments, respectively.

Listing 3.11 Setting traing arguments

```
training_args = TrainingArguments(  
    output_dir='./results',  
    num_train_epochs=1, ❶ recommended range is 3-5  
    learning_rate=3e-5, ❷ use recommended learning rate  
    per_device_train_batch_size=8, ❸ use recommended batch size  
    per_device_eval_batch_size=8,  
    gradient_accumulation_steps=1, ❹ use the recommended, 1 by default  
    warmup_steps=500, ❺ use the recommended warm-up steps  
    weight_decay=0.01, ❻ use the recommended weight decay  
    logging_dir='logs', ❼ used for logging  
    logging_steps=160,  
    evaluation_strategy='steps', ❽ steps between each evaluation  
    eval_steps=160,  
    load_best_model_at_end=True, ❾ load best model found during training  
    save_steps=160,  
    seed=42, ❿ for reproducibility  
    report_to='none' ) ⓫ for reporting platforms such as weights and biases
```

Now we can use these training arguments in our Trainer class as shown in listing 3.12. Note that there are even more settings you can use; for more information please check the documentation on Hugging Face: https://huggingface.co/docs/transformers/main_classes/trainer.

Listing 3.12 Instantiating the Trainer class

```
trainer = Trainer(  
    model=model, ❶ the defined model  
    args=training_args, ❷ previously defined arguments  
    train_dataset=subset_train, ❸ training data  
    eval_dataset=subset_test, ❹ evaluation data  
    compute_metrics=compute_rouge, ❺ custom metrics  
    data_collator=data_collator, ❻ data collator to prepare the data  
    ❼ callbacks (EarlyStopping, TenorBoard etc.)  
    callbacks=[EarlyStoppingCallback(10, 0.001)] )  
  
trainer.train()
```

NOTE It's crucial to use a custom-defined metric for the Trainer class to function. Listing 3.11 demonstrates a simple example using the ROUGE metric.

Listing 3.13 Evaluation metrics function

```
def compute_rouge(eval_prediction: EvalPrediction):  
    rouge = evaluate.load('rouge')  
    predictions = tokenizer.batch_decode(  
        eval_prediction.predictions, skip_special_tokens=True)
```

```

references = tokenizer.batch_decode(
    eval_prediction.label_ids, skip_special_tokens=True)

return rouge.compute(predictions=predictions, references=references)

```

Note that the fine-tuning process used limited configurations, with just one epoch and a 500-step warm-up, where the learning rate gradually increases. Despite these constraints, the process improved BART’s performance on text summarization, as illustrated in figure 3.18.

	rouge1	rouge2	rougeL	rougeLsum
BART before tuning	0.449934	0.206406	0.232390	0.311584
BART after tuning	0.477356	0.233620	0.257681	0.348469

Figure 3.18 Metrics for our BART model before and after fine-tuning only one epoch and 800 training samples on the *BillSum* dataset.

Throughout this chapter, we used a systematic approach to establish a solid baseline with TextRank and stressed the importance of comparing it with state-of-the-art transformer models to evaluate their relative performance. Furthermore, we delved into the fine-tuning process, which allows us to tailor a model like BART to a particular task, thereby increasing its efficacy on that specific dataset. In doing so, we not only got a better understanding of the inner workings and building blocks of these transformer models for summarization, but we also learned how to use them for different text summarization tasks.

3.7 Summary

- There are two primary types of summaries: extractive and abstractive, which differ in their coherence and the algorithms applied to generate them. Transformer-based models are primarily designed for abstractive summarization.
 - Extractive summarization: This method selects and combines the most relevant sentences or phrases from the original text to create a summary, without generating new content.
 - Abstractive summarization: This approach generates a summary by paraphrasing and rephrasing the original text, creating new sentences that convey the key ideas while maintaining coherence.
- Establishing a solid baseline is crucial for evaluating advanced models, and comparing various state-of-the-art models helps in determining the best fit for your specific needs.
- In different summarization benchmarks, cutting-edge summarization models such as BART, T5, ProphetNet, Pegasus, Longformer, and BigBird have achieved outstanding results. These models are built with particular improvements and features that

make them ideal for creating summaries. Although there are other methods, these models are currently among the most effective and commonly used choices for text summarization. Benchmarking compares models on common datasets to evaluate their performance, guiding the development of better techniques. Please note that the benchmark scores provided here are for reference, as some of the models might have already achieved higher scores.

- BART: Combines bidirectional encoding with auto-regressive decoding, excelling in tasks like summarization and translation. (CNN/Daily Mail: ≈ 43.3 ROUGE-L)
- T5: Reformulates all NLP tasks into a unified text-to-text framework, streamlining fine-tuning and performing well in tasks like summarization. (CNN/Daily Mail: ≈ 43.5 ROUGE-L)
- ProphetNet: Offers future n-gram prediction as a self-supervised goal, improving performance in sequence-to-sequence tasks such as summarization and translation. (CNN/Daily Mail: ≈ 42.5 ROUGE-L)
- Pegasus: Developed for abstractive summarization, pre-trains by hiding complete phrases in the input text, resulting in more logical and fluent summaries. (CNN/Daily Mail: ≈ 44.1 ROUGE-L)
- Longformer and BigBird: Transformer models designed for handling longer documents, using moving windows or different attention mechanisms, ideal for summarizing large documents. (Longformer: arXiv: ≈ 40.2 ROUGE-L, BigBird: arXiv: 43.0 ROUGE-L)
- Metrics, such as ROUGE and BLEU, are essential for assessing the performance of Transformer models in automatic summarization tasks.
 - ROUGE: evaluates the overlap of n-grams between the generated summary and the reference summary, commonly used for summarization quality assessment.
 - BLEU: Measures the similarity between the generated translation or summary and a set of reference translations or summaries, focusing on precision of n-grams.
- It is important to examine various datasets and understand their nuances to select the right Transformer model for specific summarization tasks, ensuring adaptability and efficacy in handling data variations for optimal performance and accuracy. Keep these points in mind:
 - Relevance: The dataset should be relevant to the domain or problem of interest. The arXiv dataset is better for summarizing scientific documents than CNN/Daily Mail.
 - Size and diversity: A larger, more diverse dataset helps the model learn subtleties and patterns, improving generalization. In complex language structures or broad themes, this is crucial.

- Quality: The sample should have solid input-output pairs (source text and human-written summaries). This ensures that the model learns to produce coherent, accurate, and useful summaries.
- Representativeness: The dataset should correctly represent the text and summary styles you expect the model to handle in real-world applications. This ensures that the model learns to produce use case-specific summaries.
- It is critical to test models on numerous samples to get a more accurate approximation of their future performance, ensuring that they can manage diverse data and generalize well across tasks and situations.
- Fine-tuning a summarization transformer using the Trainer Class from Hugging Face offers a convenient and efficient approach, streamlining the process and making it easier to optimize the model for specific tasks.

Bibliography

- [1] Mehdi Allahyari, Seyed Amin Pouriyeh, Mehdi Assefi, Saeid Safaei, Elizabeth D. Trippe, Juan B. Gutierrez, and Krys J. Kochut. Text summarization techniques: A brief survey. *CoRR*, abs/1707.02268, 2017. URL: <http://arxiv.org/abs/1707.02268>. 40
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *CoRR*, abs/2004.05150, 2020. URL: <https://arxiv.org/abs/2004.05150>. 49
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020. URL: <https://arxiv.org/abs/2005.14165>. 49
- [4] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. A discourse-aware attention model for abstractive summarization of long documents. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 2018. URL: <http://dx.doi.org/10.18653/v1/n18-2097>, [doi:10.18653/v1/n18-2097](https://doi.org/10.18653/v1/n18-2097). 65
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. URL: <http://arxiv.org/abs/1810.04805>. 50
- [6] H. P. Edmundson. New methods in automatic extracting. *J. ACM*, 16:264–285, 1969. 40
- [7] Günes Erkan and Dragomir R. Radev. Lexrank: Graph-based lexical centrality as salience in text summarization. *CoRR*, abs/1109.2128, 2011. URL: <http://arxiv.org/abs/1109.2128>. 41
- [8] Karl Moritz Hermann, Tomáš Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *NIPS*, pages 1693–1701, 2015. URL: <http://papers.nips.cc/paper/5945-teaching-machines-to-read-and-comprehend>. 61
- [9] Anastassia Kornilova and Vlad Eidelman. Billsum: A corpus for automatic summarization of us legislation, 2019. [arXiv:1910.00523](https://arxiv.org/abs/1910.00523). 67
- [10] Julian Kupiec, Jan O. Pedersen, and Francine R. Chen. A trainable document summarizer. In *Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1995. 41

- [11] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *CoRR*, abs/1910.13461, 2019. URL: <http://arxiv.org/abs/1910.13461>. 49
- [12] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL: <https://www.aclweb.org/anthology/W04-1013>. 57
- [13] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019. URL: <http://arxiv.org/abs/1907.11692>. 51
- [14] H. P. Luhn. The automatic creation of literature abstracts. *IBM Journal of Research and Development*, 2(2):159–165, 1958. doi:10.1147/rd.22.0159. 40
- [15] Rada Mihalcea and Paul Tarau. TextRank: Bringing order into text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL: <https://aclanthology.org/W04-3252>. 41
- [16] Ramesh Nallapati, Bing Xiang, and Bowen Zhou. Sequence-to-sequence rnns for text summarization. *CoRR*, abs/1602.06023, 2016. URL: <http://arxiv.org/abs/1602.06023>. 47
- [17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics. URL: <https://aclanthology.org/P02-1040>. 58
- [18] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018. 50
- [19] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. 49
- [20] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *CoRR*, abs/1910.10683, 2019. URL: <http://arxiv.org/abs/1910.10683>. 49
- [21] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. *CoRR*, abs/1704.04368, 2017. URL: <http://arxiv.org/abs/1704.04368>. 47

- [22] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, Vancouver, Canada, July 2017. Association for Computational Linguistics. URL: <https://www.aclweb.org/anthology/P17-1099>. 61
- [23] Yu Yan, Weizhen Qi, Yeyun Gong, Dayiheng Liu, Nan Duan, Jiusheng Chen, Ruofei Zhang, and Ming Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. *CoRR*, abs/2001.04063, 2020. URL: <https://arxiv.org/abs/2001.04063>. 49
- [24] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding. *CoRR*, abs/1906.08237, 2019. URL: <http://arxiv.org/abs/1906.08237>. 53
- [25] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. *CoRR*, abs/2007.14062, 2020. URL: <https://arxiv.org/abs/2007.14062>. 49
- [26] Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter J. Liu. PEGASUS: pre-training with extracted gap-sentences for abstractive summarization. *CoRR*, abs/1912.08777, 2019. URL: <http://arxiv.org/abs/1912.08777>. 49

Machine translation



This chapter covers

- Introduction to machine translation
- Evolution and approaches of machine-based translation
- Outline of multilingual Transformers
- Customizing a multilingual Transformer for a specific use case

In the previous chapter, we looked at the groundbreaking advances made by transformers in the area of text summarization, delving into the concepts and design decisions of various transformer models. In this chapter, we will expand on that knowledge by investigating the effect of transformers on machine translation, as well as discussing some common methods and challenges in the field.

Machine translation (MT) refers to the automatic translation of text from one language to another by a computer program. As a subfield of artificial intelligence in computational linguistics, machine translation contrasts with human translation, which falls under applied linguistics. In this chapter, however, our attention will be solely focused on the computational aspect of translation, particularly as it relates to transformers.

We will embark on our exploration with a discussion of the Vauquois Triangle, which portrays the evolution from classic machine translation approaches such as direct, transfer-

based, and interlingua-based methods, to modern Neural Machine Translation (NMT) models, with transformers as a notable exemplar.

Next, we'll tackle the complex challenges intrinsic to language processing. These include the handling of morphologically rich languages, word sense disambiguation, interpretation of idioms and colloquialisms, and dealing with out-of-vocabulary words. We will then explore how these challenges can be met using state-of-the-art multilingual transformer models such as mBART, XLM, and mT5.

By doing so, I will provide a brief introduction to MT, as it has been pivotal in the development of deep learning and NLP models, including transformers. The history of MT is closely linked with the evolution of various NLP techniques and models that have led to the current state of Transformers.

Despite the fact that MT may not have as wide-ranging applications as text summarization, which we discussed in the previous chapter, it remains a crucial tool for a multitude of industries such as e-Commerce, customer support, content localization, medical services, and legal services. Large corporations, in particular, may have privacy concerns and prefer to develop their own in-house MT models rather than relying on external online services. This allows them to maintain control over sensitive information and even incorporate or ban specific words and phrases to ensure compliance with their internal policies and regulations. The ability to tailor MT models to their unique requirements makes it an indispensable resource for these organizations.

Moreover, many of the foundational methods and insights drawn from MT research have contributed to the development of LLMs such as BERT and GPT. These contributions have particularly influenced the evolution of key concepts like attention mechanisms, self-attention, and positional encoding. However, it's important to recognize that these concepts have also been enriched by a larger ecosystem of natural language processing and artificial intelligence research. Now, these concepts have been extended to a variety of NLP tasks beyond MT, such as text summarization and sentiment analysis. Furthermore, through various transformer adaptations, these methods are now being extended beyond NLP and applied to other areas such as image classification, time series generation, audio processing, and reinforcement learning.

In addition, while the models, which were introduced in the last chapter, are cutting-edge text summarization algorithms, most of them are primarily designed for English or have limited multilingual support. Therefore, the use of multilingual models from this chapter can be a good starting point for adapting the summarization techniques to other languages.

4.1 Introduction to machine translation

There are two main approaches to MT: knowledge-based and data-driven. Knowledge-based MT relies on linguistic rules and dictionaries, whereas data-driven MT learns from large parallel corpora of source and target language texts. We look more closely at the differences in section 4.2. To understand the different levels of abstraction in MT systems, we will use the Vauquois triangle (13).

4.1.1 The Vauquois triangle

The Vauquois triangle was proposed by Bernard Vauquois in the 1960s, and it visually depicts the complexity trade-off between the source and target languages. The triangle has the source language at its left corner and the target language at its right corner, with an interlingua or intermediate representation at the top corner, as shown in figure 4.1.

The Vauquois triangle helps to understand various MT approaches:

- Direct translation: This method is at the bottom of the Vauquois triangle, where there is the least abstraction. In this method, the source and destination languages are directly mapped, with no abstraction or processing. Because these translation algorithms are simplistic, they may struggle to handle complex sentences, idiomatic expressions, or context-dependent translations.
- Transfer-based translation: This method lies in the middle of the Vauquois triangle and employs a higher degree of abstraction compared to direct translation. It can be divided into two types: syntactic transfer and semantic transfer. Syntactic transfer parses the source language into an intermediate representation based on its syntactic structure. For example, translating "She is reading a book" from English to French, a system might initially dissect the English sentence into a tree-like structure, marking "She" as the subject, "is reading" as the action, and "a book" as the object. This structure is then rearranged to match the sentence structure of French, thus generating "Elle lit un livre". Semantic transfer, on the other hand, creates an intermediate representation based on the meaning or semantic structure of the source language. So, the sentence "She is reading a book" might first be understood as "A female person is in the act of reading something, which is a book". This abstract understanding can then be converted into the equivalent French sentence.

Both methods involve analysis, transfer, and generation stages, the main difference being the level of abstraction used in the intermediate representation. While these systems can handle more complicated structures and linguistic phenomena, they also come with limitations.

- Interlingua-based translation: This method is at the top of the Vauquois triangle and requires the most abstraction. Both the source and destination languages are mapped to a universal, language-independent representation in interlingua-based MT. The translation is then carried out by first converting the original language into the interlingua and then the interlingua into the target language. This method has the potential to handle a broad variety of languages as well as complex linguistic structures.

If you are wondering where transformers fall into this triangle, let me answer your question. They don't fit neatly into these traditional categories. Transformers are data-driven and leverage attention mechanisms to capture semantic structure and conceptual analysis implicitly. They learn how to associate words and phrases with their corresponding translations without having to depend on explicit intermediate representations or symbolic processing.

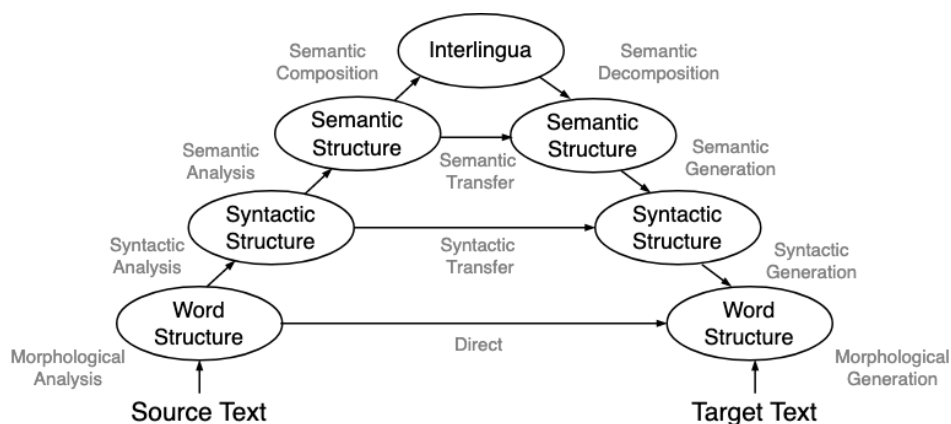


Figure 4.1 The Vauquois triangle for machine translation visualizes that the classical methods try to translate a source language either directly, via a transfer or an interlingua.

Nevertheless, transformers could be occupying a position within the Vauquois triangle that combines aspects of transfer-based and interlingua-based MT, as they are capable of learning complex linguistic structures and capturing context-dependent translations. However, transformers differ from traditional MT approaches in that they use continuous vector representations and end-to-end training, which allows them to scale better and achieve better performance on a wide range of translation tasks.

That said, while transformers can be conceptually related to the Vauquois triangle, they represent a distinct paradigm shift in MT that transcends the original categorization. As the attention mechanism in transformers helps to address some of the most challenging aspects of MT, such as long-range dependencies, idiomatic expressions, and context-dependent translations.

4.2 Machine Translation approaches

As mentioned earlier, there are two primary approaches to MT: knowledge-based and data-driven MT. Figure 4.2 provides an overview of these methods.

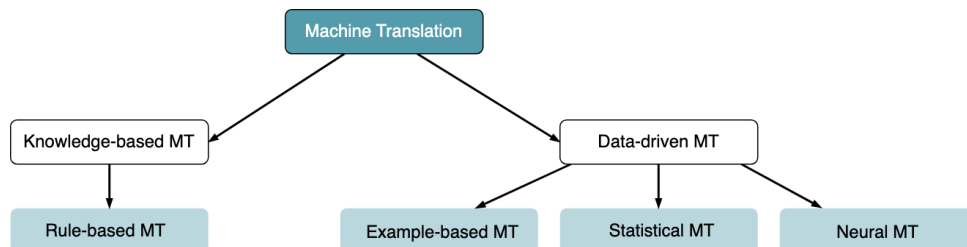


Figure 4.2 Diagram with different MT learning approaches.

In this section, we will briefly explore rule-based machine translation (RBMT), example-based machine translation (EBMT), statistical machine translation (SMT), and neural

machine translation (NMT) to help you understand their differences and recognize the advantages of using transformer models for your translation tasks.

4.2.1 Rule-based machine translation

Rule-Based Machine Translation (RBMT) is a knowledge-based approach that relies on linguistic rules and bilingual dictionaries. In RBMT, the source and target languages are analyzed by experts who create grammar rules and dictionaries to guide the translation process. RBMT systems can be highly accurate in specific domains, but they are labor-intensive and time-consuming to create and maintain. Moreover, these systems may struggle with idiomatic expressions and variations in language usage.

4.2.2 Example-based machine translation

Example-based machine translation (EBMT) is a data-driven approach to MT that relies on a database of bilingual parallel texts, where each entry consists of a sentence from the source language and its corresponding translation into the target language. Instead of using explicit linguistic rules or statistical models, EBMT identifies similar sentences or phrases from the database to generate translations for new input sentences.

4.2.3 Statistical machine translation

Statistical Machine Translation (SMT) is a data-driven approach that uses statistical models to generate translations. These models are trained on parallel corpora, which consist of texts in the source language and their corresponding translations in the target language. SMT systems are less dependent on expert linguistic knowledge and can adapt to changes in language usage more easily than RBMT systems. However, SMT systems may sometimes generate translations that are less grammatically accurate and fluent than those produced by RBMT systems. For instance, if the SMT model encounters an unfamiliar word or phrase, it might struggle to provide an accurate translation. Similarly, it could generate a grammatically awkward sentence if it has been trained on data with such constructions.

NOTE SMT was once a vast research area, and the best systems were highly complex and incorporated numerous intricate details and manually adjusted subcomponents. For each language pair, SMT required a pipeline of independently trained machine learning models, such as translation models and language models, along with extensive human feature engineering and various tables. However, with the advent of transformer models, the field of machine translation experienced a significant leap forward, surpassing the performance of traditional SMT approaches and simplifying the overall process. Take a look at: <https://www.statmt.org/> for more information.

4.2.4 Neural Machine Translation

Neural Machine Translation is another data-driven approach that leverages deep learning algorithms, such as RNNs, Convolutional Neural Networks, and transformers, to generate translations. NMT systems generally outperform SMT systems in terms of fluency and grammatical accuracy. They can also handle idiomatic expressions and learn to generate

more natural translations. However, NMT systems are more computationally intensive and require a large amount of training data to achieve high performance. This data-hunger can be a limitation, especially for low-resource languages or when computational resources are limited.

In summary, knowledge-based MT, such as RBMT, relies on expert linguistic knowledge and rules, while data-driven MT, including SMT and NMT, is based on learning from large parallel corpora. Each approach has its pros and cons, with RBMT providing more control over the translation process but requiring substantial manual effort, while data-driven MT can be more adaptable and scalable but requires large amounts of training data and computational resources.

4.3 *State-of-the-art machine translation models*

Since we reviewed models like BART, T5, and Pegasus in the previous chapter (section 3.3), which are effective for text summarization, this section will spotlight models that have gained prominence in machine translation tasks. While alternatives like Transformer-XL and Evolved Transformer exist, my focus is on Marian NMT, mBART, mBART-50, XLM, XLM-RoBERTa base, M-BERT base, and mT5. These models, are among the most widely applied and recognized in machine translation, showcasing a diverse set of top performing algorithms. Each model has demonstrated competitive performance on various machine translation benchmarks and offers unique improvements or features specifically for text translation. I'll provide a concise overview of their main distinctions in the list below. More detailed information about these models will follow in the subsequent sections, providing you with further insights into their architecture. Please note that since Marian NMT shares significant similarities with BART in terms of modeling code, and BART was introduced in the previous chapter, I will only cover the multilingual BART in the extended summary here.

- Marian NMT, a specialized neural machine translation framework based on the transformer architecture, was developed for fast experimentation and deployment and is widely used for research purposes and real-world applications. The model was originally trained by Jörg Tiedemann, (12), using the Marian C++ library. The number of parameters it uses can vary based on the model's size and configuration. Additionally, it is capable of handling different language variations, for instance, translations from English to 15 Roman-based languages.
- mBART (Multilingual BART) (8), is a variation of the BART model that has been pretrained on large-scale multilingual corpora. It excels in several machine translation tasks, including zero-shot translation. **680 million parameters, trained on 25 languages**
- mBART-50, (10) is a derived model from mBART, with a focus on 50 languages. It has been fine-tuned specifically for low-resource languages, making it more efficient and effective for a diverse set of languages. **680 million parameters, trained on 50 languages**

- XLM (Cross-lingual Language Model) (7), is a transformer model that has been pretrained on a large-scale multilingual corpus. It is designed to learn cross-lingual language understanding and translation tasks. **100 - 665 million parameters, trained on 100 languages**
- XLM-RoBERTa base (3), is a robustly optimized version of the BERT model extended to support multiple languages. It is pretrained on large-scale multilingual text corpora, making it suitable for cross-lingual understanding and machine translation. **270 million parameters, trained on 100 languages**
- M-BERT base (4), is the multilingual version of BERT, pretrained on multilingual corpora. It can be fine-tuned for a variety of tasks, including machine translation, and serves as a strong base model for multilingual applications. **110 million parameters, trained on 104 languages**
- mT5 (Multilingual Text-to-Text Transfer Transformer) (14), is a multilingual variant of the T5 model we've seen in the last chapter. It has been pretrained on a large-scale multilingual corpus, making it suitable for various machine translation tasks and other multilingual NLP applications. **min. 60 million - max 13 billion parameters, trained on 101 languages**

4.3.1 mBART

mBART (Multilingual BART) is a variation of the BART model specifically designed for multilingual applications. Building on the foundation of BART, mBART extends the model to handle multiple languages by pretraining on large-scale multilingual corpora. It excels in several machine translation tasks, including zero-shot translation, where the model is capable of translating between language pairs it was not explicitly fine-tuned on. This is made possible by the shared representations learned during the pretraining phase, which allows the model to transfer knowledge between different languages effectively.

mBART consists of **680 million parameters** and is **trained on 25 languages**. While training the model on a subset of 25 languages, from the common crawl (CC) corpus, the authors noted the imbalances in data representation across the languages. Some languages were far more prevalent in the corpus compared to others, resulting in an overrepresentation of certain languages and underrepresentation of others. To mitigate this and ensure more equitable representation across languages, they leveraged a strategy to rebalance the corpus:

$$\lambda_i = \frac{1}{p_i} \cdot \frac{p_i^\alpha}{\sum_i p_i^\alpha} \quad (4.1)$$

where p_i is the percentage of each language in the CC-25 corpus i , λ represents the rebalancing ratio, and a smoothing parameter of $\alpha = 0.7$ is used.

The mBART model is pretrained on monolingual data from multiple languages, and then fine-tuned on parallel bilingual data. This allows mBART to leverage the vast amount of available monolingual data, leading to improved translation quality, especially when fine-tuned on smaller parallel bilingual datasets. The multilingual pretraining enables the model

to learn shared representations across languages, which can be beneficial for low-resource language pairs and zero-shot translation scenarios.

During pretraining, mBART utilizes a denoising autoencoder with masked language modeling and sequence reconstruction objectives. The model learns to predict and reconstruct sentences from corrupted or masked versions, thereby learning robust and meaningful multilingual representations. The learning objective is to maximize $\mathcal{L}_\theta$, which covers K languages: $\mathcal{D} = \{\mathcal{D}_1, \dots, \mathcal{D}_K\}$ where $\mathcal{D}_i$ is a collection of monolingual documents in language i , which can be represented as follows:

$$\mathcal{L}_\theta = \sum_{\mathcal{D}_i \in \mathcal{D}} \sum_{X \in \mathcal{D}_i} \log P(X | g(X); \theta), \quad (4.2)$$

where X is an instance in language i , and P is the probability distribution over sequences defined by the sequence-to-sequence model (parameterized by θ) given the corrupted text $g(X)$. The function g is a noising function that corrupts the text X .

To handle multilingual text, mBART introduces a special language ID token, which is placed at the end of each input sequence in the encoder and at the start in the decoder. This token informs the model about the language of the input text, allowing mBART to learn language-specific representations during pretraining. As depicted in figure 4.3.

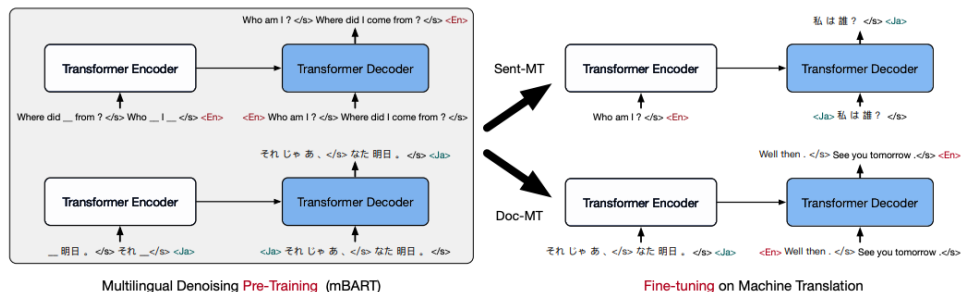


Figure 4.3 Denoising pretraining (left) and fine-tuning on MT tasks (right) with injected noise: sentence permutation and word-span masking. Language id tokens are added at encoder and decoder. Doc-MT and Sent-MT denote document-level and sentence-level MT. The image is taken from the paper: "Multilingual Denoising pretraining for Neural Machine Translation".

For fine-tuning, mBART can be trained on parallel bilingual data for specific language pairs, allowing it to specialize in translation tasks between those languages. It can also be fine-tuned for other NLP tasks like multilingual text classification or named entity recognition, benefiting from the rich multilingual representations acquired during pretraining.

4.3.2 mBART-50

mBART-50 (10) builds on the success of mBART by expanding the model's multilingual capabilities. The model is designed to be extensible, allowing researchers and practitioners to easily add support for additional languages during both pretraining and fine-tuning.

Note that mBART and mBART-50 are both multilingual BART models, sharing the same base architecture. However, they differ in the number of languages they are trained on and their focus on low-resource languages.

mBART is pretrained on 25 languages using a combination of monolingual and parallel data from a range of web sources. It excels in several machine translation tasks, including zero-shot translation, and can be fine-tuned for various NLP applications.

mBART-50 while also derived from the BART model, is specifically designed to support 50 languages. It has been fine-tuned with a focus on low-resource languages, making it more efficient and effective for a diverse set of languages and applications.

The main differences and contributions of the mBART-50 paper are:

- **Multilingual fine-tuning (ML-FT):** Unlike mBART, which uses bilingual fine-tuning, mBART-50 proposes a multilingual fine-tuning strategy where the model is fine-tuned on parallel data from multiple language pairs simultaneously. This approach allows the model to leverage the shared multilingual knowledge and generalize better to low-resource languages and zero-shot translation scenarios.
- **Improved model architecture and pretraining strategy:** The mBART-50 paper presents several modifications to the original mBART model, including improvements in the tokenizer and vocabulary, to better support a larger number of languages and achieve higher translation quality.
 - **Improved tokenizer:** The authors developed an improved tokenizer that is designed to handle languages with different scripts and morphologies more effectively, which contributes to better translation quality across a broader range of languages.
 - **Expanded language coverage:** mBART-50 extends the original mBART model's language support from 25 to 50 languages, providing a more versatile multilingual translation model.
 - **Revised vocabulary:** mBART-50 uses an updated vocabulary that better covers the languages in its training set. This allows the model to learn more accurate and expressive representations for each language, resulting in improved translation quality.
- **Multilingual translation model variants:** The paper also introduces several model variants based on different model capacities, pretraining objectives, and data sampling strategies. These model variants help explore the trade-offs between model complexity, translation quality, and computational efficiency for various translation tasks.

In the fine-tuning stage, the authors collect bitexts (sentence pairs) from various language combinations. For each language pair, they form a distinct set of these sentence pairs for each translation direction.

They then add a special token at the beginning of each sentence to indicate its language, both for the source and the target sentence. This slightly modified pair is what's used for the fine-tuning.

Initially, a pretrained mBART model is used as the starting point. This model is then fine-tuned specifically on these modified multilingual sentence pairs, which have been combined from all the different language sets.

Overall, mBART-50 is a significant advancement in multilingual machine translation, offering improved performance, extensibility, and broader language support compared to the original mBART model.

4.3.3 XLM

XLM (Cross-lingual Language Model) (7), is a transformer-based cross-lingual language model pretrained on large-scale multilingual corpora. It is designed for learning cross-lingual language understanding and translation tasks. XLM comes in different sizes from **100 to 665 million parameters** and is **trained on 100 languages**. In training the XLM model, sentences from multiple languages are picked based on a specific method that follows a multinomial distribution of probabilities. This basically means that it's a way of picking items from multiple categories (in this case, languages) based on given probabilities. Here's how it works:

- First, the model calculates what percentage of the sentences in the training dataset belong to each language. For instance, if 10% of the sentences are in French, then the base probability for selecting a French sentence is 10%.
- This base probability is then adjusted using a special value, alpha, which is set at 0.5. This adjustment is designed to give languages with fewer sentences a better chance to be included in the training.

By using this method, the XLM model ensures a fair and balanced learning process that includes a wide range of languages, even those that are usually underrepresented.

Moreover, XLM employs two main pretraining objectives: Causal Language Modeling (CLM) and Masked Language Modeling (MLM). CLM learns to predict the next token in a sequence given the previous tokens, whereas MLM learns to predict a masked token given its context. Both objectives are used to learn meaningful representations across languages.

CAUSAL LANGUAGE MODELING (CLM)

In the CLM objective, XLM uses a transformer language model to predict the probability of a word given the previous words in a sentence $P(w_t | w_1, \dots, w_{t-1}, \theta)$. For the cross-lingual setting, XLM adopts a simplified approach by leaving the first words in each batch without context. A "batch" in this context is a group of training samples that are processed together. This decision is based on the observation that, in transformer models, previous hidden states can be passed to the current batch to provide context to the first words in the batch. "Hidden states" are internal representations that the model learns and uses to make predictions.

However, this technique of passing hidden states doesn't scale well to the cross-lingual setting, primarily due to the increased complexity and variation in the input data. In a cross-lingual setting, the model has to handle multiple languages, each with its unique

structures and rules, which increases the complexity of the data. As a result, the XLM model opts for a simpler approach by leaving the first words in each batch without context. While this might seem counter-intuitive, it helps maintain computational efficiency and enables effective cross-lingual learning despite the greater variation in the data.

MASKED LANGUAGE MODELING (MLM)

In the XLM models, 15% of the Byte Pair Encoding (BPE), a tokenization method for variable-length words, tokens from the text streams are randomly sampled and replaced with a [MASK] token 80% of the time, a random token 10% of the time, and left unchanged 10% of the time. Unlike the original MLM used in BERT, XLM uses text streams of an arbitrary number of sentences instead of pairs of sentences. The model is trained with batches of 64 streams of continuous sentences, each composed of 256 tokens, as shown in figure 4.4.

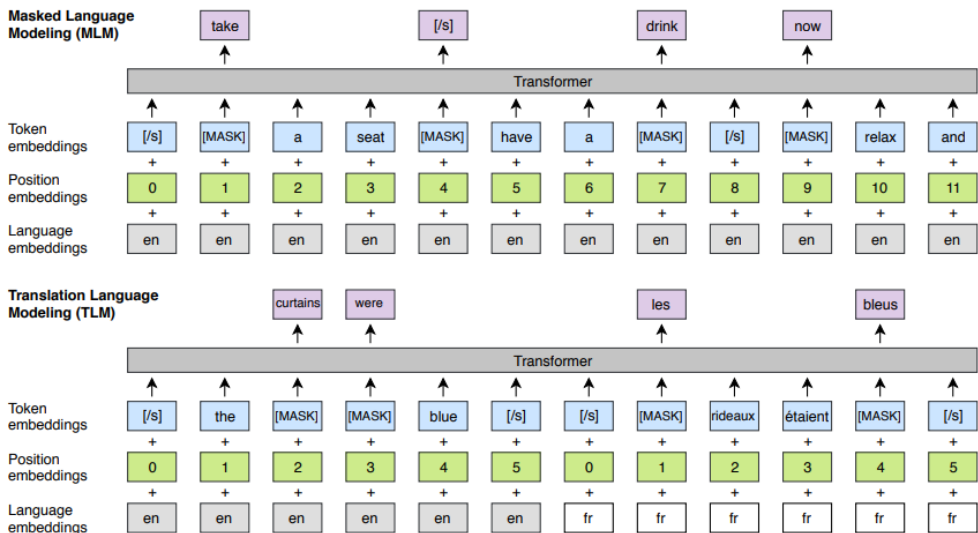


Figure 4.4 The MLM objective uses continuous text streams rather than sentence pairs. TLM extends MLM to parallel sentence pairs, encouraging alignment of English and French representations during masked word prediction. The image is taken from the paper: "Cross-lingual Language Model Pretraining".

To address the imbalance between rare and frequent tokens, XLM subsamples the frequent outputs using an approach where tokens in a text stream are sampled according to a multinomial distribution with weights proportional to the square root of their inverse frequencies.

To handle multilingual text, XLM uses special language ID tokens. These tokens, along with the standard positional embeddings used in transformer models, inform the model about the language of the input text. This allows the model to learn language-specific representations during pretraining.

For fine-tuning, XLM can be trained on parallel bilingual data for specific language pairs, allowing it to specialize in translation tasks between those languages. It can also be fine-tuned for other NLP tasks like multilingual text classification or named entity recognition and is capable of performing zero-shot translation.

4.3.4 XLM-RoBERTa

XLM-RoBERTa (3) is a scaled-up version of the RoBERTa model, specifically designed for multilingual applications. Building on the foundation of RoBERTa and XLM, XLM-RoBERTa is pretrained on a large-scale multilingual corpus, excelling in a variety of cross-lingual tasks, including zero-shot transfer. The model consists of **550 million parameters** and is **trained on 100 languages** using the CommonCrawl corpus. Figure 4.5 shows the amount of data used for the model.

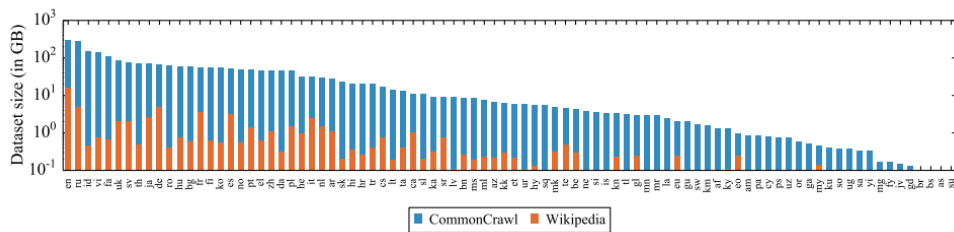


Figure 4.5 The dataset size in gigabytes (plotted in a logarithmic scale) for the 88 languages that are present in both Wiki-100 corpus utilized by mBERT and XLM-100, as well as in the CC-100 employed by XLM-R, are illustrated in the figure. Notably, the CC-100 dataset significantly augments the amount of available data, particularly for low-resource languages, elevating it by several orders of magnitude. The image is taken from the paper: "Unsupervised Cross-lingual Representation Learning at Scale".

During pretraining, XLM-RoBERTa employs the MLM objective from XLM, which predicts masked words in continuous text streams. The model also incorporates the Translation Language Modeling (TLM) objective, extending MLM to parallel sentence pairs. By predicting a masked word in one language, the model can attend to both the sentence and its translation in another language, thereby aligning the representations of the two languages.

Position embeddings of the target sentence are reset to facilitate alignment, allowing the model to learn shared representations across languages. This shared representation is beneficial for low-resource language pairs and zero-shot translation scenarios.

For fine-tuning, XLM-RoBERTa can be trained on parallel bilingual data for specific language pairs, allowing it to specialize in translation tasks between those languages. It can also be fine-tuned for other NLP tasks like multilingual text classification or named entity recognition and has the ability to perform zero-shot translation.

4.3.5 M-BERT

The BERT multilingual base model (M-BERT) is pretrained on a large corpus of multilingual data from the top 104 languages with the largest Wikipedia. M-BERT is trained using

a self-supervised approach, leveraging raw text data without any human-generated labels, allowing it to utilize vast amounts of publicly available multilingual data.

The input representation for BERT can handle both single sentences and pairs of sentences, such as question-answer pairs. WordPiece embeddings with a 30,000 token vocabulary are used. The first token of every sequence is always a special classification token ([CLS]), and the final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence, separated by a special token ([SEP]), and each token is assigned a learned embedding indicating whether it belongs to sentence A or sentence B. These special tokens are illustrated in figure 4.6.

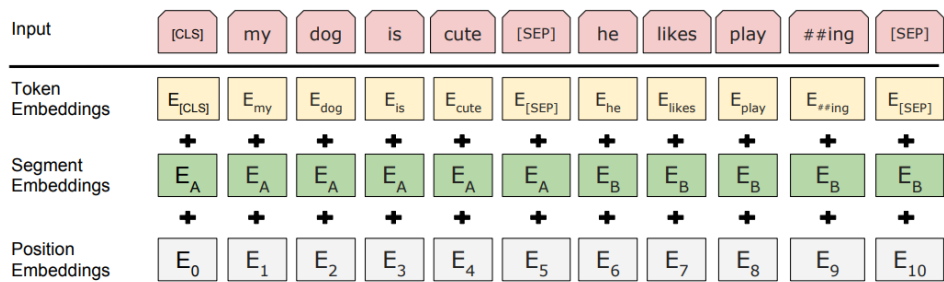


Figure 4.6 Input representation of the BERT model. The image is taken from the paper: "BERT: pretraining of Deep Bidirectional Transformers for Language Understanding".

BERT's training process involves two main objectives:

- **Masked Language Modeling:** The model takes a sentence and masks 15% of its WordPiece tokens at random. The masked tokens are replaced with the [MASK] token 80% of the time, a random token 10% of the time, and the original token 10% of the time. The model is then required to predict the original tokens for the masked positions. This approach allows BERT to learn bidirectional representations of the sentence, as it is not restricted to processing tokens sequentially or masking future tokens.
- **Next Sentence Prediction:** The model is given two masked sentences as inputs, which are sometimes adjacent in the original text and sometimes not. The model must predict whether the two sentences were originally consecutive or unrelated. This task helps BERT learn relationships between sentences.

M-BERT's ability to learn from a large multilingual corpus makes it a strong base model for various tasks, including machine translation and other multilingual applications. The model is case-sensitive, meaning it distinguishes between words based on their capitalization (e.g., "english" vs. "English"). The BERT architecture has been highly influential in the development of other transformer-based models, setting new benchmarks in a wide range of NLP tasks.

4.3.6 mT5

mT5 is a massively multilingual pretrained text-to-text transformer model designed for a wide range of natural language processing tasks. Based on the T5 model, mT5 is trained on a large multilingual dataset covering 101 languages from the mC4 (multilingual Common Crawl) corpus. The main goal of mT5 is to serve as a strong base model for various multilingual and cross-lingual tasks.

When pretraining multilingual models, data sampling from each language plays a critical role. mT5 boosts lower-resource languages by sampling examples according to the probability $p(L) \propto |L|^\alpha$, where $p(L)$ is the probability of sampling text from a given language, $|L|$ is the number of examples in the language, and α is a hyperparameter controlling the boosting of low-resource languages. mT5 uses $\alpha = 0.3$, which provides a reasonable compromise between performance on high- and low-resource languages. An example sampling is shown in image 4.7.

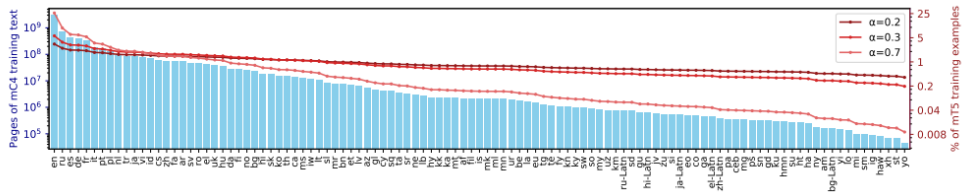


Figure 4.7 Left axis shows the page counts from the mC4 corpus and the right axis depicts the percentage of language examples and the corresponding sampling exponent α . The final model uses $\alpha = 0.3$. The image is taken from the paper: "mT5: A Massively Multilingual pretrained Text-to-Text Transformer".

To accommodate the large number of languages, mT5 employs a larger vocabulary of 250,000 wordpieces, using SentencePiece with the language sampling rates from pre-training. For languages with large character sets, such as Chinese, a character coverage of 0.99999 is used. It also uses SentencePiece's 'byte-fallback' feature, a mechanism designed to handle out-of-vocabulary words by falling back to a character-based representation, ensuring unique encoding of any string.

mT5's pretraining involves masked language modeling with denoising objectives. The model receives an input with some of its tokens randomly masked and is required to reconstruct the original text. This self-supervised approach allows the model to learn rich representations from vast amounts of unlabeled multilingual data.

The mT5 model is available in multiple sizes (small, base, large, 3B, and 13B) to accommodate varying computational requirements and use-cases. It has achieved state-of-the-art performance across a wide range of tasks, including machine translation, summarization, and question-answering, in both multilingual and cross-lingual settings.

In summary, mT5 is a powerful text-to-text transformer model that has been pretrained on a large multilingual dataset, with careful consideration given to data sampling and vocabulary size. Its ability to handle various NLP tasks and transfer learning across multiple languages makes it a highly versatile and effective model for multilingual applications.

Having gained an understanding of the architecture of state-of-the-art multilingual Transformers, let us now examine the techniques and challenges in NLP, particularly in machine translation, and how these models mitigate these challenges.

4.4 Common techniques and challenges in machine translation

Over the years, various techniques have been developed to tackle the task of MT, and the advent of deep learning and transformer-based architectures has led to significant improvements in translation quality. However, despite these advancements, MT still faces numerous challenges that need to be addressed to achieve truly human-like translation performance.

In this section, we will discuss some of the common techniques employed in modern NLP, particularly those based on transformer architectures, as well as the challenges that continue to pose difficulties for the field. We will explore topics such as transfer learning, pretraining, handling morphologically rich languages, word sense disambiguation, dealing with idioms and colloquialisms, and handling out-of-vocabulary words. Understanding these techniques and challenges is crucial for researchers and practitioners working in NMT, but also for the ones working on other NLP related tasks such as text summarization or text generation. Thus, the mentioned techniques and challenges are not limited to MT and can be considered crucial for researchers and practitioners working on text summarization and text generation as well.

Before exploring NMT techniques and challenges, let's see why combining NMT and summarization is useful. Take a hedge fund needing real-time global financial updates. A single multilingual model can handle both NMT and summarization. It translates foreign news to English and generates concise summaries, helping the company understand events like droughts or political shifts that could affect investments.

4.4.1 Benefits of pretraining in NMT and common pretraining techniques

In recent years, pretraining techniques have emerged as a powerful approach to improving the performance of NMT models. Two widely used pretraining techniques in NMT are masked language modeling and denoising autoencoders. We have already slightly discussed these techniques, since the models we used in the last chapter have adopted and built upon the concepts of MLM and denoising autoencoders, as they leverage the massive amounts of unsupervised text data available on the internet to learn general language features and structures before fine-tuning the models on specific translation tasks. This process of pretraining and fine-tuning has led to significant improvements in NMT performance across various language pairs and domains. Before we look more closely into MLM and denoising autoencoders, we will take a short discourse and have a closer look on transfer learning in multilingual models.

TRANSFER LEARNING WITH MULTILINGUAL MODELS

Transfer learning using multilingual models is an important part of NMT since it allows for the use of information learnt across different languages to increase performance on a

variety of tasks such as translation, summarization, and classification. This is particularly important in the context of NMT, where multilingual models have shown great potential.

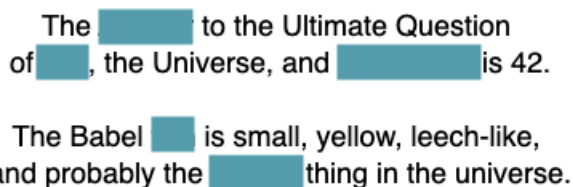
Fine-tuning a pretrained multilingual model, such as mBERT-50, on specific tasks for several languages can result in considerable performance increases. Fine-tuning mBERT-50 for a summarization assignment on German texts, for instance, helps the model to use the information it gained during pretraining across many languages. This allows the model to better understand nuances and linguistic structures unique to the German language, resulting in improved summary results. We will look at such an example in section 4.5.

Furthermore, this not only leads to better results on high-resource languages, but also allows for the development of NLP applications for languages with limited parallel data, as these models can leverage knowledge learned from high-resource languages to improve performance on tasks involving the low-resource language. This helps to mitigate the scarcity of parallel data for low-resource languages and enables the development of high-quality NLP applications for a diverse range of languages.

MASKED LANGUAGE MODELING

MLM is a pretraining technique that involves randomly masking some tokens in a sentence and training a model to predict the masked tokens based on their surrounding context. This forces the model to learn contextual representations of words, which can be fine-tuned for specific tasks like translation. BERT is a well-known example of a model that uses MLM for pretraining.

MLM is also known as the Cloze task (11). The term "Cloze task" comes from the field of psycholinguistics and educational psychology, where it was introduced as a method for measuring reading comprehension, grammar, and vocabulary knowledge. In a Cloze task, participants are presented with a text passage where certain words are removed and replaced with blanks. The participants' task is to fill in the blanks with appropriate words based on the context provided by the surrounding words. An example of this task is shown in figure 4.8, using the sentences: "The Answer to the Ultimate Question of Life, the Universe, and Everything is 42." and "The Babel fish is small, yellow, leech-like, and probably the oddest thing in the universe."



The [] to the Ultimate Question
of [], the Universe, and [] is 42.

The Babel [] is small, yellow, leech-like,
and probably the [] thing in the universe.

Figure 4.8 Illustrative example of a cloze task.

In the context of NMT, MLM can be used to pretrain a model on large-scale monolingual data before fine-tuning it on parallel bilingual data, leading to improved translation quality.

This pretraining process allows the model to learn semantic and syntactic structures present in the language, enhancing its ability to understand and generate meaningful translations.

Moreover, MLM can be extended to multilingual settings, as seen in mBERT, which is pretrained on text from multiple languages. By learning shared semantic structures across languages, mBERT can be fine-tuned for translation tasks involving multiple language pairs, further demonstrating the versatility and effectiveness of the MLM technique in NMT.

DENOISING AUTOENCODERS

Denoising autoencoders are a pretraining technique that involves corrupting an input sequence by randomly applying noise, such as token shuffling, token masking, or token replacement, and then training a model to reconstruct the original sequence from the corrupted version. This approach forces the model to learn robust representations of the input data, which can be useful for downstream tasks like translation.

The primary idea behind denoising autoencoders is that the model learns to capture the underlying structure and semantics of the input data, while being resilient to noise and variations in the input. By learning to recover the original sequence from a noisy version, the model develops an understanding of the relationships between words and their context, allowing it to generate more accurate translations. Furthermore, this pretraining strategy enables the model to learn abstract patterns and generalize better, enhancing its ability to cope with previously unseen inputs. To give you concrete example, let us look at a very simple autoencoder as shown in listing 4.1.

Listing 4.1 Simple Autoencoder

❶ Define the autoencoder model

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, 28 * 28),
            nn.Sigmoid(),
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```

To demonstrate the functioning of this autoencoder, we will use the well-known MNIST dataset, consisting of handwritten digits. Our approach involves feeding a single image into the autoencoder, adding some noise to it, and subsequently recovering the image. You can observe the output of this process in figure 4.9.

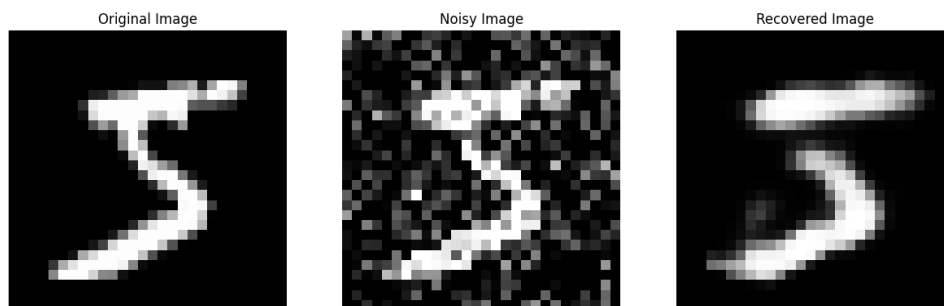


Figure 4.9 From left to right: Original image, the image with added noise and the recovered image from the autoencoder.

If we now apply this concept in the context of NLP, we can use BART as an example model, because it uses a denoising autoencoder-like pretraining approach, where the model learns to reconstruct the original sentence from a corrupted version. In BART's pretraining, words are masked, replaced, or shuffled, which introduces noise in the input:

ORIGINAL SENTENCE "The meaning of life, the universe, and everything is 42."

NOISY SENTENCE "and life, everything the universe, is of The meaning 42."

or another noise variation with masking

NOISY SENTENCE "The meaning of life, the _____, and everything is __."

RECOVERED SENTENCE "The meaning of life, the universe, and everything is 42."

In the context of NMT, models like T5 and BART use denoising autoencoders for pretraining. These models are first pretrained on large-scale monolingual data, during which they learn to denoise the corrupted input sequences. The pretrained models are then fine-tuned on parallel bilingual data for translation tasks. This two-step process of pretraining and fine-tuning helps the model to leverage the rich linguistic information obtained from monolingual data while still benefiting from task-specific supervision provided by the parallel bilingual data.

The denoising autoencoder approach has shown to be effective in NMT, as it helps the model learn contextual representations of words and phrases, which are essential for producing coherent and accurate translations. Additionally, the technique has been successfully applied to other NLP tasks, such as text summarization, sentiment analysis,

text generation, and question answering, demonstrating the versatility and effectiveness of denoising autoencoders in natural language processing.

Moreover, denoising autoencoders can be particularly beneficial in low-resource scenarios, where parallel bilingual data is scarce. By leveraging monolingual data during pretraining, the model can still learn valuable linguistic information and develop robust representations, potentially improving translation quality even when fine-tuning data is limited. This ability to adapt and perform well in diverse settings further highlights the value of denoising autoencoders in NMT and other NLP tasks.

In addition, NMT models require large amounts of parallel bilingual data for training, which can be scarce or expensive to obtain for low-resource languages and domains. To address this issue, pretraining models on large-scale unsupervised data has emerged as a promising approach to improve the performance of NMT models. This approach has several benefits, including improved translation quality, domain adaptation, and better performance for low-resource languages. The following list highlights the benefits of pretraining in NMT in more detail.

- **Improved Performance:** Pretraining allows NMT models to leverage the vast amount of available monolingual data, leading to improved translation quality, especially when fine-tuned on smaller parallel bilingual datasets.
- **Domain Adaptation:** Pretrained models can be easily adapted to different domains by fine-tuning them on domain-specific data, which is particularly useful in scenarios where parallel data for a specific domain is scarce.
- **Handling Low-Resource Languages:** Pretraining can help improve the performance of NMT models for low-resource languages, as models can leverage knowledge learned from high-resource languages during pretraining.

Ultimately, because of its potential to use large amounts of unsupervised data and increase machine translation quality, pretraining has shown to be a valuable strategy in NMT.

4.4.2 Dealing with language-related challenges

Language-related challenges are a common issue in NLP, particularly in the context of machine translation. One such challenge is handling morphologically rich languages such as Finnish or Arabic, where the inflectional morphology of the language leads to an explosion in the number of possible word forms, making it difficult for models to generalize. In the following are the main challenges listed.

- **Handling morphologically rich languages:** NMT models can struggle with languages that have complex grammatical structures and large vocabularies due to extensive inflection, agglutination, and compounding. This leads to a high number of possible word forms and difficulty in generalization.
- **Word sense disambiguation:** NMT models need to accurately determine the meaning of a word based on its context, which can be challenging when words have multiple meanings. Incorrect translations may result if the model fails to discern the appropriate

sense in a given context. Revisiting our "date" example from section 2.1, where I mentioned that the word "date" may refer to both a fruit and a social engagement, we can add another layer of complexity: "date" can also represent a point in time. These three meanings would translate into German as "Dattel" (fruit), "Treffen" (social engagement), and "Datum" (point in time) respectively. This example underscores the importance of context in discerning the correct meaning during translation.

- Dealing with idioms and colloquialisms: Idiomatic expressions and colloquial language can be difficult for NMT models to translate accurately, as their meanings often cannot be inferred from the individual words alone. Recognizing and appropriately translating idiomatic expressions remains a challenge.
- Handling out-of-vocabulary words: NMT models may struggle to handle words that were not seen during training, particularly rare or specialized terms. This can lead to issues in translation quality and difficulties in processing and representing these words.

Two widely-used algorithms to tackle these challenges in NMT models, and language-related challenges in text generation and other NLP tasks, are Byte Pair Encoding (BPE) and SentencePiece. These subword tokenization techniques help overcome various language-related problems, as they break down words into smaller units that the model can process more effectively.

BYTE PAIR ENCODING

Byte-Pair Encoding is a technique originally developed for data compression by replacing the most frequent pair of bytes with a single unused byte to save space. In natural language processing, BPE has been adapted for word segmentation for use in NMT. Here the idea is to use this for operating on characters or character sequences (like Unicode strings) instead of bytes.

BPE operates by iteratively merging the most frequent pairs of characters or character sequences in the training data to create a fixed-size vocabulary of subword units. By breaking words into subword units, BPE helps mitigate the issues of large vocabularies and the explosion of possible word forms in morphologically rich languages. It also enables the model to better represent rare or complex words, improving translation quality. For example, consider the word "inventing." BPE might break it down into smaller units like "in," "vent," and "ing." These smaller units can then be combined in different ways to create new words or understand variations of known words.

Moreover, BPE can indirectly contribute to better word sense disambiguation by capturing semantic nuances of words through subword representations, aiding in context understanding. Its subword-based representation can also help the model identify and process idiomatic expressions and colloquialisms that share common subword units.

In the paper "Neural Machine Translation of RareWords with Subword Units", the authors provide a simple Python implementation of the BPE algorithm. The code snippet given in listing 4.2 is taken from the paper, as it illustrates the main steps of the BPE algorithm using a small example vocabulary:

Listing 4.2 Simple byte pair encoding example

```
def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[symbols[i], symbols[i + 1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?<!\$)' + bigram + r'(?!\$)')
    for word in v_in:
        w_out = p.sub(' '.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out
```

① Define the initial vocabulary with words and their frequencies.

```
vocab = 'l o w </w>': 5, 'l o w e r </w>': 2,
        'n e w e s t </w>': 6, 'w i d e s t </w>': 3
num_merges = 10
for i in range(num_merges):
```

② Compute the frequencies of consecutive character pairs

```
    pairs = get_stats(vocab)
    if not pairs:
        break
    best = max(pairs, key=pairs.get)
```

③ Merge the most frequent character pair and update the vocabulary

```
    vocab = merge_vocab(best, vocab)
    print(best)
```

Which results in the following output:

```
('e', 's')
('es', 't')
('est', '</w>')
('l', 'o')
('lo', 'w')
('n', 'e')
('ne', 'w')
('new', 'est</w>')
('low', '</w>')
('w', 'i')
```

By applying the BPE algorithm to the training data, NMT models can learn a compact and efficient representation of words, which helps in handling the challenges posed by

morphologically rich languages, out-of-vocabulary words, word sense disambiguation, and idiomatic expressions.

SENTENCEPIECE

SentencePiece (6), is an unsupervised text tokenization and detokenization library that is widely used in the transformer ecosystem, including models like BERT, BART, and other transformers. It treats the input text as a raw sequence of Unicode characters and learns a subword segmentation model from the input data using either BPE from the BPE paper I earlier mentioned, or unigram language model-based tokenization (5). Similar to BPE, SentencePiece breaks words into smaller subword units, which enables the model to handle language-related challenges effectively. It also provides a consistent tokenization process across languages, making it particularly useful for multilingual models.

The following code, in listing 4.3, demonstrates the usage of the SentencePiece library to train a tokenizer on a small dataset and apply subword regularization and BPE-dropout. The dataset consists of toy sentences from "The Hitchhiker's Guide to the Galaxy" and some sentences with the word "transformers" in it. After training the tokenizer, we tokenize the word "transformers" with different subword segmentations.

Listing 4.3 Simple SentencePiece example

❶ Toy sentences

```
sentences = [ "The Answer to the ultimate question of life, the universe,
              and everything is 42.",
              # ... (omitted for brevity)
              "Transformers are widely used in machine translation tasks."
            ]
```

```
with open("hitchhikers_sentences.txt", "w") as f:
    for sentence in sentences:
        f.write(sentence)
```

❷ Train SentencePiece tokenizer

```
spm.SentencePieceTrainer.train(input="hitchhikers_sentences.txt",
                               model_prefix="hitchhikers", vocab_size=100)
```

❸ Load trained tokenizer

```
tokenizer = spm.SentencePieceProcessor()
tokenizer.load("hitchhikers.model")
```

❹ Subword regularization and BPE-dropout

```
word = "transformers"
tokenized_word_examples = []
for n in range(5):
    tokenized_word_examples.append(tokenizer.encode(word, out_type=str,
                                                    enable_sampling=True, alpha=0.5, nbest_size=-1))
```

Figure 4.10 illustrates the different subword segmentations of the word "transformers" produced by the SentencePiece tokenizer.

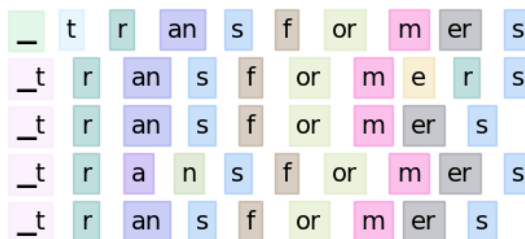


Figure 4.10 Example subword tokenization of the word "transformers". Each row represents one tokenization example, where different subword tokens are color-coded. The slight variations between the rows highlight the effects of subword regularization and BPE-dropout. These result in diverse tokenization possibilities, allowing the model to generalize better by considering multiple plausible subword segmentations.

NOTE The code for these examples as well as for the toy autoencoder can be found in the chapter folder in the books repository.

As we have seen, incorporating BPE or SentencePiece into Transformer models helps tackle the challenges in NMT, leading to better translation performance and improved handling of linguistic variations. These techniques prove valuable not only for NMT but also for various other NLP tasks, such as text classification, text generation and summarization, showcasing the versatility and robustness of Transformer models in NLP.

Eager to see these techniques in action? Let's dive into the final section of this chapter, where we explore the practical implications of these methods and models in a variety of NLP tasks.

4.5 Applications and worked examples

In this section, we'll explore two worked examples that demonstrate the adaptability of multilingual Transformer models. To be more specific, we'll use the mBART-50 model as an example to translate English text into German and finally fine-tune the same model to perform a text summarization task on German documents.

4.5.1 METEOR as evaluation metric

In the previous chapter, I introduced two evaluation metrics, BLEU and ROUGE, which are commonly used for assessing translation and summarization tasks. In this chapter, we will take a closer look at METEOR (Metric for Evaluation of Translation with Explicit Ordering) (1), as an additional metric for evaluating translation tasks.

METEOR is an evaluation metric designed to overcome some of the limitations of the BLEU metric. For example, BLEU only takes into account exact word matches, which can be problematic when dealing with synonyms or paraphrasing. BLEU also doesn't consider the order of words in the translated text. METEOR, on the other hand, takes into account factors such as word matching, stemming, and synonym matching, as well as word order. By doing so, it provides a more accurate and comprehensive evaluation of machine translations.

The primary components of the METEOR metric are:

- **Matching:** METEOR calculates matches between the translation and the reference text at the word or phrase level. It considers exact word matches, stemmed word matches (using a stemmer for the specific language), and synonym matches (using a language-specific synonym dictionary, such as WordNet for English).
- **Alignment:** METEOR computes an optimal alignment between the words in the translated and reference texts, penalizing the translation for the number of chunks or contiguous matches. This is done to account for the importance of word order in translations.
- **Scoring:** METEOR computes a final score as the harmonic mean of precision (the proportion of words in the translation that are also in the reference) and recall (the proportion of words in the reference that are also in the translation). The metric applies a tunable parameter, typically set to 9, to balance the contributions of precision and recall.

In METEOR, first, the unigram precision (P) is computed as the ratio of the number of unigrams in the system translation that are mapped (to unigrams in the reference translation) to the total number of unigrams in the system translation. Similarly, unigram recall (R) is computed as the ratio of the number of unigrams in the system translation that are mapped (to unigrams in the reference translation) to the total number of unigrams in the reference translation. The harmonic mean of P and R, Fmean, is used for the unigram matching, with a greater weight given to recall (9 times the weight of precision, as suggested in the original paper). The resulting formula is:

$$F_{\text{mean}} = \frac{(1 + \beta) \cdot \text{Precision} \cdot \text{Recall}}{\beta \cdot \text{Precision} + \text{Recall}} \quad (4.3)$$

where $\beta = 9$.

To take into account longer matches, METEOR computes a penalty for a given alignment, as follows:

$$\text{Penalty} = 0.5 \cdot \left(\frac{\# \text{ chunks}}{\# \text{ unigrams matched}} \right)^3 \quad (4.4)$$

where chunks is the number of unigram mappings in the system translation that are grouped into adjacent positions in the reference translation.

Finally, the METEOR Score for the given alignment is computed as follows:

$$\text{Score} = F_{\text{mean}} \cdot (1 - \text{Penalty}) \quad (4.5)$$

Compared to BLEU, METEOR is more linguistically motivated and has been shown to correlate better with human judgments of translation quality. However, it is computationally more expensive and requires additional resources like stemmers and synonym dictionaries for different languages.

Now that we have our evaluation metrics in place, it's time to explore the two use-cases within multilingual Transformers. So, get ready to see our models in action!

4.5.2 Generating translations

In this section, we will be leveraging the state-of-the-art mBART-50 model and the MarianMT model to generate translations from the WMT-16 dataset.

The WMT-16 dataset (2) is a machine translation dataset introduced as part of the WMT16 news shared task. It consists of parallel corpora of news articles for multiple languages, with a focus on English and German. The dataset includes news articles from a variety of sources, such as the Europarl corpus, the UN corpus, and various web crawls. The training, validation, and test sets contain approximately 4.5 million, 2,169, and 2,000 sentence pairs, respectively.

These models have been trained on large-scale multilingual datasets and have demonstrated impressive performance on various translation tasks. First we need to initialize our models as show in listing 4.4

Listing 4.4 Initialize mBART-50 and MarianMT

❶ Initialize the mBART-50 model and tokenizer

```
mbart_model = MBartForConditionalGeneration.from_pretrained(
    "facebook/mbart-large-50-many-to-many-mmt")
mbart_tokenizer = MBart50TokenizerFast.from_pretrained(
    "facebook/mbart-large-50-many-to-many-mmt")
```

❷ Initialize the MarianMT model and tokenizer

```
opus_model_name = "Helsinki-NLP/opus-mt-en-de"
opus_tokenizer = MarianTokenizer.from_pretrained(opus_model_name)
opus_model = MarianMTModel.from_pretrained(opus_model_name)
```

We then just loop over the number of examples we want to evaluate as shown in listing 4.5.

Listing 4.5 Loop over the chosen examples in the dataset

```
num_examples = 100
translations = {'mBART-50': [], 'OPUS-MT': []}

❶ Loop over the chosen examples in the dataset
for i in range(num_examples):
    ❷ Translate using mBART-50
    input_text = dataset[i]["translation"]["en"]
    target_text = dataset[i]["translation"]["de"]
    encoded = mbart_tokenizer(input_text, return_tensors="pt")
    generated_tokens = mbart_model.generate(**encoded,
        forced_bos_token_id=mbart_tokenizer.lang_code_to_id["de_DE"])
    output_text_mbart = mbart_tokenizer.batch_decode(
        generated_tokens, skip_special_tokens=True)[0]
    translations['mBART-50'].append(output_text_mbart)

    ❸ Translate using Translate using OPUS-MT
    input_text = dataset[i]["translation"]["en"]
    target_text = dataset[i]["translation"]["de"]
```

```

encoded = opus_tokenizer(input_text, return_tensors="pt")
generated_tokens = opus_model.generate(**encoded)
output_text_opus = opus_tokenizer.batch_decode(
    generated_tokens, skip_special_tokens=True)[0]
translations['OPUS-MT'].append(output_text_opus)

```

Note: MARIAN is a neural machine translation framework, and OPUS-MT is a project that provides pretrained models for machine translation using the MARIAN framework. So, the Helsinki-NLP/opus-mt-en-de, which is the model name, is actually a pretrained MARIAN model for translation between English and German.

In figures 4.11 and 4.12, respectively, are the evaluation results on the WMT-16 dataset for our two models, using the Google BLEU score as our metric. The Google BLEU score was selected due to its improved properties for single sentence evaluation, as traditional BLEU was originally designed as a corpus measure. It's worth noting the importance of evaluating a model based on multiple samples rather than relying on just one or two. This point is highlighted by comparing the performance of the models using 10 and 100 samples from the dataset.

	Model	Google BLEU Score	METEOR Score
0	mBART-50	0.2748	0.4897
1	OPUS-MT	0.2906	0.5129

Figure 4.11 mBART and OPUS-MT evaluated on 10 samples of the WMT-16 dataset.

	Model	Google BLEU Score	METEOR Score
0	mBART-50	0.4647	0.6825
1	OPUS-MT	0.4760	0.7087

Figure 4.12 mBART and OPUS-MT evaluated on 100 samples of the WMT-16 dataset.

NOTE English-German is a common language pair used for benchmarking NMT models due to the availability of abundant parallel corpora and the linguistic complexity that arises from differences in grammar, syntax, and word order between the two languages. This is because German is a more morphologically complex language with a more flexible word order compared to English.

Don't hesitate to experiment with different configurations and evaluate the resulting metrics of the models on the WMT-16 dataset by exploring the notebook provided for this section. This will allow you to gain a better understanding of how the models perform under various conditions and help you make informed decisions when selecting the best model for your specific use case.

4.5.3 Generating German summaries with mBART

In this section, we focus on generating German summaries using the powerful mBART-50 model. For this we will use the MLSum dataset (9). The MLSum dataset is a large-scale multi-domain multi-lingual dataset of news articles and summaries in 12 languages, including English, French, German, Italian, Spanish, Portuguese, Dutch, Russian, Polish, Turkish, Chinese, and Arabic. The dataset has been used in various studies on text summarization since its introduction and represents, therefore, a valid baseline for your own applications.

According to the original paper, the dataset includes more than 1.5 million articles from various news sources, such as CNN, Reuters, and The Guardian. The articles are collected between January 2012 and September 2017, and the summaries are written by professional journalists. The dataset covers various domains, including politics, economics, science, and sports. To get started, we'll need to initialize the mBART-50 model and load our dataset as shown in the listing 4.6.

Listing 4.6 Load dataset and initialize mBART-50

❶ Load dataset

```
dataset = load_dataset("mlsum", "de")
```

❷ Initialize the mBART-50 model and tokenizer

```
tokenizer = MBart50TokenizerFast.from_pretrained(
    "facebook/mbart-large-50-many-to-many-mmt")
model = MBartForConditionalGeneration.from_pretrained(
    "facebook/mbart-large-50-many-to-many-mmt")
```

We will then have to preprocess our data accordingly so we can feed that into our model as shown in listing 4.7.

Listing 4.7 Pre-processing data

```
def preprocess_function(example):
```

❶ tokenize input and output sequences

```
    input_encoding = tokenizer(
        example["text"], max_length=1024, truncation=True,
        padding="max_length", return_tensors="pt")
    target_encoding = tokenizer(
        example["summary"], max_length=56, truncation=True,
        padding="max_length", return_tensors="pt")
```

❷ include input ids, attention mask, labels

```
    return {"input_ids": input_encoding["input_ids"][0],
            "attention_mask": input_encoding["attention_mask"][0],
            "labels": target_encoding["input_ids"][0]}
```

❸ select train and test samples

```
subset_train = dataset["train"].select(range(1200)).map(preprocess_function)
subset_test = dataset["test"].select(range(400)).map(preprocess_function)
```

❹ set format to PyTorch tensors

```
subset_train.set_format(
    type="torch", columns=["input_ids", "attention_mask", "labels"])
subset_test.set_format(
    type="torch", columns=["input_ids", "attention_mask", "labels"])
```

In order to fine-tune our model, we employ the Trainer class from Hugging Face, as discussed in detail in the previous chapter. Additionally, we utilize the same *generate_sum-*

mary function and metrics that were introduced in section 3.6.3. The resulting metrics are shown in figure 4.13.

	rouge1	rouge2	rougeL	rougeLsum
mBART	0.306167	0.211556	0.262447	0.268025

Figure 4.13 mBART performance on the MLSUM dataset for summarizing German text.

NOTE All code of this section can be found in the chapter folder in the corresponding notebooks in the book repository. It’s important to acknowledge that we are using multilingual Transformers, which require a significantly larger amount of computational resources and training data compared to single-language models. In fact, while some smaller single-language models can be trained on a single GPU, multilingual variants demand more extensive computational setups due to their increased complexity and the need to handle multiple languages. For instance, multilingual models may require more epochs, larger datasets, and different batch sizes per device to achieve optimal performance, as batch sizes influence the accuracy of gradient estimation during learning. As a result, the presented code is for explanatory purposes only and not a fully real-world example, since leveraging more resources would be necessary to cover a real-world scenario.

4.6 Summary

- The Vauquois Triangle visually represents traditional methods like direct, transfer-based, and interlingua-based translation. Transformers don’t fit neatly into the triangle but signify an important shift in NMT. They transcend the original categorization and achieve improved performance across various translation tasks by leveraging attention mechanisms and continuous vector representations.
- Machine translation approaches encompass rule-based, example-based, statistical, and neural machine translation. NMT is a data-driven approach that is data-hungry, but multilingual Transformers employ innovative techniques to tackle this challenge.
- In various NMT benchmarks, state-of-the-art models like Marian NMT, mBART, mBART-50, XLM, XLM-RoBERTa base, M-BERT base, and mT5 have demonstrated exceptional performance. These models incorporate specific enhancements and features, making them suitable for multilingual machine translation tasks.
 - Marian NMT: A highly efficient NMT framework focused on research purposes, offering various pretrained models and supporting numerous language pairs. (WMT’18 English-German: ≈ 37.9 BLEU)
 - mBART: A multilingual denoising autoencoder that learns language-agnostic representations, enabling effective fine-tuning for tasks like machine translation. (WMT’19 English-German: ≈ 33.6 BLEU)

- mBART-50: An extension of mBART with support for 50 languages, leveraging a shared vocabulary and improved pretraining for multilingual machine translation. (WMT’20 English-German: ≈ 39.0 BLEU)
 - XLM: A scalable and flexible model architecture designed for cross-lingual learning, suitable for both unsupervised and supervised machine translation. (WMT’16 English-German: ≈ 38.1 BLEU)
 - XLM-RoBERTa base: A variation of XLM that incorporates the RoBERTa pre-training approach, improving cross-lingual understanding and translation capabilities. (WMT’20 English-German: ≈ 40.4 BLEU)
 - M-BERT base: A multilingual version of BERT trained on 104 languages, suitable for zero-shot cross-lingual transfer and multilingual machine translation. (WMT’19 English-German: ≈ 32.3 BLEU)
 - mT5: A multilingual version of T5, featuring a text-to-text framework that unifies NLP tasks, including machine translation, across 101 languages. (WMT’20 English-German: ≈ 41.8 BLEU)
- METEOR, designed to complement BLEU in translation tasks, differs as follows:
 - It uses an alignment-based approach for flexible word and phrase matching.
 - It considers synonyms for recognizing semantically equivalent translations.
 - It computes a weighted harmonic mean (an F-score) of precision and recall, penalizing extreme differences more than BLEU.
 - It allows adjustment of parameters like weights for precision and recall for fine-tuning.
 - Language-related challenges in NLP, particularly in machine translation, can be addressed through various techniques:
 - Handling complex languages: Transformer and multilingual models, such as mBERT or XLM, are more effective at dealing with languages that have complex grammar and large vocabularies. Additionally, subword tokenization techniques, like BPE and SentencePiece, help mitigate issues arising from languages with numerous word forms.
 - Word sense disambiguation: The self-attention mechanism in transformers, pre-trained models, and large-scale multilingual models improve context understanding and help choose the appropriate sense of words during translation.
 - Idioms and colloquialisms: Transformers, pretraining processes, and multilingual models contribute to better handling of idiomatic expressions by capturing long-range dependencies, learning underlying language structures, and leveraging shared semantic structures across languages.
 - Out-of-vocabulary words: Subword tokenization algorithms, such as BPE and SentencePiece, enable Transformers to better handle rare or unseen words, while

multilingual Transformers trained on diverse datasets and zero-shot learning help cope with out-of-vocabulary words by transferring knowledge across languages.

- Multilingual models serve as a robust foundation for a wide range of NLP applications across multiple languages. Their language-agnostic nature makes them highly adaptable, enabling them to handle tasks such as summarization, classification, named entity recognition, and more, in languages beyond English. By leveraging the shared knowledge and structure of different languages, these models can effectively learn to perform various NLP tasks with limited training data, thus providing a powerful and versatile solution for handling diverse linguistic contexts and requirements.

Bibliography

- [1] Banerjee, S. and Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments, *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, Association for Computational Linguistics, Ann Arbor, Michigan, pp. 65–72. URL: <https://www.aclweb.org/anthology/W05-0909>
- [2] Bojar, O. r., Chatterjee, R., Federmann, C., Graham, Y., Haddow, B., Huck, M., Jimeno Yepes, A., Koehn, P., Logacheva, V., Monz, C., Negri, M., Neveol, A., Neves, M., Popel, M., Post, M., Rubino, R., Scarton, C., Specia, L., Turchi, M., Verspoor, K. and Zampieri, M. (2016). Findings of the 2016 conference on machine translation, *Proceedings of the First Conference on Machine Translation*, Association for Computational Linguistics, Berlin, Germany, pp. 131–198. URL: <http://www.aclweb.org/anthology/W/W16/W16-2301> 104
- [3] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L. and Stoyanov, V. (2019). Unsupervised cross-lingual representation learning at scale, *CoRR abs/1911.02116*. URL: <http://arxiv.org/abs/1911.02116> 86, 91
- [4] Devlin, J., Chang, M., Lee, K. and Toutanova, K. (2018). BERT: pre-training of deep bidirectional transformers for language understanding, *CoRR abs/1810.04805*. URL: <http://arxiv.org/abs/1810.04805> 86
- [5] Kudo, T. (2018). Subword regularization: Improving neural network translation models with multiple subword candidates, *CoRR abs/1804.10959*. URL: <http://arxiv.org/abs/1804.10959> 101
- [6] Kudo, T. and Kazawa, H. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, <https://github.com/google/sentencepiece>. 101
- [7] Lample, G. and Conneau, A. (2019). Cross-lingual language model pretraining, *CoRR abs/1901.07291*. URL: <http://arxiv.org/abs/1901.07291> 86, 89
- [8] Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M. and Zettlemoyer, L. (2020). Multilingual denoising pre-training for neural machine translation, *CoRR abs/2001.08210*. URL: <https://arxiv.org/abs/2001.08210> 85
- [9] Scialom, T., Dray, P.-A., Lamprier, S., Piwowarski, B. and Staiano, J. (2020). ML-SUM: The multilingual summarization corpus, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online, pp. 8051–8067. URL: <https://aclanthology.org/2020.emnlp-main.647> 105

- [10] Tang, Y., Tran, C., Li, X., Chen, P., Goyal, N., Chaudhary, V., Gu, J. and Fan, A. (2020). Multilingual translation with extensible multilingual pretraining and finetuning, *CoRR abs/2008.00401*.
URL: <https://arxiv.org/abs/2008.00401> 85, 87
- [11] Taylor, W. L. (1953). Cloze procedure: A new tool for measuring readability, *Journalism Quarterly* **30**(4): 415–433.
URL: <https://doi.org/10.1177/107769905303000401> 95
- [12] Tiedemann, J. (Accessed in 2023). Joerg tiedemann - Research Portal - University of Helsinki, <https://researchportal.helsinki.fi/en/persons/j%C3%B6rg-tiedemann>. Accessed: 2023-04-02. 85
- [13] Vauquois, B. (1968). Structures profondes et traduction automatique., *Le systeme du CETA Rev. Roum. Ling.* **13**, 105130(4): 689–1176. 81
- [14] Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A. and Raffel, C. (2020). mt5: A massively multilingual pre-trained text-to-text transformer, *CoRR abs/2010.11934*.
URL: <https://arxiv.org/abs/2010.11934> 86

Text classification

This chapter covers

- Introduction to text classification
- Naïve Bayes as baseline model
- Outline of transformers for text classification
- Challenges in text classification
- Evaluation metrics for text classification
- Fine-tuning various text classification models

In the previous chapter, we delved into the fascinating world of machine translation, which to some extent, has been a driving force behind the advancements in machine learning and deep learning, ultimately leading to the invention of transformers. In this chapter, we'll explore the world of text classification, which refers to the process of assigning predefined categories to text based on its content. An abundance of use cases makes this topic both fun and engaging. While we'll cover the essential theory, it's the wide variety of practical applications that truly brings excitement and value to this chapter. Are you as eager to dive into the diverse world of text classification as I am? Let's get started with the various applications in the next section!

5.1 Introduction to text classification

When you first hear about text classification, sentiment classification might come to mind. But in reality, text classification has a broad range of applications. In this chapter, we'll explore three interesting types of text classification: sentiment classification, topic modeling, and multi-class classification. Let's briefly see what each of these involve:

- **Sentiment classification:** Identifies sentiment expressed in a piece of text. It is often used to determine whether the tone of a text is positive, negative, or neutral. This technique is widely used in social media monitoring, customer feedback analysis, and market research to understand people's opinions and attitudes towards certain topics, products, or services.
- **Topic modeling:** Categorize text into predefined topics or themes, like sports, politics, or technology.
- **Multi-class classification:** Assign a single label to a text out of several possible classes, such as star ratings for a review.

NOTE Want to dive deeper? I've jam-packed appendix ?? with a bunch of interesting applications you can use text classification for, along with quick, easy-to-understand explanations of what each one involves.

I've carefully chosen the text classification examples to gradually increase the complexity. While sentiment classification presents its own challenges, multi-class classification for reviews introduces a more nuanced problem, requiring the model to recognize subtle relationships between ratings and written text. This gradual increase in complexity showcases the versatility and strength of transformers, making them an excellent tool for tackling intricate text classification tasks. By working through these examples, you gain practical experience in applying transformers to real-world problems, enhancing your understanding of the theoretical concepts discussed in this book chapter.

To lay some groundwork, text classification is primarily concerned with understanding the semantics, or the meaning of the text, and to a certain extent, it involves reasoning. By reasoning, I mean the model's ability to infer implicit information, understand context, and make logical connections based on the given text. Models like transformers excel in deciphering the meaning of text and making predictions based on that comprehension. While more advanced models like transformers can handle complexities inherent in language processing, even simpler models like Naïve Bayes classifiers can perform impressively for such tasks. Despite their simplicity, Naive Bayes classifiers offer a level of sophistication that often surpasses rule or dictionary-based methods like those used by the TextBlob library. Their efficiency and surprising effectiveness make them an excellent baseline for comparing and evaluating the performance of more complex models like transformers.

5.1.1 Establishing a baseline for text classification: Naïve Bayes classifier

As you might remember, I've mentioned in section 3.1.3 that establishing a baseline is important to evaluate the performance of your more advanced model, like a transformer.

The chosen baseline model should be simple but still sophisticated enough to provide a solid foundation for evaluation, which is why I chose the Naïve Bayes classifier as a baseline for text classification. The Naïve Bayes classifier has shown to be one of the most efficient and effective inductive learning algorithms for machine learning and data mining, despite its fundamental assumption of conditional independence, which implies that features independently influence outcomes, rarely holding true in real-world applications.

Additionally, getting a grasp on how the Naïve Bayes classifier works allows us to simplify and visualize the process of text classification. This foundational understanding will be instrumental as we venture into the realm of more complex models. Dealing with transformers and text classification often means navigating high-dimensional spaces, which can get quite abstract. However, having the basic concept of Naïve Bayes classifier in your toolbox makes tackling these complexities a little less daunting.

Before we delve deeper into the mathematics, let's consider a simple, relatable example to better understand how the Naïve Bayes classifier works and how it handles the conditional independence assumption. Imagine you have a laundry basket filled with clean, dry socks of different types: *long*, *short*, *black*, *white*, *patterned*, and *plain*. Your task is to sort these socks into distinct categories for easier storage.

In a Naïve Bayes sense, each sock can be thought of as a document, and each characteristic (*long*, *short*, *black*, *white*, *patterned*, *plain*) is a feature. The categories you're sorting into (e.g., *long*, *black*, *plain* or *short*, *white*, *patterned*) can be thought of as the classes in a classification problem.

Now, the Naïve Bayes classifier would look at each sock independently and estimate its probability of belonging to a certain category based on its features. For example, if a sock is long, black, and plain, it would calculate the probability of it belonging to the *long*, *black*, *plain* category. This is analogous to calculating the probability of a document (like an email) being spam based on its features (words in the email).



Figure 5.1 Illustration of a Naïve Bayes classifier in action. Just as we sort socks into different piles based on their features (color), the Naïve Bayes classifier sorts text into different categories. Each sock pile represents a class that the classifier could assign a piece of text to. This sorting process, driven by learned probabilities, is at the heart of how Naïve Bayes performs text classification.

The "naïve" part comes in when considering combinations of features. Naïve Bayes assumes that each feature impacts the outcome independently. For instance, it would treat *long* and *black* as if they have no connection or influence on each other when deciding if a sock belongs in the *long*, *black*, *plain* category.

However, in reality, certain combinations of features (like *long* and *black*) might not only be more common, but also tend to occur together. This would violate the assumption of independence, as these features are dependent on each other. Despite this, the Naïve Bayes classifier still tends to perform well in practice. This is because, even though dependencies exist, they are often evenly distributed across different classes due to the varied data. This creates a kind of "canceling out" effect, as the combined effect of these dependencies does not favor one particular class significantly over others. This allows the Naïve Bayes classifier to be effective despite its "naïve" assumption.

Now, let's dive into the inner workings of the Naïve Bayes classifier to help you better understand its approach to text classification. To do this, let's put ourselves in the shoes of an algorithm and view the problem from a mathematical perspective.

In the field of machine learning, a classifier is essentially a function that assigns a class label to an instance. Each instance is a vector of n features, which, in this context, are words ($w_1, w_2, \dots, w_n$), and c is the class we aim to predict for a given instance.

The Naïve Bayes classifier works according to Bayes' theorem, under the "naïve" assumption of independence among the words. This model assumes all words in the dataset to be mutually independent, suggesting that the presence or absence of a particular word doesn't influence others. The mathematical representation of the Naïve Bayes classifier is as follows:

$$\arg \max_{c \in \text{Pos, Neg}} \prod_{i=1}^n P(w_i | c) \cdot P(c) \quad (5.1)$$

This formula signifies that for a given set of words $w_1, w_2, \dots, w_n$, the Naïve Bayes classifier will choose the class c that maximizes the product of the individual probabilities of each word given the class, multiplied by the prior probability, that is the probability of an event before new data is taken, of the class ($P(c)$).

Now that we have a solid understanding of the Naïve Bayes classifier and its mathematical foundation, it's time to bring it to life through code. Remember, coding it out finally makes those abstract mathematical concepts tangible and practical. You'll find that implementing this algorithm is quite straightforward, thanks to the simplicity of the mathematical formula. Let's explore the details of this implementation in Listing 5.1, where you'll see how each step of the algorithm is elegantly translated into code that efficiently performs text classification.

Listing 5.1 Naïve Bayes class implementation

```
class NaiveBayes:
    def fit_distribution(self, data):
        mu = np.mean(data)
        sigma = np.std(data)
```

```

dist = norm(mu, sigma)
return dist

def fit(self, X, c):
    self.classes = np.unique(c)
    ❶ Initializing the priors dict, to store  $P(c)$  for each class  $c$ 
    self.priors = {}
    ❷ Initializing the distributions dict, to store  $P(w_i | c)$  for each feature  $w_i$  and class  $c$ 
    self.distributions = {}

    for class_ in self.classes:
        X_class = X[c == class_]
        ❸ Computes  $P(c)$  for the current class  $c$ .
        self.priors[class_] = len(X_class) / len(X)
        ❹ For each feature  $i$ , compute  $P(w_i | c)$ .
        self.distributions[class_] = [self.fit_distribution(X_class[:, i])
                                     for i in range(X.shape[1])]

def predict_proba(self, X):
    proba = np.zeros((X.shape[0], len(self.classes)))

    for i, class_ in enumerate(self.classes):
        ❺ Fetches  $P(c)$  for current class.
        prior = self.priors[class_]
        ❻ Fetches  $P(w_i | c)$  for each feature  $i$ .
        likelihoods = [dist.pdf(X[:, j]) for j, dist in
                       enumerate(self.distributions[class_])]
        ❼ Multiplies all likelihoods with the prior, resulting in unnormalized  $P(c | w_1, \dots, w_n)$ .
        proba[:, i] = prior * np.prod(likelihoods, axis=0)
        ❽ Normalizes the probabilities, effectively dividing by  $P(w_1, \dots, w_n)$ .

    return proba / np.sum(proba, axis=1, keepdims=True)

def predict(self, X):
    proba = self.predict_proba(X)
    return np.argmax(proba, axis=1)

```

In this Naïve Bayes classifier implementation, the *fit method* computes and stores the prior probabilities and likelihoods for each class and word. The *predict_proba* method calculates the product of these likelihoods with the prior, resulting in an unnormalized probability for each class given a set of words. This is effectively the numerator of Bayes' theorem. It then normalizes these probabilities to sum to 1, basically dividing by the denominator of Bayes' theorem, which is the probability of the words. The *predict method* simply chooses the class with the highest posterior probability.

Let's see the Naïve Bayes classifier in action with a simple yet illustrative example. We'll use the *make_blobs* function from the widely-used ML library, scikit-learn, to generate a 2D classification dataset, as shown in Listing 5.2. This will allow us to visually inspect the results and gain an intuitive understanding of the algorithm's performance. By working through this example, you'll appreciate the ease of implementation and the effectiveness of Naïve Bayes in solving basic classification tasks. Additionally, this hands-on experience will

help solidify the theoretical concepts discussed earlier and set the stage for more advanced comparisons with other machine learning techniques, such as transformers.

Listing 5.2 Generate 2D classification dataset

```
X, y = make_blobs(n_samples=100, centers=2, n_features=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

① Train and test the Naive Bayes classifier

```
nb = NaiveBayes()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)
```

② Calculate accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: accuracy:.3f")
```

With this dataset, our Naïve Bayes classifier achieves an impressive accuracy of 1.0. As depicted in Figure 5.2, you can clearly see the data points for the two classes and the decision boundary that separates them. This visual representation not only confirms the classifier's effectiveness but also demonstrates the power and simplicity of the Naïve Bayes classifier in handling classification tasks.

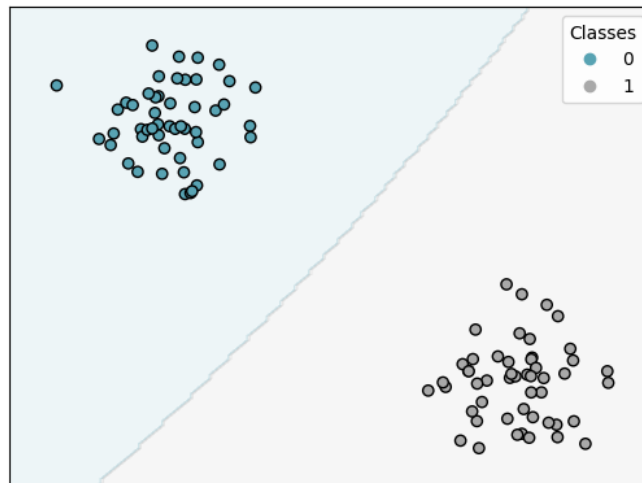


Figure 5.2 Visual representation of the decision boundary as determined by our Naïve Bayes classifier. This plot illustrates the decision-making process of the classifier, defining the regions where it would classify data points into different classes.

NOTE You can find all the code used for this code examples in the **ch05** folder within the notebook titled *ch05_text_classification_bayes.ipynb* located in the repository for this book. Don't hesitate to try out the class and experiment with it.

It's important to note, however, that real-world problems are often more complex than this example, which is why we use the Naïve Bayes classifier as a baseline to compare against more advanced methods like transformers. In the upcoming worked examples, we'll explore the performance comparison between the Naïve Bayes classifier and transformers, revealing some fascinating results and insights. By doing so, you'll gain a deeper understanding of when to use a simple algorithm like Naïve Bayes and when to opt for more sophisticated approaches, which we will introduce in the next section.

5.2 *Transformers in text classification: an overview*

In this section, we will explore a selection of state-of-the-art models that excel in the domain of text classification. With a plethora of models available, it is important to identify those that have demonstrated exceptional performance on various text classification benchmarks while also catering to the unique requirements of specific classification tasks. One of those benchmarks is the GLUE Benchmark (General Language Understanding Evaluation), (8). The GLUE Benchmark is a collection of nine text NLP tasks that focus on sentence-level or sentence-pair analysis. These tasks serve as a comprehensive evaluation for language understanding models, such as those we discuss in this section. The tasks included in the GLUE Benchmark are:

- **CoLA (Corpus of Linguistic Acceptability):** Assess if a sentence is grammatically correct or not. This dataset consists of sentences labeled as grammatically correct or incorrect.
- **MNLI (Multi-Genre Natural Language Inference):** Evaluate if a sentence entails, contradicts, or is unrelated to a given hypothesis. This dataset has two versions: one with the validation and test sets originating from the same distribution and another called "mismatched," where the validation and test sets use out-of-domain data.
- **MRPC (Microsoft Research Paraphrase Corpus):** Determine whether two sentences are paraphrases of one another or not.
- **QNLI (Question-answering Natural Language Inference):** Assess if the answer to a question is present in the second sentence or not. This dataset is derived from the SQuAD (Stanford Question Answering Dataset), which is a collection of question-answer pairs based on Wikipedia articles, designed to evaluate a model's ability to comprehend and extract relevant information.
- **QQP (Quora Question Pairs2):** Determine if two questions have the same semantic meaning or not.
- **RTE (Recognizing Textual Entailment):** Evaluate if a sentence entails a given hypothesis or not.

- SST-2 (Stanford Sentiment Treebank): Classify a sentence as having a positive or negative sentiment.
- STS-B (Semantic Textual Similarity Benchmark): Measure the similarity between two sentences on a scale from 1 to 5.
- WNLI (Winograd Natural Language Inference): Determine if a sentence containing an ambiguous pronoun and another sentence with the pronoun replaced are entailed or not. This dataset is based on the Winograd Schema Challenge dataset.

These tasks in the GLUE Benchmark provide a thorough assessment of the performance and capabilities of various models in handling diverse text classification challenges, including tasks that extend beyond traditional text classification to other areas of natural language understanding. This diversity allows for a more comprehensive and robust assessment of a model's ability to understand and classify text in different contexts, reflecting the complex nature of real-world language use.

Building on the context of the GLUE Benchmark, we now turn our focus to a selection of top-performing text classification models that have collected recognition and widespread usage in the industry. These models serve as a point of reference and a benchmark for text classification tasks, helping us measure their capabilities and effectiveness. In the following discussion, I will provide a concise overview of these models before exploring their features and capabilities in greater detail. This acts as a comprehensive summarization of each model's original paper, highlighting the important innovations of each model. Moreover, by concentrating our discussion on these models here, you gain a deep and comprehensive understanding of their differences and similarities. This facilitates a direct comparison between them. This profound knowledge, gained from directly comparing the models, will prove invaluable when we use these models in our worked examples and in your own real-world use cases. Understanding the architecture of each model and seeing their performance on the GLUE benchmark not only provides a solid foundation for interpreting the variations in their performance, but also gives you a head start in estimating their potential effectiveness for your specific tasks.

- BERT (Bidirectional Encoder Representations from Transformers), (2) is a transformer-based model pre-trained on large-scale, bidirectional language modeling tasks. It has achieved state-of-the-art results on various natural language processing tasks, including text classification. BERT comes with **110 million** parameters and is available in both cased and uncased versions.
- RoBERTa (Robustly Optimized BERT Pretraining Approach), (5) builds upon BERT by implementing several optimizations in pretraining techniques and training on a larger dataset. RoBERTa has achieved improved results on various NLP benchmarks and comes with **125 - 355 million parameters**.
- ALBERT (A Lite BERT) (4), as the name indicates, is a more efficient and lighter version of BERT that uses parameter-sharing techniques across layers to reduce the model's size and improve training efficiency. ALBERT achieves comparable results

to BERT on various NLP benchmarks while having significantly fewer parameters, with the base version containing **12 million parameters**.

- DistilBERT (7) is a smaller, faster, and more efficient version of BERT. It retains 95% of BERT's performance while reducing the model size by 40% and speeding up the training process. DistilBERT has **66 million parameters** and is available in both cased and uncased versions.
- DeBERTa (Decoding-enhanced BERT with disentangled attention) (3) is a BERT-based model that incorporates a disentangled attention mechanism to enhance the model's representation power. DeBERTa has achieved state-of-the-art results on various NLP tasks and comes with **110 million parameters**.
- ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) (1) is a transformer-based model pre-trained using a novel token replacement task. This approach allows ELECTRA to learn more efficiently and achieve state-of-the-art results with smaller model sizes. The base version of ELECTRA comes with **110 - 355 million parameters**.

5.2.1 BERT

BERT, or Bidirectional Encoder Representations from Transformers, is a cornerstone in the domain of transformer-based language models. Developed by Google AI Language, it revolutionized the field by introducing the concept of bidirectional training of transformers. BERT comes in two sizes: **BERT-base** with 110 million parameters, and **BERT-large** with 340 million parameters. Both versions are pretrained on the BookCorpus (10), and English Wikipedia datasets. During pretraining, BERT uses a Masked Language Model (MLM) objective, which allows the model to predict a masked word based on its context from both directions. This approach is an advancement over previous models that processed text unidirectionally. In addition, BERT also incorporates a Next Sentence Prediction task during pretraining, where the model learns to predict if two sentences follow each other in a document. BERT is also designed for fine-tuning, and it can be adapted to a wide range of tasks such as text classification, sentiment analysis, named entity recognition, and question-answering. The powerful pretraining approach and bidirectional context understanding enable BERT to achieve state-of-the-art results on various benchmarks.

BERT's distinct training techniques, combined with its vast scale, allowed it to achieve state-of-the-art performance on a wide variety of natural language processing tasks upon its release. Despite the introduction of newer models like RoBERTa, ALBERT, and ELECTRA, at which we will look next, BERT remains a highly influential model in the field of NLP. It continues to deliver competitive performance on many tasks, and its architecture serves as the foundation for many subsequent models.

Furthermore, BERT's pretraining-finetuning paradigm has been widely adopted in the field, with most modern LLMs following a similar approach. This, in combination with its strong performance, ensures that BERT remains at the forefront of NLP research and application.

5.2.2 RoBERTa

RoBERTa, is an optimized version of the BERT model, specifically designed to enhance the performance of pretraining tasks. It builds on the foundation of BERT by refining the training process and leveraging a larger dataset, achieving state-of-the-art results on a wide array of natural language processing benchmarks. RoBERTa comes in two sizes: **RoBERTa-base** with 125 million parameters and **RoBERTa-large** with 355 million parameters. Both versions are pretrained on the BookCorpus and English Wikipedia datasets.

During pretraining, RoBERTa employs the Masked Language Model objective from BERT, which predicts masked tokens in continuous text streams. However, RoBERTa eliminates the Next Sentence Prediction task, as it was found to negatively impact the model’s performance. Instead, RoBERTa increases the amount of training data, batch size, and learning rate to achieve better performance.

The model is trained with longer training steps and uses dynamic masking to prevent the model from learning to rely on a specific masked pattern. This results in improved downstream task performance.

For fine-tuning, RoBERTa can be adapted to various natural language processing tasks, such as text classification, sentiment analysis, named entity recognition, and question-answering. The optimized pretraining process allows RoBERTa to outperform its predecessors, achieving state-of-the-art results on numerous benchmarks. Table 5.1 shows the results on the GLUE Benchmark, scores are taken from the RoBERTa paper.

Table 5.1 Model performance comparison of BERT-large and RoBERTa on the GLUE Benchmark

Model	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST	STS	WNLI
BERT _{LARGE}	60.6	86.6	88.0	92.3	91.3	70.4	93.2	90.0	-
RoBERTa _{LARGE}	68.0	90.2	90.9	94.7	92.2	86.6	96.4	92.4	91.3

One of the key strengths of RoBERTa is its robust and optimized pretraining approach, which enables the model to effectively learn the underlying structure of natural language, leading to improved performance in various downstream tasks. The refinements implemented in RoBERTa’s training process result in a powerful and versatile model that excels in a wide array of natural language processing challenges. The availability of both base and large versions of RoBERTa ensures that researchers and practitioners can choose the model size that best fits their computational resources and task requirements.

5.2.3 ALBERT

ALBERT, is a lightweight version of the BERT model, specifically designed to address the issue of growing model size and memory requirements while retaining or even improving the performance of the original model. ALBERT achieves this by employing two primary optimization techniques: factorized embedding parameterization and cross-layer parameter sharing.

Factorized embedding parameterization separates the size of the hidden layers from the size of the input embeddings by using two separate matrix transformations. The input

embeddings, which are often large due to the size of the vocabulary, are first transformed into a smaller dimension space using a matrix. Then, this transformed embedding is further projected to the hidden dimension using a second matrix. This results in a drastic reduction in the number of parameters, especially when the vocabulary size is large and the hidden layer size is much smaller. This technique enables ALBERT to use a smaller hidden size for the embeddings while maintaining a larger hidden size for the rest of the model. And thus, this reduces the overall number of parameters in the model, leading to a more memory-efficient architecture.

Cross-layer parameter sharing involves sharing the same parameters across multiple layers of the model, as show in figure 5.3. This significantly reduces the number of unique parameters in ALBERT, resulting in a smaller and faster model compared to BERT. ALBERT comes in various sizes, with ALBERT-base having 12 million parameters and ALBERT-xxlarge having 235 million parameters.

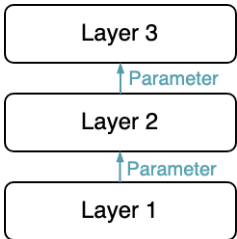


Figure 5.3 Illustration of ALBERT’s cross-layer parameter sharing mechanism. Each box represents a layer in the model, which includes a self-attention mechanism and a feed-forward network. The arrows indicate parameter sharing across layers: the same set of parameters are used in all layers. This significantly reduces the number of unique parameters in the model, resulting in a smaller and faster model compared to BERT.

During pretraining, ALBERT employs the MLM objective from BERT, predicting masked tokens in continuous text streams. However, it replaces the next sentence prediction task with a sentence-order prediction task, which predicts whether two input sentences are in the correct order. This change helps ALBERT to better model inter-sentence coherence, even surpassing BERT on several benchmarks as shown in table 5.2.

Table 5.2 Model performance comparison of BERT-large and ALBERT on the GLUE Benchmark

Model	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST	STS-B	WNLI
BERT _{LARGE}	60.6	86.6	88.0	92.3	91.3	70.4	93.2	90.0	-
ALBERT (1M)	68.7	90.4	90.2	95.2	92.0	88.1	96.8	92.7	-
ALBERT (15M)	71.4	90.8	90.9	95.3	92.2	89.2	96.9	93.0	-

For fine-tuning, ALBERT can be adapted to various natural language processing tasks, such as text classification, sentiment analysis, named entity recognition, and question-answering. The optimized architecture of ALBERT allows it to achieve competitive performance compared to its larger counterparts.

A key advantage of ALBERT lies in its capacity to offer a more resource-efficient alternative to BERT without compromising performance. The model’s design process incorporates optimization techniques that enable ALBERT to sustain high performance while remaining lightweight and computationally efficient, making it an adaptable option for a diverse set of natural language processing tasks and scenarios.

5.2.4 DistilBERT

DistilBERT, is a lightweight version of the BERT model, specifically designed for resource-constrained environments while retaining a significant portion of the original model’s performance. DistilBERT achieves this by employing knowledge distillation techniques, compressing the model to approximately half the size of BERT-base, with only 66 million parameters, and preserving over 95% of its performance. For the GLUE Benchmark it even retains 97% of the original performance of BERT on the same tasks, as shown in table 5.3, where the scores are taken from the paper "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter".

Table 5.3 Model performance comparison of BERT and DistilBERT on the GLUE Benchmark

Model	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
BERT _{BASE}	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

During the distillation process, DistilBERT is trained to mimic the behavior of the larger teacher model (BERT) by minimizing the divergence between their output distributions. This is achieved using a combination of the Masked Language Modeling objective and a distillation loss. The distillation loss measures the Kullback-Leibler divergence between the logits of the teacher and the student models, which, in simple terms, quantifies the difference in the probability distributions of the teacher and the student models. Consequently, the result is DistilBERT, a smaller and faster version of BERT that retains much of the teacher model’s performance.

DistilBERT also simplifies the architecture of BERT by removing the token-type embeddings and the pooler, as these components were found to have a minimal impact on the model’s performance. In simple terms, token-type embeddings in BERT help the model understand where a sentence begins and ends, especially when it’s processing a pair of sentences at the same time. The pooler, on the other hand, processes the output of the model’s layers to create a compressed representation of the input. This can be thought of as creating a "condensed version" of the hidden states of the model.

For DistilBERT, the token-type embeddings and the pooler were found to be less necessary. This can be seen as akin to someone reading a document - while understanding

where a sentence starts and ends (token-type embeddings) and creating a simplified, consolidated representation of the text (pooler) can be helpful, it is not strictly necessary to understand the content. By omitting these elements, DistilBERT simplifies the model without significantly impacting its ability to understand text.

Moreover, the number of layers is reduced to six, compared to 12 layers in BERT, which contributes to the model's smaller size and faster inference speed.

For fine-tuning, DistilBERT can be adapted to various natural language processing tasks, such as text classification, sentiment analysis, named entity recognition, and question-answering. Despite its smaller size, DistilBERT demonstrates competitive performance compared to its larger counterparts, making it a suitable choice for tasks where computational resources or latency are critical factors.

A significant benefit of DistilBERT is its ability to serve as a more efficient substitute for BERT with minimal performance loss. The distillation techniques applied during the model's training process allows DistilBERT to preserve high performance while being lightweight and computationally efficient, rendering it a flexible choice for various natural language processing tasks and scenarios.

5.2.5 DeBERTa

DeBERTa, stands for Decoding-enhanced BERT with disentangled attention, and it is an improved version of the BERT model, specifically designed to address the limitations of the original model's self-attention mechanism. DeBERTa introduces two key innovations: disentangled attention and an enhanced mask decoder.

Disentangled attention, as implemented in DeBERTa, refers to the separation of content and position information in the self-attention mechanism, as illustrated in Figure 5.4.

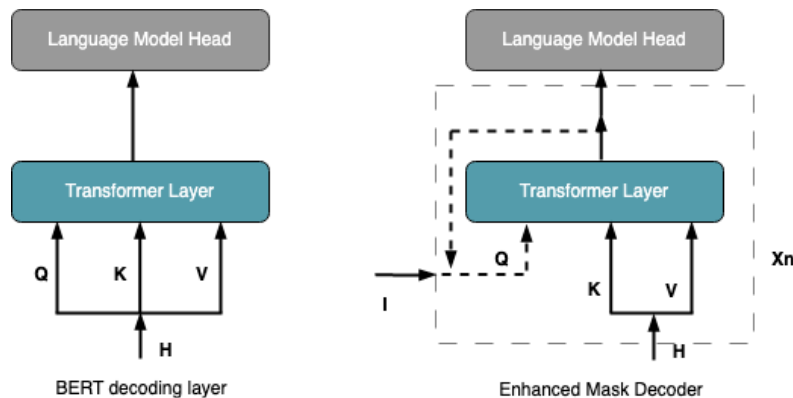


Figure 5.4 Comparison of BERT's decoder layer and the Enhanced Mask Decoder (EMD). Both start with hidden states, but EMD adds flexibility by accepting various inputs and stacking layers (denoted by X_n). Unlike BERT's single transformer layer that uses only hidden states, EMD can incorporate absolute position embedding and its own layer outputs, enabling adaptable decoding. Q , K , and V denote the query, key, and value matrices, respectively. I represents the input layer, while H signifies the hidden layer.

Unlike BERT, which does not explicitly separate these two types of information, DeBERTa independently models content and position streams and explicitly measures their interactions. This disentangled approach allows DeBERTa to capture the fine-grained interactions between words and their relative positions, which can lead to improved performance on a variety of natural language understanding tasks.

The enhanced mask decoder is an improvement over BERT’s masked language model pretraining objective. In DeBERTa, an additional denoising autoencoding loss is introduced during pretraining to minimize the reconstruction error between the original input and the reconstructed input after a certain number of masked tokens are predicted. This encourages the model to better capture the global contextual information and enhances the model’s ability to recover the masked tokens, leading to improved performance on downstream tasks. Table 5.4 illustrates the results of the performance on the GLUE downstream tasks.

Table 5.4 Model performance comparison of BERT-large and DeBERTa on the GLUE Benchmark

Model	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
BERT _{LARGE}	60.6	86.6	88.0	92.3	91.3	70.4	93.2	90.0	-
DeBERTa _{LARGE}	70.5	91.1	91.9	95.3	92.3	88.3	96.8	92.8	-

DeBERTa comes in various sizes, including DeBERTa-base with 110 million parameters and DeBERTa-large with 335 million parameters. The model is pretrained on the BookCorpus and English Wikipedia datasets, similar to BERT.

One notable strength of DeBERTa is its ability to address BERT’s self-attention limitations through disentangled attention and enhanced mask decoder techniques, making it a versatile option for various natural language processing tasks and scenarios.

5.2.6 ELECTRA

ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) is an alternative pretraining approach to the BERT model, designed to improve efficiency while maintaining high performance on natural language understanding tasks. A comparison of their performance on the GLUE benchmark is shown in table 5.5.

Table 5.5 Model performance comparison of BERT-large and ELECTRA-large on the GLUE Benchmark

Model	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
BERT _{LARGE}	60.6	86.6	88.0	92.3	91.3	70.4	93.2	90.0	-
ELECTRA _{LARGE}	69.1	90.9	90.8	95.0	92.4	88.0	96.9	92.6	-

ELECTRA introduces a novel pretraining method called the Replaced Token Detection (RTD) task, which is used to train a larger discriminator network. Unlike traditional Masked Language Modeling tasks used by models like BERT, which involves predicting a small fraction of masked tokens, the RTD task involves detecting tokens in the input sequence that have been replaced or "corrupted" by a smaller generator network. The discriminator

network, as illustrated in image 5.5, acting as a classifier, is then tasked with distinguishing between the original, unaltered tokens and the tokens replaced by the generator. This approach allows ELECTRA to make use of the full input sequence during pretraining, leading to improved efficiency.

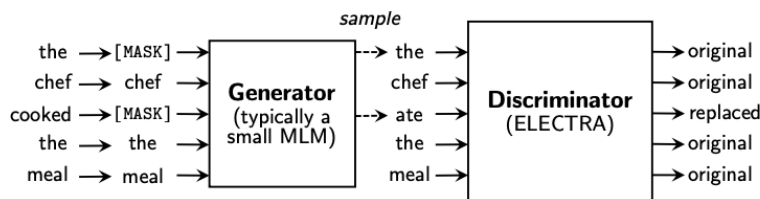


Figure 5.5 A high-level overview of replaced token detection. The generator can be any model that generates a token distribution, but the authors employed a tiny masked language model that is trained alongside the discriminator. The image is taken from the paper: "ELECTRA: Pre-Training Text Encoders As Discriminators Rather Than Generators"

ELECTRA comes in various sizes, including ELECTRA-base with 110 million parameters and ELECTRA-large with 335 million parameters. The model is pretrained on the BooksCorpus and English Wikipedia datasets, similar to BERT.

For fine-tuning, ELECTRA can be adapted to various natural language processing tasks, such as text classification, sentiment analysis, named entity recognition, and question-answering. The novel pretraining method employed by ELECTRA allows the model to achieve competitive performance compared to its larger counterparts.

A major strength of ELECTRA lies in its efficient learning from input sequences through replaced token detection, making it suitable for a wide range of NLP tasks and scenarios where computational resources or latency are crucial factors.

NOTE For a more comprehensive comparison of various models and their performance on natural language understanding tasks, be sure to check out the official GLUE (<https://gluebenchmark.com/leaderboard/>) and SuperGLUE (<https://super.gluebenchmark.com/leaderboard>) websites. These sites feature regularly up-dated leaderboards showcasing the latest and most advanced models in the field of NLP.

Having gained an understanding of each model's unique strengths and innovations, you're now ready to get your hands "dirty" and apply the models for your own tasks. In listing 5.3 you find an excerpt of an easy to use `TextClassifier` class. With it, you can try out all the models we covered in this section. You can also just replace the models in the dictionary, if you wish to try out different models. We will also employ this `TextClassifier` class throughout the following sections, which allows us to easily switch between models for evaluation and produce performance plots of those models.

I encourage you to explore the full implementation. You can find the `TextClassifier` class in the book repository, located in `utils.py`.

Listing 5.3 Evaluate different Transformer models

```
❶ We use a dictionary with the models we want to evaluate
models = {
    "BERT": "bert-base-uncased",
    "RoBERTa": "roberta-base",
    "DistilBERT": "distilbert-base-uncased",
    "DeBERTa": "microsoft/deberta-base",
    "ALBERT": "albert-base-v2",
    "ELECTRA": "google/electra-base-discriminator",
}

❷ Set the target names for our confusion matrix
financial_target_names = ['Negative', 'Neutral', 'Positive']

❸ Create a dictionary with the parameter
news_classifier_params = {
    'test_set': test_df,
    'models': models,
    'target_names': financial_target_names,
    'num_examples': 100,
    'seed_value': 0
}

❹ Which we then can easily pass into the class
financial_classifier = TextClassifier(**news_classifier_params)

❺ Call the evaluate function to plot the confusion matrix and generate a data frame
evaluation_df_financial_phrasebank_before =
    financial_classifier.evaluate_models(num_columns=3, figsize=(15, 10))
```

You may have noticed that I used the term *confusion matrix* in the code. If you were not familiar with this term, don't worry - we're going to cover it in the next section, along with other important metrics such as accuracy and F1 score. This will equip you with the necessary tools to evaluate your classification models using the metrics most appropriate for your specific problem at hand.

5.3 Evaluating classification performance

Before we explore the exciting world of hands-on examples, we'll take a brief detour to discuss some essential theory - how to evaluate our trained models effectively. In this section, I will introduce common metrics for assessing the performance of classification tasks, including the confusion matrix, accuracy, and the F1-score. With this knowledge in hand, you'll be better prepared to understand and assess the effectiveness of your classification models as we move forward with the practical examples.

5.3.1 Confusion matrix

One way to evaluate the performance of a classification model, be it binary or multi-class, is by using a confusion matrix. Although the explanation here will use a binary classification as an example, the concept can be extended to multi-class classification problems as well.

However, don't let the term "confusion matrix" confuse you! It's simply a table that helps you visualize and evaluate the performance of a classification model by showing the number of correct and incorrect predictions for each class. This understanding will enable you to assess the effectiveness of your text classification models more accurately and confidently. A confusion matrix typically consists of four elements: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).

- True Positives (TP): The number of instances where the model correctly predicted the positive class.
- False Positives (FP): The number of instances where the model incorrectly predicted the positive class.
- True Negatives (TN): The number of instances where the model correctly predicted the negative class.
- False Negatives (FN): The number of instances where the model incorrectly predicted the negative class.

A visual representation of these four components and their interrelations within a confusion matrix can be found in Figure 5.6.

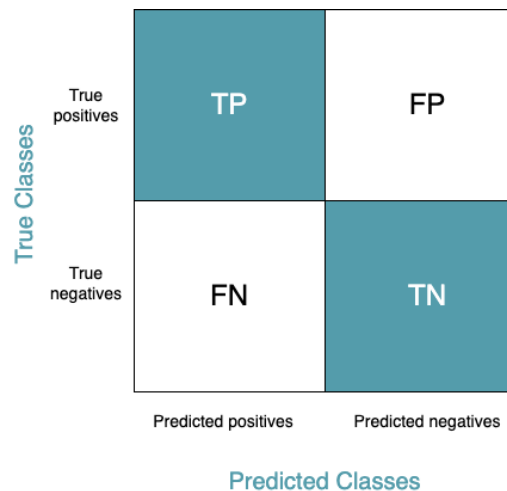


Figure 5.6 An illustration of a confusion matrix for a binary classification problem. It details the distribution of true and predicted class labels, offering insights into model performance.

To produce this plot for our models, we can easily use the build-in function from scikit-learn. An example implementation is shown in listing 5.4.

Listing 5.4 Confusion matrix plot function

```
❶ Import function from scikit-learn
from sklearn.metrics import confusion_matrix
```

```
def plot_confusion_matrix(y_true, y_pred, class_names):
    cm = confusion_matrix(y_true, y_pred)
    cmap = colors.LinearSegmentedColormap.from_list("", ["white", "#58A3B3"])
    fig, ax = plt.subplots()
    sns.heatmap(cm, annot=True, fmt='d', ax=ax, cmap=cmap, cbar=False,
                linewidths=0.2, linecolor='black')
    ax.set(xlabel="Predicted Classes", ylabel="True Classes", xticklabels=class_names,
          yticklabels=class_names)
    plt.yticks(rotation=0)
```

To try in out, we can just create two lists with example values as shown in listing 5.5.

Listing 5.5 Creating a confusion matrix plot

```
❶ Create the class labels Positive = 0, Negative = 1
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0] ❷ True labels
y_pred = [0, 0, 1, 1, 0, 0, 1, 0, 1, 0] ❸ Predicted labels
class_names = ['Positive', 'Negative']

❹ Call the plotting function
plot_confusion_matrix(y_true, y_pred, class_names)
```

This results in the plot shown in figure 5.7 for our two classes and true and predicted labels.

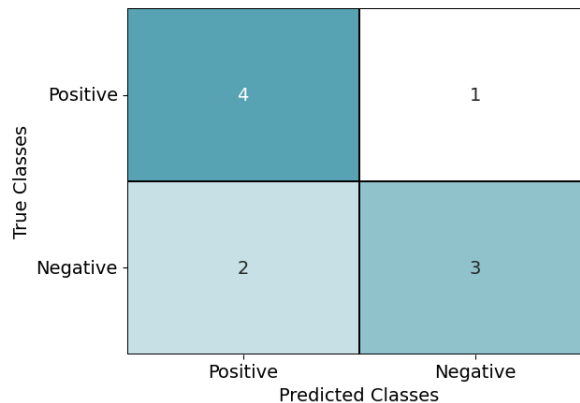


Figure 5.7 Confusion matrix plot form our toy example data.

So, there you have it! The confusion matrix demystified. Just remember, it's merely a tool to help you better understand and evaluate your classification models, so don't let it be a source of confusion.

Moreover, using these four elements, you can derive various performance metrics such as sensitivity (recall or true positive rate), specificity (true negative rate), precision, and more.

- Sensitivity (recall or true positive rate): $\frac{TP}{TP+FN}$
- Specificity (true negative rate): $\frac{TN}{TN+FP}$
- False positive rate: $\frac{FP}{FP+TN}$
- False negative rate: $\frac{FN}{FN+TP}$
- Precision: $\frac{TP}{TP+FP}$

These metrics help you assess the strengths and weaknesses of your classification model, enabling you to optimize it for different applications. Let us conveniently use again scikit-learn (listing 5.6) and the same two lists from our confusion matrix example and calculate those metrics for the two classes and true and predicted values.

Listing 5.6 Computing sensitivity, specificity and precision

```

❶ Create the class labels Positive = 0, Negative = 1
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0] ❷ True labels
y_pred = [0, 0, 1, 1, 0, 0, 1, 0, 1, 0] ❸ Predicted labels

❹ Calculating the True Positives (TP), False Positives (FP)
❺ and True Negatives (TN) and False Negatives (FN)
tn, fp, fn, tp = cm.ravel() ❻ Return a contiguous flattened, ID array

precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
specificity = tn / (tn+fp)

```

This will result in the following outputs for our our elements; true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN):

- TP: 3
- FP: 1
- TN: 4
- FN: 2

And these four elements will lead to the following performance metrics:

- Precision: 0.75
- Recall (Sensitivity): 0.6
- Specificity: 0.8

After gaining a deeper understanding of the confusion matrix and their elements, we can now turn our attention to other important performance metrics, such as the F1 score and accuracy. These metrics will help you further evaluate the performance of your text classification models and optimize them for various applications.

5.3.2 Accuracy

Accuracy is a widely used metric for evaluating the performance of classification tasks, including text classification. In simple terms, accuracy measures the proportion of correct predictions made by a classifier relative to the total number of predictions.

Mathematically, accuracy can be expressed as follows:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.2)$$

As a reminder, the elements of a confusion matrix include True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Accuracy calculates the proportion of correct predictions by considering both TP and TN in relation to the total number of predictions.

Accuracy is intuitive and straightforward to interpret, as it provides a single value that captures the overall effectiveness of a classifier. However, it has some limitations, especially when dealing with imbalanced datasets. In imbalanced datasets, where the distribution of classes is uneven, a classifier might achieve high accuracy by simply predicting the majority class, which may not be the desired outcome. In such cases, alternative metrics like precision, recall, and, as we will discuss next, the F1-score can provide more nuanced insights into the model's performance.

Nevertheless, accuracy remains a valuable metric for getting a quick overview of a text classification model's performance, especially when the class distribution is balanced or near-balanced. In the following section, we will explore the F1-score, a metric that can better handle the challenges presented by imbalanced datasets.

5.3.3 F1-score

The F1-score is a metric that combines both precision and recall, which we previously discussed while exploring the confusion matrix, to provide a single value representing the trade-off between these two performance measures. As a gentle refresher, precision measures the proportion of true positive predictions among all positive predictions, while recall, also known as sensitivity or true positive rate, measures the proportion of true positive predictions among all actual positive instances. Further, the F1-score is a metric that combines both precision and recall in a specific way, known as the harmonic mean, as show in equation 5.3

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.3)$$

With this specific combination of precision and recall, we can easily see how it is different from a simple average because it tends to penalize extreme values. In other words, to achieve a high F1-score, both precision and recall must be high.

The F1-score ranges from 0 to 1, where a score of 1 indicates perfect precision and recall, and a score of 0 indicates the complete absence of either precision or recall. The F1-score is especially useful when dealing with imbalanced datasets, as it accounts for both

false positives and false negatives, thus providing a more balanced evaluation of the model's performance.

In contrast to accuracy, which may be misleading in the presence of imbalanced datasets, the F1-score is more robust. Although it primarily focuses on the model's ability to correctly predict the positive class, by considering both precision and recall, it does provide a balanced evaluation of the model's performance, especially in the face of class imbalance.

5.4 *Applications and worked examples*

Now that we've covered the essential theory, let's solidify our understanding by exploring hands-on examples and applications. In line with the previous chapters, these examples are designed to run on a single GPU, showcasing the power and versatility of transformer models for text classification tasks across a variety of real-world scenarios, even for those with limited computational resources.

In this chapter, we will explore three diverse datasets, each offering unique challenges and opportunities for text classification. We begin with the Financial Phrasebank dataset for sentiment classification, followed by the AG_News dataset for topic modeling. Lastly, we tackle the Yelp review dataset, a more complex task that requires distinguishing between five different classes while accounting for the nuanced language present in the reviews.

5.4.1 *Fine-tuning different classification models on the Financial Phrasebank dataset*

In this section, we will begin with the Financial Phrasebank dataset and evaluate various models before using these results as a basis for fine-tuning our models on this specific task.

The Financial Phrasebank dataset (6) is a collection of financial news sentences, categorized into three sentiment classes: positive, neutral, and negative. This dataset focuses on sentiment classification within the financial domain. It consists of approximately 4,800 sentences collected from financial news articles, with 2,226 instances labeled as neutral, 1,787 as positive, and 827 as negative.

By looking at the data distribution, we can clearly see that our data is imbalanced. Dealing with imbalanced datasets is a frequent challenge in real-world applications, and knowing how to tackle this issue can greatly enhance the performance of your text classification models. One widely-used technique is undersampling, which involves reducing the number of instances from the more prevalent class in the dataset. Figure 5.8 illustrates an example of an imbalanced dataset, while figure 5.9 demonstrates the result after applying undersampling for our Financial Phrasebank dataset.

Further, if you have an imbalanced dataset, you can consider these techniques:

- **Transfer learning:** Utilize pre-trained models on a related task with a more balanced class distribution, and then fine-tune the model for the imbalanced target task. This method allows the model to benefit from the knowledge gained from the related task and can improve its performance on the target task.
- **Oversample minority classes:** Enhance the minority classes by duplicating instances or using techniques like SMOTE, which generates synthetic examples, resulting in a more balanced and representative dataset for the model to learn from. This approach

balances the class distribution and helps the model to learn from a more representative dataset.

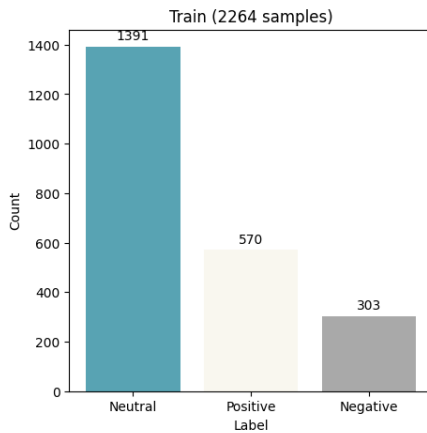


Figure 5.8 The Financial Phrasebank dataset for news sentiment classification, displaying a higher number of instances for the neutral class in comparison to the other two classes, positive and negative.

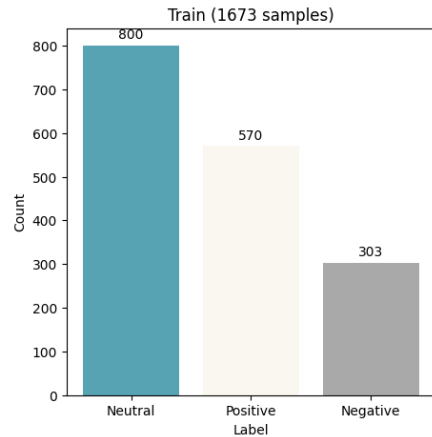


Figure 5.9 The financial phrasebank dataset after applying undersampling.

However, not every dataset is as simple as the Financial Phrasebank dataset and sometimes it is helpful to gain a better understanding of your class distributions by visualizing the data.

VISUALIZING TRAINING DATA TO GAIN MORE INSIGHTS

If you encounter more complex classification problems, which often involve numerous classes and high-dimensional spaces. You need a way to navigate these complexities and gain a better understanding of the data. For that, we can visualize the data distribution using a dimensionality reduction technique like t-SNE (t-Distributed Stochastic Neighbor Embedding), which helps us to reveal patterns and structures within the data that are otherwise hidden in the high-dimensional space. Figure 5.10 displays an example of the Financial Phrasebank dataset transformed into a 3D plot using t-SNE.

If you encounter more complex data, the following strategies can help address these complexities:

- **Model selection:** Choose a larger model architecture that is known to perform well on complex tasks. Larger transformer models have more capacity to learn complex patterns and relationships in high-dimensional spaces, where classes might not be easily distinguishable. By having more layers and parameters, these models can capture subtle nuances in the text and better distinguish between classes, even when the language is ambiguous. The benefit here lies in leveraging the proven capabilities of existing larger models.

- **Model scaling:** Increase the size of the model by adding more layers, widening the layers, or both. Newly added layers are typically configured identically to the existing ones, but different configurations may be worth exploring as part of hyperparameter tuning. This approach allows you to tailor the model size based on your specific task's needs and data availability. Be cautious about the risk of overfitting and ensure that you have sufficient data to train the larger model effectively.

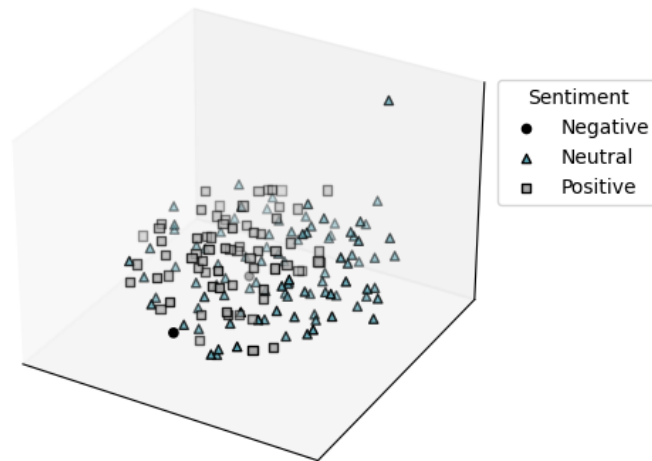


Figure 5.10 3D plot of the data distribution of the Financial Phrasebank dataset, visualized using a dimensionality reduction technique called t-SNE. The goal of t-SNE is to take a set of points in a high-dimensional space and find a close representation of those points in a lower-dimensional space, typically a 2D plane or a 3D space, providing an insightful look into the structure and patterns inherent in the data.

NOTE The t-SNE algorithm is a nonlinear dimensionality reduction technique that is particularly well-suited for embedding high-dimensional data into a space of two or three dimensions. It maintains local similarities between data points, enabling us to visualize complex relationships in lower-dimensional spaces. Another technique known as UMAP (Uniform Manifold Approximation and Projection) has also gained popularity for similar purposes. While t-SNE excels in preserving local structures, UMAP is known for its speed and preservation of the global data structure. Both can be useful depending on the specifics of your dataset and task. For your reference, you can find the code for the function `plot_tsne_3d`, which applies the t-SNE technique, in `plotting.py`. This can be used as a template for visualizing other classification datasets using t-SNE.

Using the undersampled Financial Phrasebank dataset we prepared earlier, we are now ready to put our TextClassifier class (listing 5.3) to work. When we apply this class to the dataset, the output will be six distinct confusion matrices, one for each model we have chosen to evaluate. These confusion matrices serve as an excellent tool for visualizing and understanding the performance of each model against our specific dataset. They clearly

display the number of correct and incorrect predictions for each class, enabling us to determine the strengths and weaknesses of each model. Refer to figure 5.11 for a visual comparison of these six confusion matrices. As we progress, you'll see how these visual representations aid us in fine-tuning our models and optimizing them for the tasks at hand.

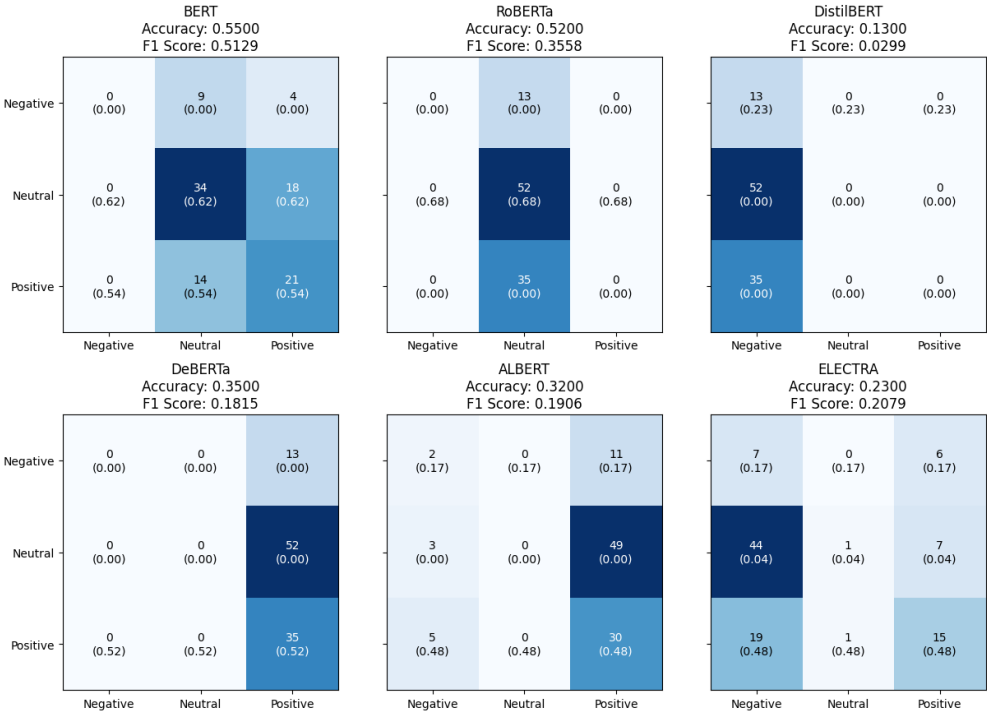


Figure 5.11 Overview of the performance of different Transformer models before fine-tuning, using weighted F1 scores.

We observe that BERT and RoBERTa demonstrate the best performance among the six models evaluated. Furthermore, the performance of these models varies across negative, neutral, and positive predictions. However, we will fine-tune RoBERTa and ELECTRA, and compare them to our Naïve Bayes classifier, emphasizing the importance of evaluating different models even when their initial performance might suggest otherwise.

NOTE We also utilize our custom *TextClassifier* class to generate data frames, enabling us to conveniently review each model’s performance and assess its overall effectiveness. This streamlined approach allows for a more efficient comparison of different models, leading to a better understanding of their strengths and weaknesses. Furthermore, it gives us the ability to visualize the performance metrics, thereby making the differences among models more tangible. Ultimately, this facilitates informed decision-making when selecting the best model for a specific task.

Before fine-tuning our models, we need to prepare the Financial Phrasebank dataset for use with our Transformer models. As we have the dataset in a dataframe format, the access method will differ, as demonstrated in Listing 5.7, compared to directly using the Hugging Face dataset.

Listing 5.7 Create dataset for fine-tuning

```
class FinancialPhrasebankDataset(Dataset):
    def __init__(self, data, tokenizer, max_length):
        self.data = data
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):
        ❶ We use the iloc and index to get the text and labels
        text = self.data.iloc[index]['sentence']
        label = self.data.iloc[index]['label']
        ❷ Tokenize the text, add special tokens, pad or truncate the sequence to the specified max_length
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_length,
            padding='max_length',
            truncation=True,
            ❸ Return the attention mask and input tensors
            return_attention_mask=True,
            return_tensors='pt'
        )
        ❹ Flatten the input_ids and attention_mask tensors and return them along with the label
        return
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
```

We proceed by loading our model and the corresponding tokenizer for the model, defining the maximum sequence length and batch size, and setting the number of classes for the specific classification task in the model, as demonstrated in listing 5.8. This process ensures that our data is properly prepared and formatted for the specific model we are using, taking into account the unique requirements of the classification task at hand.

Listing 5.8 Prepare model, tokenizer and dataset

```
❶ Load the RoBERTa tokenizer
tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
```

② **Define the RoBERTa model for sequence classification with the number of labels**

```
model = RobertaForSequenceClassification.from_pretrained(  
    'roberta-base', num_labels=3)
```

③ **Define the maximum sequence length and batch size**

```
max_length = 128  
batch_size = 16
```

④ **Create the training, validation, and test datasets**

```
train_dataset = FinancialPhrasebankDataset(train_df, tokenizer, max_length)  
val_dataset = FinancialPhrasebankDataset(val_df, tokenizer, max_length)  
test_dataset = FinancialPhrasebankDataset(test_df, tokenizer, max_length)
```

As a next step, we define the *EarlyStoppingCallback*, to avoid overfitting during the fine-tuning process as shown in listing 5.9.

Listing 5.9 Setting up early stopping

```
early_stopping_callback = EarlyStoppingCallback(  
    ① Stop training if the metric doesn't improve for 10 epochs  
    early_stopping_patience=10,  
    ② Minimum change in the monitored metric to be considered as an improvement  
    early_stopping_threshold=2e-5)
```

Throughout this section, we will use accuracy as a metric during the fine-tuning process, as shown in listing 5.10. In practice, it is beneficial to employ custom metrics tailored to the specific needs of the project. For instance, when dealing with imbalanced datasets, the F1-score should be considered as an alternative metric. By adopting such metrics, you can better assess your model's performance and optimize it for your particular application.

Listing 5.10 Custom evaluation metric

```
accuracy_metric = evaluate.load("accuracy")  
  
def compute_metrics(eval_pred):  
    logits, labels = eval_pred  
    predictions = np.argmax(logits, axis=-1)  
    return accuracy_metric.compute(predictions=predictions, references=labels)
```

With all the essential prerequisites established for training our chosen models on the downstream task, we can now proceed to initialize the training arguments and the trainer class. This process, which is demonstrated in Listing 5.11, ensures that our models are well-prepared to tackle the specific classification problem effectively and efficiently, taking full advantage of the Transformer architecture's capabilities.

Listing 5.11 Fine-tuning RoBERTa

① **Define training arguments**

```
training_args = TrainingArguments(  
    output_dir="./results",
```

```
overwrite_output_dir=True,
num_train_epochs=5,
per_device_train_batch_size=batch_size,
save_strategy="epoch",
save_total_limit=2,
evaluation_strategy="epoch",
learning_rate=3e-5,
weight_decay=0.01,
logging_strategy="epoch",
logging_steps=10,
load_best_model_at_end=True)
```

2 Initialize the trainer with the compute_metrics function

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
    callbacks=[early_stopping_callback] )
```

Once the fine-tuning is complete, we can compare the results from the different models and our baseline, as illustrated in Figure 5.12.

	Accuracy	F1 Score	Negative F1	Neutral F1	Positive F1
Model					
BERT	0.550000	0.512865	0.000000	0.623853	0.538462
RoBERTa	0.520000	0.355789	0.000000	0.684211	0.000000
DistilBERT	0.130000	0.029912	0.230088	0.000000	0.000000
DeBERTa	0.350000	0.181481	0.000000	0.000000	0.518519
ALBERT	0.320000	0.190609	0.173913	0.000000	0.480000
ELECTRA	0.230000	0.207854	0.168675	0.037037	0.476190
Fine-tuned RoBERTa	0.982143	0.982207	0.983051	0.988235	0.971963
Fine-tuned ELECTRA	0.988095	0.988095	0.965517	1.000000	0.981132
Naive Bayes	0.720238	0.671115	0.129032	0.863388	0.655738

Figure 5.12 Comparison of the weighted results of all evaluated models before fine-tuning, as well as our fine-tuned models, RoBERTa and ELECTRA, and our baseline. 'Negative F1', 'Neutral F1', and 'Positive F1' refer to the F1 scores achieved on the negative, neutral, and positive sentiment classes, respectively. We can clearly see the improvements of ELECTRA after fine-tuning, yielding now an impressive F1-Score of 0.988.

As emphasized earlier, evaluating various models for a specific task is crucial. Although ELECTRA was not one of the top-performing models in our confusion plot shown in

figure 5.11, it performed the best after being fine-tuned for this particular classification task. Furthermore, our baseline model exhibited surprisingly strong results, with an accuracy of 0.72 and an F1-Score of 0.67.

NOTE The process for obtaining the ELECTRA model and its tokenizer is identical to the RoBERTa example. To determine the appropriate name for the *from_pretrained* function, simply refer to the corresponding model card on the Hugging Face website, which will provide the necessary information for each specific model. This ensures that you can easily and efficiently access the desired model and tokenizer without encountering any compatibility issues.

5.4.2 Fine-tuning a classification model on the AG_News Dataset

In this section, we delve into a slightly more intricate task, shifting our focus from sentiment classification to topic modeling. While sentiment classification primarily involves discerning the positive, negative, or neutral sentiments conveyed in a piece of text, topic modeling is a more complex undertaking. Here, we are tasked with classifying a text based on the specific topic or subject it relates to.

The dataset we'll be using for this task is the AG_News dataset (9), a comprehensive collection of news articles from the AG's corpus. The news articles in this dataset are categorized into four distinct topics: World, Sports, Business, and Science/Technology. It is a balanced dataset, with 30,000 articles in the training set and 1,900 articles in the testing set for each category, totalling to 120,000 training samples and 7,600 testing samples. This extensive and well-organized dataset makes it an excellent resource for topic modeling tasks.

Similar to our previous task, we start by evaluating various models using our custom *TextClassifier* class. This step allows us to understand the performance of each model and decide which one would be the most suitable for our current task. Based on our evaluation, we select the BERT model for fine-tuning on the AG_News Dataset.

To streamline our discussion, I will not repeat the steps involved in creating the dataset, initializing the model, and fine-tuning it, as these are essentially the same as those we undertook for the Financial Phrasebank dataset. Nevertheless, I strongly encourage you to review these steps in the notebook `ch05_text_classification_ag_news`. This notebook, which includes all the relevant steps and code, can be found in the book repository under the **CH05** folder.

With this process in place, we're able to optimize our selected model for the topic modeling task. The results of this fine-tuning process are presented in figure 5.13, where we compare the performance of our models both before and after fine-tuning BERT on the topic modeling task. Through this comparison, we can appreciate the improvements we achieved by fine-tuning BERT, underlining its value in enhancing model performance for specific tasks.

NOTE Moreover, the importance of efficient topic modeling extends far beyond the realm of news categorization. In a world overwhelmed by information, it is a vital tool

for sorting, managing, and understanding vast amounts of text data. From classifying customer reviews for businesses to analyzing social media trends or understanding scholarly literature, topic modeling provides valuable insights. The fine-tuned model can help in rapidly processing and categorizing new documents, thereby making information retrieval more efficient and aiding in the comprehension of large textual datasets.

	Accuracy	F1 Score	World F1	Sports F1	Business F1	Sci/Tech F1
Model						
BERT	0.160	0.114231	0.000000	0.076923	0.215686	0.195122
RoBERTa	0.370	0.199854	0.000000	0.000000	0.000000	0.540146
DistilBERT	0.330	0.203815	0.464000	0.307692	0.000000	0.000000
DeBERTa	0.300	0.138462	0.461538	0.000000	0.000000	0.000000
ALBERT	0.200	0.070588	0.000000	0.336134	0.000000	0.000000
ELECTRA	0.280	0.174292	0.446429	0.000000	0.000000	0.109091
Fine-tuned BERT	0.918	0.917863	0.933884	0.973384	0.879346	0.882236
Naive Bayes	0.887	0.886835	0.895277	0.965649	0.836653	0.845996

Figure 5.13 This table compares the results of all evaluated models before fine-tuning, our fine-tuned model, BERT, and our baseline. The F1 scores for each class ('World', 'Sports', 'Business', 'Sci/Tech') show how well each model performed on different categories of the dataset. It's noteworthy to observe the substantial improvement in the overall F1-Score of BERT, from 0.114 to 0.918, achieved through fine-tuning.

Our base model performed impressively, yet further performance enhancement for our transformer model is achievable through hyperparameter optimization, a topic I'll cover in Chapter 9. Particularly in use cases like stock price prediction using sentiment classification, even a 2-3% improvement can significantly impact outcomes.

5.4.3 Fine-tuning a classification model on the Yelp Dataset

In this section, we will fine-tune the base and large variants of RoBERTa on the Yelp Dataset. The Yelp Review dataset is a large-scale dataset containing customer reviews from the Yelp platform, categorized into five different star ratings (1 to 5 stars). It is designed to challenge models with the task of predicting the rating based on the nuances in the language used in the review. The dataset contains over 8 million reviews, with each review having a variable length, reflecting the diversity of the review content. This dataset presents a more complex task, requiring models to distinguish between the five classes while accounting for the subtle differences in language used by customers to express their opinions.

This task is more complex due to having five classes and nuanced language. To deal with such complex classification tasks, it's helpful to employ class weighting. Class weighting

assigns higher weights to the minority class during fine-tuning by using class weights in the loss function.

This strategy encourages the model to pay more attention to the minority class examples and can enhance its performance on imbalanced datasets. However, class weighting can also be used to address issues arising from classes that are more difficult to predict due to the nuances in the language, as it is the case with the Yelp Review dataset. By assigning higher weights to these classes, the model is encouraged to pay closer attention to them during training, improving its ability to make accurate predictions. We can achieve this by customizing the Hugging Face trainer class, as shown in Listing 5.12.

Listing 5.12 Class weighting example

```
class CustomTrainer(Trainer):  
    ❶ Rest of code omitted for brevity  
    def compute_loss(self, model, inputs, return_outputs:  
        labels = inputs.pop("labels")  
        outputs = model(**inputs)  
        logits = outputs.logits  
  
        ❷ Use the CrossEntropyLoss with class weights  
        loss_fct = CrossEntropyLoss(weight=class_weights)  
        loss = loss_fct(logits.view(-1,  
            self.model.config.num_labels), labels.view(-1))  
  
        return (loss, outputs) if return_outputs else loss  
  
    ❸ Assign custom weights  
    class_weights = torch.tensor(  
        [1.0, 1.55, 1.55, 1.55, 1.05]).to(training_args.device)
```

NOTE As a simple guideline, examining the classification report or confusion matrix can help identify classes with many false positives, indicating that class weighting may be beneficial for those classes.

In addition we incorporate the ReduceLROnPlateau learning rate scheduler in our custom trainer class. The excerpt provided in listing 5.13 demonstrates how the scheduler is initialized within the custom trainer class. For a complete understanding of the implementation, please refer to the provided code in the notebook in the book repository.

Listing 5.13 Using a learning rate scheduler

```
class CustomTrainer(Trainer):  
    ❶ Other functions are omitted for brevity  
  
    ❸ Use the ReduceLROnPlateau scheduler  
    def training_epoch_end(self, outputs):  
        if self.lr_scheduler.__class__.__name__ == "ReduceLROnPlateau":  
            eval_metrics = self.evaluate()  
            metric_to_check = eval_metrics["eval_accuracy"]
```

```

        if metric_to_check > self.best_metric: ❶ Check if the metric improved
            self.best_metric = metric_to_check

        ❷ Update the learning rate scheduler with the evaluation metric
        self.lr_scheduler.step(metric_to_check)
    else:
        super().training_epoch_end(outputs)

```

Don't worry if you're not yet familiar with learning rate schedulers; we will cover them in detail in chapter 9, where we will discuss hyperparameter optimization, learning rate schedulers, gradient descent, and other techniques to optimize your transformer model for your chosen downstream task. For now, let's look at a brief, illustrative example to explain these concepts.

Picture the Swiss Alps as the hyperparameter space, where each point represents a particular combination of hyperparameters for a transformer model. The goal is to find the best combination of hyperparameters that results in the lowest error or loss for the model. To better visualize this concept, I've prepared a graphical representation of such a hyperparameter space, shown in figure 5.14.

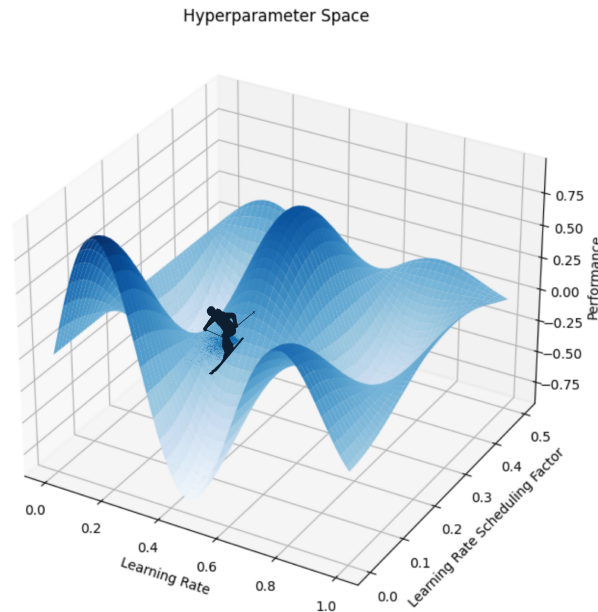


Figure 5.14 An imaginative illustration of gradient descent and learning rate scheduling in the context of a skiing adventure. The skier's path represents the optimization process, with the learning rate akin to the skier's controlled speed, adjusting to the gradient terrain to seek the lowest possible point. While the exact global minimum might not be reachable, the objective is to find a 'good' local minimum, which yields a performance close to the optimal one.

Just like the vast and varied terrain of the Swiss Alps, each point on this plot represents a distinct combination of hyperparameters for a transformer model. The goal is to find the optimal point, yielding the lowest error.

Relating to our imaginative illustration, think of gradient descent as a heli-skiing adventure. You're dropped off at a random peak in the Swiss Alps, and your objective is to ski down the mountain to the lowest point in the valley, representing the optimal combination of hyperparameters with the lowest loss. You don't have a map, but you can use the slope of the terrain to guide your descent.

Now, imagine learning rate scheduling as your skiing speed control technique. While skiing down the mountain, you need to maintain an optimal speed to ensure you have enough momentum to overcome flat areas or small valleys (local minima) but not so fast that you lose control and miss the global minimum. Adjusting your skiing speed corresponds to changing the learning rate during the optimization process, allowing the model to make larger or smaller updates to the gradients and therefore the learned weights depending on the situation. By controlling your speed, you can prevent getting stuck in suboptimal areas or having to take off your skis and walk. This technique helps you efficiently find the best slope leading to the global minimum, improving the optimization process.

With this understanding of learning rate scheduling, let's return to our fine-tuning results. To maintain brevity, I do not present the steps for creating the dataset, initializing the model, and fine-tuning, as they follow a similar process as the previous datasets. However, I encourage you to explore these steps in the corresponding notebook within the book repository. The comparison of our models before we introduced the weights into the training process as well as after fine-tuning the RoBERTa base and large variants with weighting on the Yelp classification task can be seen in the figure 5.15.

	Accuracy	F1 Score	1-Star F1	2-Star F1	3-Star F1	4-Star F1	5-Star F1
Model							
BERT	0.270	0.151779	0.424779	0.060606	0.000000	0.148148	0.000000
RoBERTa	0.240	0.092903	0.387097	0.000000	0.000000	0.000000	0.000000
DistilBERT	0.140	0.050724	0.000000	0.068966	0.000000	0.000000	0.234234
DeBERTa	0.140	0.034386	0.000000	0.000000	0.000000	0.000000	0.245614
ALBERT	0.230	0.101684	0.000000	0.000000	0.105263	0.382609	0.000000
ELECTRA	0.210	0.091636	0.381818	0.000000	0.000000	0.000000	0.000000
Fine-tuned RoBERTa no weights	0.570	0.574591	0.583333	0.518519	0.586066	0.478149	0.691293
Fine-tuned RoBERTa large	0.662	0.664448	0.781491	0.621359	0.606516	0.544974	0.748815
Fine-tuned RoBERTa base	0.605	0.610282	0.713881	0.583691	0.550000	0.474667	0.709360
Naive Bayes	0.485	0.484025	0.666667	0.457983	0.392947	0.431472	0.460526

Figure 5.15 This figure compares the performance of all evaluated models before fine-tuning, our larger model, and the model with and without class weights applied during the fine-tuning process. Notably, RoBERTa large outperforms the other fine-tuned models, suggesting its superior ability to handle the dataset's complexity compared to RoBERTa base.

To illustrate the step-by-step process of iterating over a problem and evaluating model improvements, we will examine the confusion matrices, as shown in 5.16, before and after incorporating weights into the model’s fine-tuning process, as well as after opting for a larger model for the task.

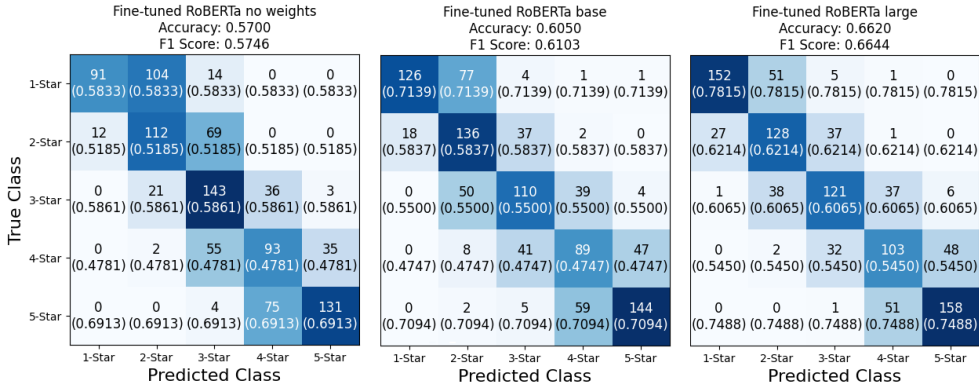


Figure 5.16 (Left) As we can see in the confusion matrix, the unweighted RoBERTa model struggles particularly with the 2-Star and 4-Star classes, misclassifying many instances as 3-Stars and 5-Stars, respectively. The diagonal entries in a confusion matrix represent true positives (correct predictions), while off-diagonal entries represent misclassifications. In this case, for the 2-Star class, we have 112 true positives, and for the 4-Star class, we have 93 true positives. (Center) With the addition of class weights, the model shows improved performance, particularly for the 2-Star class. (Right) The larger RoBERTa model shows further improvement, yielding better F1-scores for all classes, indicating its superior suitability for this complex task.

Dive Deeper: Fine-Tuning RoBERTa on the Yelp Dataset

To delve deeper into the fine-tuning processes of the different RoBERTa variants highlighted in this section, I invite you to explore the corresponding Jupyter Notebooks in the **CH05** folder of the book’s repository. Each notebook is uniquely named for ease of identification:

- `ch05_text_classification_yelp_roberta_base_no_weights.ipynb`
- `ch05_text_classification_yelp_base.ipynb`
- `ch05_text_classification_yelp_roberta_large.ipynb`

I encourage you to experiment with these notebooks — adjust the class weights or the number of epochs for training the model, for instance, and observe how these changes influence the performance of the classification. Engaging in this hands-on way will offer you a deeper understanding of how seemingly minor adjustments can significantly impact the classification performance on different classes, especially when dealing with complex classification problems.

After examining these confusion matrices closely, we can effectively assess the impact of using different approaches and opting for larger models for complex tasks. The insights gained from comparing the matrices highlight the importance of exploring various strategies, such as incorporating weights and utilizing more substantial models, to tackle difficult problems and achieve better performance. However, it's worth noting that further improvements can be achieved by optimizing hyperparameters, increasing the number of training epochs, and adding more training data. It's essential to monitor the performance of each experiment carefully to make informed decisions and achieve the best possible results.

5.5 Summary

- Text classification plays a prominent role in various real-world applications, including sentiment analysis, topic modeling, emotion recognition, and market research. Due to its extensive range of use cases and practical significance, it is beneficial for anyone working with textual data to be versatile in text classification techniques.
- The GLUE Benchmark is a collection of nine text NLP tasks that focus on sentence-level or sentence-pair analysis.
- Having a benchmark like the GLUE Benchmark is valuable as it provides a starting point for choosing an appropriate model for various text classification tasks. The GLUE Benchmark consists of diverse datasets, testing a range of linguistic skills, and allows for thorough evaluation of model performance. Additionally, there is an extended version called SuperGLUE, which builds upon GLUE by introducing more challenging tasks and evaluation metrics, pushing the boundaries of natural language understanding models.
- In various text classification benchmarks, state-of-the-art models such as BERT, RoBERTa, ALBERT, DistilBERT, DeBERTa, and ELECTRA have demonstrated exceptional performance. These models have unique features and improvements that make them highly effective for text classification tasks. These models, benchmarked on the GLUE dataset provide valuable insights into their effectiveness and capabilities for text classification tasks, making them popular choices in the industry and giving you a starting point to evaluate them for your own projects.
 - BERT: A transformer-based model pre-trained on bidirectional language modeling tasks, achieving state-of-the-art results on numerous NLP tasks, including text classification. BERT has a GLUE score of **80.5**.
 - RoBERTa: Builds upon BERT with pretraining optimizations and a larger dataset, resulting in improved results on various NLP benchmarks. RoBERTa has a GLUE score of **88.5**.
 - ALBERT: A more efficient and lighter version of BERT using parameter-sharing techniques across layers, achieving comparable results to BERT with significantly fewer parameters. ALBERT has a GLUE score of **86.4**.

- DistilBERT: A smaller, faster, and more efficient version of BERT, retaining 95% of BERT’s performance with a 40% reduction in model size. DistilBERT has a GLUE score of **82.2**.
 - DeBERTa: A BERT-based model with a disentangled attention mechanism, enhancing representation power and achieving state-of-the-art results on various NLP tasks. DeBERTa has a GLUE score of **89.9**.
 - ELECTRA: A transformer-based model pre-trained using a novel token replacement task, allowing for efficient learning and achieving state-of-the-art results with smaller model sizes. ELECTRA has a GLUE score of **87.6**.
-
- Transformer models excel in handling long-range dependencies but still face challenges, such as imbalanced datasets and complex classification problems. Addressing imbalanced data can involve techniques like undersampling, oversampling, class weighting, or transfer learning. For complex classification problems, larger model architectures or scaling existing models can help improve performance.
 - In evaluating classification tasks, essential metrics include the confusion matrix, accuracy, and F1-score. The confusion matrix visualizes correct and incorrect predictions for each class, while accuracy measures the proportion of correct predictions, and the F1-score balances precision and recall for a more comprehensive performance assessment.
 - When fine-tuning a model for a specific downstream classification task, it’s crucial to initially evaluate its zero-shot prediction capabilities, assess the model’s architecture, and validate its performance using common benchmarks such as GLUE. Enhancing the model’s performance, particularly for complex tasks, can be achieved by incorporating weighting or a learning rate scheduler during the fine-tuning process. Additionally, it’s essential to carefully weigh the advantages and disadvantages of employing a larger model for more intricate tasks.

Bibliography

- [1] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. ELECTRA: pre-training text encoders as discriminators rather than generators. *CoRR*, abs/2003.10555, 2020. URL: <https://arxiv.org/abs/2003.10555>, [arXiv:2003.10555](https://arxiv.org/abs/2003.10555). 120
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. URL: <http://arxiv.org/abs/1810.04805>, [arXiv:1810.04805](https://arxiv.org/abs/1810.04805). 119
- [3] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. *CoRR*, abs/2006.03654, 2020. URL: <https://arxiv.org/abs/2006.03654>, [arXiv:2006.03654](https://arxiv.org/abs/2006.03654). 120
- [4] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. *CoRR*, abs/1909.11942, 2019. URL: <http://arxiv.org/abs/1909.11942>, [arXiv:1909.11942](https://arxiv.org/abs/1909.11942). 119
- [5] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019. URL: <http://arxiv.org/abs/1907.11692>, [arXiv:1907.11692](https://arxiv.org/abs/1907.11692). 119
- [6] Pekka Malo, Ankur Sinha, Pyry Takala, Pekka J. Korhonen, and Jyrki Wallenius. Good debt or bad debt: Detecting semantic orientations in economic texts. *CoRR*, abs/1307.5336, 2013. URL: <http://arxiv.org/abs/1307.5336>, [arXiv:1307.5336](https://arxiv.org/abs/1307.5336). 132
- [7] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *CoRR*, abs/1910.01108, 2019. URL: <http://arxiv.org/abs/1910.01108>, [arXiv:1910.01108](https://arxiv.org/abs/1910.01108). 120
- [8] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. URL: <https://aclanthology.org/W18-5446>, [doi:10.18653/v1/W18-5446](https://doi.org/10.18653/v1/W18-5446). 118
- [9] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, pages 649–657, 2015. 139
- [10] Yukun Zhu, Ryan Kiros, Richard S. Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. *CoRR*, abs/1506.06724,

2015. URL: <http://arxiv.org/abs/1506.06724>, arXiv:1506.06724. 120

Part 3

Advanced models and methods

Large Language Models excel at processing and understanding natural language, enabling them to produce remarkable text outputs. This includes generating creative prompts for poems, crafting factual prompts with clarity, and effectively summarizing given texts in an organized and logical way.

In Part 2, we took a deep dive into fundamental NLP tasks covering text summarization, neural machine translation, and text classification. We discussed the challenges faced when applying LLMs to our use cases and touched slightly on how to optimize these models for specific tasks.

In this third part of the book, we will expand upon this knowledge. We will explore how LLMs are able to produce stunning text generation outputs, what influences their creativity, and how you can control the generated text outputs, including methods to create your own sophisticated chat model.

Moreover, we will delve into prompt engineering, or so-called in-context learning. This involves controlling your LLM's output without retraining it, or in simpler terms, without making fundamental changes to their underlying algorithms. We will not only explore how to apply prompt engineering effectively but also address how to prevent attacks via prompting. Such attacks can exploit an LLM to reveal private data, posing significant security risks.

Further, we will take a deep dive into optimizing your model for a given downstream task. As a final chapter of this part, we will look into the caveats of using large language models and discuss how to make them more ethical and responsible in their use. Preparing these architectures for the upcoming regulatory rules is crucial, as we will need to adhere to these rules to ensure the responsible and ethical use of LLMs.

Text generation

This chapter covers

- Introduction to text generation
- Outline of text generation models
- Techniques for text generation
- Challenges in text generation

Welcome to the first chapter of the third part of this book. We're about to embark on a journey through some of the most exciting and widely discussed topics in the field of NLP today - text generation. This journey will unfold over the next three chapters. In the previous part, we explored fundamental NLP tasks, including text summarization, neural machine translation, and text classification. These discussions were grounded in the foundational concepts introduced in the first part of the book, where we dived deep into the original transformer architecture from the paper "Attention Is All You Need." Armed with this knowledge, we're well-prepared to dive into the newer models and cutting-edge techniques used in the mind-blowing field of text generation.

6.1 Introduction to text generation

The history of text generation stretches back as far as the 1960s, with rudimentary rule-based chatbots paving the way. Joseph Weizenbaum introduced the first chatbot, ELIZA, marking the dawn of computer-based natural language conversation. This pioneering sys-

tem offered many their first taste of interactive computing, providing a user experience that, even by today's standards, approximates what one might expect from a real conversation with a computer.

However, it's important to note that ELIZA, while innovative for its time, wouldn't pass the Turing Test. The Turing Test, proposed by Alan Turing, evaluates a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human. Despite its limitations, ELIZA serves as an important milestone in our journey of text generation. For a nostalgic glimpse into the past and a measure of how far technology has come, I recommend taking ELIZA for a spin at <https://www.masswerk.at/eliza/>. This early chatbot still retains its charm and serves as a humble reminder of the origins of our current text generation journey. Let us look at some common techniques and how text generation has evolved.

6.1.1 From rule-based chatbots to Turing Test passing bots

From the early days of rule-based chatbots, such as ELIZA, to the deep learning models of today, i.e. ChatGPT, the development of text generation technology has seen a gradual shift from NLP towards natural language understanding (NLU). Researchers and engineers started exploring the potentials of machine learning to create more sophisticated and responsive systems. Chatbots like Jabberwacky and Cleverbot were among the first to embrace these new techniques, learning and evolving their responses from human interactions. IBM Watson marked a significant milestone in this development, showcasing the power of natural language processing and machine learning to a wider audience by winning Jeopardy!

However, with the rise of LLMs, we began to see a new class of language generation models. OpenAI's GPT-4 and Meta's LLaMA 2 are among these, utilizing transformer-based architectures to create remarkably human-like text. These models represent the current state-of-the-art (SOTA) in text generation, but as technology continues to advance, we can expect to see even more powerful and versatile language models in the future. Here is a short explanation of each type:

- **Rule-based chatbots:** These are the earliest types of chatbots, which operate based on pre-determined rules on which they were initially programmed. They can only respond to specific commands and lack the ability to understand and learn from user inputs. ELIZA is an example of a rule-based chatbot.
- **Corpus-based chatbots:** These chatbots are a significant step forward as they are capable of learning from past interactions. They use a so called "corpus", a large dataset of dialogue examples, to generate their responses. The versatility of a chatbot increases with the size and diversity of its corpus. Examples of such chatbots include Cleverbot and Jabberwacky.
- **Machine learning-based chatbots:** These chatbots utilize machine learning algorithms to learn from user inputs and improve their responses over time. They can adapt to the user's language and provide more personalized responses. IBM's Watson is a prime example of a machine learning-based chatbot.

- Transformer-based models: The current state-of-the-art in text generation is exemplified by models such as GPT-4, Falcon and LLaMA. Unlike prior approaches such as RNNs and LSTMs, transformers can handle much longer sequences, which enables them to process and generate larger pieces of text. What distinguishes them from the previous chatbots, is, these transformer-based models can generate context-aware embeddings, empowering them to understand the context, extract nuanced meanings, and generate coherent, relevant responses.

As you can see, the evolution of text generation isn't merely a story of technological advancement; it also marks a significant shift in the field's underlying goals. The initial focus was on NLP, with systems designed to understand and manipulate human language to perform tasks like translation, summarization, and sentiment analysis. However, the field has since transitioned towards NLU, involving the more complex tasks of grasping context, interpreting sentiment, and extracting nuanced meanings. These advancements have led to the emergence of what can be called "next-generation" models, which we will focus on in this chapter. Now, with a better understanding of how we got here, let's dive deeper into these sophisticated models.

6.2 *Transformers in text generation: An overview*

At the time of writing this book, the field of LLMs, particularly models capable of text generation, is one of the most rapidly evolving areas in artificial intelligence. New models are being developed and released with such frequency that keeping pace with all the advancements presents a challenge.

This section provides a snapshot of this dynamic landscape by shedding light on some recent developments and revisiting a few *milestone* models that have shaped the discourse in this area. However, it's important to note that this is by no means an exhaustive overview. But this section aims to reflect the rapid pace of the innovations as a testament to the exciting and ever-evolving nature of this field.

In the following sections, I will provide an overview of a selection of these influential text generation models, diving into their unique features and capabilities in more detail. Please note that I will only cover GPT models up to and including GPT-3 in this chapter. The GPT-4 architecture will be discussed in chapter 8, as GPT-4, also known as Chat-GPT, introduces multimodal capabilities that were not present in the earlier models. Additionally, while BERT also plays a significant role in text generation, I have already covered it in detail in section 5.2.1, and I won't repeat that discussion here.

To ensure you get the most out of this section, I'll initially present a concise snapshot of several significant text generation models, providing key information about each one. After this overview, we'll delve deeper into the specifics of each model, offering an easy-to-digest distillation of their primary research papers. This two-tiered approach serves as a guide, helping you navigate the world of these fascinating models.

- GPT-1 (Generative Pretrained Transformer 1) (8) is an early model that leveraged a transformer-based architecture for language generation tasks in a novel way. Unlike traditional transformer models that included both encoder and decoder com-

ponents, GPT-1 featured a decoder-only structure. This model employed masked self-attention heads to focus solely on preceding words in the text during the prediction process. The model comprises **117 million parameters**

- GPT-2 (Generative Pretrained Transformer 2) (9) was a substantial expansion of its predecessor, offering significantly improved performance. It introduced the concept of "zero-shot" learning, wherein the model generates appropriate responses without any prior task-specific training. GPT-2 comprises **1.5 billion parameters**, allowing it to generate remarkably coherent and contextually relevant outputs.
- GPT-3 (Generative Pretrained Transformer 3) (2) is among the larger language models, with **175 billion parameters**. It further enhanced the capabilities of GPT-2, producing even more accurate and contextually appropriate text. GPT-3 can generate creative content, answer questions, translate languages, and perform a number of other NLP tasks.
- InstructGPT (6) is a variant of the GPT architecture that has been fine-tuned for instruction following. Unlike the base GPT models, which are typically trained on a diverse range of internet text, InstructGPT was trained with a dataset that includes demonstrations of correct behavior, making it particularly adept at generating human-like responses to instructions in the text. InstructGPT was developed in three different sizes: **1.3B, 6B, and 175B parameters**. It has demonstrated improvements in truthfulness and reductions in toxic output generation.
- GPT-NeoX-20B (1) model architecture closely mirrors that of GPT-3, but incorporates several critical improvements, including rotary positional embeddings, parallel attention and feed-forward layers, and exclusive use of dense layers. Trained on the Pile dataset, it offers robust general-purpose language modeling capabilities. All training code, model architecture, and weights are open-sourced for both research and commercial use. The model has **20B parameters**.
- Llama (Open and Efficient Foundation Language Models) (10) is a collection of transformer-based language models ranging from **7 billion to 65 billion parameters**. Unlike some other large-scale models, Llama is trained exclusively on publicly available data, demonstrating state-of-the-art performance on numerous benchmarks. For instance, Llama-13B outperforms GPT-3 (175 billion parameters) on many tasks.
- RedPajama is a collaborative project launched by Together, Ontocord.ai, ETH DS3Lab, Stanford CRFM, and Hazy Research, RedPajama aims to create leading, fully open-source language models. A crucial milestone achieved by this collaboration is the reproduction of the Llama training dataset, a comprehensive data repository of over 1.2 trillion tokens. RedPajama is primarily designed to bridge the quality gap between closed commercial models and fully open-source models. The model is available with **3 billion and 7 billion parameters**. RedPajama also provides an instruct/chat model variant designed for chat applications

- Alpaca (A Strong, Replicable Instruction-Following Model) was developed by Stanford University, with the intent to address challenges in instruction-following models such as GPT-3.5 (text-davinci-003) and ChatGPT. Alpaca is a fine-tuned model based on Meta's Llama 7B. The model is trained on 52K instruction-following demonstrations and is specifically designed to mimic the behaviors exhibited by OpenAI's GPT-3.5.
- Dolly was launched by Databricks. Dolly was one of the initial truly open-source instruction fine-tuned large language models. The latest version, Dolly 2.0, is a comprehensive model with **12 billion parameters**, grounded in the Pythia model family from EleutherAI. It is comparative to other LLMs of their size, such as BLOOM and Llama models. The entire Dolly 2.0 model, including the training code, dataset, and weights, is open-sourced, supporting both research and commercial use.
- Falcon, developed by The Technology Innovation Institute of Abu Dhabi (TII), is a decoder-only transformer model. It is trained on an enriched corpus of 1,000B tokens from RefinedWeb, supplemented with curated corpora. The model is available in two sizes with **6.7 billion and 40 billion parameters**, respectively. In addition to its standard model, Falcon also provides an instruct model variant designed for chat applications.

Let us now look into these models in more detail.

6.2.1 GPT-1 to GPT-3

The development of the Generative Pretrained Transformer (GPT) series has marked significant advancements in language models. GPT-1 was the first model in the GPT series with its unique generative pretraining technique on a diverse corpus of unlabeled text and subsequent discriminative fine-tuning. It employed a transformer-based architecture with 12 transformer layers and 117 million parameters, introducing a unidirectional approach to language modeling which means that it sequentially predicts the next word from left to right.

The pretraining was conducted on the BooksCorpus dataset, enabling the model to optimize for predicting the next word in a sequence. The mathematical objective during this pretraining is to maximize the following likelihood:

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta) \quad (6.1)$$

where k denotes the context window size, which refers to the range of words around a target word that a model considers, and Θ represents the model parameters. This objective requires the model to estimate the probability of a word given its preceding context.

In terms of architecture, GPT-1 uses a multi-layer transformer decoder, as depicted in figure 6.1, applying self-attention and feedforward networks to process the input tokens and predict the next ones.

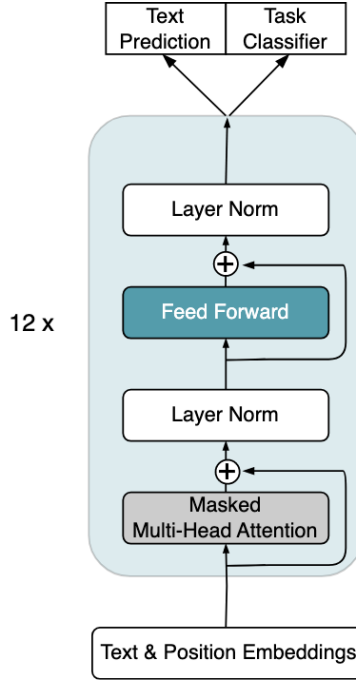


Figure 6.1 Illustration of the multi-layer Transformer decoder of GPT-1 with 12 layers, which is a variant of the original Transformer architecture from the paper "Attention is all you need".

The mathematical representation of the model's operations is as follows:

$$\begin{aligned}
 h_0 &= UW_e + W_p \\
 h_l &= \text{transformer_block}(h_{l-1}) \quad \forall l \in [1, L] \\
 P(u) &= \text{softmax}(h_L W_e^T)
 \end{aligned} \tag{6.2}$$

where U is the context vector of tokens, W_e represents token embeddings, W_p is for position embeddings, and L is the number of layers. The transformer block produces a series of hidden states h_l , leading to a probability distribution over the next tokens $P(u)$.

The fine-tuning phase introduces an additional objective:

$$P(y \mid x^1, \dots, x^m) = \text{softmax}(h_L^m W_y) \tag{6.3}$$

where h_L^m is the last layer's hidden state, and W_y is the output layer's weight matrix. This phase adjusts the model parameters to maximize the log-probability of the correct label given the input sequence.

The auxiliary language modeling objective during fine-tuning is also considered:

$$L_3(C) = L_2(C) + \lambda * L_1(C) \tag{6.4}$$

which combines the primary objective with the language modeling objective, balanced by a weighting factor λ .

Through these methods, GPT-1 established a strong foundation for subsequent models in the GPT series, enabling its successful application to various language tasks.

Despite its unidirectional nature, GPT-1's framework laid the groundwork for subsequent models, achieving effectiveness in various tasks such as text classification and translation.

Building on this foundation, GPT-2 was introduced, scaling the model to 1.5 billion parameters across 48 Transformer layers. Trained on the more diverse WebText dataset, GPT-2 showed enhanced capabilities, particularly in zero-shot settings where the model performs tasks without task-specific training. The transition from GPT-1 to GPT-2 showcased a leap in language model development, improving versatility and expanding application potential.

Following this data-centric approach, GPT-3 introduces improvements in the model architecture. While the underlying architecture remains similar to GPT-2, GPT-3 uses alternating dense and locally banded sparse attention patterns within the transformer layers. This approach allows the model to handle larger context windows effectively. Context window refers to the range of words a model can process. The parameters are now scaled to 175 billion across 96 transformer layers, using an enriched mixture of datasets, including a filtered version of the Common Crawl and an expanded WebText, along with other curated data. The dataset and the weight in the training mix is shown in table 6.1.

Table 6.1 Datasets utilized in the training of GPT-3.

Dataset	Quantity (tokens)	Weight in training mix	Epochs per 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

In the table, "Weight in training mix" refers to the fraction of examples during training that are drawn from a given dataset, which was intentionally not proportional to the size of the dataset. As a result, when the model is trained for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

Similar to its predecessors, GPT-3 operates on a left-to-right training objective. However, due to its increased model size and improved attention mechanism, GPT-3 can generate and comprehend text with more sophistication. Moreover, its zero-shot learning capabilities enable it to understand and perform various NLP tasks without any specific task training or further gradient updates to its weights, utilizing only the context provided in the prompt.

6.2.2 *InstructGPT*

InstructGPT, an offshoot of the GPT-3 model released in March 2022, is designed to follow instructions in a text prompt and provide detailed responses. Like GPT-3, InstructGPT employs the same transformer architecture but differentiates itself through a unique training

process involving three crucial steps: supervised fine-tuning (SFT), reward model (RM) training, and reinforcement learning via proximal policy optimization (PPO). PPO, a policy optimization method used in reinforcement learning, allows an agent to learn a policy that maximizes reward with efficient and stable learning dynamics. A more detailed discussion on PPO is provided in section 7.1.1.

During the supervised fine-tuning stage, a GPT-3 model that has already been pretrained is fine-tuned with examples that demonstrate exactly how it is expected to behave. This process is known as supervised fine-tuning or SFT. Afterward, a set of prompt responses is generated by the model and compared. These comparisons are used to train another model, called a reward model, which learns to identify which responses are preferred by humans.

Once this reward model is set up, it assigns scores to the generated responses, similar to points in a game. The better a response aligns with the human-classified preferred responses, the higher the score it receives. These scores are then used to further refine the original GPT-3 model using PPO. This iterative process repeats, leading to continuous improvements, which enhance the model's ability to understand and respond to prompts in a way that aligns more closely with human expectations.

Despite this new training method, the model still may sometimes generate incorrect or nonsensical answers, signaling the ongoing challenges in developing reliable and accurate AI models. However, it's worth noting that InstructGPT has shown improvements in truthfulness and reduced toxicity over GPT-3, producing truthful and informative answers about twice as often and generating about 25% fewer toxic outputs when instructed to be respectful.

Moreover, InstructGPT incorporates a specialized technique to safeguard the model's original strengths while it undergoes fine-tuning for specific tasks. This technique is particularly designed to avoid "performance regression", which can occur when a model, after being fine-tuned, starts to perform worse on the general tasks it was initially trained on. By blending PPO-based weight updates with updates that reinforce the weights of the model's initial pretraining, this approach carefully adjusts the model without overriding its foundational abilities. This ensures that the fine-tuning process retains the model's core competencies on standard tasks while it learns to respond to the nuances of new instructions.

Additionally, InstructGPT introduces a methodology for minimizing performance regressions on public NLP datasets observed during the fine-tuning process. This method mixes PPO updates with updates that increase the log likelihood of the pretraining distribution, which significantly reduces performance regressions without compromising labeler preference scores. "Performance regression" is a term used in the context of machine learning to refer to a situation where a model's performance decreases or degrades.

InstructGPT's training involved gathering data from a team of carefully selected contractors. This expanded the diversity of inputs, which is critical for the model to effectively navigate prompts that are controversial or sensitive. The contractors were selected after a thorough screening process, designed to identify individuals skilled in detecting and addressing sensitive content. Yet, incorporating human data introduces the risk of subjective

bias, as the data is influenced by the perspectives and interpretations of those involved in the process. Moreover, handling such data also introduces the need for stringent privacy measures. Therefore, while human contributions significantly enhance the model's ability to handle a wide range of prompts by providing nuanced feedback, these contributions must be managed with caution to mitigate bias and ensure privacy.

6.2.3 GPT-NeoX-20B

The GPT-NeoX-20B model is engineered not just to scale up the capabilities of language models, but also to enhance the precision and depth of their responses. It exhibits exceptional proficiency in few-shot reasoning, delivering a notable improvement in five-shot performance when compared to a similarly scaled GPT-3 model.

The GPT-NeoX-20B model, like its predecessor, employs the transformer architecture, which leverages self-attention mechanisms for more efficient computation. It is an autoregressive transformer decoder model whose architecture largely follows that of GPT-3. However, the novelty lies in the model's increased parameters and the way it scales to larger datasets and computational resources. This model comes with 20 billion parameters.

The backbone of the GPT-NeoX-20B architecture is the "Megatron" technology. Megatron, a technique for training massive transformer models, emphasizes parallelization, allowing for efficient scaling of training across multiple GPUs. This makes the training of such an enormous model feasible and efficient.

One of the unique aspects of GPT-NeoX-20B is the use of model parallelism and pipeline model parallelism. Model parallelism is a method of parallelization that distributes the model's layers across multiple devices, allowing for efficient computation even with larger models. Similarly, pipeline model parallelism is a technique that further improves training efficiency by overlapping the computation of different mini-batches on different GPUs.

An important aspect of training GPT-NeoX-20B is the use of Zero Redundancy Optimizer (ZeRO) to reduce memory footprint and increase the number of trainable parameters per GPU. ZeRO allows each GPU to hold unique parameters and optimizer states, thus enabling a larger number of parameters to be trained without exhausting memory resources.

In terms of data, GPT-NeoX-20B was trained on a combination of Common Crawl, Wikipedia, and other large publicly available text corpora. This diverse dataset enabled the model to acquire a broad understanding of human language and knowledge, encompassing a wide variety of topics, styles, and perspectives.

For tokenization, GPT-NeoX-20B employs a Byte Pair Encoding (BPE) based tokenizer, much like the one used in GPT-2, maintaining an identical total vocabulary size of 50,257.

However, this model introduces three significant modifications to the tokenization process. Firstly, the team trained a new BPE tokenizer on the Pile dataset, utilizing its wide array of text sources to develop a more versatile and general-purpose tokenizer. This approach enhances the model's ability to handle a vast range of linguistic styles and topics.

Secondly, unlike the GPT-2 tokenizer, which treats tokenization at the start of a string as a non-space-delimited token, the GPT-NeoX-20B tokenizer enforces consistent space delimitation. A comparison of both model tokenizations is illustrated in figure 6.2.

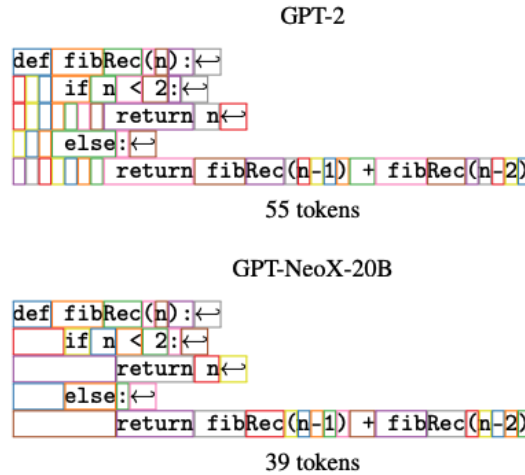


Figure 6.2 Comparison of tokenization between GPT-2 and GPT-NeoX-20B. The GPT-NeoX-20B tokenizer exhibits enhanced management of whitespace, proving especially beneficial for types of text like source code. Image is taken from the paper: "GPT-NeoX-20B: An Open-Source Autoregressive Language Model"

This modification eliminates inconsistencies regarding the presence of prefix spaces in the tokenization input, thus providing a more standardized tokenization procedure.

Thirdly, the GPT-NeoX-20B tokenizer incorporates tokens for repeated space tokens for all positive integers up to and including 24. This feature enables the tokenizer to manage text with substantial amounts of whitespace using fewer tokens, proving beneficial in situations where whitespace is crucial for interpretation, such as in program source codes or LaTeX source files.

The enhancements in tokenization in GPT-NeoX-20B reflect a continued commitment to improving the versatility and effectiveness of language models, contributing to their capacity to understand and generate human-like text.

6.2.4 Llama

Llama (Open and Efficient Foundation Language Models) and Llama 2 is a collection of transformer-based, pretrained and fine-tuned language models ranging from 7 billion to 70 billion parameters.

Llama, drawing inspiration from other large language models as well as the original transformer architecture, implements several novel modifications to enhance its performance and training stability. Following the pre-normalization technique introduced in GPT-3, the Llama model normalizes the input rather than the output of each transformer sub-layer, applying the RMSNorm normalizing function.

The model replaces the ReLU non-linearity, used in the original transformer architecture, with the SwiGLU activation function. SwiGLU has been recognized for improving performance in the PaLM model and belongs to the family of gated linear unit (GLU) activation functions. Within Llama, SwiGLU is designed to work with a reduced dimen-

sionality of its weight matrices, specifically using $2/3$ of the input dimension, as opposed to the fourfold version used in PaLM. This modification optimizes the model's efficiency while retaining the performance benefits of SwiGLU activation. To put it simply, the original transformer's ReLU activation involved calculations with two sets of weights, Llama's SwiGLU configuration involves three sets of weights, with the third acting as a gate to process the input data effectively.

In an adaptation from GPTNeo, Llama replaces absolute positional embeddings with rotary positional embeddings and applies these at each layer of the network.

For model optimization, Llama utilizes the AdamW optimizer, with a specific set of hyper-parameters: $\beta_1 = 0.9$, $\beta_2 = 0.95$. It applies a cosine learning rate schedule (refer to section 5.4.3 to understand what learning rate schedulers do), such that the final learning rate equals 10% of the maximal learning rate, alongside a weight decay of 0.1 and gradient clipping of 1.0. The model employs 2,000 warm-up steps and modifies the learning rate and batch size according to the model size, as shown in table 6.2.

Table 6.2 Model sizes, architectures, and optimization hyper-parameters.

Model	Params	Dimension	n heads	n layers	Learning rate	Batch size	n tokens
Model	6.7B	4096	32	32	3.0e-4	4M	1.0T
Model	13.0B	5120	40	40	3.0e-4	4M	1.0T
Model	32.5B	6656	52	60	1.5e-4	4M	1.4T
Model	65.2B	8192	64	80	1.5e-4	4M	1.4T

Impressively, Llama 1-13B outperforms GPT-3 (175 billion parameters) on zero-shot tasks on the common sense reasoning tasks, as shown in table 6.3.

Table 6.3 Comparison of Performance between GPT-3 and Llama 1.

Model	Params	BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3	175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
LLaMA	7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
LLaMA	13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
LLaMA	33B	83.1	82.3	50.4	82.8	76.0	80.0	57.8	58.6
LLaMA	65B	85.3	82.8	52.3	84.2	77.0	78.9	56.0	60.2

Common sense reasoning tasks refer to a category of tests that evaluate a model's ability to use and understand implicit, everyday knowledge that humans generally consider "common sense." This might involve predicting or explaining ordinary events, answering questions that rely on an understanding of the world, or making logical deductions based on broad human experiences. These tasks present a challenge for AI models because they often

require intuitive understanding and reasoning that goes beyond the explicit information available in a given dataset.

Unlike some other large-scale models, Llama 1 is trained exclusively on publicly available data, shown in table 6.4, demonstrating state-of-the-art performance on numerous benchmarks.

Table 6.4 Pre-training data utilized in the training of LLaMA.

Dataset	Sampling proportion ¹	Epochs
Common Crawl	67%	1.10
C4	15%	1.06
Github	4.5%	0.64
Wikipedia	4.5%	2.45
Books	4.5%	2.23
ArXiv	2.5%	1.06
StackExchange	2.0%	1.03

¹ Each subset specifies the sampling proportion and the number of epochs performed on the subset during the training on 1.4T tokens. The pre-training conducted on 1T tokens follows the same sampling proportions.

As for LLaMA 2, which is an enhanced iteration of Llama 1, was developed using a new mix of publicly accessible data. Moreover, the volume of the training corpus was increased by 40%, the model’s context length was doubled, and the researchers incorporated grouped-query attention. Which is a blend of multi-head and multi-query attention, where each subgroup of query heads has a single key and value head, as shown in figure 6.3.

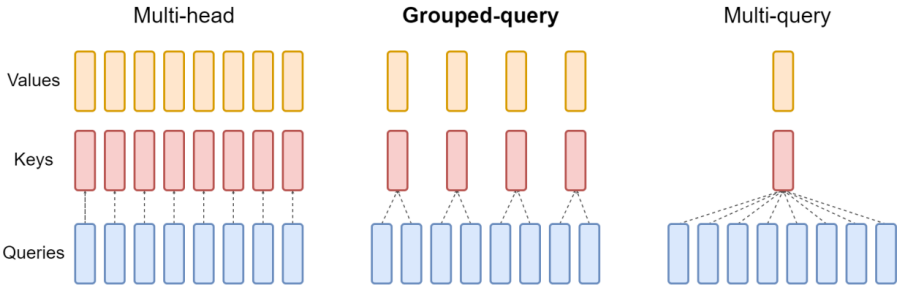


Figure 6.3 An overview of the grouped-query approach: Multi-head attention consists of H sets of query, key, and value heads. In multi-query attention, all query heads share one key and one value head. On the other hand, grouped-query attention allocates a single key and value head to every cluster of query heads, creating a blend between multi-head and multi-query attentions. Image is taken from the paper: "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints"

Llama 2 comes with 7B, 13B, and 70B parameters. However, there exist also a 34B variant which wasn’t released. Additionally, Llama 2-Chat is a refined model of Llama 2,

specifically tailored for dialogue applications. This model also has versions with 7B, 13B, and 70B parameters.

Overall, LLaMA is a significant effort in progressing transformer-based language models, incorporating the best of previous models, while introducing further improvements. Also, with LLaMA 2, the researchers made combined efforts to enhance transparency and shed light into the underlying causes of potential biases in downstream tasks ensuring a more reliable and safe usage of LLMs by providing additional information about the pretraining data.

6.2.5 *RedPajama*

RedPajama represents a notable endeavor to design an array of premier, fully open-source models. It emerges as a response to the limitations imposed by today's most advanced foundation models like GPT-4, which are typically bound by commercial APIs or only partially open. This project's prime objective is to create a platform where research, customization, and interaction with sensitive data are not restricted, thus bridging the quality gap between open and closed models.

NLP is currently experiencing its "Linux moment", where open-source technologies are not only challenging the quality of commercial offerings but fostering an environment of creativity through global community participation. This momentum has extended to the field of large language models with open source models like LLaMA, Alpaca, Vicuna, and Koala, as well as fully open models such as Pythia, OpenChatKit, Open Assistant, Dolly , and Falcon.

RedPajama is an initiative stemming from a collaboration between several leading research entities including Together, Ontocord.ai, ETH DS3Lab, Stanford CRFM, and Hazy Research. It embodies a three-pronged approach:

- Creation of high-quality pre-training data with wide-ranging coverage.
- Development of base models that undergo extensive training on this data.
- Production of instruction tuning data and models, which refine the base model to enhance its usability and safety.

As of the writing of this book, RedPajama has successfully completed its first phase, unveiling the pre-training data component. This involves the reproduction of the LLaMA training dataset, comprising over 1.2 trillion tokens. The LLaMA model, with its 7 billion parameters, served as the project's initial inspiration due to its quality and ability to run on a broad spectrum of GPUs, making it valuable for the open community.

RedPajama seeks to replicate LLaMA in a completely open-source format, and with that tries to advance the open-source AI movement, ushering in a new era of innovation and exploration.

6.2.6 *Alpaca*

Alpaca is an instruction-following language model, fine-tuned from Meta's LLaMA 7B model. Despite the extraordinary capabilities of instruction-following models they still exhibit notable deficiencies such as the generation of incorrect information, propagation of

social stereotypes, and creation of harmful language. The Alpaca model aims to address these issues, providing a foundation for the academic community to further research and improve upon.

Developed by a dedicated team of researchers from the Center for Research on Foundation Models (CRFM) with support from the Stanford Institute for Human-Centered AI (HAI) and the Stanford Natural Language Processing (NLP) group.

Alpaca has been fine-tuned on 52,000 instruction-following demonstrations, emulating the self-instruct methodology utilizing text-davinci-003. The performance of the Alpaca model mirrors that of OpenAI's text-davinci-003, but it stands out due to its smaller size, making it easier and cheaper to reproduce.

In a bid to engage the academic community in understanding and further enhancing the behavior of the Alpaca model, the researchers have released their training methodologies and data. They also intend to release the model weights.

However, it is essential to note that the Alpaca model is intended strictly for academic research. Commercial use is prohibited due to the licensing restrictions on the LLaMA model and OpenAI's text-davinci-003, which were instrumental in the development of Alpaca. The model also lacks comprehensive safety measures, making it unsuitable for general use.

The research team addressed two significant challenges during the development of Alpaca: the need for a strong pre-trained language model and high-quality instruction-following data. The LLaMA models from Meta provided the solution for the first challenge. For the latter, they devised a method to automatically generate instructional data using an existing potent language model. This approach resulted in Alpaca, a language model fine-tuned using supervised learning from the LLaMA 7B model on 52K instruction-following demonstrations generated from OpenAI's text-davinci-003.

Alpaca's performance, when compared to text-davinci-003, was found to be similar in a blind pairwise comparison. This surprising result, considering the smaller model size and the modest amount of instruction following data, has sparked optimism in the research community.

6.2.7 Dolly

Dolly is an instruction-following large language model. As one of the first truly open-source instruction fine-tuned LLMs, Dolly has been instrumental in demonstrating the potential of open-source developments in LLMs.

Dolly 2.0 is a significant enhancement over the original model. It is a 12B parameter language model based on the Pythia models, from EleutherAI and has been fine-tuned on a crowdsourced dataset built by Databricks employees. These employees generated over 15,000 high-quality human-generated prompt/response pairs, which served as a robust dataset for training Dolly 2.0.

It is the first open-source, instruction-following LLM fine-tuned on a human-generated instruction dataset licensed for research and commercial use. The model, training code, and dataset of Dolly 2.0 are all open-source, meaning any organization can utilize and customize this model without having to pay for API access or share data with third parties.

The databricks-dolly-15k dataset generated for the instruction tuning of large language models, is the first open-source, human-generated instruction dataset designed to enable LLMs to exhibit interactivity similar to ChatGPT.

The creation of the databricks-dolly-15k dataset arose out of a need for a commercially viable dataset for training LLMs. The initial version of Dolly was trained using a dataset created by the Stanford Alpaca team with the OpenAI API. However, the terms of service of this dataset restricted commercial use, prompting the creation of a new "untainted" dataset.

The Databricks team took inspiration from OpenAI's research paper, which mentioned that the InstructGPT model was trained on a dataset of 13,000 instruction-following demonstrations. However, creating a similar dataset proved challenging due to the need for original content, free from copying from ChatGPT or any other web source. The team overcame this hurdle by incentivizing Databricks employees to contribute to the dataset, resulting in a unique, high-quality instruction-following dataset for Dolly 2.0.

The crowdsourcing process was structured around seven specific tasks, including open Q&A, closed Q&A, extraction of information from Wikipedia, and summarization of information from Wikipedia. These tasks ensured a diverse and comprehensive dataset to cover a wide range of behaviors that an LLM may encounter. This approach increases the model's versatility and robustness, making it more applicable to both academic research and commercial use.

6.2.8 *Falcon*

The Falcon model family includes various fully open-sourced models: Falcon-40B and its smaller variant, Falcon-7B. The Falcon-40B model requires around 90GB of GPU memory to function, a substantial but less demanding requirement compared to the LLaMA-65B, a model which Falcon surpasses in performance. Falcon-7B, requiring only about 15GB of memory. And the currently largest open-access available model, Falcon-180B

Three instruction/chat-based versions, Falcon-7B-Instruct, Falcon-40B-Instruct and Falcon-180B-Chat, have also been developed by the TII. These models, fine-tuned with instructional and conversational data, are especially suited for assistant-style tasks. The flexibility of these models allows for customization using a wide variety of community-developed datasets. All models are fully open source, since TII has waived all restrictions for commercial usage, and available via the Hugging Face API.

The models are designed for the causal language modeling task, which involves predicting the subsequent token in a sequence. While the architecture is predominantly adapted from the GPT-3 model, there are a few distinct modifications, such as:

- Positional embeddings: The models make use of rotary positional embeddings, similar to LLaMA.
- Attention: The models utilize multiquery attention and FlashAttention.
- Decoder-block: A unique feature is the parallel attention/MLP (Multi-Layer Perceptron) with two-layer norms.

In the multiquery attention mechanism, each tensor parallel degree uses independent keys and values. Meaning, a shared key and value are used across all heads, in contrast to the typical multi-head attention scheme.

The models support a variety of languages. They offer robust capabilities in English, German, Spanish, and French, while also providing limited proficiency in Italian, Portuguese, Polish, Dutch, Romanian, Czech, and Swedish. This means that it might not perform well with other languages. Moreover, since its training is based on vast web data, it may reflect the common biases and stereotypes found online. Meaning, one would have to fine-tune the models accordingly to address inherent biases, so that the model adheres with the required objectives.

The training processes for Falcon-7B and Falcon-40B involved 1.5 trillion and 1 trillion tokens respectively, mirroring the trend of modern models geared towards optimizing inference. The superior performance of the Falcon models is largely due to their training data, with over 80% originating from RefinedWeb, a new comprehensive web dataset derived from CommonCrawl. The data distribution is shown in table 6.5. To improve data quality, TII implemented large-scale deduplication and stringent filtering, reducing the dependence on curated sources, such as Reddit conversational data. TII has also publicly released a subset of RefinedWeb comprising 600 billion tokens for community use in their own LLM projects.

Table 6.5 Data distribution for the training of Falcon 40 billion on 1,000B tokens.

Data source	Fraction	Tokens	Sources
RefinedWeb-English	75%	750B	massive web crawl
RefinedWeb-Europe	7%	70B	European massive web crawl
Books	6%	60B	
Conversations	5%	50B	Reddit, StackOverflow, HackerNews
Code	5%	50B	
Technical	2%	20B	arXiv, PubMed, USPTO, etc.

This dataset, comprising five trillion tokens, is completely extracted from web data and refined using an architecture known as MDR (MacroData Refinement). This architecture filters and removes duplicate web data from CommonCrawl on a massive scale. Through its emphasis on large scale, meticulous deduplication, and neutral filtering, the MDR is able to generate a high-quality dataset that matches the standards of aggregated corpora used in training cutting-edge models. This enables the creation of datasets suitable for training models with 40200 billion parameters, without heavily relying on time-consuming human curation processes.

NOTE Make sure you check out the Hugging Face Open LLM Leaderboard which aims to track, rank and evaluate open LLMs and chatbots. Link: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

6.3 Common techniques in text generation

Like every task in NLP or machine and deep learning, text generation comes with its own set of techniques and challenges. In this section, we will discuss the most common techniques used in transformer-based text generation.

To produce their stunning, human-like, and at times almost uncanny text, modern transformer models rely on a diverse set of methods. In the following, I provide a brief overview of these techniques. We will dive into each of these techniques in more detail later, with the aim of giving you a solid understanding of the tools at the disposal of these advanced models.

- Contextual word embeddings: Transformer models generate what are called "contextual embeddings". That is, the embedding for a word can change based on the context in which it is used. This is a significant advancement over traditional methods where each word has a single, static representation.
- Decoding strategies: These strategies guide how the model forms its output sequence. Here are a few common decoding strategies:
 - Greedy search: This method picks the most probable word at each time step and then moves on to the next one, without reconsidering past choices.
 - Beam search: This strategy maintains a "beam" of the most probable sequences and expands all of them at each time step, keeping only the most likely sequences.
- Sampling methods: These methods introduce randomness into the decoding process to generate diverse outputs:
 - Top-k sampling: This method randomly picks the next word only from the top k most likely words.
 - Nucleus sampling (also known as Top-p Sampling): This strategy chooses the smallest set of top words such that their total probability exceeds a certain threshold (p), and then samples the next word from this set.
 - Temperature sampling: Here, a parameter called "temperature" is used to control the randomness of the sampling process. A high temperature leads to more randomness, while a low temperature makes the output closer to greedy search.

Now that we have a basic understanding of what each method entails, let's take a closer look at each one.

6.3.1 Contextual word embeddings

Simple word embeddings such as Word2Vec or fastText struggle to capture the meaning of word combinations or the negation in a sentence. In other words, these simple word embeddings don't take context into account. This limitation becomes evident when we revisit our "date" example from section 2.1, and from section 4.4.2. We already know that the context is crucial, and the model chosen for tasks such as Neural Machine Translation needs to understand the context of a sentence to find the correct translation of the words.

This requirement for context understanding is also true for text generation. Contextual embedding methods strive to map a vector to each word based on the entire sentence or context. In short, we build a vector for each word conditioned on its context! Looking at this from a mathematical perspective, $w_1 \dots w_n$ represents each word or token in a sentence, and $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ is a high-dimensional vector associated with each token. These vectors are the contextual word embeddings. The function f then maps each token to its corresponding embedding based on the context, represented by the other words in the sentence, as shown in equation 6.3.1:

$$f : (w_1, w_2, \dots, w_n) \longrightarrow \mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d \quad (6.5)$$

where $w_1 \dots w_n$ is the word (token) that gets mapped to the embedding vectors $x_1 \dots x_n$. With this function, we can then compute the contextual vector as follows:

$$\mathbf{C}_k = f(w_k \mid w_1, w_2, \dots, w_n) \in \mathbb{R}^d \quad (6.6)$$

This technique enables the model to understand the different meanings of the word **date** in these three example sentences:

"Please check the expiration **date** on the milk carton."

≠

"I'm really looking forward to my **date** with Jane this evening."

≠

"We enjoyed the freshly baked **date** cake."

In short, the power of contextual word embeddings lies in their ability to capture the meaning of words as they are used in specific sentences or contexts. This has a direct impact on transformer-based text generation. These contextual word embeddings are learned during the model's pre-training phase when the model is exposed to vast amounts of text data.

During this pre-training phase, the model not only learns to understand the intricacies of language, including syntactic structures, semantic relationships, and various other linguistic patterns, but also learns a rich representation for each token in its vocabulary. These token representations, combined with the learned contextual patterns, form what we refer to as contextual word embeddings. Hence, each token is associated with a high-dimensional vector that encapsulates its meaning in the context of other words. A simplified version of this process is illustrated in Figure 6.4.

Importantly, these representations are dynamic, changing based on the context in which a token appears. This allows the model to differentiate between multiple meanings or usages of the same token based on context, which is crucial for generating coherent and meaningful text. In a sense, this technique, combined with the attention mechanism, provides the model with a solid understanding of the language and its nuances, forming the foundation for its ability to generate meaningful and coherent text. Even when faced with new or unseen

data, the model can use these embeddings to infer context and maintain the coherence of the generated text.

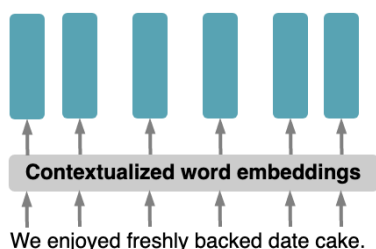


Figure 6.4 An illustration on how each word in a sentence is processed by the transformer model. Each word is mapped to its own unique contextualized word embedding (represented by the vertical rectangles). These embeddings capture the meanings of the words based on the specific context in which they are used.

In contrast, traditional text generation techniques may produce text suffering from issues like repetition, irrelevant sentences, or loss of context. But, with the help of contextual word embeddings, transformers can generate more coherent, relevant, and meaningful sentences. Moreover, this specific contextual-awareness enables transformers to generate text that adapts to the narrative, exhibiting human-like characteristics, as we have already seen with models like ChatGPT, such as maintaining the theme, adhering to the writing style, and displaying an understanding of the nuances and subtleties of language. This has established a new era of modern text generation.

NOTE Contextual word embeddings are crucial for language tasks that involve generating text, such as summarization and machine translation, as well as for other forms of text generation, such as chatbot models. By providing a nuanced understanding of the context in which words are used, these embeddings contribute significantly to the performance of models in these tasks. However, they are particularly relevant for text generation. This is because, unlike in tasks such as text summarization or machine translation where there's a reference text, in text generation the model often has to generate text from scratch or based on a short prompt. Therefore, understanding the context and maintaining coherence throughout the generated text is critical. It's for this reason that I focus on contextual word embeddings in this chapter on text generation.

In conclusion, the ability of transformer-based models to generate contextual word embeddings and combine this technique with the attention mechanism during their pre-training phase is key to their success in text generation tasks. As we continue to explore more techniques for text generation, we will deepen our understanding of why LLMs are able to produce such coherent and sometimes uncannily accurate text generation results.

6.3.2 Greedy Search decoding for text generation

In text generation, we face a similar problem to machine translation: The need to consider multiple possible word sequences, while maintaining syntactic correctness and semantic coherence. This is due to the fact, that in language, words and phrases don't exist in isolation, but in relation to each other. In other words, this means that the meaning of a word can change based on its context, leading to a multitude of possible word sequences for any given prompt.

This creates a large search space, which can lead to a combinatorial explosion. To navigate this space, we employ decoding strategies, one of which is greedy search.

Before delving into the technical details of greedy search, let's conceptualize it with an illustrative analogy. Picture yourself in a maze, with the objective to find the exit in the quickest and shortest way. In the context of a greedy search strategy, what you would do is consistently choose the path that appears to be the shortest and seemingly directs you towards the exit, without giving much thought to the overall layout of the maze. The shortcoming of this approach is that it could easily lead you into dead ends or inadvertently longer paths, as you're not considering the broader maze structure.

Now let us look at the technical "translation" of this analogy. The greedy search decoding algorithm selects the most probable next word at each step in the generation process. If we define y_{t+1} as the word to be generated at time step $t+1$ and $P(y_{t+1}|y_{1:t}, x)$ as the conditional probability of word y_{t+1} given previous words $y_{1:t}$ and input x , the greedy search algorithm can be mathematically formulated as follows:

$$y_{t+1} = \arg \max_y P(y|y_{1:t}, x) \quad (6.7)$$

While this approach might seem logical, it doesn't always produce the most coherent or contextually appropriate results. That's because the decision at each step is made independently, without considering future implications. It's a bit like choosing the path of least resistance at every junction, without considering the overall destination. To clarify this approach more, let us look at a pseudo-code example to see how this algorithm works.

Algorithm 1: Greedy Search for Text Generation

- 1: Initialize the initial input (e.g., a start token or a prompt)
 - 2: $h :=$ initial input
 - 3: Repeat until an end token is generated:
 - 4: $P :=$ Compute the next token probabilities using the model
 - 5: $y := \arg \max_y P(y|y_{1:t}, x)$
 - 6: $h := h \cdot y$
 - 7: Set the selected token as the new input
 - 8: Return the generated sequence h
-

In simple terms, the algorithm follows these steps:

- Step 1 and 2: Initialization

- Step 3: The loop runs until we reach an end token which indicates the end of the sentence in language models.
- Step 4: In each iteration, the model generates the probability distribution P over all possible next tokens.
- Step 5: The token y with the maximum probability is selected as the next token in the sequence. This corresponds to the equation $y_{t+1} = \arg \max_y P(y|y_{1:t}, x)$
- Step 6: The selected token y is appended to the generated sequence h .
- Step 7: The selected token y is set as the new input to the model for the next iteration.
- Step 8: The final generated sequence is returned after we reach an end token.

Now it is clear, that this approach might not always be the best choice to generate consistent and meaningful text. This is where other, more sophisticated search strategies, such as beam search, come in, as we'll see in the following section.

Text generation with Hugging Face

The `model.generate` function provided by the Hugging Face *transformer library* employs greedy search as the default algorithm for text generation. This choice is reasonable for many use-cases due to its computational efficiency and simplicity. However, it's crucial to note that the default parameters might not always be the best choice for every specific task.

To accommodate a variety of needs, this function allows for a high degree of customization, offering us the flexibility to choose from a range of text generation strategies like beam search decoding, top-k sampling, and top-p sampling. Each strategy comes with its own strengths and limitations, and it's often beneficial to experiment with these different decoding or sampling strategies to identify the one that best aligns with your specific task and objectives.

To make it more tangible, how this works, let us shortly look at a concrete example in listing 6.1.

Listing 6.1 Greedy search implementation in `model.generate`

```

❶ Instantiate the model and tokenizer
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

❷ Define an input prompt
prompt = "In a world where AI has become ubiquitous, "

❸ Encode the input prompt and prepare it for the model
input_ids = tokenizer.encode(prompt, return_tensors='pt')
```

④ Generate text output with default greedy search setting

```
greedy_output = model.generate(  
    input_ids,  
    max_length=100  
)
```

This results in the following output:

OUTPUT In a world where AI has become ubiquitous, it's hard to imagine how it will ever be able to replace humans. "We're going to have to see how we do it," said Dr. Michael S. Hirsch, a professor of psychology at the University of California, San Francisco. "We're going to have to see how we do it in a way that's not just a matter of human beings, but also of machines." The AI revolution is already happening.

Although the generated text may seem coherent in this case, it's important to note that with greedy search, the results can still be somewhat unpredictable and may not always produce the most fluent or meaningful sentences. For instance, the phrase "We're going to have to see how we do it in a way that's not just a matter of human beings, but also of machines" is somewhat unclear, and the opening sentence "In a world where AI has become ubiquitous, it's hard to imagine how it will ever be able to replace humans" presents a contradiction. These examples demonstrate that while the model can generate text that adheres to general grammatical rules, it might struggle with maintaining a consistent and coherent narrative.

As we progress through this chapter, you'll gain a deeper understanding of the limitations and strengths of various approaches. This will allow us to better understand the complexity of text generation and the ingenuity of the methods used to address it. As we look deeper into these decoding and sampling methods in the upcoming sections, you'll gain further insights into why LLMs are capable of producing such coherent and engaging text.

6.3.3 Beam search decoding for text generation

Beam search like many algorithms in computer science, finds its roots in graph theory. Specifically, it can be thought of as a pruned version of the classic graph traversal algorithm, breadth-first search (BFS).

BFS is a strategy for traversing or searching tree or graph data structures. It begins at the root in tree-based structures, or at an arbitrarily chosen node in graph-based structures, and explores all of the neighbor nodes at the current level before moving on to nodes at the next depth level. BFS is a cornerstone of computer science due to its robustness and versatility, finding applications in many fields such as network routing protocols or peer-to-peer networks.

Like with greedy search, let's conceptualize the algorithm with the maze analogy. With beam search, you would always consider a fixed number of the most promising paths at each decision point. This would be like standing at a junction in the maze and considering a few paths that seem most promising.

Now, if we imagine our sentence as a graph where each word leads to all other possible next words, we get a tree-like structure with the first word at the root and subsequent words forming the branches. However, due to the vast number of words in a language, this would result in an incredibly wide tree. This is where beam search comes in. It prunes the tree, or in other words, narrows the search space by only considering a set number of probable sequences at each step. Such a simplified graph with two beams is shown in figure 6.5.

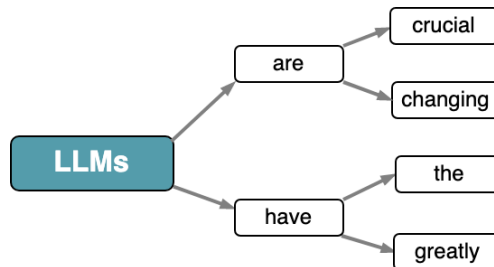


Figure 6.5 This illustration provides a simplified example of beam search in action, focusing on a scenario with two beams (i.e., beam width equals two). We start by initializing the process with "LLMs" as the first word. From here, the beam search algorithm evaluates the two most probable next words.

In mathematical terms, at each time step $t - 1$, we have a set of B most likely sequences of words, denoted as Y_{t-1} . This set includes sequences $(y_{1,[t-1]}, \dots, y_{B,[t-1]})$, where each $y_{b,[t-1]}$ is a sequence of $t - 1$ words.

At the next sequence step, the algorithm expands each sequence by one possible token from the vocabulary V . This results in a set of potential new sequences, Y_t , which is the Cartesian product of Y_{t-1} and V , meaning that we pair each sequence from Y_{t-1} with every possible token from V .

Beam search then evaluates the likelihood of all these new sequences and selects the B sequences with the highest probabilities to form the new Y_t , under the condition that the sequences are unique within the current set. This selection process continues iteratively until each sequence has reached a predefined maximum length T , or a termination condition such as the end-of-sequence token is encountered. The final output of the beam search is the sequence with the highest overall probability.

While this approach allows multiple sequences to be explored in parallel, it's important to note that the output often tends to consist of minor perturbations of a single sequence. This is where trade-offs and tweaks can be made to promote more diversity in the output, forming the basis of methods like diverse beam search.

To understand how this effect influences the search, we could extend our maze analogy to illustrate this efficiency. Imagine that each path in the maze is a potential sequence of words, and the exit represents the most meaningful and coherent sequence. When you use greedy search, you might reach a dead-end faster but might miss the best exit, which is the highest-quality output. On the other hand, beam search, by keeping multiple promising paths open, is like exploring several potentially successful routes at once. While this method may take slightly longer than greedy search due to its breadth, it can lead you to a more

suitable exit, i.e., a more high-quality output. Consequently, it manages to strike a balance between computational efficiency and output quality.

Let's put this into code with listing 6.2, a simple, yet illustrative coding example to better understand this concept and show how beam search can generate more varied and high-quality text than greedy search.

Listing 6.2 Beam search implementation in `model.generate`

```
❶ Instantiate the model and tokenizer
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

❷ Define an input prompt
prompt = "In a world where AI has become ubiquitous,"

❸ Encode the input prompt and prepare it for the model
input_ids = tokenizer.encode(prompt, return_tensors='pt')

❹ Generate text output with beam search setting
beam_outputs = model.generate(
    input_ids,
    max_length=100,
    num_beams=5, ❺ Number of beams to keep
    no_repeat_ngram_size=2,
    num_return_sequences=5, ❺ How many outputs to generate
    early_stopping=True,
    temperature=1.5, # increase the temperature
    top_k=50, # consider only top 50 tokens at each step
)
```

This results then in the following output:

OUTPUT 1 In a world where AI has become ubiquitous, it's hard to imagine a better time to be a part of it than right now. "I think we're going to see a lot more of that in the next few years," he said.

OUTPUT 2 In a world where AI has become ubiquitous, it's hard to imagine a better time to be a part of it than right now. "I think we're going to see a lot more of that in the next few years," he says.

OUTPUT 3 In a world where AI has become ubiquitous, it's hard to imagine a better time to be a part of it than right now. "I think we're going to see a lot more of that in the next few years," he said. "It's a very exciting time."

OUTPUT 4 In a world where AI has become ubiquitous, it's hard to imagine a better time to be a part of it than right now. "I think we're going to see a lot more of that in the next few years," he said. "It's a very exciting time for AI."

OUTPUT 5 In a world where AI has become ubiquitous, it's hard to imagine a better time to be a part of it than right now. "I think we're going to see a lot more of that in the next few years," he said. "It's a very exciting time for AI, and I think that's what's driving it."

Let's examine these outputs more closely to better understand the variations introduced by beam search:

- **Output 1:** This sentence features a direct quote attributed to an unspecified individual. The past tense "said" is used, indicating the quote was made in the past. The prediction is limited to the quote itself, not expanding upon it further.
- **Output 2:** The structure of this sentence is similar to output 1 but uses the present tense "says," implying that the quote is a current statement or opinion. The change in tense subtly alters the perceived immediacy and relevance of the statement.
- **Output 3:** This output expands upon output 1 by adding another sentence to the quote. The use of "said" suggests both sentences are part of the same past statement. The added sentence provides more details about the speaker's feelings towards the situation.
- **Output 4:** Like output 3, this output extends the quote, and it further specifies the source of excitement being related to "AI." This slight change introduces more specificity, focusing the speaker's enthusiasm on the field of AI.
- **Output 5:** This is the longest and most detailed output. It not only extends the quote but also introduces an additional layer of information - an opinion about what's driving the excitement. It suggests that the speaker is not just expressing excitement but also analyzing the situation.

These examples illustrate how the application of beam search can diversify the outputs of a model, introducing variety in tense, tone, and perspective. By considering multiple potential sequences, the model has the ability to generate different sentence structures and expressions, leading to a wider range of meaningful and diverse outputs. Now that we better understand how decoding methods influence the output of our generated text, let's move on to explore how different sampling methods affect a model's output in the following sections.

6.3.4 Top-k sampling for Text Generation

Top-k sampling, also known as k-sampling or top-k decoding, is a decoding strategy employed in language models for generating text. Unlike beam search, which keeps track of multiple hypotheses (beams), top-k sampling maintains a single hypothesis and expands it with a stochastic approach.

Let us again, apply our maze analogy to understand this in an illustrative way, before diving into the more technical details. With top-k sampling, from all the paths available, you consider the top k most promising ones. Your next move would then be chosen randomly

from this selection, leading to a good balance between exploration (checking new paths) and exploitation (following the most promising paths).

To elaborate on this analogy, top-k sampling selects the next word of a sequence from the top k most likely candidates given by the model. In the traditional way of generating text, each token is sampled from the full distribution of the model's vocabulary. However, in top-k sampling, the token is sampled only from the top k probabilities, which can lead to more meaningful and coherent text.

Let's imagine we have a sentence, and we are trying to predict the next word. Our model outputs probabilities for all the words in our vocabulary, then we pick the top k words with the highest probabilities and sample from this subset to select our next word. This approach allows for some randomness in the text generation process, and thus, can produce diverse outputs. This process is simplified illustrated in figure 6.6.

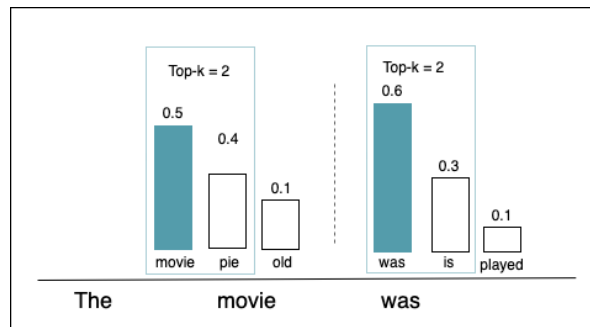


Figure 6.6 If we assume we want to model the probability of the next word to choose for our text generation, we would do: $P(w | \text{"The"})$, which will then be followed the probability $P(w | \text{"The", "movie"})$ and so on. The choice of the model will be limited to the 2 k-probable words, so, only movie or pie will be considered for the first probability and was or is for the next choice, respectively.

In terms of a code implementation, we can use the `model.generate()` function in Hugging Face's transformers library, with the `do_sample` and `top_k` parameters set to enable top-k sampling. Listing 6.3 shows an example of how to use top-k sampling with the GPT-2 model.

Listing 6.3 Top-k sampling implementation in `model.generate`

```
❶ Instantiate the model and tokenizer  
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')  
model = GPT2LMHeadModel.from_pretrained('gpt2')  
  
❷ Define an input prompt  
prompt = "In a world where AI has become ubiquitous,"  
  
❸ Encode the input prompt and prepare it for the model  
input_ids = tokenizer.encode(prompt, return_tensors='pt')  
  
❹ Generate text output with top-k sampling setting  
top_k_outputs = model.generate(  

```

```

input_ids,
max_length=100,
num_return_sequences=5,
do_sample=True, ⑤ Enable random sampling
top_k=50, ⑥ Number of top tokens to consider at each step
temperature=1.5 ⑦ Set the temperature
)

```

Using this method, we might generate this output:

OUTPUT In a world where AI has become ubiquitous, there are growing concerns both over the value the machine as a platform for science discovery, and over what the tech industry can expect in the future as a technological infrastructure for more sophisticated computation, it'd make sense, from an investment, to have an AI as the chief developer of machine learning. The key thing to recognize with this shift from a traditional software developer to the most mature tech developer is the need that AI is an integrated, flexible, robust

NOTE To maintain brevity, the full set of five model outputs will not be included in the text as in the previous example. Complete outputs for the methods discussed in this chapter are available in the accompanying Jupyter Notebooks.

The outputs generated by top-k sampling display a significant degree of variability and creativity. However, this method can lead to responses that might seem nonsensical or incomplete at times due to the inherent randomness of the approach. This is particularly noticeable when the token limit is reached before the narrative can reach a logical endpoint, causing sentences to be cut off.

In contrast, beam search is optimized for sentence coherence, resulting in more focused and usually shorter responses that adhere to common phrase structures. Therefore, it is less likely to hit the token limit in the middle of a sentence, and the outputs often appear more "complete" or "coherent". These differences highlight the trade-off between diversity and coherence that underlies these two different methods of text generation.

That said, while top-k sampling can produce diverse and fluent sentences, its inherent randomness being non-deterministic, as opposed to the deterministic beam search leads to less predictable outcomes. This unpredictability arises because each execution can yield different results, even with the same initial context. Furthermore, the quality and diversity of the output can significantly vary depending on the choice of k. Setting a higher k value will make the output more diverse but less focused, while a lower k value will make the output more focused but potentially less diverse. Therefore, fine-tuning the k value is essential to optimize the balance between diversity and focus in the generated text.

Now, having learned about both beam search and top-k sampling, we can see that both strategies have their strengths and weaknesses, and the choice between them will largely depend on the specific requirements of your application. In the next sections, we'll explore more sampling strategies and compare their performances.

6.3.5 Nucleus sampling for text generation

Nucleus sampling, also known as top-p sampling, dynamically selects the smallest possible set of words whose cumulative probability exceeds a predetermined threshold p , offering a more adaptive approach than top-k sampling. While top-k sampling always chooses from a fixed number of the most probable words, nucleus sampling's set size varies. If the model is confident, it narrows down to a few highly probable words, but it expands the set when less certain, enhancing creativity without compromising coherence. The threshold p is typically set between 0.8 and 0.95, fine-tuning the balance between variety and predictability in the generated text, image 6.7 visualizes this concept.

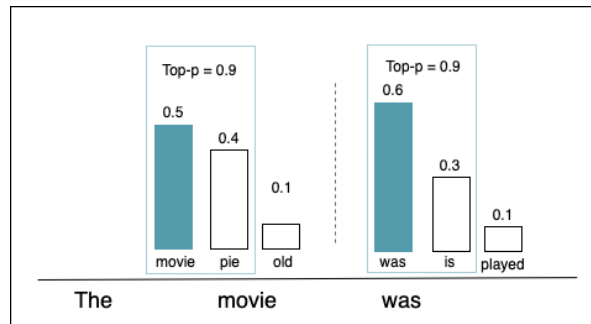


Figure 6.7 If we assume we want to model the probability of the next word to be chosen with top-p (nucleus) sampling, we would again consider the following: $P(w \mid \text{"The"})$, which will then be followed the probability $P(w \mid \text{"The", "movie"})$ and so on. But now the choice of the model will be limited to the threshold of cumulative probability of 0.9 for the words which can be selected, so again, only movie or pie will be considered for the first probability and was or is for the next choice, respectively.

To visualize this using the maze analogy, imagine that with nucleus sampling, you aren't restricted to a fixed number of paths. Instead, you have a dynamic "pool" of promising routes. The size of this pool shifts according to the potential of current paths, sometimes only a small number of straightforward paths and other times a broader array of routes when more exploration is needed.

To illustrate nucleus sampling with code, we again use Hugging Face's transformers library, specifically the `model.generate()` function. The `do_sample` and `top_p` parameters are set to enable nucleus sampling. Listing 6.4 shows an example of how to use nucleus sampling with the GPT-2 model.

Listing 6.4 Nucleus sampling implementation in `model.generate`

❶ **Instantiate the model and tokenizer**

```
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')
```

❷ **Define an input prompt**

```
prompt = "In a world where AI has become ubiquitous,"
```

❸ **Encode the input prompt and prepare it for the model**

```
input_ids = tokenizer.encode(prompt, return_tensors='pt')
```

```
④ Generate text output with nucleus sampling setting  
output = model.generate(  
    input_ids,  
    max_length=100,  
    do_sample=True, ⑤ Enable random sampling  
    top_p=0.92, ⑥ Threshold for cumulative probability  
    top_k=0 ⑦ No limit on the number of top tokens considered  
)
```

Running the above code, we might generate sentences like this one:

OUTPUT In a world where AI has become ubiquitous, this is not the case. The most recent example is Google's own self-driving car, which, at the time of this writing, was able to successfully navigate a highway in Los Angeles without any human intervention. Now that its self-driving car is on the road, it's almost certain that autonomous driving will be the next major industry. That's bad news for the American auto industry, who have been trying to make it safe

With nucleus sampling, the generated text is expected to be diverse, yet more controlled compared to pure random sampling or top-k sampling. This is due to the dynamic adjustment of the probability threshold that includes a varying number of tokens at each step. Depending on the threshold, the output can be more focused or more diverse, offering a flexible trade-off between the two.

In conclusion, nucleus sampling is a powerful method for generating diverse and high-quality text. Depending on the task at hand and the desired level of randomness and coherence, it can be a great tool in the toolkit of a natural language processing practitioner. In the next sections, we will continue to explore more advanced techniques.

6.3.6 Temperature Sampling for Text Generation

Temperature sampling is another text generation technique that controls the randomness of predictions by scaling the logits before applying softmax.

In temperature sampling, a parameter called the "temperature" is used to control the randomness of the predictions. When the temperature is close to 0, it tends towards greedy decoding, and the model will generate the most likely next word. As the temperature approaches infinity, the model's output approximates random sampling, choosing words from the vocabulary with equal likelihood. At a temperature of 1, the model uses the predicted probabilities without any modification, as shown in image 6.8.

This approach makes it possible to fine-tune the balance between exploiting the model's knowledge (i.e., selecting the most probable words) and exploring different possibilities (i.e., generating less likely words).

Let's illustrate temperature sampling in code. We can adjust the temperature parameter in the `model.generate()` function from the transformers library, as shown in Listing 6.5.

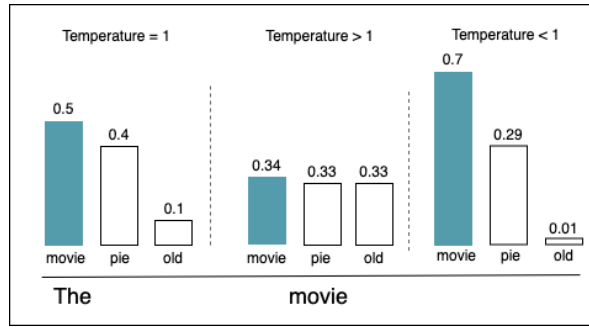


Figure 6.8 With temperature sampling, if we chose the temperature to be 1, the probabilities won't change. However, if we set it to be greater 1, let's say 2, it will equalize the probabilities more, and if we have the temperature smaller than 1 the probabilities will be more extreme.

Listing 6.5 Temperature sampling implementation in model.generate

```

❶ Instantiate the model and tokenizer
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

❷ Define an input prompt
prompt = "In a world where AI has become ubiquitous,"

❸ Encode the input prompt and prepare it for the model
input_ids = tokenizer.encode(prompt, return_tensors='pt')

❹ Generate text output with temperature sampling setting
output = model.generate(
    input_ids,
    max_length=100,
    do_sample=True, ❺ Enable random sampling
    temperature=0.7 ❻ Control randomness with the temperature parameter
)

```

This is the output from the temperature sampling:

OUTPUT 1 In a world where AI has become ubiquitous, it's easy to see the potential for it to become a problem. "It's very important to understand the potential for AI to change, to develop," says Kishore Kumar, president of the AI Foundation. "The AI Foundation is making a conscious effort to make sure that people come out and say, 'We should all be working with this AI.' We have no idea how it will come to be. "We have to

By adjusting the temperature, we can fine-tune the level of randomness in our generated text. A higher temperature leads to more diverse outputs, while a lower temperature results in more deterministic and focused outputs.

Addressing truncated text in non-deterministic text generation

When using the *model.generate* function from Hugging Face for text generation, particularly with non-deterministic or stochastic methods (e.g., temperature sampling, nucleus sampling), a challenge often emerges: the generated text can appear abruptly truncated. Contrasting, deterministic methods like greedy decoding offer consistent results without the abrupt cut-off problem, while methods such as beam search introduce some variability but typically produce more coherent endings.

In the field of text generation, non-deterministic methods introduce a degree of randomness, allowing the model to produce varied, and often more creative, outputs for the same input on different runs. This approach contrasts with deterministic methods, which provide more predictable results.

The inherent randomness, while enabling creative outputs, often doesn't account for the structural or logical endings we humans expect, leading to outputs that may seem cut off or incomplete.

To better manage the termination of these outputs, one effective strategy involves refining the model's generated content, as demonstrated in listing 6.6. The end-of-sentence (EOS) token can be especially beneficial in this context, offering more control over where the generated content concludes.

Listing 6.6 Prevent sentence cut off

```
❶ Check if EOS token exists in the output and trim up to that
if '<|endoftext|>' in generated_text:
    generated_text = generated_text.split('<|endoftext|>')[0].strip()
else:
❷ Alternatively, trim up to the last period for coherence
    generated_text = '. '.join(generated_text.split('. ')[::-1]) + '.'
```

The challenge of truncated outputs can still persist, underscoring the intricate balance between creativity and coherence in AI-generated text.

As with all the other methods we've discussed, the choice between these approaches depends on the specific requirements of the task at hand. Temperature sampling offers a flexible means of controlling the balance between diversity and focus in the generated text.

6.4 Challenges in transformer-based text generation

Generating text with transformers comes with its own set of challenges, here we'll look at some of the prominent challenges faced by these often billion-parameter models.

A significant challenge in working with large language models is their necessity to be trained on vast amounts of internet data. These large-scale models have the capacity to memorize extensive information, which consequently raises a potential risk. There is a possibility that downstream tasks might be inadvertently influenced if their test or develop-

ment datasets are encountered during the pre-training phase. Given these complexities, for certain tasks, it may be more suitable to opt for smaller, "classical" Transformer models and use the "traditional" approach of fine-tuning them on a specific downstream task. This approach often requires fewer resources and may yield comparable or even better results. To help guide your decision-making when choosing between large, billion-parameter models and smaller ones, we will delve into these and other challenges in this section, beginning first with a brief overview of the most prevalent issues.

- Creating quality training data: Designing unbiased, comprehensive, and diverse training datasets is a key challenge. Avoiding overlap between training and testing sets to ensure model generalization is crucial, especially in the billion-parameter models.
- Hallucination: Text generation models sometimes generate content that seems plausible but isn't grounded in reality, a phenomenon known as "hallucination".
- Model interpretability: Deep learning models are often seen as "black boxes" due to their complexity, and developing tools and techniques to understand their inner workings is a significant challenge.
- Bias in AI: AI systems can inherit and amplify biases present in their training data, leading to unfair outcomes.

Now, let's delve deeper into the first two points in the list: creating quality training data and hallucination. The final two points in our list, model interpretability and bias in AI, will be comprehensively discussed in chapter 10.

6.4.1 High quality training data

The cornerstone of training Large Language Models (LLMs) with billions of parameters, like GPT-4, LLaMA, RedPajama and Falcon, is the utilization of high-quality data. Conventionally, such models have been trained on a blend of carefully selected high-quality corpora, including content from books and technical papers, and filtered web data. This curation process, aimed at ensuring wide-ranging zero-shot generalization abilities, has been a key determinant of effective model performance. However, as model sizes grow and the requirement for training on an expanding number of tokens escalates, this process confronts challenges such as content duplication and inherent bias.

A common resource for such massive model pre-training is CommonCrawl, a publicly available web scrape that amasses petabytes of data spanning more than a decade. Processing such a diverse dataset presents unique challenges, including the need to eliminate low-quality or undesirable content. This typically involves intricate data pipelines encompassing language identification, filtering rules and heuristics, ML-based quality filtering, and deduplication. However, a careful balance must be struck here: while stringent filtering is crucial to ensure the data's quality, excessive or improperly calibrated filtering can inadvertently remove content that represents minority perspectives. Such an imbalance in the data can lead to biases, as the model may underrepresent or misrepresent these groups.

Given the complexities associated with managing training and testing data overlap, curating training data for billion-parameter models becomes a critical task. Deduplication plays

a significant role in improving language models by mitigating issues such as memorization and repeated data, which can compromise model quality as the parameter count increases.

In this context, the recent publication of RefinedWeb, (7), which uses a MacroData Refinement (MDR) pipeline to enhance the quality of web data, is a significant development. Figure 6.9 illustrates the pipeline.

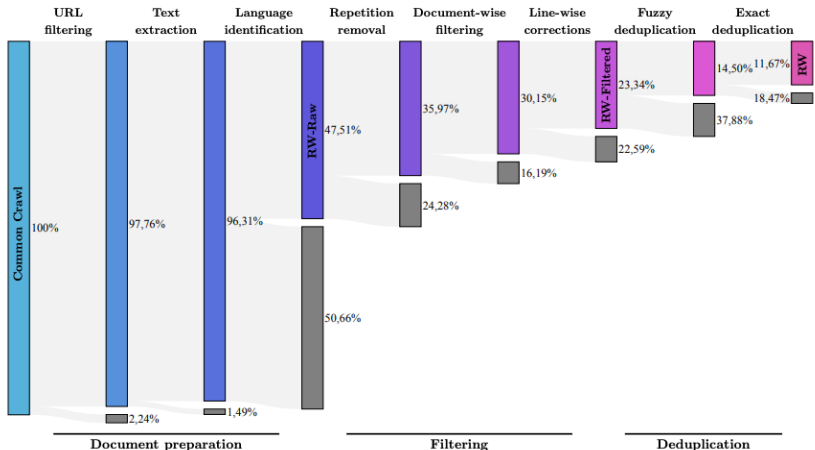


Figure 6.9 Through the stages of Macrodata Refinement, approximately 90% of CommonCrawl’s initial documents are discarded. Half are dropped for being non-English, around a quarter for poor quality, and 12% for duplication. The reported removal and retention rates are measured first in documents, then in tokens. Image is taken from the paper: “The RefinedWeb Dataset for Falcon LLM”.

As the illustrations shows, MDR uses strict filtering and deduplication to prepare the web data from CommonCrawl on a large scale. The principles guiding MDR are:

- **Emphasis on Scale:** MDR aims to generate datasets suitable for training models with 40-200B parameters, necessitating trillions of tokens, (4). Consequently, labor-intensive human curation processes are set aside in favor of focusing on Common-Crawl.
- **Rigorous Deduplication:** The pipeline incorporates stringent deduplication, employing both exact and fuzzy methods inspired by the work of (5)).
- **Neutral Filtering:** To avoid introducing further undesirable biases, (3), (11), into the model, ML-based filtering is applied solely to language identification. Simple rules, heuristics, and URL filtering. The URL filtering targets fraudulent and/or adult websites.

This facilitated the creation of the RefinedWeb dataset, a pretraining dataset made up entirely of high-quality web data.

Therefore, considering these challenges, it is essential to understand the data on which a model was trained. This knowledge can provide invaluable insights into the model’s potential biases, informing on the appropriate methods on how to address these biases,

whether through reinforcement learning from human feedback, prompt engineering, or fine-tuning.

6.4.2 Hallucination

Hallucination, as it relates to LLMs or machine learning models in general, is a term that describes a phenomenon where the model generates outputs that don't align with the facts presented in the input, or aren't grounded in the real world. Essentially, the model is "making things up". This can happen in various tasks, including text summarization, translation, and text generation. Even state-of-the-art models are prone to producing invented facts, which can be particularly problematic in domains where the models need to perform multiple steps of reasoning, like coding or solving mathematical equations. Since one simple error is enough to populate throughout the whole solution.

We can distinguish between two hallucinations: open-domain hallucination and closed-domain hallucination.

In this context, "open-domain hallucination" generally refers to the model generating statements or facts that aren't true or didn't exist in its training data. It may fabricate events, people, or concepts that simply don't exist in reality. This is considered "open-domain" because it could potentially involve any topic, concept, or subject area.

On the other hand, "closed-domain hallucination" refers to the model generating content that is inconsistent with the provided prompt or input, even though it may be internally coherent. It's "closed" to the specific domain of the conversation or interaction. The model isn't adhering to the context it's been given and instead is going off on a tangent or making up information that's not relevant or accurate to the specific prompt.

As closed-domain hallucinations typically involve the model not adhering to the specific instructions or prompt. This is an area where techniques such as reinforcement learning from human feedback can be particularly useful. By providing the model with feedback on its outputs, the model can learn to better align its responses with the provided context and adhere more closely to the prompt. Similarly, creating and training models on a dataset that includes demonstrations of instruction following behavior, like the case with InstructGPT or Dolly from sections 6.2.2 and 6.2.7, can help the model to better understand and follow specific prompts, thereby reducing the likelihood of closed-domain hallucinations.

However, addressing open-domain hallucinations can be more challenging, as these hallucinations can involve a broader range of topics and the model making things up that don't exist in its training data or in reality.

For both hallucination problems there is still ongoing research in this area on how to address it, a recent paper called "Let's Verify Step by Step", tries to address hallucinations by training a reward model on a dataset of 800,000 step-level human feedback labels to provide the model with process supervision while solving a task.

Given the complexities and challenges in mitigating hallucinations in LLMs, understanding the limitations of these models in generating text is essential. That said, it is crucial to handle the generated content with caution and thoroughly review and fact-check the outputs, especially when they're being used for tasks requiring factual accuracy and logical consistency. Nonetheless, ongoing research in this area continues to provide new methods

and approaches for addressing these issues which will most likely help to improve the reliability and accuracy of text generated by LLMs.

6.5 Summary

- Text generation has evolved from the 1960s rule-based chatbots like ELIZA to today's advanced transformer models like GPT-4 and LLaMA 2, marking a shift from simple language processing to deep language understanding and context-aware responses.
- In the continuously evolving domain of text generation, a plethora of LLMs has emerged, each showcasing distinctive features and abilities. Models like BERT, GPT-1, GPT-2, GPT-3, InstructGPT, GPT-NeoX-20B, LLaMA, RedPajama, Alpaca, Dolly, and Falcon have left a lasting mark in the field, proving their ability in generating coherent and contextually relevant text.
 - **BERT**: A transformer-based model with **110 million parameters**, renowned for bidirectional language modeling. Available in cased and uncased versions.
 - **GPT-1**: An early transformer model with **117 million parameters** featuring a decoder-only structure, emphasizing solely on preceding words during prediction.
 - **GPT-2**: A successor to GPT-1 with **1.5 billion parameters**, introducing "zero-shot" learning for enhanced context-aware text generation.
 - **GPT-3**: A successor to GPT-2 with **175 billion parameters**, renowned for generating creative content and executing diverse NLP tasks.
 - **InstructGPT**: A GPT variant fine-tuned for instruction following, available in sizes of **1.3B, 6B, and 175B parameters**.
 - **GPT-NeoX-20B**: Mirrors GPT-3's architecture but introduces several innovations and is trained on the Pile dataset with **20B parameters**.
 - **LLaMA**: A collection of models ranging from **7 billion to 70 billion parameters**, notable for their performance on numerous benchmarks.
 - **RedPajama**: An open-source initiative aimed at bridging the gap between commercial and open-source models, available in **3 billion and 7 billion parameters**.
 - **Alpaca**: A fine-tuned model from Stanford based on Meta's LLaMA **7B**, designed for instruction-following tasks.
 - **Dolly**: An open-source model by Databricks grounded in EleutherAI's Pythia model family, boasting **12 billion parameters**.
 - **Falcon**: Developed by TII, a decoder-only model trained on an enriched corpus and available in sizes of **6.7 billion and 40 billion parameters**.
- The *creativity* of a text generation models output is adjustable via different search and sampling methods.

Bibliography

- [1] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. Gpt-neox-20b: An open-source autoregressive language model, 2022. [arXiv:2204.06745](https://arxiv.org/abs/2204.06745). 153
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020. URL: <https://arxiv.org/abs/2005.14165>, [arXiv:2005.14165](https://arxiv.org/abs/2005.14165). 153
- [3] Jesse Dodge, Maarten Sap, Ana Marasovi, William Agnew, Gabriel Ilharco, Dirk Groeneveld, Margaret Mitchell, and Matt Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus, 2021. [arXiv:2104.08758](https://arxiv.org/abs/2104.08758). 182
- [4] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. [arXiv:2203.15556](https://arxiv.org/abs/2203.15556). 182
- [5] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better, 2022. [arXiv:2107.06499](https://arxiv.org/abs/2107.06499). 182
- [6] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. [arXiv:2203.02155](https://arxiv.org/abs/2203.02155). 153
- [7] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only, 2023. [arXiv:2306.01116](https://arxiv.org/abs/2306.01116). 182
- [8] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018. 152

- [9] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. 153
- [10] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. [arXiv:2302.13971](https://arxiv.org/abs/2302.13971). 153
- [11] Johannes Welbl, Amelia Glaese, Jonathan Uesato, Sumanth Dathathri, John Mellor, Lisa Anne Hendricks, Kirsty Anderson, Pushmeet Kohli, Ben Coppin, and Po-Sen Huang. Challenges in detoxifying language models, 2021. [arXiv:2109.07445](https://arxiv.org/abs/2109.07445). 182

Controlling generated text

This chapter covers

- Reinforcement learning from human feedback
- Direct Preference Optimization
- Prompt engineering

This chapter builds upon the previous one on text generation, where we learned about some of the fundamental and newer Large Language Model architectures currently available. We also looked into how to influence the creativity of our LLM using various sampling and decoding strategies. In this chapter, we will delve into how you can nudge your LLM to produce your preferred text output, exploring two distinct approaches.

The first approach involves updating the weights of our model through methods like reinforcement learning or Direct Preference Optimization. These techniques allow for more direct control over the model's behavior by adjusting its text output based on specific performance feedback.

The second approach is known as in-context learning. Here, we guide the model on how to "behave" using prompt examples, without altering the model's underlying architecture. This technique is particularly powerful when working with an API, where direct access to modify the model's parameters is not feasible.

Both methods offer unique advantages and applications, providing valuable tools for tailoring LLM outputs to specific needs and contexts.

7.1 *Improving LLMs with reinforcement learning from human feedback*

LLMs such as Llama 2 gain significantly in being safe and helpful, when they are augmented with SFT from human preferences. Training large language models with reinforcement learning from human feedback (RLHF) stands out as a predominant method for this. This section dives into the nitty-gritty of this approach. We'll kick off with the foundation: Markov decision processes, and then, after we understand all the basics, we'll look into reinforcement learning using Proximal Policy Optimization (PPO).

7.1.1 *From Markov decision processes to reinforcement learning*

A Markov decision process (MDP) is a stochastic control process. It provides a mathematical framework for modeling decision-making. The MDP consists of five key components:

- Agent
- States
- Actions
- Rewards
- Policy

The agent acts within an environment, transitioning between various states. MDP outlines how specific states and corresponding actions guide the agent to subsequent states. The agent earns rewards based on its actions and the resulting state. In the MDP model, the policy indicates the agent's next action based on its present state. To make this more intuitive, let us consider an illustrative example.

Imagine taking your dog through a dog training obstacle course, teaching it tricks and helping it navigate challenges. This training scenario can be associated with the key components of an MDP:

- **Dog (Agent):** Takes decisions and performs actions.
- **Obstacle Course (Environment):** The setting in which the dog operates and faces challenges.
- **Obstacles (States):** Specific situations or challenges the dog encounters.
- **Actions:** Decisions the dog makes, such as jumping, crawling, or sidestepping obstacles.
- **Rewards:** Positive reinforcements, like treats, when the dog performs well, or gentle scolding for mistakes.

As the dog navigates the obstacle course, it learns from its interactions, aiming to maximize the number of treats it receives. Similarly, in an MDP, an agent learns to take actions in various states with the goal of maximizing its total rewards - this is often formalized using the "argmax" function. Over time, both the dog and the agent develop strategies (or policies) that optimize their outcomes.

Now, let's connect this analogy to RLHF in text generation. Imagine the dog (agent) is a large language model, and the parcour (environment) represents the space of possible

text outputs. Each obstacle (state) is a specific point in the text generation process, with the dog's actions equivalent to adding words or phrases. Rewards come from human feedback, guiding the LLM in generating desired outputs.

In the Proximal Policy Optimization (PPO) context for LLMs, we can compare the training process with how the dog adapts to the parcourse. Let's consider the obstacles. If we start with incredibly high obstacles, the dog might get discouraged and not attempt to jump at all. But if we begin with manageable hurdles and gradually raise them as the dog gets more trained and confident, the dog's "policy" or behavior strategy adjusts gradually. This mirrors the essence of PPO. The idea is to enhance the stability of the policy by ensuring we don't make overly drastic changes during training. The reasons for this are:

- Smaller, incremental updates are empirically known to converge better to an optimal solution.
- Making a dramatic change can lead to unfavorable outcomes in policy, which can take a long time to correct, if they can be corrected at all.

To solidify this conservative approach, PPO introduces the concept of "proximal" policy. In essence, it ensures that the newly updated policy doesn't deviate too far from the previous one. Think of it as guiding the dog on the parcourse in such a way that its new behaviors are close or *proximal* to what it previously learned. We achieve this by comparing the new and old policies using a specific measure, and if they diverge too much, we clip or adjust the new policy to ensure it stays within $[1 - \epsilon, 1 + \epsilon]$ depending on whether the advantage is positive or negative. This "proximity" ensures a consistent and stable progression in training, whether for a dog navigating obstacles or an LLM generating text. Hence the name *Proximal* Policy Optimization.

7.1.2 Improving models with human feedback and reinforcement learning

In this section, we will apply the previously learned theory training an LLM with RLHF. To accomplish this, we'll employ the trlX library, a distributed training framework emphasizing the fine-tuning of large language models with reinforcement learning. It supports the use of either a predefined reward function or a reward-labeled dataset. The library can fine-tune both causal and T5-based language models up to 20B parameters. For models larger than 20B, trlX offers NVIDIA NeMo-backed trainers that utilize efficient parallelism techniques. Furthermore, the library currently implements Proximal Policy Optimization and Implicit Language Q-Learning as reinforcement learning algorithms. For this demonstration, however, I'll use PPO and opt for the GPT-2 model to minimize computational overhead. For our training data, we'll use the Financial Phrasebank dataset from 5.4.1. Alternatively, you could substitute it with an instruction or prompt dataset to guide the model's behavior. Let us start with loading the model and the dataset as shown in listing 7.1.

Listing 7.1 Loading the model and dataset

❶ Load GPT-2

```
model = GPT2LMHeadModel.from_pretrained("gpt2")
```

② Load the Financial Phrasebank dataset

```
financial_dataset = load_dataset('financial_phrasebank', 'sentences_allagree')
```

NOTE The provided code is also compatible with other models. To use a different model, simply replace the model name. For example, to use Falcon-1B, you would write: `model = "tiiuae/falcon-rw-1b"`.

As the next step (listing 7.2), we will define our reward function. This function will simply reward longer and more diverse responses. Alternatively, you might employ a dataset consisting solely of negative news. This approach can be used to produce additional negative samples, particularly useful for datasets with an imbalance as we have seen in 5.4.

Listing 7.2 Defining the reward function

```
def reward_fn(output):
    length = len(output)
    diversity = len(set(output.split()))
    return length + 2 * diversity
```

In the following step, we will preprocess the dataset and generate the rewards.

Listing 7.3 Preprocess the dataset

```
samples = [example["sentence"] for example in financial_dataset["train"]]
rewards = [reward_fn(sample) for sample in samples]
```

After preprocessing our data we are ready to train our model with RLHF.

Listing 7.4 Preprocess the dataset

① Define the config

```
default_config = default_ilql_config().to_dict()
default_config['train']['tracker'] = None
default_config['train']['batch_size'] = 16
default_config['train']['epochs'] = 20
config = TRLConfig.update(default_config, )
```

① Train the model

```
trainer = trlx.train(
    model,
    samples=samples,
    rewards=rewards,
    eval_prompts=[
        "The S&P has shown",
        "The market trends indicate",
        "The economic indicators for the quarter are",
        "According to recent financial reports"
    ] * 20,
    config=config,
```

)

To use the model with a prompt we do the following:

Listing 7.5 Using the trained model

```
input_str = 'The market trends indicate,'
trainer_output = trainer.generate_eval(
    **trainer.tokenizer(input_str, return_tensors='pt')) [0]
print(trainer.tokenizer.decode(trainer_output))
```

This code will generate for instance the following text:

OUTPUT The market trends indicate, based on the current market situation, that a continued expansion in the segments's volume would be expected.

In this section of this chapter we looked at how straightforward the integration of RLHF with the trlX library is. I encourage you to experiment with the provided code in the books repository for this chapter and change the dataset and the model to compare the results. In Chapter 9, we will dive further into this subject, exploring the capabilities of QLoRA and other quantization methods to reduce memory usage. These advanced methods offers the advantage of minimizing memory consumption, making it feasible to fine-tune a model with 65B parameters on a single 48GB GPU. Impressively, these methods achieve this without compromising the performance of the fine-tuning tasks.

7.2 Aligning LLMs with Direct Preference Optimization

In the previous section, we explored RLHF, a method used for refining the responses of chatbot systems like ChatGPT, Alpaca, and StableVicuna. However, RLHF is a complex method that requires first fitting a reward model reflecting human preferences and then fine-tuning the LLM using RL to maximize the policy. Direct Preference Optimization (DPO) offers a streamlined alternative by directly optimizing the LLM's policy. DPO employs the same objective as RLHF, where we aim to maximize the reward with a Kullback-Leibler divergence constraint. This statistical measure quantifies the difference between one probability distribution and a second, reference probability distribution. It's often used as a constraint in various optimization tasks to ensure that solutions do not deviate excessively from a desired distribution. Leveraging exactly this constraint, DPO now just updates the relative log probability of preferred to dis-preferred prompt responses, effectively transforming the challenge into a classification objective. In simpler terms, we now have a classification task at hand. Which involves the following steps:

- Supervised fine-tuning step (same as for RLHF)
- Annotating data with preference labels (same as for RLHF)
- DPO-Step

- Prompt which consists of the context prompt which is given to a model at inference time for text generation.
- Preferred generated response to the corresponding prompt.
- Response which is not preferred or should not be the sampled response with respect to the given prompt

As you can see, the DPO formulation bypasses the reward modeling step and directly optimizes the language model on the preference data. Image 7.1 shows a simplified comparison of RLHF and DPO.

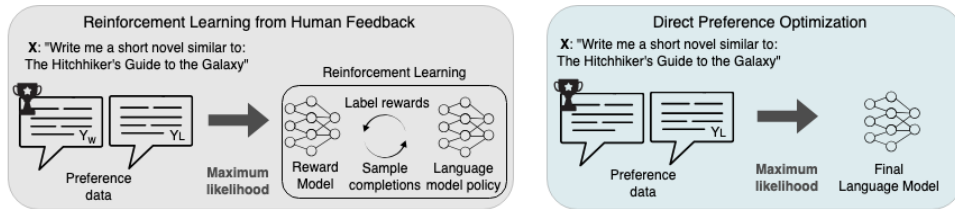


Figure 7.1 A simplified comparison of RLHF and DPO and how DPO directly optimizes for the policy without an explicit reward function.

After having learned about the theoretical side of DPO, let us now turn to the practical side of it. For this we will use the transformer, peft and trl library from Hugging Face and an open source library called unsloth, which helps us speeding up the training of our LLM. Further, we will log our trainings to Weights and Biases to track our experiments. We will follow the steps I outlined earlier:

- SFT step
- DPO-step to train the model with the preference labeled prompts.

For the first step, the SFT, we load, as usual, the model we want to train, as shown in listing 7.6

Listing 7.6 Loading the model for SFT

```

1 load the model and tokenizer
model, model_tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/mistral-7b-bnb-4bit",
    2 Define the the max sequence length
    max_seq_length = 4096,
    3 Set type detection to auto
    dtype = None,
    4 Use 4bit quantization to reduce memory usage.
    load_in_4bit = True
)

```

After having loaded the model into our Notebook, we have to add the LoRA adapters (listing 7.7), in order to be able to just update a portion of the model.

Listing 7.7 Add LoRA adapters for model

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 64,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 64,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = True,
    random_state = 42,
    max_seq_length = 4096,
)
```

Now that we have set this part up, we need to prepare our data. We will use the cleaned Alpaca dataset from Hugging Face (<https://huggingface.co/datasets/yahma/alpaca-cleaned>) for this. We have to prepare the a prompt template as demonstrated in listing 7.8 and load and format our dataset (7.10).

Listing 7.8 Create the prompt instructions

```
alpaca_template = """Write a response that completes the task from below,
following the instruction.

### Instruction:

### Input:

### Response:
"""

def prepare_prompts(data):
    texts = [alpaca_template.format(inst, inp, out) for inst, inp, out in
              zip(data["instruction"], data["input"], data["output"])]
    return {"text": texts}
```

Listing 7.9 Load and prepare Alpaca data

```
alpaca_dataset = load_dataset("yahma/alpaca-cleaned", split="train")
formatted_dataset = alpaca_dataset.map(prepare_prompts, batched=True)
```

Now we train our LLM as we have already seen in the previous chapters by initializing the training arguments for the Trainer Class:

Listing 7.10 Initialize training arguments

```
training_args = TrainingArguments(
    per_device_train_batch_size = 2,
```

```

        gradient_accumulation_steps = 4,
        warmup_steps = 5,
        max_steps = 60,
        learning_rate = 2e-4,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 1,
        report_to = "wandb",
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "cosine",
        seed = 42,
        output_dir = "outputs"
    )

```

And then we feed the arguments into our SFTTrainer class which we obtain from the trl library:

Listing 7.11 Train model with SFTTrainer

① Initialize the SFTTrainer

```

trainer = SFTTrainer(
    model = model,
    train_dataset = formatted_dataset,
    dataset_text_field = "text",
    max_seq_length = 4096,
    args = training_args
)

```

② Call the the trainer class and train the model

```

trainer_stats = trainer.train()

```

Because we used LoRA, which means we only update a small portion of the model, we can train the model with a T4 GPU in a Colab Notebook. This will take about 8-10 minutes to complete.

If we now want to take our newly trained model for a test-spin we can run the inference as shown in listing 7.12

Listing 7.12 Run inference on trained model

① Prepare the prompt

```

prompt = model_tokenizer(
    [
        alpaca_template.format(
            ② Instruction
            "What is the iconic symbol of freedom at the US east coast?",
            ③ Input
            " ",
            ④ Output
            " ",

```

```

    )
] * 1, return_tensors="pt").to("cuda")

5 Model's generation settings
generation_parameters =
6 Maximum number of new tokens to generate
    "max_new_tokens": 256,
7 Whether to use past key values for attention
    "use_cache": True
8 Generate outputs using the model and the specified generation parameters
outputs = model.generate(**prompt, **generation_parameters)
9 Decode the generated outputs
decoded_outputs = model_tokenizer.batch_decode(outputs, skip_special_tokens=True)

```

This will result in the following output:

Instruction:

What is the iconic symbol of freedom at the US east coast?

Input:

Response:

The Statue of Liberty is the iconic symbol of freedom at the US east coast. It is a colossal copper statue, designed by French sculptor Frédéric Auguste Bartholdi, and is located on Liberty Island in New York Harbor. The statue was a gift from France to the United States and was dedicated on October 28, 1886. The statue is a symbol of freedom, democracy, and the United States welcoming of immigrants. It has become an iconic symbol of the United States and is a popular tourist destination.

We save our model and continue with the next step, the DPO training. We load our previously trained model, in the same way as shown in listing 7.6 and add our LoRA adapters as shown in listing 7.7. Now for the DPO training, we use the UltraFeedback Binarized dataset (https://huggingface.co/datasets/HuggingFaceH4/ultrafeedback_binarized). And because we want to do DPO, we only use the preference modelling (prefs) samples of the dataset to train our model. How you can accomplish this is shown in listing 7.13 and listing 7.14.

Listing 7.13 Function to load the dataset with desired split

```

def get_sampled_datasets(dataset_name, splits, fraction, shuffle=True):
    raw_datasets = DatasetDict()
    for split in splits:
        dataset = load_dataset(dataset_name, split=split)
        if shuffle:
            dataset = dataset.shuffle(seed=42)
        sampled_dataset = dataset.select(range(int(fraction * len(dataset))))
        raw_datasets[split] = sampled_dataset
    return raw_datasets

```

Listing 7.14 Load the dataset for DPO training

❶ **Dataset name**

```
dataset_name = "HuggingFaceH4/ultrafeedback_binarized"
```

❷ **Only use the preference modelling samples**

```
splits = ["train_prefs", "test_prefs"] (prefs) splits of the dataset
```

❸ **The fraction of the dataset to sample**

```
fraction = 0.01
```

❹ **Get sampled datasets**

```
raw_datasets = get_sampled_datasets(dataset_name, splits, fraction)
```

Since we are using DPO, our dataset need to have the columns *chosen*, *rejected*, *prompt*. These are the needed columns for the DPOTrainer class to run properly, if we don't name the columns in with that naming convention, we will run into problems. To prepare the dataset according to the naming convention we adjust our dataset as shown in listing 7.15.

Listing 7.15 Function to prepare training data

```
def apply_chat_template(example, tokenizer, assistant_prefix=""):
```

```
    def _strip_prefix(s, pattern):
```

```
        return re.sub(f"^{re.escape(pattern)} ", "", s)
```

```
    def _concatenate_messages(messages):
```

```
        return ' '.join(msg['content'] for msg in messages)
```

```
    if all(key in example for key in ('chosen', 'rejected')):
```

❶ **Process chosen field**

```
        if isinstance(example['chosen'], list):
```

```
            example['chosen'] = _strip_prefix(
```

```
                _concatenate_messages(example['chosen']
```

```
                [1:]), assistant_prefix)
```

❷ **Process rejected field**

```
        if isinstance(example['rejected'], list):
```

```
            example['rejected'] = _strip_prefix(
```

```
                _concatenate_messages(example['rejected'] [1:]),
```

```
                assistant_prefix)
```

❸ **Process prompt field**

```
        if 'prompt' in example and isinstance(example['prompt'], list):
```

```
            example['prompt'] = _strip_prefix(_concatenate_messages(example['prompt']),
```

```
            assistant_prefix)
```

```
    return example
```

We then map and transform the dataset as shown in listing 7.16.

Listing 7.16 Prepare training data

```
transformed_datasets = raw_datasets.map(
    lambda example: apply_chat_template(example, tokenizer),
    remove_columns=[col for col in raw_datasets["train_prefs"].column_names
                    if col not in ['chosen', 'rejected', 'prompt']],
    desc="Formatting prompt template",
)
```

Then again, we prepare our training arguments as shown in listing 7.17.

Listing 7.17 Initialize training arguments for DPO training

```
training_args = TrainingArguments(
    per_device_train_batch_size = 2,
    gradient_accumulation_steps = 4,
    warmup_ratio = 0.1,
    num_train_epochs = 2,
    learning_rate = 5e-6,
    fp16 = not torch.cuda.is_bf16_supported(),
    bf16 = torch.cuda.is_bf16_supported(),
    logging_steps = 1,
    report_to = "wandb",
    optim = "adamw_8bit",
    weight_decay = 0.0,
    lr_scheduler_type = "cosine",
    seed = 42,
    output_dir = "outputs",
)
```

Now we use the DPOTrainer class from the trl library to train our model:

Listing 7.18 Initialize training arguments for DPO training

```
dpo_trainer = DPOTrainer(
    model=model,
    ref_model=None,
    args=training_args,
    beta=0.1,
    train_dataset=transformed_datasets["train_prefs"],
    eval_dataset=transformed_datasets["test_prefs"],
    tokenizer=tokenizer,
    max_length=1024,
    max_prompt_length=512,
)

# Continue with your training process
longcomment/1/Train the model with DPO/
dpo_trainer.train()
```

You can then save your model either locally with:

Listing 7.19 Save model locally

```
model.save_pretrained("your_model_name")
```

Or push it to Hugging Face Hub:

Listing 7.20 Save model in Hugging Face Hub

```
model.push_to_hub("your_name/your_model_name")
```

NOTE The code for the SFT and the DPO training can be found in the books repo: <https://github.com/Nicolepcx/Transformers-in-Action> in the following Notebooks:

- CH07_SFT.ipynb
- CH07_DPO.ipynb

To now run the inference of the model we can leverage the same code as in 7.12:

Listing 7.21 Run inference on trained model

1 Prepare the prompt

```
prompt = model_tokenizer([
    alpaca_template.format(
        2 Instruction
        "What is the iconic symbol of freedom at the US east coast?",
        3 Input
        " ",
        4 Output
        " ",
    )
] * 1, return_tensors="pt").to("cuda")
```

5 Model's generation settings

```
generation_parameters =
```

6 Maximum number of new tokens to generate

```
"max_new_tokens": 256,
```

7 Whether to use past key values for attention

```
"use_cache": True
```

8 Generate outputs using the model and the specified generation parameters

```
outputs = model.generate(**prompt, **generation_parameters)
```

9 Decode the generated outputs

```
decoded_outputs = model_tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

The response will be the same as in the SFT example. But your models behaviour will, of course, change based on the training dataset. So, if you, for instance, have a labeled

dataset where your model is guided to adhere to a certain company policy, your model will now follow this guideline.

This completes our section on DPO training. Feel free to experiment with the provided code. You can exchange the models and the used datasets and/or change the hyperparameters in the training arguments to see how this influences your models responses. In the next section we will look at how you can control your LLMs output without having to alter the model weights.

Evaluating text generation performance

Assessing text generation poses significant challenges due to the complex and varied nature of human language. BERT Score provides a quantitative method and is frequently used in academic research, however, it's not a catch-all solution to this intricate problem. A significant consideration when using BERT Score, is the need for a reference text that aligns closely with the context and intent of the generated text. This requirement may pose challenges in real-world text generation tasks, where suitable reference texts may not be readily available or even possible to create.

Despite these limitations, knowing how to use BERT Score can be a useful tool for understanding and assessing the effectiveness of your text generation models. At the end, it's important to remember that human judgment often provides the most reliable way to evaluate text generation quality.

BERT SCORE

BERT Score is a metric for evaluating the quality of generated text in natural language processing tasks. Unlike conventional evaluation metrics such as BLEU, METEOR, NIST, ROUGE, and CIDER, which simply measure the n-gram overlap between the generated and reference translations, BERT Score operates at a more granular level, considering word and sentence embeddings to capture the contextual information within the text.

In essence, BERT Score calculates the cosine similarity between the BERT embeddings of the candidate and reference sentences. The scoring function computes the precision (P_{BERT}), recall (R_{BERT}), and F1 score (F_{BERT}) for each candidate sentence against each reference sentence. The computation is defined as follows:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \quad (7.1)$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \quad (7.2)$$

$$F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \quad (7.3)$$

Here, x represents the set of words in the reference sentence, and $\hat{x}$ denotes the set of words in the candidate sentence. $\mathbf{x}_i$ and $\hat{\mathbf{x}}_j$ represent the BERT embeddings of the words in the reference and candidate sentences, respectively. The BERT Score calculates the cosine similarity between these embeddings, which results in a value ranging from -1 to 1. The final BERT Score is obtained by averaging the cosine similarity values across all words in the sentence and then shifting and scaling the average to the range [0, 1]. This helps in better handling sentences with different lengths.

R_{BERT} is the recall, which measures how many of the actual positive cases (i.e., correctly predicted words) our model was able to catch. P_{BERT} is the precision, reflecting the proportion of the positive identifications (i.e., the predicted words) that were indeed correct. F_{BERT} is the F1 score, which represents the harmonic mean of precision and recall, providing a balance between them.

Furthermore, BERT Score incorporates importance weighting to reflect the varying significance of different words in a sentence. It also applies baseline rescaling to account for the difference in score distributions across languages.

ALPACAEVAL

AlpacaEval is a GPT-4-based comparison. The leaderboard can be accessed via: https://tatsu-lab.github.io/alpaca_eval/, this leaderboard helps as a general baseline which model you might want to choose for your own task. If you want to evaluate your custom model, i.e. after refining a LLM either with RLHF, RLAIF or DPO, you can follow the instructions provided here https://github.com/tatsu-lab/alpaca_eval#contributing.

7.3 Prompt engineering: The art of prompting

In section 1.3 we've shortly looked into zero-shot and few-shot learning and that the billion-parameter models, like Llama 2, can produce stunning outputs using these methods. In this section we will take a deep dive into these techniques and while doing so, shed some light onto common techniques, to improve a model's prompt output, like Chain-of-Thought and Tree of Thoughts.

We use what are called *prompts* to "talk" to LLMs to perform a task. A prompt is text that a user types in for the model to respond to. This text can be in the form of questions, instructions, or any kind of input, depending on what you aim to achieve with the model. With multimodal models, like ChatGPT, handling inputs from images and text, prompts can also be in the form of images. Or a combination of text and images.

To get the most out of LLMs we use a technique called *prompt engineering*. This term refers to the process of carefully crafting prompts to generate a specific output from our model. We distinguish between the following strategies:

- Zero-shot prompting
- One- and few-shot prompting
- Chain-of-Thought prompting
- Contrastive Chain-of-thought prompting

- Tree of Thought prompting
- Thread of Thought

In the following sections will take a close look into each of these techniques, showing how and when to use them.

7.3.1 Zero-shot prompting

Zero-shot prompting, also known as direct prompting, enables users to generate an output from an LLM with minimal overhead and it is the simplest type of a prompt. This technique provides no examples to the model for the task at hand. This is possible because these billion parameter models can efficiently use in-context information, as shown in image 7.2.

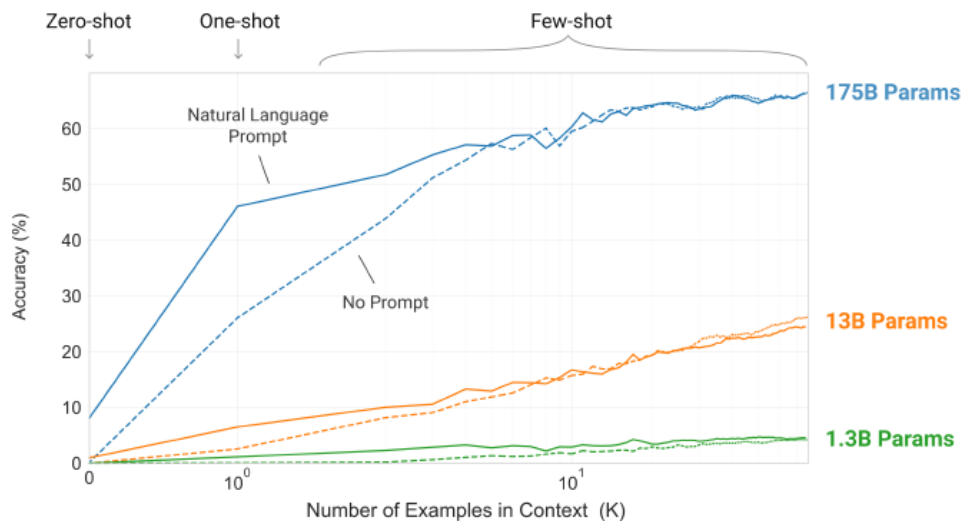


Figure 7.2 This figure illustrates how larger models progressively use in-context information more efficiently. In-context refers to the information immediately preceding a task which provides some guidance or context for the task itself. Here, it demonstrates the performance of learning on a basic task where the model must remove random symbols from a word. The task is executed both with and without a natural language task description. It can be observed that the steeper learning curves for larger models are indicative of their enhanced capacity to learn tasks using this in-context information. This behavior pattern is consistent across a broad spectrum of tasks. Image is taken from the GPT-3 paper.

To use zero-shot prompting we have to provide the following information to the model:

- Instruction
- Some context

The LLM will based on this generate it's output. For instance, we could tell the model to do a sentiment classification:

- **Instruction:** Sentiment classification

- **Context:** Classify the given text: "The movie was not bad." into the following classes: positive, negative.

This is the output generated from using this set-up to prompt ChatGPT with this task:

OUTPUT The text "The movie was not bad." should be classified as **positive**. This classification is based on the use of the phrase "not bad," which typically indicates a positive sentiment, especially when used in informal contexts like movie reviews. In such cases, "not bad" is often understood as a colloquial way of saying something is good or satisfactory.

It's important to note that even though the model did classify the sentiment correctly, there are some limitations to zero-shot prompting. However, even the most advanced language models may still face challenges with complex multi-step reasoning tasks, including solving math word problems and understanding commonsense reasoning. This is where other techniques like Chain-of-Thought and Tree of Thought come in.

7.3.2 One- and few-shot prompting

The GPT-3 paper was titled "Language Models are Few-Shot Learners", this paper introduced not only GPT-3, but also expanded on how these large models are performing better compared to others, as shown in image 7.3.

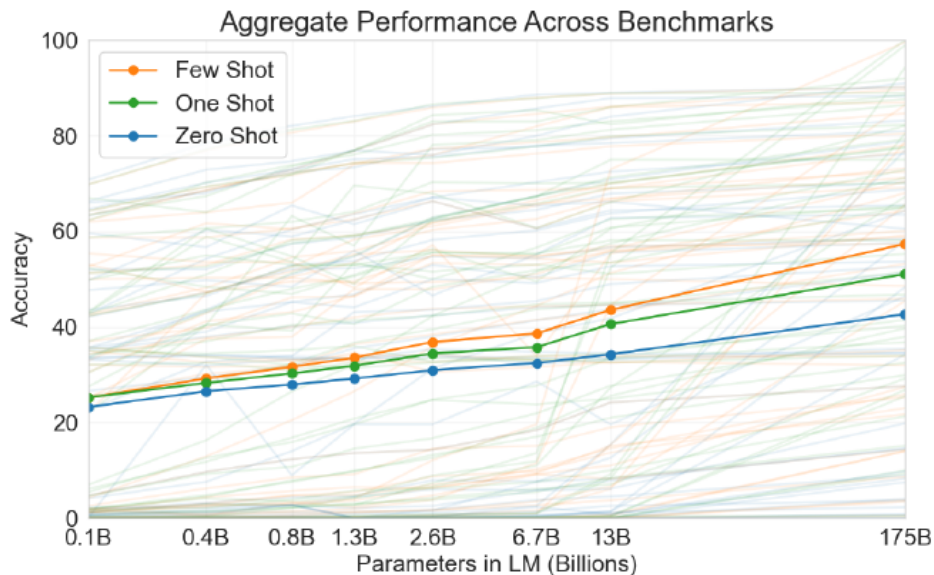


Figure 7.3 In this plot we can clearly see how zero-shot performance steadily improves with model size, however, compared to few-shot, the performance increases faster. The chosen Benchmark for evaluating the models was SuperGLUE, a standard NLP benchmark suite.

To leverage this method fully, let us take a closer look how you can optimize the output from an LLM using one- and few-shot prompting. The difference between zero-shot and

one- or few-shot prompting is, we now use examples and show these to the model. That is, with one-shot we provide the model with one example, and for few-shot learning, we guide the model with a couple of examples. Let us revisit our simple text classification example from zero-shot prompting. We now would modify the prompt as follows, using few-shot prompting:

- **Instruction:** Sentiment classification
- **Context:** Classify the given text: "The movie was not bad." into the following classes: positive, negative.
- **Examples:** A wonderful little production: positive; Phil the Alien is one of those quirky films where the humor is based around the oddness of everything: negative;

We will get the same response, from our model, that the sentiment of "The movie was not bad." is positive. This structure can be expanded to more nuanced texts and tasks. This method is sufficient for most tasks, but if we want to enable the LLM to perform reasoning tasks, we need to use more advanced techniques, which we will look at in the following sections.

7.3.3 Chain-of-Thought prompting

We’ve seen in the previous section how we can guide the model with some examples to solve simple tasks. However, if we want the model to perform complex reasoning, as it is, for instance, necessary with some text math problems, we need a new way of guiding the model. This can be done by showing the model a few chain of thought demonstrations as examples via prompting. Figure 7.4 shows this method in comparison to standard prompting.

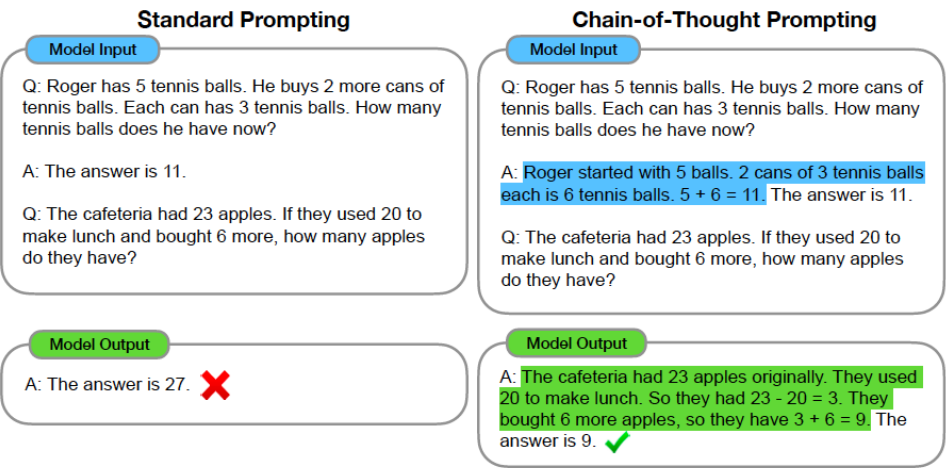


Figure 7.4 This example demonstrates how chain-of-thought prompting enables LLMs to be able to tackle complex tasks such as arithmetic commonsense reasoning. The processes for chain-of-thought are highlighted. Image is taken from the paper: "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models".

Chain-of-thought (CoT) prompting is a series of natural reasoning steps which lead to the final, desired output. This technique is an inspiration from our thought process. We human beings tend to decompose a complex problem, and solve each intermediate step before we give the final answer. The programmers among the readers might wryly recognize this as the "divide and conquer" approach, a mantra often repeated exhaustively, in coding lectures. That said, in simple terms, Chain-of-Thought allows LLMs to decompose complex problems into intermediate steps that can be solved individually. Now, the question might arise: Why is it not sufficient to only show the model the correct answer, as we saw in figure 7.4? The answer is simple, it is hard for the LLM to directly translate all of the semantics into a single equation. So, if we look at the problem we've seen in the comparison between standard and chain-of-thought prompting, the model lacks the ability to derive all semantics to answer the question correctly from just seeing: "The answer is 11.". However, if we guide the model with: "Roger started with 5 balls, 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$." The model can link the semantics from the questions to the answer. A note of caution, while chain-of-thought prompting is applicable for any text-to-text task and outperforms standard prompting in reasoning tasks, it is more useful if the tasks requires multi-step reasoning. CoT prompting can be optimized by using a method call self-consistency. This method basically just prompts the model with the same prompt multiply times and then takes the majority as the final result. To achieve this, self-consistency follows three steps:

- Using Chain-of-Thought prompting to prompt an LLM.
- Sampling from the LLM's decoder to generate various reasoning paths.
- Sort out the unimportant answers and aggregate the most consistent answers.

So, in simple terms, self-consistency is an ensemble approach which returns the most frequent output to get the final answer. In the next section we are continuing exploring more advanced prompting techniques.

7.3.4 **Contrastive Chain-of-Thought Prompting**

In the previous section, we learned about CoT prompting and how it enhances the reasoning of language models. However, in some specific cases, we want to inform our LLM about the mistakes it should avoid during its reasoning process. Let us revisit our previous example: "Roger started with 5 balls, 2 cans of 3 tennis balls each, which is 6 tennis balls. $5 + 6 = 11$." and how this would be formulated as a contrastive CoT prompt for our model. Example model input for contrastive CoT:

- **Question:** Roger starts with 5 balls. If he adds 2 cans containing 3 tennis balls each, how many tennis balls does he have in total?
- **Correct Explanation:** Roger adds the contents of the two cans to his original 5 balls. Each can has 3 tennis balls, so two cans have $2 \times 3 = 6$ tennis balls. Therefore, the total is $5 + 6 = 11$ tennis balls.
- **Wrong Explanation:** If Roger adds 2 cans of tennis balls without considering the quantity in each can, one might incorrectly add the number of cans to the original 5

balls, resulting in $5 + 2 = 7$ tennis balls. Hence, this does not account for the fact that each can contains 3 balls, not 1.

This way of guiding the model can be specifically helpful if we know of specific negative results we want to avoid. For instance, consider the use case where you are an investment company investing in private equity, and you want to get a non-emotional evaluation of an investor's pitch from an LLM. Here, you know that your analysts tasked with this often made some mistakes in evaluating the pitch deck. You can use this inside information to feed it into the model and get a better evaluation of the pitch deck.

7.3.5 Tree of Thought prompting

If we have even more complex tasks where initial decisions play a pivotal role or a more exploratory and strategic approach is needed, we can leverage tree of thoughts (ToT) prompting. ToT does generalize over the previously introduced CoT prompting by enabling the LLM to explore coherent text units which serve as an intermediate problem solving step. Image 7.5 shows an illustration of this method.

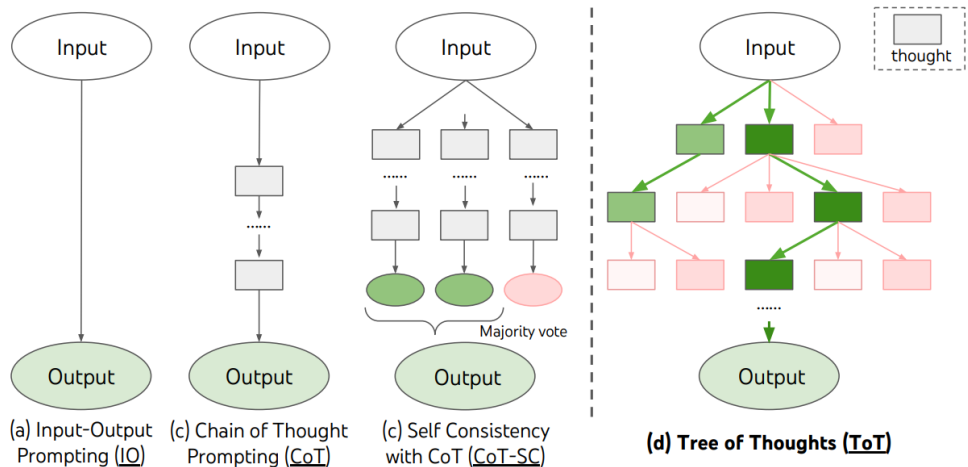


Figure 7.5 Comparison of different prompting methods, where each rectangle corresponds to a language sequence. Image is taken from the paper: "Tree of Thoughts: Deliberate Problem Solving with Large Language Models".

ToT uses the LLMs ability to evaluate coherent language sequences ("thoughts") in combination with search algorithms for data structures such as depth-first search (DFS) or breadth-first search (BFS). Some of you may have heard about these two algorithms. They are commonly used to search in tree or graph data structures where we traverse through nodes. Both BFS and DFS start at the root, but BFS explores first all nodes at the current level of the tree before going to the next one, while DFS explores first as far as possible along each branch. As you can see, BFS and DFS allow a systematic exploration of ToT.

Let us look how the method can be applied. For that, we follow an example from the authors of the paper. They evaluated ToT with "Game of 24", which is an online

mathematical reasoning challenge where the goal is to manipulate 4 integer numbers with basic arithmetic operations (+-*/) in such a way that we get 24. Image 7.6 illustrates how ToT can be used for Game of 24 and how the steps look like if we had the numbers "4 9 10 13" as a given input.

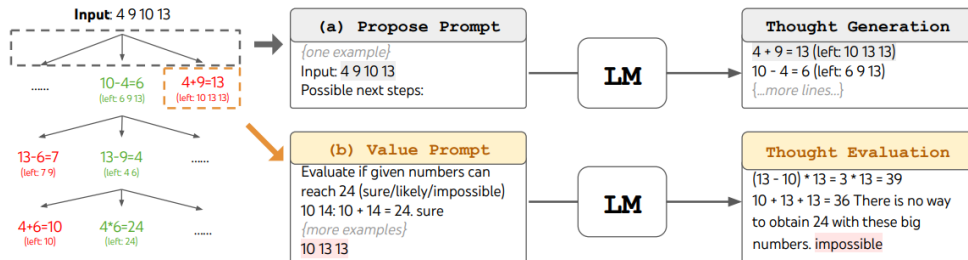


Figure 7.6 Illustration how ToT can be used, where a corresponds to the thought generation and b to the evaluation of the thought. Image is taken from the paper: "Tree of Thoughts: Deliberate Problem Solving with Large Language Models".

The tree part of the illustration demonstrates how, at each tree node, the thoughts get evaluated by prompting the LLM to evaluate each intermediate equation and to eliminate impossible partial solutions by choosing "too big" or "too small" and just keep the ones labeled "likely" or "sure".

Let's see how this is working in practice. Listing 7.22 demonstrates how you can run the same experiment as in the image from the paper, given the input "4 9 10 13".

Listing 7.22 Using ToT with GPT-4

- 1 **Install the necessary package**

```
!pip install tree-of-thoughts-llm -q
```
- 2 **Import the necessary packages, we are using the BFS search algorithm**

```
import os
import argparse
from tot.methods.bfs import solve
from tot.tasks.game24 import Game24Task
```
- 3 **Set up OpenAI API key and store in the environment variable**

```
# Replace 'your-api-key' with your actual OpenAI API key.
os.environ['OPENAI_API_KEY'] = 'your-api-key'
```
- 4 **Parse the arguments to run the experiment**

```
args = argparse.Namespace(backend='gpt-4', temperature=0.7, task='game24',
naive_run=False, prompt_sample=None, method_generate='propose',
method_evaluate='value', method_select='greedy', n_generate_sample=1,
n_evaluate_sample=3, n_select_sample=5)
```
- 5 **Call the functions to run the experiment**

```
task = Game24Task()
```
- 6 **999 is the index from the repo's CSV file to select the number input: 4 9 10 13**

```
ys, infos = solve(args, task, 999)
print(ys[0])
```

This will lead to this final output:

```
[ '13 - 9 = 4 (left: 4 4 10)10 - 4 = 6 (left: 4 6)4 * 6 = 24 (left: 24): 4 * (10 - (13 - 9)) =
24', '10 - 4 = 6 (left: 6 9 13)13 - 9 = 4 (left: 4 6)4 * 6 = 24 (left: 24): (10 - 4) * (13 - 9) =
24', '13 / 4 = 3.25 (left: 3.25 9 10)10 / 3.25 = 3.08 (approx) (left: 3.08 9)9 - 3.08 = 5.92
(left: 5.92)5.92 * 2 = 11.84 (left: 8 8 11.84 14)', '13 / 4 = 3.25 (left: 3.25 9 10)10 / 3.25 =
3.08 (approx) (left: 3.08 9)9 - 3.08 = 5.92 (left: 5.92)5.92 + 2 = 7.92 (left: 7.92 8 8 14)',
'13 / 4 = 3.25 (left: 3.25 9 10)10 / 3.25 = 3.08 (approx) (left: 3.08 9)9 - 3.08 = 5.92 (left:
5.92)14 - 5.92 = 8.08 (left: 2 8 8 8.08)'] 13 - 9 = 4 (left: 4 4 10)
10 - 4 = 6 (left: 4 6)
4 * 6 = 24 (left: 24)
Answer: 4 * (10 - (13 - 9)) = 24
```

NOTE If you want to try it out yourself, make sure you check out the corresponding notebook in **chapter 7** of the books repository: <https://github.com/Nicolepcx/Transformers-in-Action>. However, in order to run the code you will have first to setup an API for OpenAI: <https://platform.openai.com/api-keys> and you will have to increase your limit in your API account and add some funds and credit card to your API account here <https://platform.openai.com/usage>. Note: Running this short experiment will amount for about \$1.5.

It's also important to consider the limitations of every prompting method. One of the limitations of ToT is that it requires a GPT-4 API and is therefore more costly, because we have to pay for the API access and we will have more computational costs, since we will have to prompt the model several times. Moreover, we need to keep in mind that such specific search algorithms are not needed for most tasks. Nonetheless, for task which involve analytical reasoning, like for instance coding or solving mathematical problems, this method is a good choice.

7.3.6 Thread of Thought prompting

Thread of Thought (ThoT) prompting, much like many methods in machine learning, is inspired by human cognitive processes. It systematically analyzes and segments the context before selecting relevant information. The fundamental concept is to emulate how humans process vast amounts of information while maintaining a continuity of ideas and selecting pertinent details from the context. ThoT is adaptable and compatible with various LLMs and a range of prompting techniques.

ThoT follows these simple steps:

- Initiating the reasoning of the LLM by prompting it with a sentence like this: "Walk me through this context in manageable parts step by step, summarizing and analyzing as we go."

- Refining the conclusion by combining the initial prompted text with the model's response and a conclusion marker such as: "Therefore, the answer:"

This approach enhances the LLMs capacity of navigating chaotic context in helping it to organize the context into organized chunks of thoughts. Image 7.7 illustrates how ThoT can be used and how it compares to CoT.

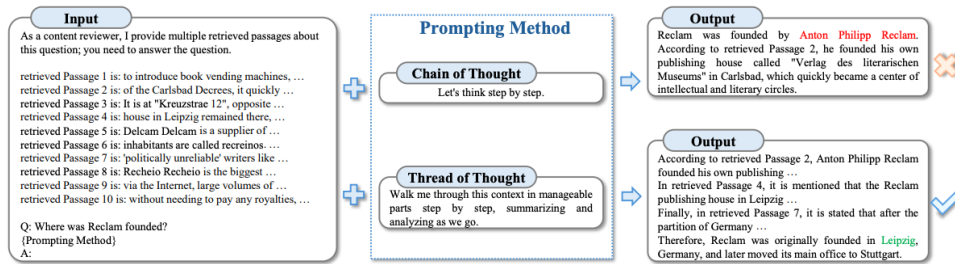


Figure 7.7 Thread of Thought prompting empowers LLMs to conquer chaotic context problems. In the illustration, text in green represents the accurate response, whereas text in red signifies the incorrect prediction. Image is taken from the paper: "Thread of Thought Unraveling Chaotic Contexts".

To conclude this section, ThoT offers a straightforward way to enhance the reasoning capabilities of LLMs for large and chaotic texts.

Tips for efficient prompting

One task per prompt

Similar to the way we *divide and conquer* in programming, we can split the tasks for an LLM into just having one task for each prompt. This helps to avoid confusing the model by giving it precise instructions. If you need to have more information or you need to have more tasks in one prompt aim to use methods such as CoT or ToT to help the LLM with it's reasoning and organizing textual sequences.

Be explicit

Again, taken from programming, be explicit. Think of how you would name your functions and your variables to be self-documenting. Or how you would explain a task step-by-step to a 5-year old child.

7.4 Summary

- To refine text generation models, leveraging RLHF is a common method. At its core is the concept of Markov decision processes, which emphasizes the relationship between actions, states, and rewards. Proximal Policy Optimization ensures that the model's learning trajectory remains stable by making incremental adaptations rather than drastic changes.

- DPO is another way to refine your language model's behavior and align it with your preferences. In contrast to RLHF, with DPO, we now only have a classification objective. Hence, this method is more straightforward in its overall implementation.
- In-context learning refers to the way an LLM "learns" from prompts without altering the model weights. This approach is specifically helpful if one has only API access to a language model.
- Prompt Engineering helps in guiding the language model for specific tasks. There exist different prompting methods from zero-shot (no examples are given to the model) to one- and few-shot prompting, where the prompt also contains example responses.
- Advanced prompting techniques like Chain-of-thought, Contrastive Chain-of-Thought, Tree of Thought, and Thread of Thought prompting help the models if the given task involves reasoning as is the case with mathematical problems.

Multimodal models

This chapter covers

- Introduction to multi-modality
- Challenges with multimodal language models
- Overview of multimodal models
- Creating a multimodal chatbot

In the last chapter, we learned how you can control the content of your large language model. You might remember that I mentioned in the intro of chapter 6 that your journey would unfold over three chapters. To recap, chapter 6 focused on text generation and covered some of the state-of-the-art models. However, I intentionally left some top-notch models out. The reason is that they fall under a different category, they are known as multimodal models. Therefore, in this chapter, we will take a close look at these multimodal models. I will first cover what multimodal means, then explain the challenges with multimodal language models, and finally, how you can leverage these versatile models to create a chatbot which takes an image and text as input.

8.1 Getting started with multimodal models

Humans possess five senses: sight, hearing, touch, smell, and taste. These capabilities enable us to interpret and interact with our environment. Consequently, "multimodal" refers to the integration of various information streams to achieve a comprehensive understanding

of our surroundings. For instance, as a toddler or child, we learn through engagement in a multimodal learning process. Think of a toddler playing with their teddy bear; they usually put things in their mouth. Or if they grow up with pets, they mimic their sounds. This process of learning in humans served as inspiration for researchers, trying to blend diverse modalities in the training of large language models. If we now compare our LLM with how a child uses all senses to learn about their environment, we can draw similarities. Multimodal large language models (MLLMs) integrate various types of data inputs, such as visual images, text, and audio, among others, to enhance their learning and understanding.

To give you a simple example where this might be helpful, consider an online clothing shop with an LLM-based chatbot. If a customer asks to see some white shirts, the model will be able to handle the text input, map the images from the shop to the query, and provide the customer with appropriate findings.

Fundamentally, this relies on the model being tasked with refining an objective defined by a loss function. The minimization of the loss is achieved through gradient descent. For this process to be successful, we need to have and align the numerical input data. Multimodal tasks often involve handling unstructured data like images or text. Thus, the initial challenge is converting such input into a numerical format. Another complexity in multimodal tasks is the effective combination of different modalities.

Having understood the basic principles of multi-modality, we will look, in the next section, into the specific challenges and important considerations that come with these complex multimodal tasks.

8.2 Challenges and considerations for multimodal models

One of the challenges in pre-training multimodal models is the high computational cost, primarily due to the requirement of end-to-end training using large-scale models and extensive datasets. This challenge has led to a significant shift in the field, particularly in vision-language pre-training (VLP). Rather than building entirely new models from scratch, there's an increasing trend towards leveraging SOTA unimodal models that specialize in either language or vision. This approach not only reduces computational demands but also enhances the efficacy of the models.

This strategy has significantly influenced the development of multimodal models incorporating other modalities, such as audio or video. MLLMs, which integrate LLMs to process diverse data types including text, vision, and audio, are typically built with a focus on modality alignment. This alignment is crucial for bridging the gap between the inherently language-centric LLMs and other modalities. A key component in achieving this is a specifically designed alignment method, which is designed to align and synchronize different data types. This method facilitates the LLM's understanding of non-textual data, enabling it to interpret and integrate information from various sources coherently.

Such alignment can be either done by a converter facilitating the transformation into objects comprehensible or learnable by LLMs. An illustration of such a converter is shown in image 8.3.

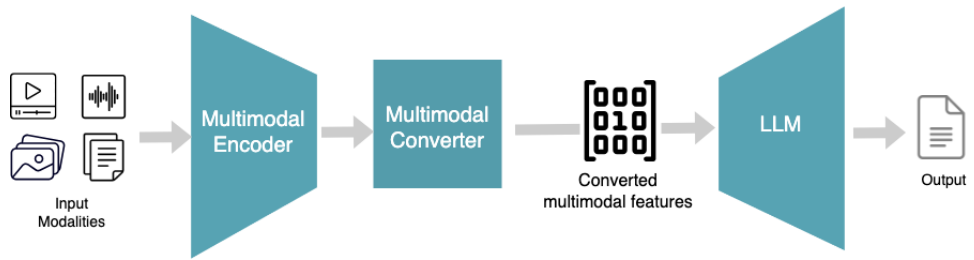


Figure 8.1 This illustration shows the data flow in a multimodal language model. The multimodal encoder processes various input modalities, which could represent a deep neural network capable of handling different types of data such as images, audio, and text. The processed data is then passed through a multimodal converter, which aligns and translates these features into a representation that a language model can understand.

Or a perceiver, which is designed to primarily enhance the perceptual abilities towards an understanding of multimodal information from different data types by transforming multimodal features into multimodal tokens, as shown in 8.2.

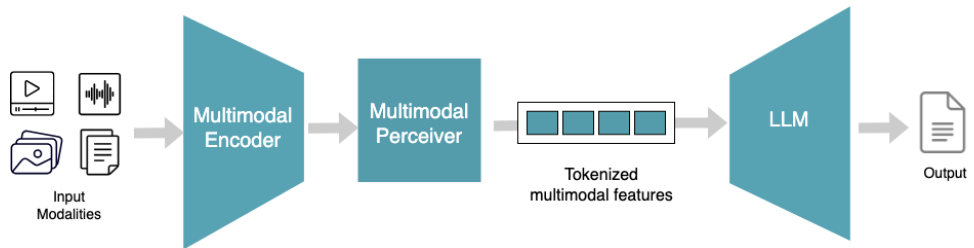


Figure 8.2 This illustration shows the data flow in a multimodal language model using a perceiver, that serves as a connector between the textual modality and additional modalities, converting multimodal attributes into tokens compatible with the representation space of LLMs.

The implications of this development are significant. By effectively aligning different data types, MLLMs are not only more computationally efficient but also become more capable in contextually understanding and interpreting multimodal data. This advancement holds immense potential across various domains, from enhancing the capabilities of AI assistants to improving content analysis tools in multimedia.

Choosing an appropriate modality alignment method is crucial for the success of MLLMs. The effectiveness of these models depends on the seamless integration of different data types, which requires a nuanced understanding of the strengths and limitations of each modality.

For instance, in vision-language models, the alignment method must effectively synchronize visual data with textual interpretation, ensuring that the model comprehensively understands visual elements in the context of relevant text. Similarly, in models integrating audio, the method must accurately translate and integrate auditory information with linguistic data.

Current state-of-the-art MLLMs employ a variety of techniques to bridge the semantic gap, some models use attention mechanisms, which allows the model to focus on specific parts of the data from different modalities, enhancing the relevance and accuracy of the model's responses. Others employ neural network architectures specifically designed for multimodal data, which can process and integrate information from different sources more efficiently. Techniques like transfer learning are also prevalent, where a model trained on one type of data is adapted to understand another, leveraging the knowledge it has already acquired.

The choice of alignment method also depends on the specific application and the nature of the data being processed. For instance, models designed for multimedia content analysis might require different alignment techniques compared to those used in interactive AI assistants. As this field continues to evolve, the development of more sophisticated and efficient alignment methods is a key area of research, promising to further enhance the capabilities and applications of MLLMs. An example of different methods is shown in image 8.3.

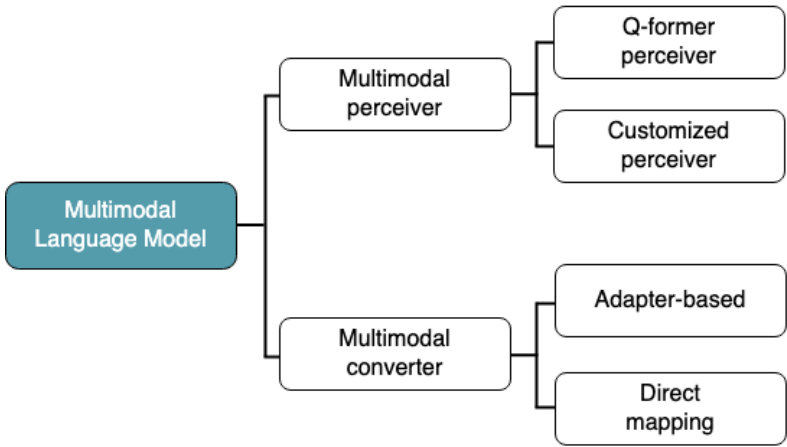


Figure 8.3 This diagram shows the different methods that can be integrated within a multimodal language model system. The illustration distinguishes two main methods: the perceiver and the converter. The perceiver module connects, for instance, a vision encoder to the frozen language model. The converter, on the other hand, converts the various inputs in such ways that the LLM can digest it.

In the following sections I will explain the different methods in details, so you have a thorough understanding of each method, which will set the stage for your understanding of the different available MLLMs, we look at in section 8.3.

8.2.1 Perceiver-based multimodal methods

The perceiver-based methods aim to minimize the gap of the different modalities by a perception module. This module transforms the data features from other modalities, such as images, into tokens which are compatible with the embeddings of the LLMs.

Q-FORMER PERCEIVER

LLMs are, due to their pre-training on textual data, not capable of understanding information from a multimodal feature space. The querying transformer (Q-former) perceiver aims to bridge this modality gap. The Q-former was first introduced for BLIP-2 (Bootstrapping Language-Image Pre-training) (4), a vision-language multimodal model. To achieve the alignment between the two modalities, language and vision, the researchers explored two LLMs: FlanT5, a decoder-encoder-based instruction-tuned LLM, and OPT, a decoder-based, unsupervised-trained LLM. Image 8.4 compares the two different architectures and illustrates their inner workings.

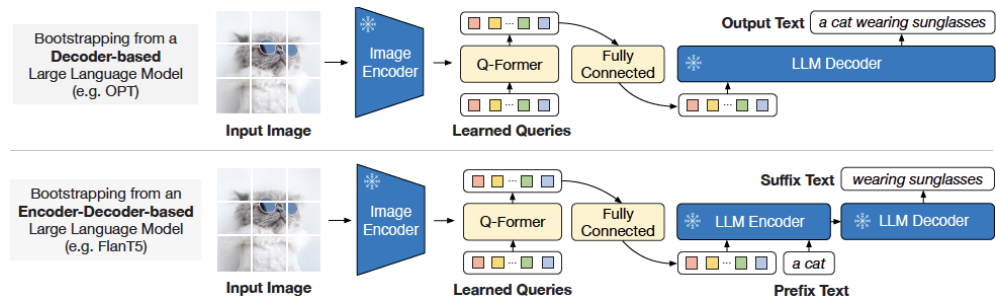


Figure 8.4 Illustration of the vision-to-language generative pre-training. For both LLMs, the fully-connected layer acts as an adapter between the output dimension of the Q-Former to the input dimension of the LLM. Image is taken from the paper: "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models"

For the vision models, they explore two distinct architectures: ViT-L/14 from CLIP, which is a vision transformer (ViT) model pre-trained using contrastive learning to align images and text, and ViT-g/14 from EVA-CLIP, which is an enhanced ViT model integrating attention mechanisms for more effective visual understanding and alignment in multimodal contexts. With image-text contrastive learning, the model contrasts the image-text similarity of a positive pair against those of negative pairs. And by doing so learns to maximize the alignment of the image-text representations.

To align the two modalities, the Q-former perceiver employs shared self-attention layers, facilitating an efficient interaction between the image and text modalities. The image transformer and the text transformer act as submodules, and work in tandem, with the image transformer receiving input from both learnable query embeddings and image features encoded by an image encoder. This interaction occurs through a process of cross-attention, allowing for a seamless integration of image-text modalities. Furthermore, the learnable queries also engage with the input text via shared self-attention layers, promoting a cohesive multimodal understanding.

To further facilitate the Q-former’s capability in multimodal understanding, it employs a two-stage training strategy with frozen unimodal models. In this context, *frozen* models refer to pre-trained models whose parameters are not updated during the training of the Q-former. Essentially, these models are used as-is, with their learned weights and biases remaining unchanged. This approach allows the Q-former to leverage the established strengths and knowledge encapsulated in these models, while focusing the training process on adapting and fine-tuning its own parameters to bridge the modality gap. By using frozen models, the system can efficiently integrate and use the robust capabilities of these pre-trained components without the computational cost and complexity of retraining them. In the first stage, the Q-former, acting as a lightweight transformer, uses learnable query vectors to extract visual features from a frozen image encoder. This stage focuses on vision-language representation learning, enabling the Q-former to learn visual representations that are most relevant to the text. In the second stage, vision-to-language generative learning is performed. Here, the output of the Q-former, containing visually represented data, is connected to a frozen LLM. This stage trains the Q-former to output visual representations that can be effectively interpreted by the LLM, enhancing the alignment between visual features and textual data. This two-stage training process not only enhances the multimodal integration but also sets a precedent for future MLLM research, particularly in vision-language tasks.

CUSTOMIZED PERCEIVER

In the field of MLLMs, researchers have explored various perceivers to achieve effective alignment across different modalities. One development of such an innovative method are multimodal prompt generators for MLLMs. These generators are designed to produce prompts that guide the LLM in processing and interpreting multimodal data. For example, VisionLLM uses a language-guided image tokenizer to create visual prompts based on instructions, whereas MACAW-LLM formulates an alignment module that maps features from images, audio, and videos into the LLMs embedding space.

Furthermore, the modality-adaptive module, as seen in mPLUG-Owl2, represents another significant advancement in the field. This module projects various modalities into a shared semantic space, thereby achieving a cohesive coordination of different modalities. Such a module is crucial for models that aim to integrate and interpret diverse data types, ensuring that the models understanding remains consistent and accurate across all modalities.

A standout example in the domain of perceivers is the Flamingo model (1), which employs a perceiver resampler module. This module acts as a critical interface between the vision encoder and the LLMs, using a mechanism akin to the multi-layer decoders found in transformers. The perceiver resamplers architecture is designed to take flattened visual features and a set number of learnable latent queries as input. These latent queries function as the queries within the module, while the visual features are concatenated as keys and values. An overview of this architecture is shown in image 8.5.

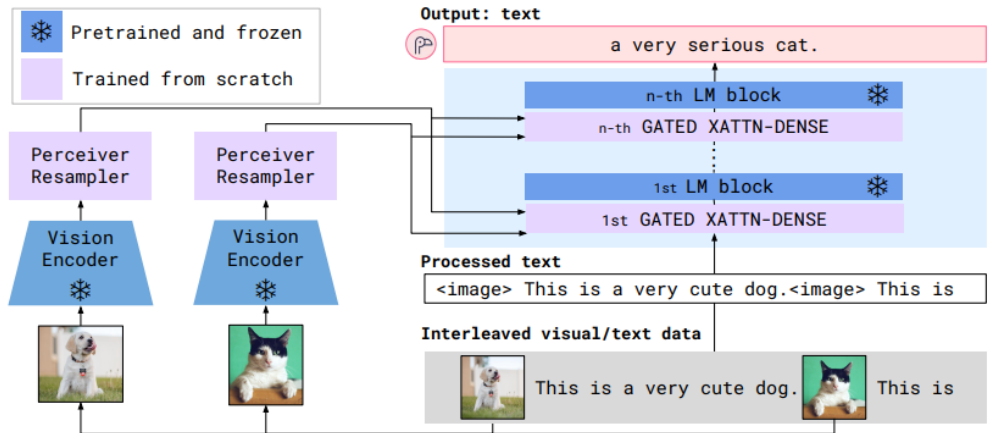


Figure 8.5 Illustration of the Flamingo architecture, showing how visual data interleaves with text to produce a textual output. Image is taken from the paper: "Flamingo: a Visual Language Model for Few-Shot Learning"

This design, similar to the querying mechanism in the Q-former perceiver, is specifically tailored to transform multiple visual modalities, such as images or sequences of images in videos, into a fixed number of visual tokens. The key objective of Flamingo's perceiver resampler is to boost the alignment efficiency between its vision-text modalities, with alignment occurring within the gated xattn-dense layers, illustrated in image 8.6.

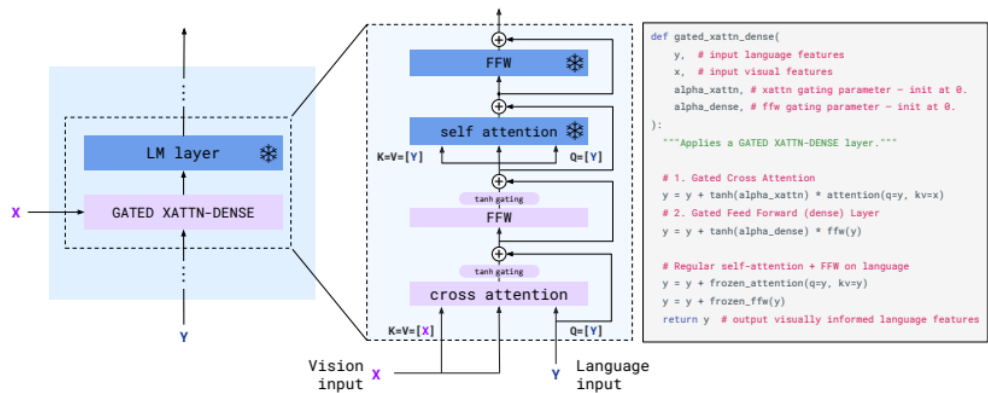


Figure 8.6 To enable the language model to process visual inputs, an additional cross-attention layers amidst the pre-existing, unaltered layers of the LM is integrated. The information for these layers' keys and values comes from vision-related features, whereas the queries originate from text inputs. Subsequent to these layers are dense, feed-forward layers. Image is taken from the paper: "Flamingo: a Visual Language Model for Few-Shot Learning"

After undergoing training on diverse datasets that include interleaved image-text pairs and video-text pairs, Flamingo has demonstrated state-of-the-art performance in numerous image and video understanding tasks, particularly excelling in few-shot learning scenarios.

In summary, the development of custom multimodal perceivers like Flamingo and the implementation of multimodal prompt generators and modality-adaptive modules reflect the ongoing evolution in the field of MLLMs. These innovations not only enhance the models ability to align and interpret data from different sources but also pave the way for more natural, human-like communication and understanding in chat-based AI systems.

8.2.2 Converter-based multimodal methods

The most straightforward way of enabling LLMs with multimodal capabilities is to directly input multimodal features into LLMs. But since LLMs are pre-trained purely on text this approach means that we need to find a way of dealing with that semantic gap. One way of overcoming that semantic gap is to map the multimodal features space into a feature space which aligns with the one of LLMs. There exist two distinct methods, direct mapping and adapter-based, which are used in SOTA MLLMs. For instance LLaVA, employs direct mapping, whereas LLaMA adapter and LLaMA adapter 2 employ the adapter-based approach.

DIRECT MAPPING CONVERTER

The direct mapping converter approach is the simplest converter approach and enables the models to leverage conventional image encoding methodologies to capture and process visual data.

LLaVA, or Large Language-and-Vision Assistant (6), which stands out for its innovative integration of visual and language understanding, is a prime example of the effectiveness of this method.

In LLaVA, the direct mapping strategy is employed to connect a vision encoder with a LLM, such as Vicuna. This connection is achieved using a simple projection matrix, demonstrating a method to align the visual features with the language model. An illustration of the network architecture is shown in image 8.7.

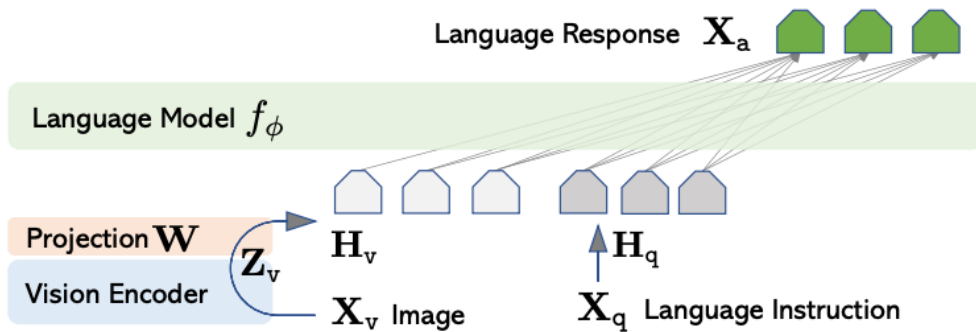


Figure 8.7 LLaVA Model Architecture: The diagram illustrates the process where an input image X_v is encoded into visual features Z_v by a CLIP visual encoder, which are then projected through a linear layer W into visual tokens H_v . These tokens are combined with language instruction embeddings H_q to generate a language response X_a using Vicuna as their language model f_ϕ .

LLaVA's approach is characterized by its two-stage instruction-tuning procedure. The first stage focuses on pre-training for feature alignment, where only the projection matrix is updated. This initial stage lays the groundwork for effective feature alignment between different modalities. The second stage involves fine-tuning the model end-to-end, where both the projection matrix and the LLM are updated. This stage is applied to different scenarios, like visual chat and Science QA, enabling the model to fine-tune its understanding based on the specific requirements of these applications.

LLaVA's ability to perform well in a variety of tasks, such as visual chat and Science QA, demonstrates the effectiveness of the direct mapping approach in facilitating multimodal understanding. The model achieves impressive results, sometimes exhibiting behaviors akin to multimodal GPT-4 on unseen images and instructions. This performance indicates the potential of the direct mapping method in multimodal settings, particularly when the model is fine-tuned on specific datasets and tasks.

Beyond LLaVA, other models like FROMAGE, DreamLLM, VisionLLM, Fuyu, and VideoLLM also use direct mapping strategies, combining visual embeddings of images directly with textual data as input for LLMs. These models extend the method to various types of visual data, including video content, by encoding and arranging video frames in chronological order and then combining these encoded features with textual data.

However, it's important to note that despite the promising capabilities of direct mapping converters like LLaVA, they may face challenges in fully comprehending complex multimodal data. This is because the method involves directly inputting encoded features as vectors into LLMs, which might not always align perfectly with the intricate nature of multimodal information.

Nonetheless, the direct mapping converter, as introduced by LLaVA, represents a significant advancement in the field of MLLMs, offering new research directions and possibilities. By enabling LLMs to process and understand multimodal data more effectively, this approach sets the the groundwork for more sophisticated and contextually aware multimodal language models.

ADAPTER-BASED CONVERTER

Adapters are used to process and learn from the multimodal features, such as images. Here features are encoded as an adjustment information to influence the text generation process. The LLaMA-adapter (8) appends a set of learnable adaption prompts into the higher layers of LLaMA. With that, new instructions can be injected to the frozen LLaMA. By using a zero-initialized attention, it is possible to preserve the original knowledge of the LLM, but still incorporate, on top, instructional signals during training.

The LLaMA-Adapter can be summarized as follows:

- The model uses a fraction of the 7B parameters, approximately 1.2 million, which are trained while keeping the larger pre-trained model fixed, allowing for efficient instruction-following capabilities.
- Training the adaptation modules is fast, requiring only about an hour on specialized hardware, making the process significantly quicker than similar models.

- It's designed to incorporate specialized adapters for various knowledge domains, making it versatile without needing to replicate the entire base model.
- The adapter is applicable for multimodal reasoning, for instance by adding image tokens into the adaption prompts, the model performs competitively on reasoning tasks such as ScienceQA and COCO Caption.

This process is illustrated in image 8.8

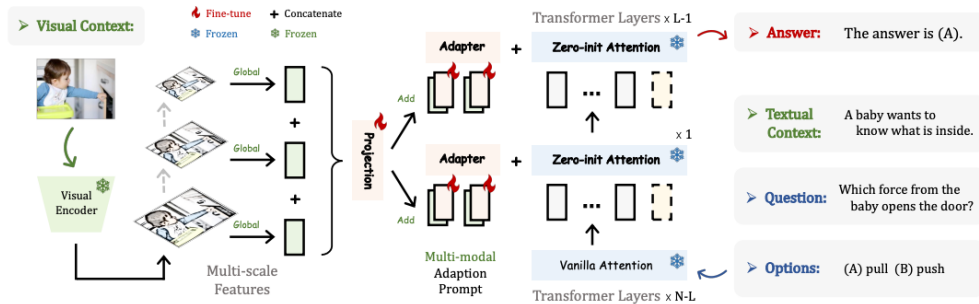


Figure 8.8 Illustration of multimodal reasoning within the LLaMA-Adapter. In the ScienceQA benchmark, LLaMAAdapter has been expanded into a multimodal version tailored for image-based question answering. This approach involves using an image as the visual context, from which a global image token is derived through multi-scale aggregation. This token is then added to the adaptation prompts on an element-by-element basis, enabling the model to follow visual instructions effectively. Image is taken from the paper: "LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention".

Building upon the foundational LLaMA-Adapter, the LLaMA-Adapter V2 (3) represents a leap forward in the transformation of LLMs into multimodal instruction followers. The LLaMA-Adapter V2 aims to expand the potential of LLMs for multimodal instruction following, a task that remains relatively uncharted compared to traditional language processing. This iteration of the adapter enriches LLMs with visual processing capabilities, addressing the limitations of its predecessor, which struggled with open-ended visual tasks and lagged behind the more advanced GPT-4.

LLaMA-Adapter V2 enhances the LLM's performance in language instruction following, making it capable of handling multi-turn dialogues, a significant step up from the previous model. The improvement comes from a parameter-efficient tuning strategy and the use of high-quality language instruction data. These advancements enable the model to interpret and act upon language instructions more effectively than before.

LLaMA-Adapter V2 marks its superiority over the original by introducing only 14M additional parameters over LLaMA. The framework not only exhibits a robust capability in language instruction following but also excels in interactive chat scenarios, demonstrating its comprehensive understanding of both language and vision. The V2 iteration advances the adapter-based approach through several contributions:

- **Enhanced Language Model:** It demonstrates a notable leap in language instruction-following proficiency, even capable of engaging in multi-turn dialogues. This signifies a substantial improvement in its utility as a language instruction model.

- **Visual Instruction Tuning:** The V2 version introduces an early fusion strategy, shown in 8.9, which mitigates the conflict between image-text alignment and language instruction. This approach enables the model to better handle visual instructions without the need for specialized multimodal instruction data.
- **Expert System Integration:** In contrast to models that require extensive pre-training on large image-text datasets, LLaMA-Adapter V2 incorporates a modular design philosophy. This allows the integration of external expert systems, such as OCR or captioning tools, which can enhance the model's image understanding capabilities during inference. This modular approach significantly reduces the need for vast amounts of training data and ensures that the model remains parameter-efficient.

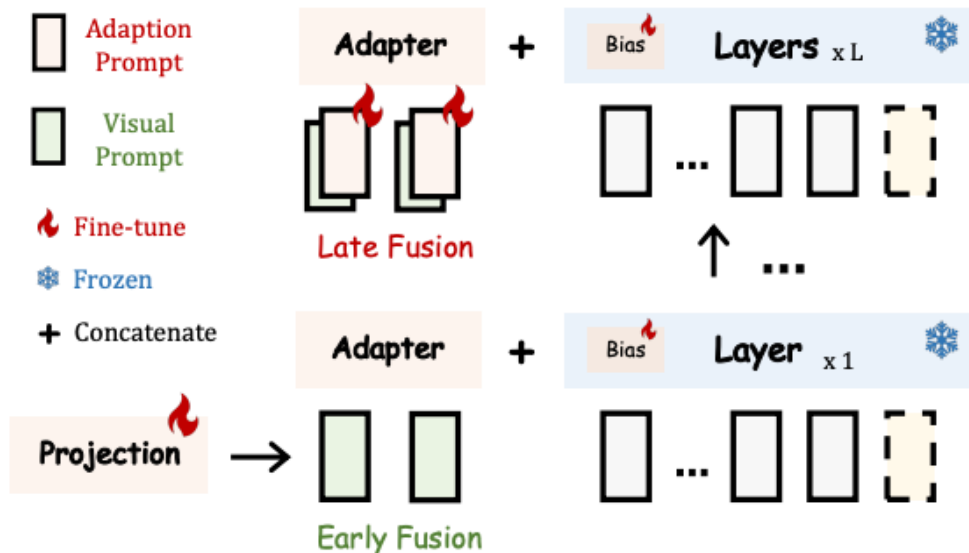


Figure 8.9 Early fusion of visual knowledge. Inspired by LLaMA-Adapter, LLaMA-Adapter-2 uses fixed adaptation prompts within the final L layers. For visual prompts, these are incorporated at an earlier stage within the large language model, separate from the adaptation. Image is taken from the paper: "LLaMA-Adapter V2: Parameter-Efficient Visual Instruction Model".

To summarize, LLaMA-Adapter V2 stands out as a transformative development in the field of MLLMs, fostering stronger language models that can interact with and understand the visual world.

Now that you have a clear understanding of the commonly used methods to align multimodalities within MLMs, it's time to explore the current state-of-the-art MLLMs in the next section.

8.3 Model Overview

In this section, we will examine various MLLMs. Each model incorporates different modalities in distinct ways. However, most ongoing research in MLLMs revolves around vision-text models. Therefore, this section will primarily focus on these vision-text models. Moreover, I won't cover models like DALL-E, SimCLR, BigBiGAN, and VQ-VAE-2, while advanced and capable in their respective domains, they don't fit the typical definition of MLLMs. These models are primarily vision-focused, with language understanding as a component, but they don't typically output text as part of their primary function.

As with the previous chapters, I will first provide a short overview of the models and then dive deeper into the overall architecture of each model.

- BLIP (Bootstrapping Language-Image Pre-training) (5) is a model designed for various multimodal tasks including visual question answering, image-text retrieval, and image captioning. It uses a unique VLP framework which is effective in both vision-language understanding and generation tasks. The vision part uses a ViT encoder, and the text encoder is the same as BERT and has 583M trainable parameters.
- BLIP-2 builds upon the original BLIP. It incorporates a lightweight 12-layer transformer encoder, bridging pre-trained image encoders and large language models. BLIP-2 is available as 3.1B, 3.4B, 3.8B, 4.1B, 7.8B and 12.1 B variants.
- CLIP (Contrastive Language-Image Pretraining) by OpenAI is a widely recognized model that efficiently connects text and images. It is trained to understand images in the context of natural language, allowing it to perform tasks like zero-shot classification. The original research paper on CLIP detailed several model configurations, focusing on variations of the ViT and ResNet architectures for the image encoder, and a transformer-based architecture for the text encoder.
- X-CLIP (7) is built upon the foundation of the CLIP model but extends its capabilities to handle video data. It has a cross-frame vision encoder to capture long-range dependencies across frames in a video. As well as a multi-frame integration transformer allowing the model to create a coherent understanding of the entire video sequence, rather than just individual frames.
- Flamingo is a family of vision-language models with 3B, 9B and 80B parameters. It was developed by DeepMind and specializes in multimodal learning, particularly in handling tasks that involve both vision and language processing.
- OpenFlamingo (2) is an open-source variant of Flamingo, the models range from 3B to 9B parameters and perform 80-89% of the corresponding Flamingo performance on seven vision-language tasks.
- GPT-4 with vision (GPT-4V) allows users to direct GPT-4 to examine images they provide. GPT-4V inherits the strengths and weaknesses of both text and vision modalities, yet it also introduces new abilities that arise from the combination of these modalities and from the enhanced reasoning capabilities of large-scale models.

- Large Language and Vision Assistant (LLaVA) represents an innovative approach in the multimodal field. It uses instruction tuning with machine-generated language-image instruction-following data, aiming to connect a vision encoder with a LLM for versatile visual and language understanding. LLaVA uses Vicuna as the LLM and the visual encoder, ViT-L/14, from CLIP.

8.3.1 BLIP

BLIP is a framework in the multimodal AI domain, designed to refine vision-language understanding and generation. Developed to make use of noisy web-scale image-text pairs, BLIP introduces a cohesive VLP approach, encapsulated in its Multimodal Encoder-Decoder (MED) architecture. At the core of BLIP's architecture is the MED, which handles both unimodal and multimodal tasks. The visual part employs a ViT as the image encoder, processing images into patches for sequence embedding, a method favoring computational efficiency over traditional pre-trained object detectors. The text encoder mirrors BERT's structure, enhancing the model's language understanding capabilities. Image 8.10 shows the overview of the model architecture.

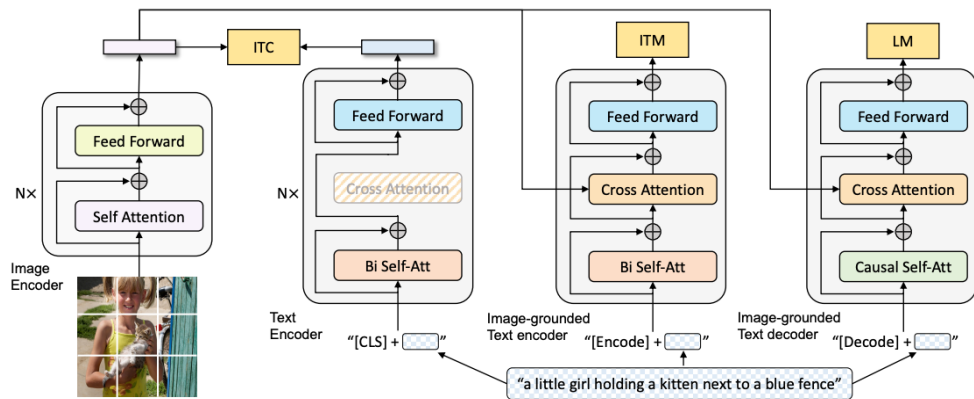


Figure 8.10 Architecture illustration of BLIP. Which uses an unimodal encoder using image-text contrastive loss to synchronize visual and linguistic representations. An image-grounded text encoder that employs cross-attention layers to process vision-language interactions. And an image-grounded text decoder with causal self-attention, sharing the encoder's cross-attention and feed-forward networks, trained on language modeling loss to create captions from images. Image is taken from the paper: "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation".

BLIP's MED operates in three distinct modes:

- As a unimodal encoder for separate image and text processing.
- An image-grounded text encoder for multimodal representation, leveraging cross-attention layers.
- An image-grounded text decoder for generating text from visual inputs, using causal self-attention for narrative coherence.

The model is fine-tuned with a trio of objectives, balancing understanding with generation. image-text contrastive loss and image-text matching Loss focus on aligning and matching image-text pairs for accurate multimodal representation. Meanwhile, language modeling loss is dedicated to generating descriptive text from images, signifying BLIP’s generative ability. To enhance dataset quality, BLIP introduces CapFilt a two-step method involving caption generation for web images and subsequent filtering of incongruent image-text pairs. This method significantly improves the learning material’s relevance and quality, facilitating more effective pre-training.

Further, BLIP is pre-trained on a blend of human-annotated and web-collected datasets, ensuring a broad spectrum of visual and textual knowledge. The model’s architecture and training are designed for efficiency, enabling it to achieve remarkable results in various multimodal tasks with less computational demand compared to previous models.

8.3.2 BLIP-2

BLIP-2 (Bootstrapping Language-Image Pre-training with Frozen Unimodal Models) innovates the field of VLP by bridging the gap between pre-trained unimodal models for enhanced cross-modal alignment. This advancement addresses the challenge of integrating LLMs, which traditionally do not process images, with vision models in a cohesive framework without the need for end-to-end retraining.

BLIP-2 uses a two-stage pre-training approach. The first stage focuses on vision-language representation learning, optimizing the Q-Former to select visual representations most relevant to accompanying text. The subsequent stage shifts to vision-to-language generative learning, refining the Q-Former’s output to ensure it can be effectively interpreted by the LLM for text generation tasks. The queries between the text transformer and image transformer interact with each other through self-attention layers, image 8.11 illustrates the self-attention masking strategy for each query-text interaction.

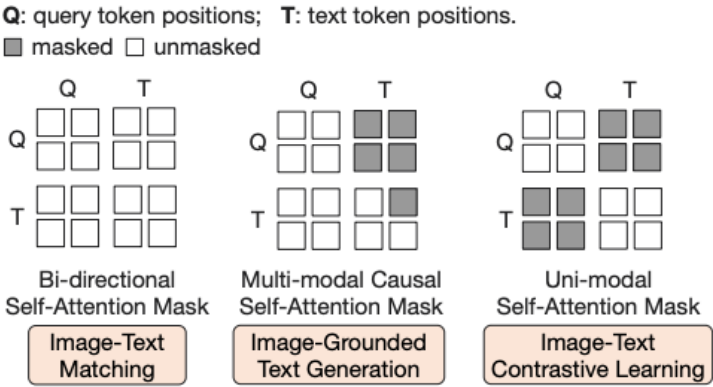


Figure 8.11 The strategy for self-attention masking is tailored to each objective to manage the interaction between queries and text. Image is taken from the paper: "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models".

Central to BLIP-2's architecture is the Q-Former, a component designed to act as an intermediary between the vision and language modalities. The Q-Former, through its learnable query vectors, efficiently extracts and distills visual features from a frozen image encoder, presenting them to a frozen LLM in a manner that facilitates seamless vision-language integration.

The model leverages the robust capabilities of existing pre-trained image and language models, achieving state-of-the-art performance across a variety of vision-language tasks such as visual question answering, image captioning, and image-text retrieval. Moreover, BLIP-2 introduces the capacity for zero-shot image-to-text generation, opening avenues for applications like visual knowledge reasoning and conversational AI. Importantly, the use of frozen models paired with the lightweight Q-Former makes BLIP-2 significantly more compute-efficient than its predecessors.

8.3.3 CLIP

CLIP (Contrastive Language-Image Pre-training), developed by OpenAI, introduces an approach to understanding images in the context of natural language, facilitating a wide range of applications from zero-shot classification to complex reasoning tasks. At the heart of CLIP's design is a dual-encoder framework, comprising a ViT or a ResNet architecture for image encoding, and a transformer-based model for text encoding. This design enables CLIP to process and embed images and text into a shared representation space, leveraging the strength of contrastive learning to tightly couple the two modalities.

Contrastive learning, as implemented in CLIP's training methodology, is a technique designed to maximize the similarity between related (or positive) pairs of data while minimizing the similarity between unrelated (or negative) pairs. In the setup of CLIP, this involves teaching the model to align the embeddings of corresponding image-text pairs closer together in the shared representation space, while pushing the embeddings of non-corresponding pairs further apart.

This process begins with CLIP's dual-encoder framework, where one encoder processes images (using either a ViT or a ResNet architecture) and the other processes textual descriptions (using a transformer-based model). During training, CLIP is presented with batches of image-text pairs, each consisting of a genuine pairing from its diverse, internet-sourced dataset and multiple mismatched pairings.

For each genuine image-text pair, the model calculates similarity scores between the image embedding and the correct text embedding versus embeddings of mismatched texts. The goal is for the model to learn feature representations that maximize the similarity score for genuine pairs and minimize it for mismatched pairs. This is typically achieved through a contrastive loss function, such as the triplet loss or the noise contrastive estimation loss, which quantifies the difference in similarity scores and guides the model's optimization process.

The contrastive learning approach enables CLIP to develop a nuanced understanding of the relationships between visual and textual content without direct supervision for specific tasks. As a result, the model learns a broad, generalizable understanding of visual concepts and their natural language descriptions. By leveraging this large-scale, pre-trained

knowledge, CLIP can accurately categorize images into various classes it was not explicitly trained on, showcasing its flexibility and efficiency across a wide array of applications.

8.3.4 X-CLIP

X-CLIP introduces an approach to enhancing video recognition capabilities by integrating cross-frame communication and video-specific prompting into existing language-image pre-trained models. This enables the seamless extension of these models to handle video data, preserving their original performance while broadening their applicability. The name X-CLIP derives from leveraging and expanding upon the set of existing image-level cross-modality pre-trained models, specifically CLIP, Florence and eXpand, and extending their application to the video domain.

X-CLIP introduces a video-specific prompting technique that automatically generates instance-level textual representations. This technique leverages the content information within videos to produce more contextually relevant and discriminative textual prompts, thereby improving the alignment between video representations and their textual counterparts. This addresses the challenge of generating effective textual prompts for videos, which is crucial for tasks that involve matching or relating video content with text. Image 8.12 shows an overview of the model's architecture.

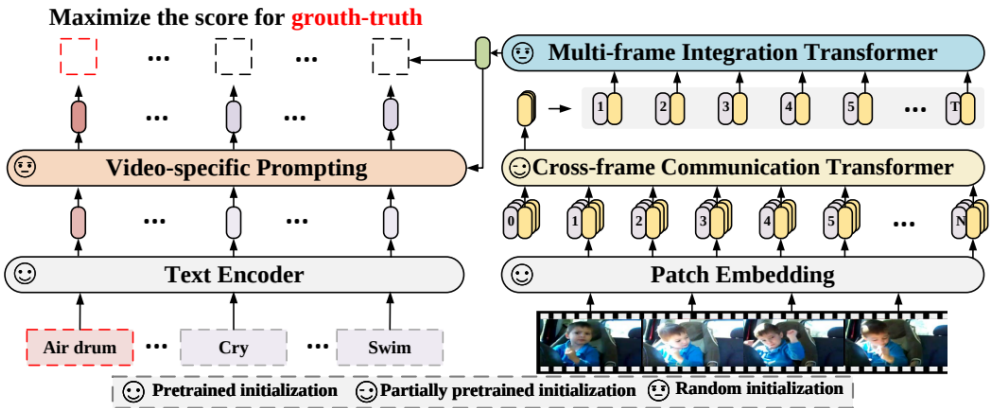


Figure 8.12 Overview of the architecture of X-CLIP, and how the model aligns video and text representations through a video encoder, text encoder, and a video-specific prompt generator, leveraging pretrained language-image models with video temporal modeling for improved semantic understanding. Image is taken from the paper: "Expanding Language-Image Pretrained Models for General Video Recognition".

Another central feature of X-CLIP's design is its cross-frame communication attention mechanism. This module facilitates temporal modeling by allowing information exchange across video frames, enhancing the model's ability to understand and represent video content dynamically. By incorporating this mechanism, X-CLIP can capture the nuanced temporal relationships inherent in video data, setting it apart from traditional models that may only consider spatial aspects.

8.3.5 *Flamingo*

Flamingo stands out for its capability to process visual data and text in an interleaved manner, generating text outputs in a contextually rich and open-ended format. At the core of Flamingo's architecture is the integration of pre-trained vision and language models through the perceiver resampler and initialized cross-attention layers. This setup allows for efficient handling of spatio-temporal features from visual inputs (images or videos) and their seamless integration with text, enabling the model to generate language based on visual context.

A key component, the perceiver resampler, facilitates the transformation of large feature maps from the vision encoder into a concise set of visual tokens. This mechanism reduces the computational load during the vision-text cross-attention process, enhancing the model's efficiency without compromising its ability to understand complex visual inputs. Moreover, Flamingo leverages gated cross-attention dense layers, inserted within a frozen pre-trained language model to condition language generation on visual inputs. This allows Flamingo to generate language that is not only contextually relevant to the visual inputs but also rich and coherent, bridging the gap between visual perception and linguistic expression.

8.3.6 *OpenFlamingo*

OpenFlamingo is designed to enhance in-context learning by processing interleaved image-text sequences and currently does not support video inputs. OpenFlamingo adopts the Flamingo architecture, using dense cross-attention modules attached to the layers of a frozen, autoregressive language model. This architecture allows text tokens to attend to their corresponding images, enabling the model to predict the next text token based on both visual and textual contexts. A key component, the perceiver resampler, transforms patch features from images, processed by a frozen vision encoder (CLIP ViT-L/14), into a manageable number of visual tokens, streamlining the vision-text cross-attention process.

The model is trained on a mix of open-source image-text pairs and interleaved image-text sequences, namely LAION-2B and Multimodal C4 (MMC4). This training setup allows OpenFlamingo to learn from a diverse range of visual and textual inputs, significantly enhancing its adaptability to various tasks. The inclusion of synthetic data, generated through ChatGPT, further enriches the training dataset, providing a broader spectrum of image-text contexts.

8.3.7 *GPT-4 with vision*

Just like GPT-4, GPT-4V completed its training phase in 2022, with early access rolling out in March 2023. Leveraging GPT-4's foundational technology for visual understanding, GPT-4V underwent a similar training process. Initially, it was trained to anticipate the subsequent word in texts, utilizing a vast amount of text and image data sourced from the internet and various licensed databases. Subsequently, it underwent refinement through RLHF, enhancing its output to align with human preferences.

The approach to preparing for deployment focused on evaluating and addressing risks associated with images of people, including issues like identifying individuals, biases in outputs from people's images that could lead to representational or allocational harms, and

the possibility of the model’s performance increasing significantly in high-risk areas such as medicine and scientific expertise.

8.3.8 LLaVA

LLaVA (Large Language and Vision Assistant) focuses on bridging the capabilities of pre-trained LLMs with advanced visual understanding. This facilitates a new era of instruction-following agents capable of comprehending and acting upon complex multimodal inputs across various tasks, including visual question answering, visual reasoning, and more, through end-to-end training.

Demonstrating state-of-the-art performance in few-shot learning on multimodal tasks, LLaVA sets new benchmarks in visual question answering, visual reasoning, and conversational AI. The comparison of instruction-following capabilities on the LLaVA-Bench (In-the-Wild) is presented in 8.1, with average scores and standard deviations ($\pm$). Across the first three rows, outcomes from three separate inference attempts are disclosed. Notably, LLaVA outperforms competing models.

Table 8.1 Performance comparison across tasks

Model	Conversation	Detail Description	Complex Reasoning	All
OpenFlamingo [5]	19.3 $\pm$ 0.5	19.0 $\pm$ 0.5	19.1 $\pm$ 0.7	19.1 $\pm$ 0.4
BLIP-2 [28]	54.6 $\pm$ 1.4	29.1 $\pm$ 1.2	32.9 $\pm$ 0.7	38.1 $\pm$ 1.0
LLaVA	57.3 $\pm$ 1.9	52.5 $\pm$ 6.3	81.7 $\pm$ 1.8	67.3 $\pm$ 2.0
LLaVA [†]	58.8 $\pm$ 0.6	49.2 $\pm$ 0.8	81.4 $\pm$ 0.3	66.7 $\pm$ 0.3

LLaVA leverages the concept of GPT-assisted visual instruction data generation, addressing the scarcity of multimodal instruction-following data by utilizing GPT-4/ChatGPT to expand image-text pairs into rich, instruction-based datasets. This method not only enhances data diversity but also deepens the reasoning and contextual understanding required for multimodal instruction following.

The core of LLaVA’s architecture is a seamless integration between a pre-trained visual encoder (CLIP’s ViT-L/14) and Vicuna, which is an open-source chatbot trained by fine-tuning LLaMA, connected via a simple yet effective linear projection layer. This design allows the model to convert visual inputs into language embeddings, enabling the LLM to process and respond to visual data as if it were text.

A pivotal component of LLaVA’s methodology, visual instruction tuning, is aimed at refining the model’s ability to follow complex instructions across both visual and textual domains. Unlike visual prompt tuning, which focuses on model adaptation efficiency, instruction tuning enhances the model’s capacity to interpret and act on detailed multimodal instructions, pushing the boundaries of its generative and reasoning capabilities.

LLaVA undergoes a two-stage instruction tuning process, beginning with feature alignment pre-training on curated image-text pairs to align visual features with LLM embeddings. The subsequent fine-tuning stage leverages the generated multimodal instruction-following dataset to give the model robust instruction-following abilities.

8.4 Applications and worked examples

Now it's time to take some of these MLLMs you've learned about in the previous section for a test spin. First, we will compare the performance of three different vision-language MLLMs, namely BLIP-2, LLaVA, and GPT-4. Then, we will look into how you can use Gradio, an open-source Python package that allows you to quickly build a demo or a web application for your machine learning model, to build a multimodal chatbot.

8.4.1 Comparison of different MLLMs for visual reasoning and chat capabilities

In this section we will use a rather unusual image and compare the visual reasoning and chat capabilities of BLIP-2, LLaVA and GPT-4. For this comparison, I will use the same image, as in the LLaVA paper, which is shown in image 8.13.



Figure 8.13 Test-image for the comparison of different MLLMs.

Upon examining the image, it becomes clear that identifying the activity depicted may pose a challenge for most multimodal models due to the unconventional nature of the scene. Let us look how the models classify the scene. To do so, I will ask the model this question: "What is unusual about this image?". But first, let's prepare the models to be able to answer the question. Listing 8.1 shows how you can initialize the BLIP-2 model in your notebook with quantization.

Listing 8.1 Initialize BLIP-2 with quantization

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16
)

model_id = "Salesforce/blip2-opt-2.7b"
```

```
processor = AutoProcessor.from_pretrained(model_id)
model = Blip2ForConditionalGeneration.from_pretrained(model_id,
quantization_config=quantization_config, device_map="auto")
```

Listing 8.2 shows how you can create a pipeline with Hugging Face and prompt the model with the question.

Listing 8.2 Prompt BLIP-2 with question

```
pipe = pipeline("image-to-text", model=model_id,
model_kwargs={"quantization_config": quantization_config})

prompt = "Question: What is unusual about this image? Answer:"

outputs = pipe(image, prompt=prompt, generate_kwargs={
    "max_new_tokens": 300, "do_sample" : True, "temperature" : 0.8})

print(outputs[0]["generated_text"])
```

I'll omit the model's answer here and proceed directly with the next model, LLaVA, so we can compare all the outputs in one place. To initialize LLaVA, the process (8.3) is the same as with BLIP-2, only the model_id needs to be changed.

Listing 8.3 Initialize LLaVA with quantization

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16
)

model_id = "llava-hf/llava-1.5-7b-hf"

processor = AutoProcessor.from_pretrained(model_id)
model = LlavaForConditionalGeneration.from_pretrained(model_id,
quantization_config=quantization_config, device_map="auto")
```

The next step is also mostly the same as with BLIP-2, but for the prompt we need a different template. This is one important thing you have to consider when you are using models. Usually, if you use models via Hugging Face, you find on the model card page a short description telling you how you have to prompt the model.

Listing 8.4 Prompt LLaVA with question

```
pipe = pipeline("image-to-text", model=model_id,
model_kwargs={"quantization_config": quantization_config})

prompt = "USER: <image> What is unusual about this image? ASSISTANT:"

outputs = pipe(image, prompt=prompt, generate_kwargs={
    "max_new_tokens": 300, "do_sample" : True, "temperature" : 0.8})

print(outputs[0]["generated_text"])
```

Let us now compare the responses from each model.

BLIP-2 it's a man in a yellow shirt standing on the back of a taxi

LLaVA In this image, there is something unusual happening on a busy street. A man is standing on the back of a yellow taxi cab, performing a laundry service while the taxi drives down the road. This is not a common sight, as people typically do not stand on the roof of a moving vehicle. It might raise questions about the safety and legality of such an action. Moreover, the image also features traffic, including a truck, which demonstrates the bustling nature of the street.

CHATGPT-4 The image depicts a rather unusual and humorous situation where a man appears to be ironing clothes on an ironing board that is mounted on top of a yellow taxi. Typically, ironing is a domestic task performed within the home or a laundry facility, not on the move atop a vehicle, especially not in the middle of what appears to be a busy city street. The juxtaposition of a stationary household chore with the dynamic environment of city traffic creates a surreal and comical effect.

Interestingly, despite being only trained on a modest-sized multimodal instruction-following dataset (approximately 80,000 unique images), LLaVA achieves reasoning outcomes that closely match those of multimodal GPT-4. It's worth noting that even though these images are beyond LLaVA's initial training scope, the model is still capable of comprehending the scenes and adhering to the question instructions to deliver a plausible answer, showcasing its zero-shot capability. In contrast, BLIP-2 tends to describe the image rather than responding directly to my question/instruction.

Since the LLaVA shows a remarkable result in visual question answering, let us use this model to build a multimodal chatbot. Here are several reasons why using Gradio makes sense for deploying a multimodal chatbot, especially one powered by a model like LLaVA:

- **Ease of Use:** Gradio allows you to quickly create web interfaces for your machine learning models with just a few lines of code. This means you can focus more on the model itself and less on the intricacies of web development.
- **Interactive Demos:** With Gradio, you can create interactive demos that let users input text, images, and other data types, then see the outputs generated by your model. This interactivity is crucial for a multimodal chatbot, as it enables users to interact with the model in real-time, providing a practical demonstration of its capabilities. And you can analyze the model errors during testing, since you can use a debug mode.
- **Customization and Flexibility:** Despite its simplicity, Gradio offers a degree of customization that allows you to tailor the look and feel of your demo to match your needs. You can customize the interface to highlight the features of your multimodal chatbot, making it more engaging for users.

To use Gradio with LLaVA, you have again to first initialize the model as before. This is shown in listing 8.5.

Listing 8.5 Initialize LLaVA with quantization

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16
)

model_id = "llava-hf/llava-1.5-7b-hf"

processor = AutoProcessor.from_pretrained(model_id)
model = Blip2ForConditionalGeneration.from_pretrained(model_id,
    quantization_config=quantization_config, device_map="auto")
```

To use Gradio with its chat interface, there is a need for a function called *update_conversation*. Listing 8.7 shows an example of such a customized function.

Listing 8.6 Customized update conversation function for Gradio

```
def update_conversation(new_message, history, image=None):
    if image is None:
        return "Please upload an image, since this is an image to text model."
    ,

    prompt = "USER: <image>"

    for user_message, assistant_response in history:
        prompt += f"{user_message}ASSISTANT: {assistant_response}USER: "

    prompt += new_message + "ASSISTANT: "

    outputs = pipe(image, prompt=prompt, generate_kwargs=
{"max_new_tokens": 200, "do_sample": True,
"temperature": 0.7})[0]['generated_text']

    return outputs[len(prompt)-5:]
```

Now, to launch the chatbot, you need to call the Blocks() function and then launch the demo as shown in listing.

Listing 8.7 Launch Gradio chatbot in demo mode

```
with gr.Blocks() as demo:
    with gr.Row():
        image = gr.Image(type='pil', interactive=True)

        chat = gr.ChatInterface(
            update_conversation,
            additional_inputs=[image]
        )

demo.launch(debug=True)
```

The "demo.launch(debug=True)" will launch an interactive UI in your notebook, as shown in image 8.14.

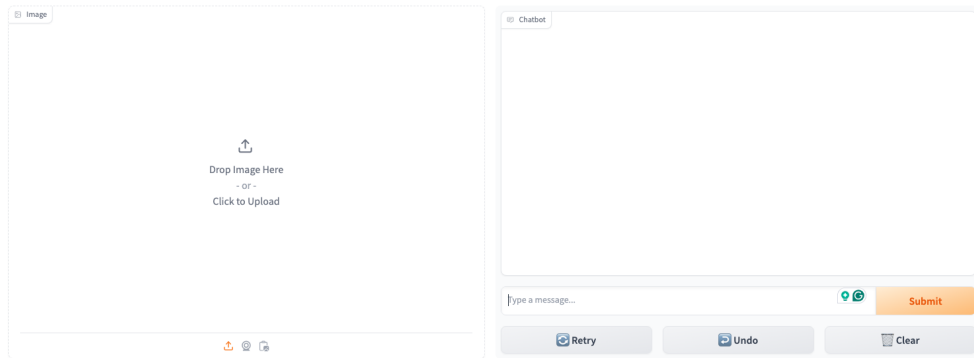


Figure 8.14 Launched Gradio UI for the chatbot using LLaVA as a multimodal model.

Or you can use the provided link to share it via a public URL. If you want to deploy the model, you would have to run it via a bash with "gradio deploy", after that it will be permanent freely hosted on Hugging Face Spaces (<https://huggingface.co/spaces>). This integration facilitates easy sharing and deployment of your models without the need for setting up your own server infrastructure. It also places your work within a community of AI researchers and practitioners, increasing its visibility.

Within that launched UI of the chatbot you can now upload an image and ask the model a question about the image or task it with creating an image caption for you. In image 8.15 you can see the image I questioned the other models before as well as my question to the deployed LLaVA model and its answer to my question.

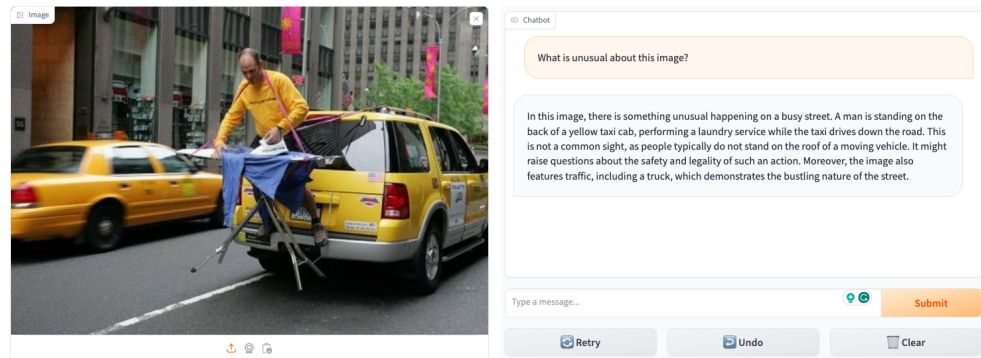


Figure 8.15 Launched Gradio UI for the chatbot using LLaVA as a multimodal model.

Closing this section, I'd like to point out something pretty cool about controlling how your model comes up with its answers. In the example I showed earlier (check out listing 8.5), there's this neat option called "generate_kwargs=" in the model's pipeline setup.

This feature is super handy because it lets you tap into the power of any model from the Hugging Face Hub for a bunch of tasks, whether it's about language, vision, speech, or even combining them. The best part? You don't need to be a pro in these areas or know all the nitty-gritty of the model to make this work for you.

With the `"generate_kwargs="` option, you can play around with different ways for the model to generate responses. Whether you're into experimenting with sampling methods or fine-tuning how the model picks its next words, this feature has you covered. You can enable various generation strategies like multinomial sampling or even get fancy with things like Top-K and Top-p sampling. I dived into all these strategies starting from section 6.3.2 through to section 6.3.6, so you can revisit it there, if you need a short refresher, what each strategy does for you.

For a deeper dive into how the pipeline function works and all the settings you can tweak for your model's generation strategy, check out these links: the pipeline tutorial at https://huggingface.co/docs/transformers/pipeline_tutorial and a guide on generation strategies at https://huggingface.co/docs/transformers/en/generation_strategies.

8.5 Summary

- Multi-modality in AI has been adapted from the way humans perceive their environment, having five senses: sight, hearing, touch, smell, and taste.
- To bridge the semantic gap and enable large language models to understand different modalities, there exist two distinct methods: the perceiver and the converter. The perceiver module, for instance, connects a vision encoder to the frozen language model. The converter, on the other hand, converts various inputs in such a way that the LLM can digest them.
- Current MLLM research is mostly focused on vision-text. Some of the state-of-the-art models include: BLIP-2, CLIP, X-CLIP, Flamingo, GPT-4, and LLaVA, with the last two achieving similar results in visual reasoning and question-answering tasks, despite LLaVA being trained only on a small multimodal instruction-following dataset.
- Gradio is an easy-to-use, open-source, Python-based package that speeds up the rapid prototyping and demonstration of an AI system. With this setup, you can quickly iterate over different models or model configurations, get immediate feedback from users, and refine your chatbot accordingly.

Bibliography

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning, 2022. [arXiv:2204.14198](#). 216
- [2] Anas Awadalla, Irena Gao, Josh Gardner, Jack Hessel, Yusuf Hanafy, Wanrong Zhu, Kalyani Marathe, Yonatan Bitton, Samir Gadre, Shiori Sagawa, Jenia Jitsev, Simon Kornblith, Pang Wei Koh, Gabriel Ilharco, Mitchell Wortsman, and Ludwig Schmidt. Openflamingo: An open-source framework for training large autoregressive vision-language models, 2023. [arXiv:2308.01390](#). 222
- [3] Peng Gao, Jiaming Han, Renrui Zhang, Ziyi Lin, Shijie Geng, Aojun Zhou, Wei Zhang, Pan Lu, Conghui He, Xiangyu Yue, Hongsheng Li, and Yu Qiao. Llama-adapter v2: Parameter-efficient visual instruction model, 2023. [arXiv:2304.15010](#). 220
- [4] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023. [arXiv:2301.12597](#). 215
- [5] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation, 2022. [arXiv:2201.12086](#). 222
- [6] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023. [arXiv:2304.08485](#). 218
- [7] Bolin Ni, Houwen Peng, Minghao Chen, Songyang Zhang, Gaofeng Meng, Jianlong Fu, Shiming Xiang, and Haibin Ling. Expanding language-image pretrained models for general video recognition, 2022. [arXiv:2208.02816](#). 222
- [8] Renrui Zhang, Jiaming Han, Chris Liu, Peng Gao, Aojun Zhou, Xiangfei Hu, Shilin Yan, Pan Lu, Hongsheng Li, and Yu Qiao. Llama-adapter: Efficient fine-tuning of language models with zero-init attention, 2023. [arXiv:2303.16199](#). 219

Optimize and evaluate large language models

This chapter covers

- Deep dive into hyperparameters
- Hyperparameter optimization with Ray
- Effective strategies for experiment tracking
- Model pruning and distillation
- Parameter efficient fine-tuning
- Various quantization techniques
- Model sharding

Large language models have transformed how we approach tasks ranging from translation to content generation. However, their size presents unique challenges, and with that the need for innovative optimization strategies to ensure efficiency.

Therefore, this chapter aims to equip you with the knowledge and tools to refine and evaluate your language models, and provide you with an understanding of the most recent advancements in efficiency, scalability, and performance, by bridging theoretical concepts with practical applications.

I begin by demystifying hyperparameters, shedding light on their pivotal role in model performance. This is why you need tools for optimization and need to know how you

can track your experiment, and evaluate the model performance. Effective experiment tracking is crucial for iteratively refining your models and systematically understanding their behavior under different configurations.

In the subsequent sections, I introduce targeted optimization techniques such as pruning, distillation, quantization, and sharding. Through practical examples, I illustrate how these optimization techniques can be effectively implemented, providing you with actionable insights for your own projects.

Let's start with hyperparameters, and the exploration how their careful adjustment can lead to significant improvements in model efficiency and performance. This foundational knowledge paves the way for more advanced optimization techniques, such as pruning and sharding in the later sections to come.

9.1 *Deep dive into hyperparameters*

Hyperparameters play a pivotal role in controlling your LLM's performance. Hyperparameters are different from model parameters. We must set our hyperparameters before we start training; they can't be learned during training. Typical hyperparameters are learning rate and its decay, or the number of epochs you train your model. Model parameters, on the other hand, are the parameters, your model learns based on your data. With LLMs, for instance the attention weights, which are important for your model to understand words and generate later on meaningful outputs. Both, model parameters and model hyperparameters are important for the model optimization process, that is, for finding the minimum of a loss function. To optimize the loss function, we use gradient descent. Its goal is to minimize the loss function, and, thereby improving the model's predictions. Let us look how parameters and hyperparameters factor into gradient descent in the next section.

9.1.1 *How parameters and hyperparameters factor into gradient descent*

Model parameters are the weights and biases in the model that are adjusted during training through the process of gradient descent and are learned directly from the data the model processes during training. The algorithm computes the gradient of the loss function with respect to each parameter, indicating the direction in which the parameter should be adjusted to minimize the loss. The model parameters are then updated in the opposite direction of the gradient, scaled by a step size known as the learning rate (which is a hyperparameter).

While model parameters are learned directly from the data during training, hyperparameters are set before training begins and dictate how the training progresses. Key hyperparameters in gradient descent include:

- **Learning rate:** Determines the size of the steps taken during the update of model parameters. A too high learning rate can cause the model to overshoot the minimum, while a too low learning rate can result in a long training process or the model getting stuck in a local minimum.
- **Batch size:** Influences how many data points are used to calculate the gradient at each step. It affects the stability and speed of the convergence.

- **Number of epochs:** Dictates how many times the entire dataset is passed through the network. Training a model for a greater number of epochs can lead to better learning, up to a point. Beyond this point, the model might start overfitting.

Overfitting occurs when a model "sees" the data too often and begins to memorize it rather than learning to generalize from it. This memorization means the model performs well on the training data but poorly on new, unseen data because it has not truly learned the underlying patterns. To mitigate overfitting, we employ regularization methods such as early stopping. Early stopping interrupts the training process if it detects that the model is no longer improving on a validation metric for a specified number of epochs. This approach ensures that the model is trained just enough to learn from the data without memorizing it, striking a balance between learning and generalization.

In the process of training your LLM, the choice of learning rate is also a critical decision that affects how quickly a model learns and whether it learns successfully. The learning rate determines the size of the steps the model takes towards the minimum of the loss function. The following two figures, 9.1 and 9.2, respectively, offer a visual comparison between two learning rates: one with $lr=0.1$ and the other with $lr=0.01$.

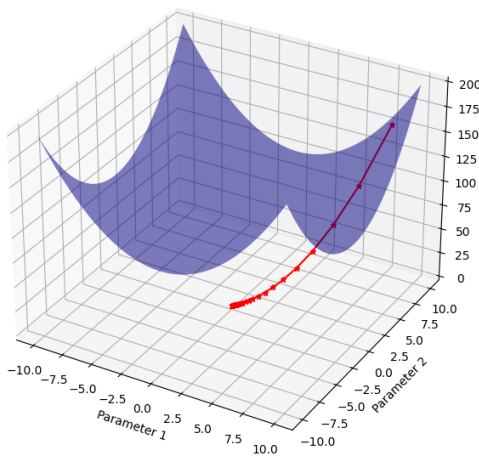


Figure 9.1 Gradient descent with learning rate=0.1, demonstrating faster convergence.

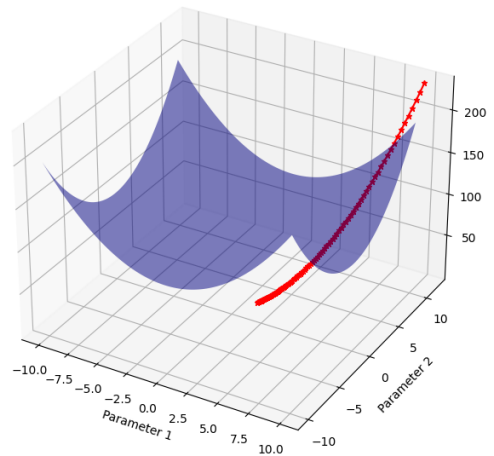


Figure 9.2 Gradient descent with learning rate=0.01, illustrating stability in convergence.

Image 9.1 shows that the steps taken by the gradient descent algorithm are relatively large, which is indicative of a higher learning rate. This allows the optimization process to move quickly towards the minimum of the loss function. The potential risk associated with this approach is that if the learning rate were any higher, it might cause the algorithm to overshoot the minimum, bouncing around it without settling down, or, in the worst case, diverging entirely.

Image 9.2 shows a much finer trajectory with smaller steps towards the minimum, characteristic of a lower learning rate. The advantage of this more cautious approach is that it reduces the risk of overshooting and provides a more stable convergence to the minimum.

However, the trade-off is speed with such small steps, the algorithm will take a longer time to reach the minimum, which means more iterations and potentially more computational resources.

That "little" 0 in the learning rate that turns 0.1 into 0.01 may seem minor, but as these images demonstrate, it has a profound effect on the behavior of the gradient descent optimization. With $lr=0.1$, the model might converge quickly but with less precision, while with $lr=0.01$, the model converges more slowly but with potentially greater accuracy.

As you can see, hyperparameters significantly influence the training process's efficiency and the final model's performance. Hence, proper tuning of hyperparameters can lead to faster convergence in the loss function and better generalization of the model to new data. Techniques such as grid search, random search, and Bayesian optimization are commonly used for hyperparameter tuning to find the optimal settings.

LLMs have hundreds of millions, if not billions, of parameters, making the optimization process computationally intensive and highlighting with that the importance of efficient hyperparameter tuning. Moreover, advanced variants of gradient descent, such as Adam (adaptive moment estimation), are often used to optimize LLMs due to their ability to adapt the learning rate for each parameter, illustrating the nuanced interplay between parameters and hyperparameters in these models. After having learned what hyperparameters are and why they are important let us now look at some practical examples in the next section.

9.2 *Model tuning and hyperparameter optimization*

Now it is time to put the learned theory from the previous section into practice and look at how you can tune your LLM's hyperparameter. For this, I will use Ray, which is an open-source framework designed for scaling AI applications. It offers a parallel processing compute layer, so you don't need to know anything about distributed systems. From Ray, I will use Tune (<https://docs.ray.io/en/latest/tune/index.html>), which is a Python library for experiment execution and hyperparameter tuning at any scale. And Ray Data (<https://docs.ray.io/en/latest/data/data.html>), which is a scalable data processing library for ML workloads. For this example, I will use the SuperGLUE benchmark dataset, which I introduced in section 5.2, more information about this benchmark can be found here <https://super.gluebenchmark.com/>. Specifically, I will use the CommitmentBank (cb) task. The CommitmentBank is a collection of 1,200 natural discourses, each concluding with a sentence that includes a clause-embedding predicate beneath an operator that cancels entailment (such as a question, modal, negation, or the antecedent of a conditional). The dataset has the following fields:

- Premise: a string feature.
- Hypothesis: a string feature.
- idx: a int32 feature.
- label: a classification label, with possible values including entailment (0), contradiction (1), neutral (2).

To use Ray data with a Hugging Face dataset, we first have to load the dataset from Hugging Face and then convert it into a Ray data dataset. This process is illustrated in listing 9.1. Note all following code examples can be found in <https://github.com/Nicolepcx/Transformers-in-Action/tree/main/CH09>

Listing 9.1 Use Ray data with Hugging Face

❶ **Load dataset from Hugging Face**

```
datasets = load_dataset("super_glue", "cb")
```

❷ **Convert to Ray dataset**

```
ray_datasets =  
    "train": ray.data.from_huggingface(datasets["train"]),  
    "validation": ray.data.from_huggingface(datasets["validation"]),  
    "test": ray.data.from_huggingface(datasets["test"]),
```

Next we tokenize the input sentences, premises and hypotheses, respectively. This process is shown in listing 9.2.

Listing 9.2 Tokenize input sentences

```
def tokenize_fn(samples):  
    outputs = tokenizer(  
        samples["premise"],  
        samples["hypothesis"],  
        truncation=True,  
        padding="longest",  
        return_tensors="pt"  
    )
```

❶ **Add labels to outputs and convert to tensor**

```
outputs["labels"] = torch.tensor(samples["label"], dtype=torch.long)
```

❷ **Move all tensors in outputs to GPU**

```
outputs = {key: value.to('cuda') for key, value in outputs.items()}  
  
return outputs
```

Next we need to define a `train_func` this is required by Ray to run properly. How you can set up this function is given in listing 9.3.

Listing 9.3 Setup training function

```
def train_func(config):  
    ❶ Load metric  
    metric = load_metric("super_glue", task)  
    ❷ Get model and tokenizer  
    tokenizer = AutoTokenizer.from_pretrained(model_checkpoint, use_fast=True)  
    model = AutoModelForSequenceClassification.from_pretrained(  
        model_checkpoint, num_labels=num_labels
```

```
)
```

3 Prepare the dataset

```
train = ray.train.get_dataset_shard("train")
eval = ray.train.get_dataset_shard("eval")

train_iterable = train.iter_torch_batches(
    batch_size=batch_size, collate_fn=tokenize_fn
)
eval_iterable = eval.iter_torch_batches(
    batch_size=batch_size, collate_fn=tokenize_fn
)
```

4 Initialize the training arguments

```
args = TrainingArguments(
    name,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    per_device_train_batch_size=config.get("batch_size", 64),
    per_device_eval_batch_size=config.get("batch_size", 64),
    learning_rate=config.get("learning_rate", 2e-5),
    num_train_epochs=config.get("epochs", 6),
    weight_decay=config.get("weight_decay", 0.001),
    push_to_hub=False,
    max_steps=max_steps_per_epoch * config.get("epochs", 6),
    disable_tqdm=False,
    no_cuda=not torch.cuda.is_available(),
    report_to='wandb',
    run_name='superclue_cb'
)
```

5 Custom evaluation function

```
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return metric.compute(predictions=predictions, references=labels)
```

6 Initialize trainer class

```
trainer = Trainer(
    model,
    args,
    train_dataset=train_iterable,
    eval_dataset=eval_iterable,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)
```

7 Add a callback

```
trainer.add_callback(RayTrainReportCallback())

trainer = prepare_trainer(trainer)
trainer.train()
```

Now that you have your training function setup, you can use the `TorchTrainer` class from Ray to control the workers and resources used for the training, as illustrated in listing 9.4.

Listing 9.4 Instantiate the `TorchTrainer` class

```
trainer = TorchTrainer(
    train_func,
    scaling_config=ScalingConfig(num_workers=num_workers, use_gpu=use_gpu),
    datasets=
        "train": ray_datasets["train"],
        "eval": ray_datasets["validation"],
    ,
    run_config=RunConfig(
        checkpoint_config=CheckpointConfig(
            num_to_keep=1,
            checkpoint_score_attribute="eval_loss",
            checkpoint_score_order="min",
        ),
    ),
)
```

Now with this setup in place you can use the `Tuner` class from Ray to do the hyperparameter search for you. How you do this, is shown in listing 9.5.

Listing 9.5 Instantiate the `Tuner` class

```
tuner = Tuner(
    trainer,
    param_space={
        "train_loop_config": {
            "learning_rate": tune.grid_search([2e-5, 2e-4, 2e-3, 2e-2]),
            "epochs": tune.choice([2, 4, 6, 8]),
            "batch_size": tune.choice([16, 32, 64, 128]),
            "weight_decay": tune.grid_search([0.0, 0.01, 0.1, 0.001])
        }
    },
    tune_config=tune.TuneConfig(
        metric="eval_loss",
        mode="min",
        num_samples=1,
        scheduler=ASHAScheduler(
            max_t=max([2, 4, 6, 8]),
            grace_period=1,
            reduction_factor=2,
        ),
    ),
    run_config=RunConfig(
        name="tune_transformers",
```

```

        checkpoint_config=CheckpointConfig(
            num_to_keep=1,
            checkpoint_score_attribute="eval_loss",
            checkpoint_score_order="min",
        ),
    ),
)

```

To use the hyperparameters, from the search, you can use the `get_best_result` function and pass the parameters in the `TorchTrainer` class as shown in listing 9.6.

Listing 9.6 Use best hyperparameter for training

```

best_trial = tune_results.get_best_result(metric="eval_loss", mode="min")
train_loop_config = best_trial.config['train_loop_config']

```

❶ Preparing the configuration with the best hyperparameters

```

trainer = TorchTrainer(
    train_loop_per_worker=train_func,
    train_loop_config=train_loop_config,
    scaling_config=ScalingConfig(num_workers=num_workers, use_gpu=use_gpu),
    datasets=
        "train": ray_datasets["train"],
        "eval": ray_datasets["validation"],
    ,
    run_config=RunConfig(
        checkpoint_config=CheckpointConfig(
            num_to_keep=1,
            checkpoint_score_attribute="eval_loss",
            checkpoint_score_order="min",
        ),
    ),
)

```

❷ Start the training process with the best hyperparameters

```

result = trainer.fit()

```

Now you can save the newly trained model from the best checkpoint as shown in 9.8.

Listing 9.7 Use best hyperparameter for training

```

checkpoint: Checkpoint = result.checkpoint

```

❶ Using the checkpoint to access the saved model

```

with checkpoint.as_directory() as checkpoint_dir:
    model = AutoModelForSequenceClassification.from_pretrained(checkpoint_dir)
    print(f"Model loaded from checkpoint_dir")

```

You can now save the model to Hugging Face hub as shown in listing 9.8.

Listing 9.8 Use best hyperparameter for training

❶ **Login to Hugging Face hub**

```
notebook_login()
```

❷ **Login to Hugging Face hub**

```
model.push_to_hub()
```

This model will be stored as a public model. If you are working for a company or you just want to store your model privately, you can use enterprise hub <https://huggingface.co/enterprise>, which allows you to distribute private models and datasets for collaboration both within teams and among different groups. In the next section I will show you how you can track the experiments and why this might be beneficial to do.

9.2.1 Track experiments

You might have noticed throughout the book that I have already integrated Weights & Biases (W&B) here and there into the training arguments of the Trainer class. For instance, in section 7.10, where I showed you an example of SFT and DPO. And I used it also in section 9.2 for hyperparameter optimization. Tracking your experiments is important, as it lets you analyze your model development in an organized manner. In addition, you can create a report and share the results of your training runs and inference with your team. Moreover, you can monitor your system resources, for instance, GPU memory which was allocated, as shown in the image 9.3.

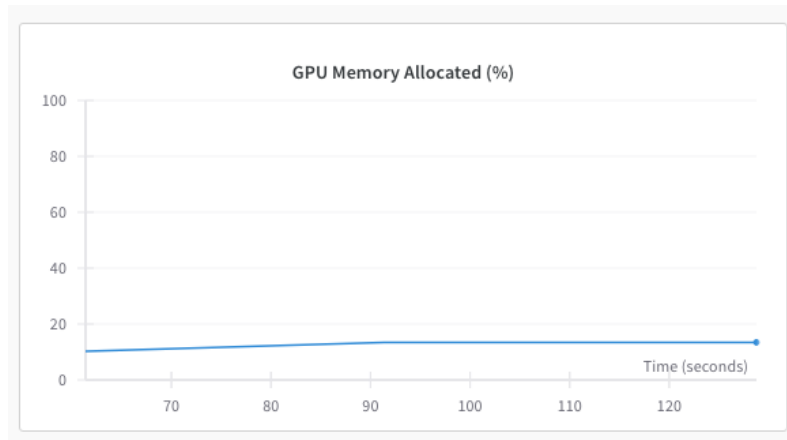


Figure 9.3 Example of how to analyze system resources with Weights & Biases for model training.

Monitoring GPU usage is vital for accurately assessing a models resource needs. If, for example, the GPU utilization does not exceed 60% during training on an A100, one could consider using a less powerful and more cost-effective GPU, such as the V100. This strategic approach to resource allocation is essential for optimizing training performance and can lead to significant cost savings in both training and inference phases of your models

lifecycle. Also, you can use this information as a basis to estimate the costs associated with training and inference of your model.

Another benefit of using W&B is, that it offers a robust implementation of hyperparameter sweeps that can be used with any machine learning framework, including PyTorch and Hugging Face Transformer library. The sweeps feature of W&B allows you to systematically search through combinations of hyperparameters to find the most effective ones for your model, regardless of the underlying framework. W&B allows you to systematically evaluate and identify the best model based on hyperparameter sweeps because it logs the details of each experiment run in a granular manner. This systematic logging includes not just the hyperparameters used in each run but also the metrics generated during training and validation, such as loss and accuracy, among others. To use W&B sweeps, you start by defining a sweep configuration, which specifies the hyperparameters to explore, including their ranges or sets of values to try. You also define the metric to optimize, such as minimizing loss or maximizing accuracy, as shown in listing 9.9

Listing 9.9 Define the sweep configuration

```
sweep_config = {
    'method': 'bayes',
    'metric': {
        'name': 'eval_loss',
        'goal': 'minimize'
    },
    'parameters': {
        'learning_rate': {
            'min': 1e-5,
            'max': 5e-4
        },
        'num_train_epochs': {
            'values': [2, 3, 4]
        },
        'per_device_train_batch_size': {
            'values': [8, 16, 32]
        },
        'model_name_or_path': {
            'values': ['bert-base-uncased', 'distilbert-base-uncased']
        }
    },
    'early_terminate': {
        'type': 'hyperband',
        'min_iter': 3,
        'eta': 2,
        's': 2
    }
}
```

When you start the sweep, as shown in listing 9.10, W&B automates the process of initiating multiple training runs, each with a different combination of hyperparameters based on the strategy you've chosen (e.g., random search, grid search, Bayesian optimization).

Listing 9.10 Start Weights & Biases sweep

```
sweep_id = wandb.sweep(sweep_config, project="huggingface_sweeps")
```

During each run, W&B logs the hyperparameters and the performance metrics for each epoch or step of the training process. This data is sent to the W&B servers and visualized in real-time on the W&B dashboard. An example of the dashboard from a sweep run is shown in image 9.4.

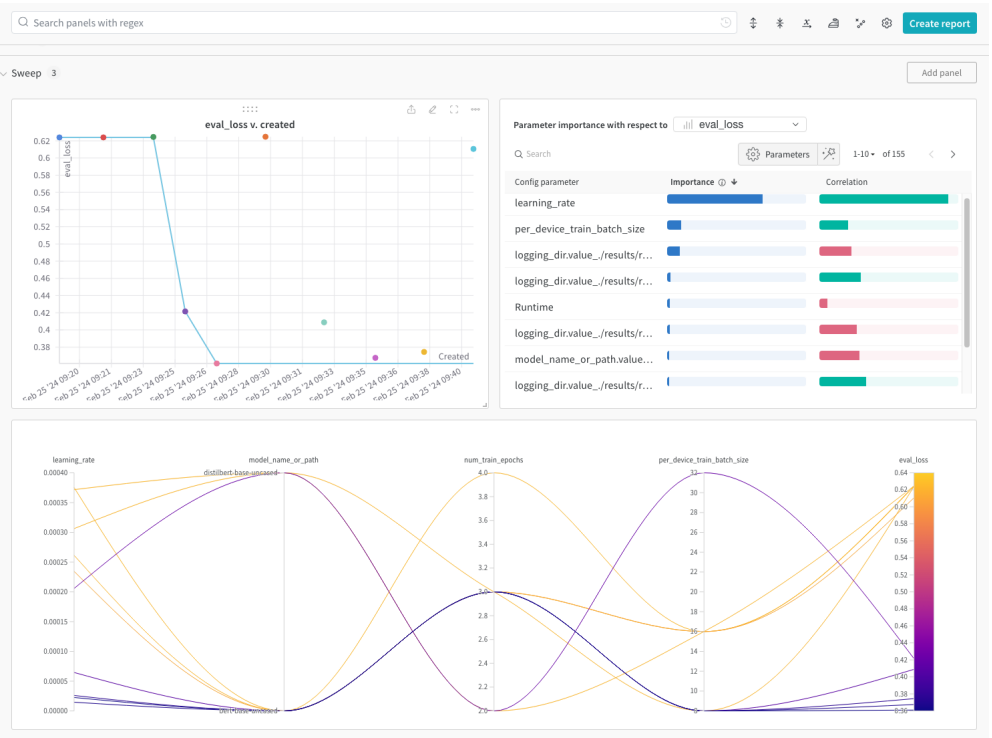


Figure 9.4 Example of W&B dashboard to compare the performance of different hyperparameter combinations.

After the sweep runs are completed (or even while they're running), you can use this dashboard to compare the performance of different hyperparameter combinations. W&B provides various tools for analysis such as the parallel coordinates plot, as shown in 9.5. This visualization allows you to see how different hyperparameters relate to the performance metrics, helping you identify which hyperparameters have the most impact.

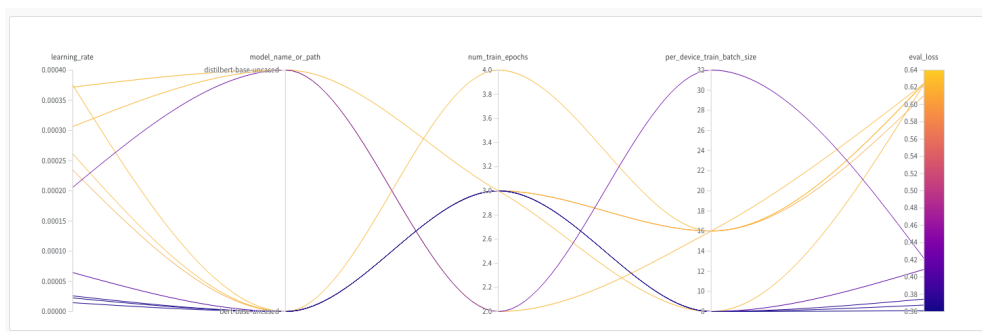


Figure 9.5 Example of parallel coordinates plot from W&B, where the different hyperparameters can be compared to the objective function.

Further, W&B can automatically analyze which hyperparameters were most predictive of success, helping you focus on tuning the most influential parameters, as shown in 9.6

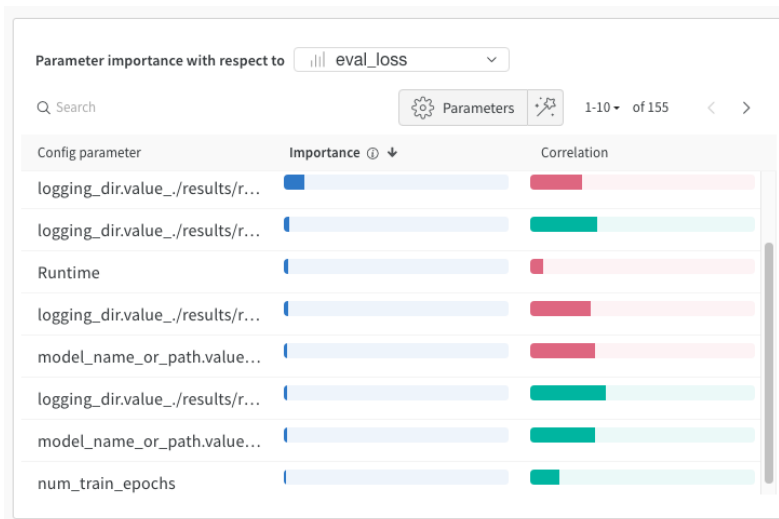


Figure 9.6 A tabular view of all the runs in the sweep, where you can sort and filter based on different metrics and hyperparameters.

Based on the analysis, you can identify which hyperparameter combination led to the best performance based on your chosen metric. You can then use this combination for further experiments or in your final model.

9.3 Techniques for model optimization

Optimizing large language models for efficiency is essential, especially when deploying them on hardware with limited resources. This section dives into three powerful techniques for model optimization: model pruning, model distillation, and quantization. While diverse in approach, these techniques share a common goal: they streamline the underlying LLM

to be less resource-intensive, enabling deployment and inference on less resource intensive hardware with minimal compromise to performance.

9.3.1 Model Pruning

Pruning reduces model complexity by identifying and removing parts of the network that contribute the least to its output, akin to trimming branches from a tree to improve its shape and health. This process makes the LLM more compact, enhancing the inference efficiency of these models. There are two primary ways to perform pruning on your language model: structured and unstructured pruning, a comparison is shown in image 9.7. Both techniques can be applied to LLMs, and the choice between them depends on specific goals.

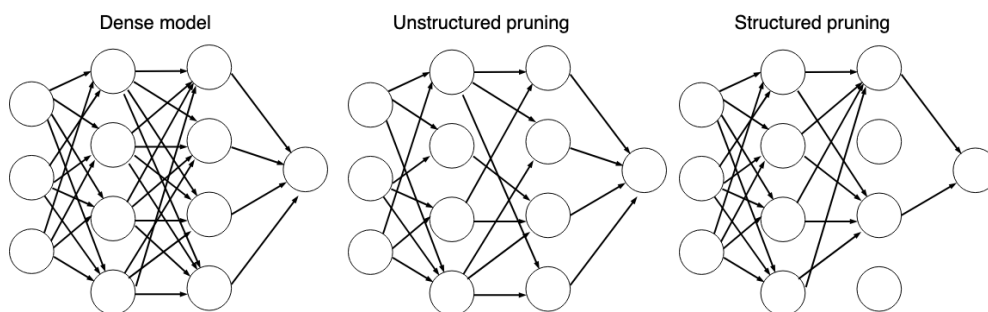


Figure 9.7 The leftmost diagram shows a fully connected, or dense, network where each neuron is connected to every neuron in the subsequent layer. The center diagram represents unstructured pruning, where individual connections are removed, rather than whole neurons or layers. The network retains its original architecture, but with many connections missing. The rightmost diagram illustrates structured pruning, by showing the removal of entire neurons (and their associated connections). This illustrates how structured pruning removes specific parts of a network, such as a neuron in a layer or an entire transformer block.

The illustration is a simplified overview how the two different pruning methods work, for LLMs, which are typically composed with attention mechanisms, the concept of structured pruning could extend to the pruning of attention heads or even entire transformer blocks, whereas unstructured pruning might involve the removal of individual attention weights or feed-forward network parameters.

The following outlines the main differences between both techniques:

- **Unstructured pruning:** This involves removing individual weights or connections within the neural network. This approach can lead to significant reductions in the number of parameters and can compress the model effectively. However, the resulting sparse matrices might not lead to computational efficiency improvements on all hardware, especially if the hardware is optimized for dense matrix operations. Unstructured pruning is more granular and can be more flexible in identifying and removing redundant or less important connections. This is usually used to put the emphasis on maximizing model compression.
- **Structured pruning:** This technique involves removing entire units or groups within the network, such as neurons in linear layers, or attention heads. Structured pruning

is more about reducing the complexity of the model in a way that aligns better with hardware optimizations, often leading to actual computational speedups. It simplifies the network architecture by reducing the number of channels, layers, or other structural components, which can make the model smaller and faster to run but might result in a more significant impact on the model's performance compared to unstructured pruning. This technique is applied if one wants to ensure computational efficiency, while maintaining hardware compatibility.

It's important to note that both pruning methods typically require a careful balance between model size reduction and the retention of sufficient model accuracy. This often requires iterative cycles of pruning followed by retraining or fine-tuning.

9.3.2 *Model Distillation*

Model distillation, also known as knowledge distillation, is a technique used to compress the knowledge of a large, complex model (often called a "teacher" model) into a smaller, more efficient one (called a "student" model). The process involves training the student model to mimic the behavior of the teacher model. The student model is typically a smaller or simplified version of the teacher model, designed to be faster and more resource-efficient. You might remember that I introduced in section 5.2.4 DistilBERT, which is a distilled version of BERT. DistilBERT is trained to mimic the behavior of the larger teacher model (BERT) by minimizing the divergence between their output distributions. This loss measures the Kullback-Leibler divergence between the logits of the teacher and the student models, the result is DistilBERT, a smaller and faster version of BERT that retains much of the teacher model's performance. However, one limitation that knowledge distillation often faces is its reliance on large amounts of unlabeled data.

To tackle this problem, "distilling step-by-step" (3) presents an innovative method that aims to train smaller models to surpass the performance of large language models (LLMs) using less training data than required by standard fine-tuning or distillation techniques, by reducing the need for large unlabeled data by distilling not just labels but also the teachers rationales.

DISTILLING STEP-BY-STEP

This method extracts informative natural language rationales, intermediate reasoning steps, from LLMs. These rationales explain the connections between input questions and their corresponding outputs, thereby providing rich, supervisory signals for training smaller models in a more data-efficient manner.

The process is as follows:

- 1 **Rationale Extraction:** Utilizing few-shot CoT prompting, which was introduced in section 7.3.3, natural language rationales are drawn from an LLM. For example, when presented with a complex problem, the LLM generates intermediate rationales that articulate the reasoning process leading to the answer.
- 2 **Multi-task Training Framework:** The extracted rationales are then used to train small, task-specific models within a multi-task learning framework. This framework

employs task prefixes to inform the model of the type of output required, whether it be a label prediction or a rationale generation.

The implementation of this approach involves two main stages:

- 1 In the first stage, CoT prompting, comprising exemplars with inputs, rationales, and outputs, guides the LLM to generate rationales for new inputs in a similar structured format.
- 2 The second stage involves incorporating the rationales into the training of small models. This is done by framing the training as a multi-task problem, with the model learning to generate both the label and the reasoning steps leading to the label.

Image 9.8 illustrates this process.

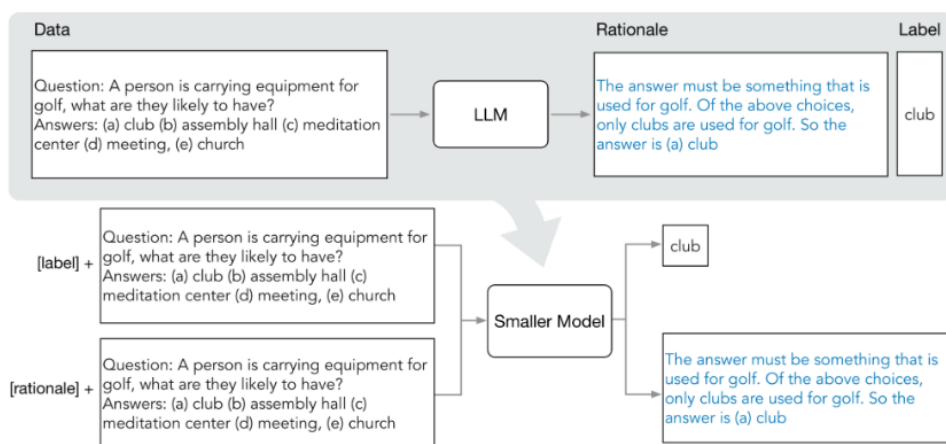


Figure 9.8 Overview of distilling step by step. Initially, chain-of-thought prompting is used to elicit detailed rationales from a large language model. These rationales are then leveraged to inform the training of a compact, task-specific model within a multi-task learning setup. During this training process, specific task-related prefixes are attached to the input examples. This instructs the model to modify its output according to the task indicated by the prefix, effectively enabling the model to distinguish and adapt to a variety of tasks. Image is taken from the paper: "Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes"

The results from the paper show that "distilling step-by-step" can achieve superior performance with significantly less data compared to standard fine-tuning, highlighting its potential to create efficient yet powerful smaller models.

The code for this method can be found here: <https://github.com/google-research/distilling-step-by-step>.

9.4 Parameter efficient fine-tuning LLMs

Parameter efficient fine-tuning LLMs (PEFT), encompasses a set of strategies designed to adapt LLMs to new tasks or domains with minimal adjustments to their parameters. Unlike traditional fine-tuning, which often involves updating a vast number of weights across the

entire network, PEFT techniques aim to achieve comparable performance enhancements by only modifying a small subset of the model's parameters. This approach not only preserves the generalizable knowledge embedded in the pre-trained models but also significantly reduces the computational cost and risk of overfitting associated with adapting these often billion parameter models to new tasks. I will introduce the following methods in this section:

- Low-rank adaptation (LoRA) (4) introduces low-rank matrices that interact with the original weights of the model, allowing for efficient updates to specific parts of the network without the need to retrain the entire model.
- Weight-decomposed low-rank adaptation (DoRA) (5) decomposes the pre-trained weights into two components, magnitude and direction.
- Quantization is the process of converting the model weights from a higher precision numerical format to a lower precision one.
- Quantized low rank adapters (QLoRA) (1) introduces 4-bit NormalFloat (NF4), a new data type that is information theoretically optimal for normally distributed weights and double quantization to reduce the average memory footprint by quantizing the quantization constants.
- Quantization-aware low-rank adaptation (QA-LoRA) (6) introduces group-wise operations which increases quantization flexibility and reduces adaption parameters to achieve better balance and allows end-to-end INT4 quantization without post-training quantization.
- Low-rank plus quantized matrix decomposition (LQ-LoRA) (2) uses an iterative algorithm to decompose each pre-trained matrix into a high-precision low-rank component and a memory-efficient quantized component.

Now let's dive into the specifics of how these different methods enhance model performance. Each approach, while distinct in its mechanism, shares the common goal of minimizing parameter adjustments while maximizing model efficiency and effectiveness.

Beginning with LoRA, I explain the nuances of low-rank matrix adaptation, providing insights into how it enables selective parameter updates that preserve the integrity and generalizability of pre-trained models. Next I'll cover DoRA, which enhances the learning capacity and stability of LoRA.

Subsequently, you will learn how LoRA can be combined with quantization, covering QLoRA, QA-LoRA, LQ-LoRA. By understanding the specific advantages and applications of these methods, you will gain a deeper understanding what each method does and when you want to use it.

9.4.1 Low-rank adaptation

LoRA freezes the pretrained model weights and introduces trainable matrices derived from rank decomposition into each layer of the transformer architecture. This strategy significantly reduces the number of parameters that require training for downstream tasks.

Conceptually, if you are training a transformer model, the training of the downstream task can be mathematically represented as:

$$W_0 + \Delta W \quad (9.1)$$

Here, W_0 denotes the original pre-trained weight matrix of the transformer model, and ΔW symbolizes the weight updates from the downstream task.

W_0 is a weight matrix with dimensions $d \times k$, where d is the dimension of the output space, and k is the dimension of the input space. In layers that handle self-attention within the transformer model, the input and output space dimensions are identical, yielding a matrix of dimensions $d_{\text{model}} \times d_{\text{model}}$. During adaptation, the matrix W_0 remains fixed. LoRA then formulates the update as:

$$W_0 + \Delta W = W_0 + BA \quad (9.2)$$

The update matrix ΔW is defined as the product of two matrices B and A , where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. The rank r is a hyperparameter that governs the rank of the update matrix ΔW , chosen so that $r \ll \min(d, k)$, guaranteeing that ΔW is indeed low-rank.

So, the essence of LoRA is, to update the original weight matrix W_0 in a parameter-efficient manner by learning a low-rank update $\Delta W = BA$, rather than directly modifying ΔW . Below is a simplified example illustrating how this decomposition can approximate ΔW .

Listing 9.11 Illustrative example of low-rank adaptation

```
def lora_decomposition(W, r):
    ❶ Initialize B with zeros (d x r) and A with random Gaussian values (r x d)
    B = np.zeros((W.shape[0], r))
    A = np.random.normal(0, 1, (r, W.shape[1]))

    ❷ For this illustrative example, I use SVD for the optimal low-rank approximation
    U, S, Vt = np.linalg.svd(W, full_matrices=False)
    B[:, :r] = U[:, :r] * np.sqrt(S[:r])
    A[:r, :] = np.sqrt(S[:r])[ :, np.newaxis] * Vt[:r, :]

    ❸ Compute the low-rank approximation of W
    W_approx = B @ A

    return B, A, W_approx

❹ Create a random integer square matrix W with dimension d x d
d = 6
❺ Dimension of the square matrix
W = np.random.randint(0, 10, size=(d, d))

❻ Rank r for the low-rank approximation, with r < d
r = 5 # Rank for the decomposition

❼ Perform the decomposition
B, A, W_approx = lora_decomposition(W, r)
```

Note that in the code, singular value decomposition (SVD) is used to find the optimal low-rank approximation of matrix W . By selecting the top r singular values and their corresponding singular vectors, B and A are constructed in such a way that their product approximates W . This is an example of a rank- r approximation. In neural network weight training, SVD is not typically used as B and A are intended to be learned during training. Nevertheless, this code serves to illustrate the concept of low-rank approximations and their potential closeness to the original matrix.

To demonstrate how closely we can approximate W , let's consider the original matrix W and its approximations through the matrices B , A , and W_{approx} as shown in the following figures:

```
[[2 8 2 4 5 1]
 [7 4 6 7 1 5]
 [7 1 8 7 0 3]
 [5 1 0 0 0 5]
 [5 3 1 0 5 7]
 [7 5 3 2 1 2]]
```

Figure 9.9 Original matrix W before decomposition, with dimension 6×6 .

For matrix B (after decomposition) I get the matrix as shown in 9.10:

```
[[-1.63975843 -1.33957697 -2.07707937 -0.16573653  0.18276492]
 [-2.69725086  0.70080261 -0.06759739 -0.29920634  0.81155888]
 [-2.43915942  1.84683617  0.07585973 -0.41680288 -0.57780471]
 [-1.08102782 -0.56800259  1.53387208  0.50825529  0.6897052 ]
 [-1.67257669 -1.79246668  1.13205129 -0.86851545 -0.54666945]
 [-1.85552222 -0.31598708 -0.07997718  1.61607965 -0.49072672]]
```

Figure 9.10 Matrix B as part of the decomposition, with dimension 6×5 ($d \times r$).

And for A (after decomposition) I get the matrix as shown in 9.11:

```
[[-2.82949436 -1.78893697 -1.98157622 -1.98198396 -0.90532274 -1.85518664]
 [ 0.12216945 -1.4565141  1.45308667  1.26912081 -1.63691814 -0.89096464]
 [ 1.08890724 -1.54181904 -0.38426074 -1.0558866 -0.61174412  1.66300157]
 [ 1.06453149  0.77740217 -0.37838641 -0.62396052 -0.98449913 -0.82204055]
 [-0.35179518  0.35596623 -0.68899414  0.67894206 -0.734205  0.56217305]]
```

Figure 9.11 Matrix A as part of the decomposition, with dimension 5×6 ($r \times d$).

If I now, approximate W as W_{approx} , I get the matrix as shown in 9.12:

```
[[ 1.97355752  8.02323197  2.03771342  3.97055002  4.97691009  1.02037504]
 [ 7.03984842  3.96498977  5.94316643  7.04438069  1.03479616  4.96929512]
 [ 6.96938812  1.02689514  8.04366001  6.9659064 -0.02673069  3.02358774]
 [ 4.95803279  1.03687176  0.05985548 -0.04674046 -0.03664631  5.0323375 ]
 [ 5.01401415  2.98768737  0.9800124  0.01560809  5.01223734  6.98920151]
 [ 7.01750091  4.98462396  2.97503944  2.01949142  1.01528202  1.98651482]]
```

Figure 9.12 Low-rank approximation of W , with dimension 6×6 .

To make it easier to compare, I round the matrix to integer representation as shown in 9.13 and compare it to the original matrix W 9.14.

```

[[2 8 2 4 5 1]
 [7 4 6 7 1 5]
 [7 1 8 7 0 3]
 [5 1 0 0 0 5]
 [5 3 1 0 5 7]
 [7 5 3 2 1 2]]

```

Figure 9.13 Low-rank approximation of W converted to integers.

```

[[2 8 2 4 5 1]
 [7 4 6 7 1 5]
 [7 1 8 7 0 3]
 [5 1 0 0 0 5]
 [5 3 1 0 5 7]
 [7 5 3 2 1 2]]

```

Figure 9.14 Original matrix W before decomposition.

Now that you understand the basics of LoRA, let us look at an improved method, called DoRA, in the next section.

9.4.2 Weight-decomposed low-rank adaptation

DoRA reparameterizes model weights into magnitude and directional components, aiming to closely examine and compare the changes from fine-tuning (FT) and LoRA in both magnitude and direction. This approach is grounded in the theory that gradient optimization with weight reparameterization can achieve faster convergence. By deconstructing the weight matrix into distinct magnitude and directional elements, DoRA reveals the inherent learning pattern differences between FT and LoRA, providing insightful analysis into their respective adjustments. Illustration 9.15 shows an overview of DoRA.

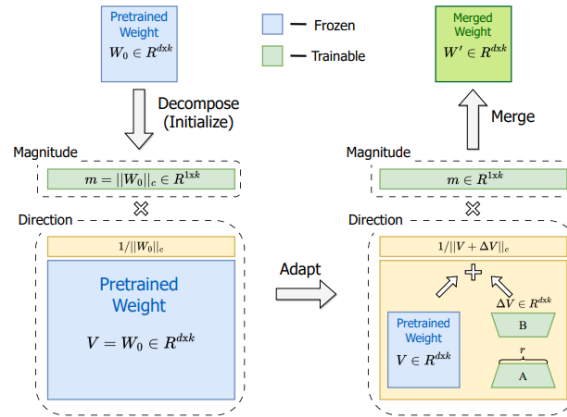


Figure 9.15 And overview of weight-decomposed low-rank adaptation (DoRA), which segments the pre-trained weight into separate magnitude and direction elements for fine-tuning purposes, utilizing LoRA to effectively refine the direction aspect. It's important to note that $\|\cdot\|_c$ represents the vector-wise norm across each column vector in a matrix. Image is taken from the paper: "DoRA: Weight-Decomposed Low-Rank Adaptation".

Initial findings demonstrate that DoRA, akin to FT, can facilitate more nuanced and effective learning adjustments, contrasting with LoRA's proportional changes in direction and magnitude. This can lead to faster and more efficient convergence in model training, by enabling separate, focused tuning of magnitude and direction components. The mathematical foundation of DoRA's weight decomposition is formulated as follows:

$$W = m \frac{V}{\|V\|_c} = \|W\|_c \frac{W}{\|W\|_c} \quad (9.3)$$

where $m \in \mathbb{R}^{1 \times k}$ is the magnitude vector, $V \in \mathbb{R}^{d \times k}$ is the directional matrix, and $\|\cdot\|_c$ denotes the vector-wise norm of a matrix across each column. This means, division by $\|\cdot\|_c$ isn't literal division but is meant to denote element-wise scaling of the matrix by the column-wise norm, resulting in each column being a unit vector.

Distinguishing itself from weight normalization, which trains weight components from scratch and is thus sensitive to initialization. DoRA leverages pre-trained weights to overcome these initialization issues, as demonstrated in the following initialization strategy:

$$W' = m \frac{V + \Delta V}{\|V + \Delta V\|_c} = m \frac{W_0 + BA}{\|W_0 + BA\|_c} \quad (9.4)$$

Here, W_0 is the pre-trained weight as outlined in equation 9.4.2, where $m = \|W_0\|_c$ and $V = W_0$ after initialization. And ΔV represents the incremental directional update learned by multiplying two low-rank matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, aligning with LoRA's strategy to ensure that before fine-tuning. Furthermore, V is kept frozen and m is a trainable vector.

The matrices A and B are initialized in line with LoRA's strategy, reinforcing the seamless integration of DoRA's adjustments into the pre-trained model framework. This approach not only preserves the original model's latent knowledge but also enhances its adaptability and performance across various tasks without the overhead of extensive retraining or additional computational costs.

9.4.3 Quantization

Quantization is a method used to decrease both the computational and memory demands of the inference of neural networks, such as LLMs. This is achieved by encoding weights and activations in lower-precision formats, such as 8-bit integers (int8), as opposed to the standard 32-bit floating-point (float32) types.

By decreasing the bit depth, the modified model will use less memory, theoretically uses less power, and benefits from quicker computational operations, such as matrix multiplication, due to the efficiency of integer arithmetic. Additionally, this enables the deployment of models on embedded systems that may only accommodate integer data types.

Table 9.1 Comparison of Data Types

Data Type	Bit Width	Range	Memory Reduction from FP32
float32	32 bits	$\approx \pm 3.4 \times 10^{38}$	-
float16	16 bits	$\approx \pm 6.5 \times 10^4$	50%
int8	8 bits	-128 to 127	75%
int4	4 bits	-8 to 7	87.5%

The memory reduction achieved by transitioning from a 32-bit floating point (float32) to other data types can be computed based on the bit-width of each type. Below are the calculations for each transition: float32 uses 32 bits, and float16 uses 16 bits. The memory reduction is computed as:

$$\text{Reduction} = 1 - \frac{\text{Size of float16}}{\text{Size of float32}} = 1 - \frac{16 \text{ bits}}{32 \text{ bits}} = 0.5 \text{ or } 50\% \quad (9.5)$$

Where 1 represents the full memory usage of the original type.

int8 uses 8 bits, the memory reduction is computed as:

$$\text{Reduction} = 1 - \frac{\text{Size of int8}}{\text{Size of float32}} = 1 - \frac{8 \text{ bits}}{32 \text{ bits}} = 0.75 \text{ or } 75\% \quad (9.6)$$

int4 uses 4 bits, the memory reduction is computed as:

$$\text{Reduction} = 1 - \frac{\text{Size of int4}}{\text{Size of float32}} = 1 - \frac{4 \text{ bits}}{32 \text{ bits}} = 0.875 \text{ or } 87.5\% \quad (9.7)$$

These reductions in memory requirements are crucial to improve the efficiency of model inference, and make it also possible to fine-tune a 7B model with the free Google Colab version. However, reducing the precision can have an impact on the accuracy of the model. Let us consider two examples why this might be tricky, quantization from float32 to float16 and float32 to int8. The first one, float32 to float16, is simple, because both data types are the same (float).

Let me visualize how much memory we can save if we load, for instance, falcon 7B with and without quantization. To load the model I use the library bitsandbytes: <https://github.com/TimDettmers/bitsandbytes>

Listing 9.12 Loading Falcon 7B with bitsandbytes

```
model_id = "tiiuae/falcon-7b"

❶ Ensure CUDA is available
if torch.cuda.is_available():
    ❷ Reset peak memory statistics
    torch.cuda.reset_peak_memory_stats()

    ❸ Capture initial GPU memory usage
    device = torch.device("cuda")
    initial_memory = torch.cuda.memory_allocated(device)

    ❹ BitsAndBytes configuration
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True, ❺ Load model in 4-bit
        load_in_8bit=False, ❻ If set to true, model loads in 8-bit
        bnb_4bit_use_double_quant=False,
        bnb_4bit_quant_type="fp4",
        ❼ Use float16 for computation, such as fine-tuning or DPO
        bnb_4bit_compute_dtype=torch.bfloat16
    )
```

```

⑧ Load tokenizer and model with BnB configuration
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id,
                                              quantization_config=bnb_config)

⑨ Capture GPU memory usage after loading the model
final_memory = torch.cuda.memory_allocated(device) / (1024**2)

⑩ Peak memory during the process in GB
peak_memory = torch.cuda.max_memory_allocated(device) / (1024**2)

⑪ Calculate the difference
memory_difference = final_memory - initial_memory

print(f"Initial GPU Memory Usage: initial_memory/1024 GB")
print(f"Final GPU Memory Usage: final_memory/1024 GB")
print(f"Memory Difference (Model Load Impact): memory_difference/1024 GB")
print(f"Peak GPU Memory Usage: peak_memory/1024 GB")
else:
    print("CUDA is not available. Please check your PyTorch and GPU setup.")

```

Running the code will result in this output:

```

Initial GPU Memory Usage: 0.0 GB
Final GPU Memory Usage: 4.074565887451172 GB
Memory Difference (Model Load Impact): 4.074565887451172 GB
Peak GPU Memory Usage: 4.609706878662109 GB

```

If I load the model without the use of bitsandbytes it will result in the following memory usage:

```

Initial GPU Memory Usage: 0.0 GB
Final GPU Memory Usage: 25.876148223876953 GB
Memory Difference (Model Load Impact): 25.876148223876953 GB
Peak GPU Memory Usage: 25.876148223876953 GB

```

However, a potential issue with reduced precision is the inability to represent very small or very large numbers that fall outside the range of the reduced precision format, as showed in table 9.1. This can lead to NaN values and is a critical point to understand the trade-offs between memory efficiency and numerical accuracy. Using libraries such as bitsandbytes, which use 4-bit quantization for the pretrained model weights while allowing for computations (such as training or fine-tuning "on top" of these pretrained models) to occur in higher precision formats like 16-bit floating-point (float16) or bfloat16, we can achieve several beneficial outcomes:

- **Memory Efficiency:** The 4-bit quantization dramatically reduces the memory footprint of the model's weights. This reduction is crucial for deploying large models on hardware with limited memory resources or for applications that require running multiple models simultaneously.
- **Performance Preservation:** Despite the significant reduction in memory usage, the precision for training or inference computations can be maintained at a higher level (e.g., float16). This approach ensures that the quantization process does not lead to

a significant loss in model accuracy or performance. Higher precision formats like float16 offer a good compromise between computational efficiency and numerical precision, enabling faster computation than float32 without the substantial accuracy loss that might occur with lower precision formats.

The landscape of quantization techniques, can be categorized into the following categories:

- **Uniform quantization:** Applies equal-sized quantization intervals across the entire range of values. This simplicity makes it more straightforward to implement on hardware, leading to increased efficiency in terms of computational resources and energy consumption.
- **Non-uniform quantization:** Adopts variable-sized intervals, which can be tailored to the distribution of the data and applied differently across various weights or layers within the model. This approach is advantageous for minimizing quantization error, thereby potentially improving model performance, especially in scenarios where precision is crucial. **Quantization-aware training (QAT):** Involves retraining the model with quantized weights and activations, allowing the model to adapt to the quantization-induced changes. This can lead to better retention of performance, even at extremely low precision levels (e.g., 2-bit quantization). The approach, however, is computationally intensive due to the additional training required.
- **Post-training quantization (PTQ):** Applies quantization to a pre-trained model without further training. This method is less computationally demanding than QAT but typically does not achieve as low levels of quantization (limited to 8-bit or 6-bit). Despite its limitations, PTQ can be advantageous in scenarios where computational resources are limited or when rapid deployment is necessary.

Now that we've covered in depth what quantization is and how it improves the efficiency of model inference, let's explore QLoRA in the next section. QLoRA, a quantized version of LoRA, takes these efficiency enhancements further by specifically tailoring the quantization process to the unique structure of LoRA, offering even greater performance and memory usage improvements.

9.4.4 Efficient fine-tuning of quantized LLMs with QLoRA

QLoRA, achieves a significant reduction in memory usage, enabling the fine-tuning of a 65-billion parameter model on a single 48GB GPU while preserving 16-bit fine-tuning precision. This is made possible through the introduction of the 4-bit NormalFloat (NF4), a new data type that is information-theoretically optimal for representing weights that follow a normal (Gaussian) distribution, which is common among neural network weights. This optimality stems from information theory, which deals with the efficient quantification, storage, and communication of information. In essence, NF4 is designed to "pack" weight values into a 4-bit format as densely as possible, following the principle that more common values (according to a normal distribution) are assigned shorter codes. Quantile quantization operates by approximating the quantile of the input tensor using the empirical cumulative distribution function. Moreover, QLoRA uses a technique called double quantization. This technique applies a secondary layer of quantization to the constants used in the initial

quantization step, further reducing the memory footprint. By optimizing the storage of these constants, QLoRA minimizes additional memory demands, enabling more efficient use of available resources. To address the challenge of memory spikes during gradient checkpointing, a common issue that can lead to out-of-memory errors during fine-tuning, QLoRA introduces paged optimizers. This solution leverages the concept of memory paging, traditionally used in managing computer memory, to dynamically allocate and manage memory during the training process. By efficiently moving data between the CPU and GPU, paged optimizers ensure smooth and uninterrupted model optimization, even under tight memory constraints. Image 9.16 shows an overview how QLoRA compares to fine-tuning and to LoRA.

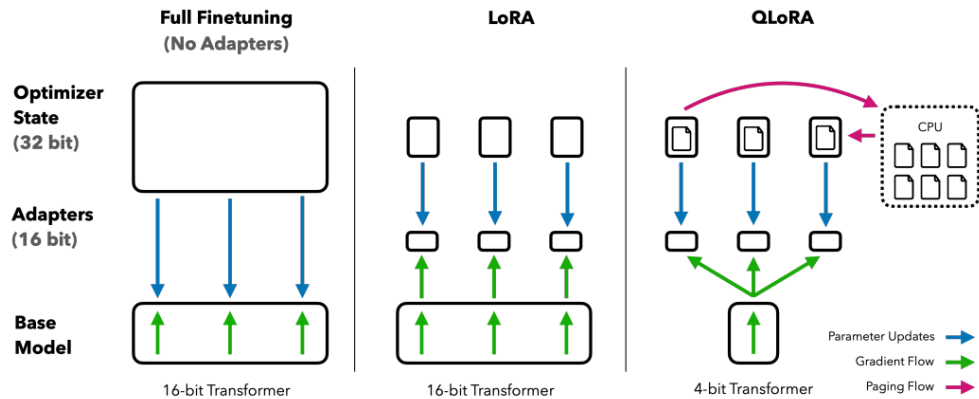


Figure 9.16 Overview of fine-tuning methods, with on the left the "usual" fine-tuning, without any optimization, in the middle LoRA and on the right QLoRA. Image is taken from the paper: "QLORA: Efficient Finetuning of Quantized LLMs".

Listing 9.13 shows how to load your model with this method.

Listing 9.13 Loading Falcon 7B with bitsandbytes

```
model_id = "tiiuae/falcon-7b"

❶ Ensure CUDA is available
if torch.cuda.is_available():
    ❷ Reset peak memory statistics
    torch.cuda.reset_peak_memory_stats()

    ❸ Capture initial GPU memory usage
    device = torch.device("cuda")
    initial_memory = torch.cuda.memory_allocated(device)

    ❹ BitsAndBytes configuration
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True, ❺ Load model in 4-bit
        bnb_4bit_use_double_quant=True,
```


The key takeaway here is that both bfloat16 and float32 allocate 8 bits to the exponent, which means they can represent a comparable range of magnitudes. The significant difference lies in the precision, where float32, with its 23 bits for the mantissa, offers much finer granularity and closer value representation than bfloat16's 7 mantissa bits. The mantissa in the floating data point is used to represent the fraction of a number, so for instance, if we take pi (3.14159265), the numbers after 3 will be represented with the mantissa.

Therefore this distinction in mantissa size directly impacts the precision of the represented numbers:

- Float32 can differentiate between numbers that are very close together, thanks to its higher number of mantissa bits. This makes it suitable for applications requiring high numerical precision, such as scientific computations.
- BFloat16, with fewer mantissa bits, has less precision for individual numbers but maintains the broad range necessary for many machine learning algorithms. This makes bfloat16 particularly useful for neural network training, where the hardware efficiency and memory bandwidth savings from using a 16-bit format can significantly speed up computation, and the exact precision of every operation is often less critical.

The reason why bfloat is used for neural networks is, that the larger exponent allows bfloat16 to represent both very large and very small numbers. This is essential for capturing the wide range of values that, for instance, gradients can take on, especially in deep networks or complex models where the gradients may vary greatly in scale across different layers and weights.

9.4.5 Quantization-aware low-rank adaptation

QA-LoRA is a quantization-aware low-rank adaptation algorithm. This method aims to achieve two goals:

- 1 Allowing LLMs to be fine-tuned using the minimum number of GPUs necessary in the fine-tuning phase, as the pre-existing weights W are converted into a low-bit format.
- 2 Facilitate the deployment of LLMs with enhanced computational efficiency, this is achieved by letting the combined weights W remain in a quantized state.

The first goal is similar to what you've read about QLoRA, where the higher precision was converted into the new format nf4. However, QLoRA introduced significant advancements, including double quantization and optimized memory management through paged optimizers. These innovations specifically address memory spikes and ensure efficient data movement between the CPU and GPU. In comparison, QA-LoRA advances further by optimizing the balance between quantization and adaptation. It refines the quantization process to accommodate the unique demands of LLMs, ensuring that each quantization step contributes positively to the model's efficiency and accuracy. This balance is critical in maintaining the delicate balance between computational demands and performance.

At the core of QA-LoRA's methodology is its use of group-wise operations, which not only optimize quantization but also ensure that the low-rank adaptations are computation-

ally efficient. This efficiency is achieved without the high memory cost typically associated with fine-tuning LLMs, marking a notable improvement over traditional techniques. The adaptation process within QA-LoRA is tuned to leverage the strengths of low-bit representation, ensuring that the quantized weights seamlessly integrate with the adaptation mechanism to produce a model that is both accurate and efficient. Figure 9.18 illustrates the differences between LoRA, QLoRA and QA-LoRA, respectively.

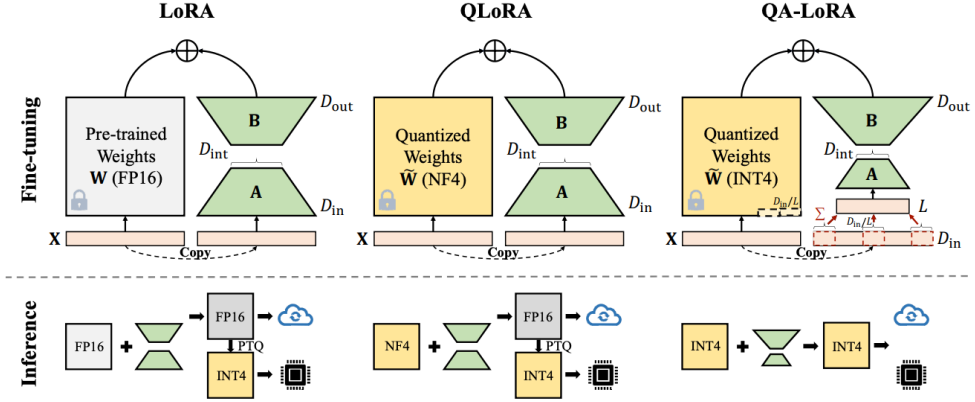


Figure 9.18 The illustration demonstrates the goal of QA-LoRA. That is, unlike previous adaptation techniques, such as LoRA and QLoRA, this method is more computationally effective during both the fine-tuning and inference phases. Importantly, it avoids a decrease in accuracy as it eliminates the need for post-training quantization. Although the figure presents int4 quantization, QA-LoRA is adaptable to both int3 and int2. Image is taken from the paper: QA-LoRA Quantization-Aware Low-Rank Adaptation of Large Language Models.

QA-LoRA partitions weight matrices into groups and applies quantization and adaptation at this granular levels. This granular approach allows for the tailored adaptation of each group, maintaining high precision, while reducing memory footprint.

By adjusting the granularity of quantization and focusing on group-wise adaptation, QA-LoRA not only preserves the model's precision and efficiency but also enhances the LLM's ability to be fine-tuned on new data effectively.

9.4.6 Low-rank plus quantized matrix decomposition

LQ-LORA uses a simple factorization scheme to break down each pre-trained matrix into a component of high precision but low rank and another component that is quantized for memory efficiency. Throughout the fine-tuning phase, the quantized component is kept constant while updates are only applied to the low-rank component. This approach is rooted in the understanding that traditional methods, like LoRA, which reparameterize a pre-trained matrix $W_0 + \Delta W = W_0 + BA$ and initialize A with Gaussian values and B to zero, can maintain the model's initial output constancy at the onset of fine-tuning. However, this strategy may not be ideal when dealing with a quantized version of W , especially considering the potential significant discrepancy between W and its quantized version when quantizing to lower bits.

Recognizing the limitations of conventional initialization, which overlooks the inherent structure of W in deciding adaptation subspaces, LQ-LoRA adopts a matrix factorization perspective. This perspective aims to factorize the original matrix into a component that is quantizable and a low-rank component that captures high-variance directions. Such decomposition not only aligns with the principles of robust principal components analysis but also adapts its iterative algorithms for effective application, toggling between optimizing the low-rank components and the quantized component for optimal reconstruction.

Throughout the fine-tuning phase, the focus is on adjusting only the low-rank component, keeping the quantized component unchanged. This selective update strategy ensures that the quantized element contributes to memory efficiency without compromising the adaptation process's quality or the model's performance. Each step of this refined algorithm, including randomized SVD for low-rank approximation followed by quantization for the second component, is executed on contemporary GPUs.

9.5 Sharding LLMs for memory optimization

This section explores the concept of sharding as a strategy for memory optimization. Sharding, a technique traditionally associated with databases to distribute data across multiple servers, can also be applied to distribute the computational and storage load of LLMs across multiple devices or nodes. By partitioning the model's parameters into smaller, manageable chunks, or shards, it becomes feasible to train and deploy models that were previously unattainable due to hardware constraints. Essentially, sharding an LLM entails segmenting it into more manageable pieces. Figure 9.19 illustrates this concept.

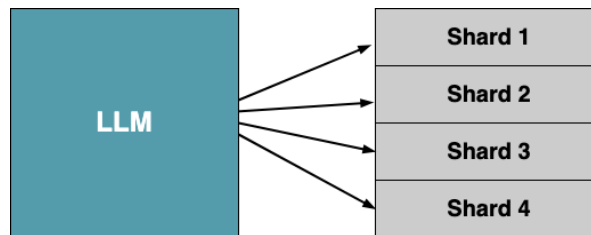


Figure 9.19 Illustration of the concept of sharding, where an large language model is broken down in pieces to distribute its computational and memory load more efficiently. Each shard can be processed independently, or on different devices or processors, which can lead to improvements in memory efficiency, inference speed, and scalability of the LLM.

Memory efficiency is one of the benefit of sharding, allowing for the operation of large models on hardware with limited memory capacity. This technique circumvents the need to load the entire model into memory simultaneously, allowing only the needed segments to be loaded and processed. As a result, memory requirements are significantly reduced. Additionally, sharding facilitates the distribution of computations across multiple devices, achieving parallel processing that notably speeds up inference times. This acceleration is particularly helpful for large models, which would otherwise exhibit slow performance on a single device.

Furthermore, sharding enhances the scalability of model deployment on distributed systems equipped with multiple GPUs or across computing clusters. This capability is important for managing substantial workloads and tasks on a grand scale. Sharding also plays a pivotal role in distributed inference, especially within large-scale distributed systems where processing power is spread across numerous nodes or GPUs. By optimizing the use of computational resources and minimizing communication overhead, sharding ensures a more efficient and streamlined operation, making it a vital technique for memory optimization in the training and deployment of LLMs.

However, while sharding significantly enhances the scalability of model deployment on distributed systems and optimizes computational resource use, it introduces complexities such as the need for efficient synchronization and potential bottlenecks in data communication. These challenges, inherent to distributing workloads across multiple GPUs or computing clusters, require careful management to ensure the seamless operation of sharded LLMs.

To use sharding with Hugging Face, you can use the `accelerate` library. The following listings 9.14 - 9.17 show a step-by-step approach to use sharding.

Listing 9.14 Print system resources

```
def print_system_resources():
    num_cpus = os.cpu_count()
    print(f'Number of CPU cores: {num_cpus}')
    print(f'Total CPU RAM: psutil.virtual_memory().total / (1024**3):.2f GB')

    if torch.cuda.is_available():
        num_gpus = torch.cuda.device_count()
        print(f'Number of GPUs: {num_gpus}')
        for i in range(num_gpus):
            print(f'GPU {i} Name: {torch.cuda.get_device_name(i)}')
            print(f'GPU {i} RAM: {torch.cuda.get_device_properties(i).
                    total_memory / (1024**3):.2f GB}')
    else:
        print('No GPUs found')
```

This code will print out the number of CPU cores and the number of GPUs. In listing 9.15 is the function to load and shard the model

Listing 9.15 Sharding Falcon 7B with accelerate

```
def shard_and_load_model(model_name, save_directory, max_shard_size, device_map):
    ❶ Initialize tokenizer and model
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)

    ❷ Initialize Accelerator
    accelerator = Accelerator()

    ❸ Save model with sharding
```

```

accelerator.save_model(model=model, save_directory=save_directory,
                       max_shard_size=max_shard_size)

```

④ Load model from checkpoint with device map

```

model = load_checkpoint_and_dispatch(
    model, checkpoint=save_directory, device_map=device_map,
    no_split_module_classes=['Block']
)

return model, tokenizer

```

As a next step, in listing 9.16 is the function to generate the model outputs.

Listing 9.16 Generate model outputs

```

def generate_outputs(model, tokenizer, input_text, max_new_tokens):
    inputs = tokenizer(input_text, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=max_new_tokens,
                             return_dict_in_generate=True, output_scores=True)
    return tokenizer.decode(outputs.sequences[0], skip_special_tokens=True)

```

After having set up all our needed functions, we can now use a main function to load the model and ask it a question as demonstrated in listing 9.17.

Listing 9.17 Main function to generate model output

```

def main():
    model_name = "tiiuae/falcon-7b"
    save_directory = '/content/model'
    device_map = "": 'cpu'

    ① Print system resources
    print_system_resources()

    ① Shard and load the model
    model, tokenizer = shard_and_load_model(model_name,
                                             save_directory, "2GB", device_map)

    ② Generate outputs
    raw_input_text = "Tell me something about falcons"
    generated_text = generate_outputs(model,
                                      tokenizer, raw_input_text, 100)

    print(f'Generated Text: {generated_text}')

```

NOTE Note that I use as device CPU, for that reason the computation will be slower as with a GPU, if you have multiple GPUs you can change this to gpu.

In the following is the model's answer to the prompt "Tell me something about falcons". Falcons are birds of prey. They are carnivores and they hunt for their food. They are very strong and they can kill their prey with their sharp talons. They are very fast and they can fly very fast. They are very good hunters. They hunt for small animals like rabbits, mice,

squirrels, birds, etc. They hunt for their food in the wild. They are very good at hunting. They are very good at flying.

This section has demonstrated the practical steps required to implement sharding using the Hugging Face's accelerate library, from checking system resources to sharding and loading the model, and finally generating outputs. The potential of sharding extends beyond just memory efficiency; it enables the deployment of massive models in distributed systems.

9.6 Summary

- You must set hyperparameters before you start the training and can not be learned during training. They model hyperparameters are important for the model optimization process, that is, for finding the minimum of a loss function.
- Ray and its libraries Data and Tune offer a way of parallizing and organizing your hyperparameter search without having to understand anything about distributed systems.
- Tracking experiments is important to monitor and evaluate your LLMs during training and inference. Frameworks such as Weights & Biases offer a structured way to generate reports what can be shared with the developement team. Moreover Weights & Biases offers a robust implementation of hyperparameter sweeps that can be used with any machine learning framework, including PyTorch and Hugging Face Transformers.
- Pruning reduces model complexity by identifying and removing parts of the network that contribute the least to its output. There are two pruning methods: unstructured pruning and structured pruning. The first method removes individual weights to significantly reduce model size without guaranteeing improved computational efficiency due to potential hardware optimization issues for sparse matrices. the second method eliminates entire units or groups within the model to align with hardware optimizations, potentially speeding up computation but possibly affecting performance more substantially.
- Model distillation is a technique used to compress the knowledge of a large, complex model (often called a "teacher" model) into a smaller, more efficient one (called a "student" model). The process involves training the student model to mimic the behavior of the teacher model.
- Parameter efficient fine-tuning LLMs (PEFT), encompasses a set of strategies designed to adapt LLMs to new tasks or domains with minimal adjustments to their parameters. Common methods are Low-rank adaptation (LoRA) and Weight-decomposed low-rank adaptation (DoRA).
- Quantization is the process of converting the model weights from a higher precision numerical format to a lower precision one. Libraries such as bitsandbytes make it easy to leverage quantization for common methods such as Quantized low rank adapters (QLoRA).

- Sharding is a method to distribute the computational and storage load of LLMs across multiple devices or nodes. This facilitates the distribution of computations across multiple devices, achieving parallel processing that speeds up for the inference.

Bibliography

- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. [arXiv:2305.14314](#). 253
- [2] Han Guo, Philip Greengard, Eric P. Xing, and Yoon Kim. Lq-lora: Low-rank plus quantized matrix decomposition for efficient language model finetuning, 2024. [arXiv:2311.12023](#). 253
- [3] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes, 2023. [arXiv:2305.02301](#). 251
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL: <https://arxiv.org/abs/2106.09685>, [arXiv:2106.09685](#). 253
- [5] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation, 2024. [arXiv:2402.09353](#). 253
- [6] Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models, 2023. [arXiv:2309.14717](#). 253

appendix

Get the most out of this book

- how to run the code

This book is designed not only to provide you with a solid theoretical foundation but also to ensure that you have the practical skills to apply the concepts in real-world scenarios. To help you gain hands-on experience, I have prepared an extensive code repository that complements the content of each chapter. You can access the code repository with this link: <https://github.com/Nicolepcx/Transformers-in-Action>.

The code repository is organized into individual folders for each chapter, as shown in figure A.1, ensuring that you can easily locate the relevant code snippets and examples for the concepts discussed in the book.

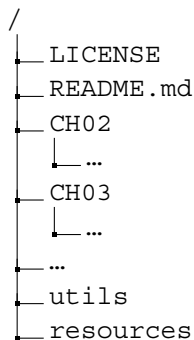


Figure A.1 Directory structure of the repository

I have chosen Jupyter Notebooks as the primary medium for sharing the code, making it simple and interactive for you to follow along and experiment with the models.

Moreover, given the computational requirements of the models and examples provided in this book, it is necessary to have access to a GPU to execute the code. I have made an intentional effort to bring you closer to real-world scenarios, and this often means dealing with larger datasets and more complex models. These tasks can be computationally intensive and require higher GPU resources, particularly a GPU with a large amount of RAM.

Platforms like Paperspace and Google Colab provide these resources in the cloud, making it more accessible to a broader audience. By clicking on the button, as shown in figure A.2, the Jupyter Notebooks will automatically open in your chosen cloud service, even directly from the GitHub repository.

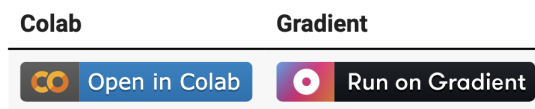


Figure A.2 Just click on one of these buttons to easily open the notebook within your preferred platform.

Running these notebooks in cloud services like Paperspace or Google Colab is not just a matter of convenience. Given the nature of the computations involved in our examples, it's essential to have access to a GPU. That's where the beauty of this setup comes into play. By opening the notebooks in your chosen cloud service, you gain access to powerful GPUs, and the required dependencies for each chapter are automatically installed for you. Since I've programmed specific functions into the notebooks to manage this installation process, so you can focus on learning and experimenting.

All you need to do is connect the notebook to the GPU on the platform. With this approach, you can easily and efficiently run the code directly in your browser, providing you with a seamless learning experience.

In addition, some of the code is organized into classes and functions and imported into the notebooks for better structure and ease of use. When running the notebooks on your chosen cloud service, the GitHub repository will be automatically cloned, ensuring that all necessary code files are available, making it even more convenient for you to focus on learning and experimenting with the models.

That said, I encourage you to actively engage with the code as you progress through the book. This hands-on approach will not only deepen your understanding of the concepts but also enable you to develop the skills necessary to implement and adapt these models to your specific use cases. So, dive in, explore the code, and don't hesitate to modify and experiment with it to solidify your knowledge and expertise in working with Transformers.